IMPERIAL

Department of Chemical Engineering

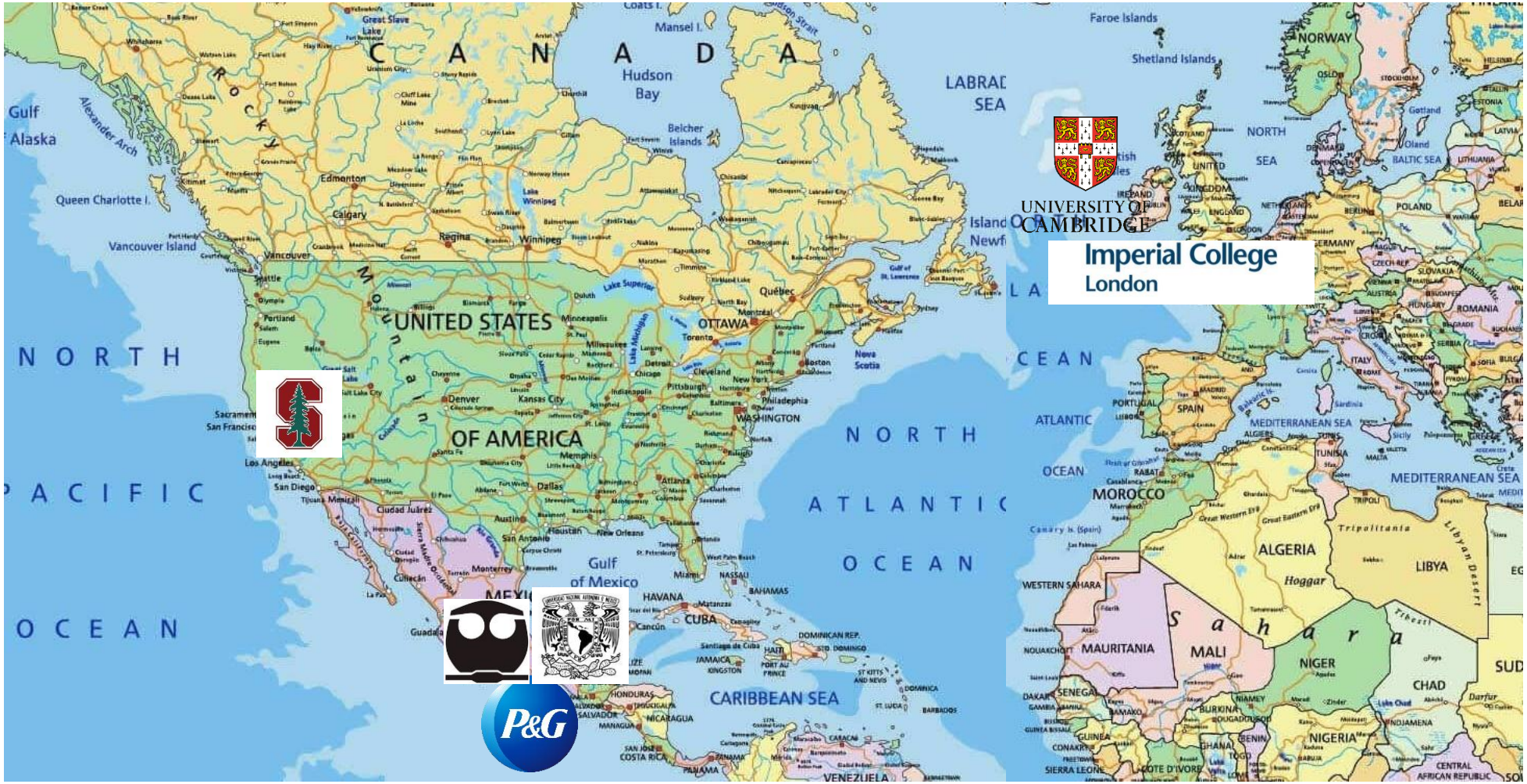
AI & Autonomous Industrial Systems

Lecture – Data-driven optimisation

Antonio del Rio Chanona

a.del-rio-chanona@imperial.ac.uk

A bit about me



Optimisation and Machine Learning for Process Systems Engineering

OPTIMLPSE

Sargent Centre for Process Systems Engineering

PhD Students



Akhil Ahmed

Adaptive Modelling, Control and Optimization of Large-Scale Systems using Machine Learning



Abdullah Bahamdan

Decarbonization of Energy Systems



Max Bloor

Deep Reinforcement Learning for Process Control and Scheduling



Miguel Ángel de Carvalho Servia

Automated Knowledge Discovery Methods in Reaction Engineering



Friedrich Hastedt

Supervised Machine Learning for Retrosynthesis and Synthesis Route Planning



Buse Sibel Korkmaz

Advancing Large Language Models for Comprehensive Scientific Assistance



Niki Kotecha

Multi-Agent Reinforcement Learning for Supply Chain Operations



Yong Lee

Generative Design and Optimisation of Functional Small Molecules



Matthew Marsh

Robust Data Driven Modelling, Optimisation and Control



Mathias Neufang

Statistical Machine Learning and Optimization for Solvent Selection



Ilya Orson Sandoval

Knowledge-driven Autonomous Systems - Neural ODEs and Reinforcement Learning



Emma Pajak

Digitalisation of Chemical Value Chains



Tom Savage

Applied Data-Driven Optimisation



David Tsang

High-throughput Synthetic Cell Engineering using Robotics & Machine Learning



Damien van de Berg

Data-Driven Optimization for the Integration of Interconnected Process Systems



Haiting Wang

Hybrid modeling for bioprocesses - merging first principles and machine learning



Luxi Yu

Towards online quality control of biotherapeutics through soft sensing of intracellular states



Zhengang Zhong

Data-Driven Distributionally Robust Control

Postdoctoral & Affiliated Researchers



Dr. Ana Marisa Arias

Research Associate



Daniel Cristiu

Research Assistant



Laura Helleckes

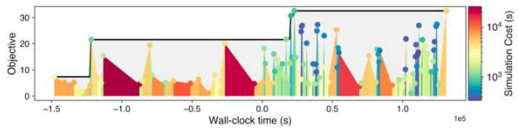
AI Postdoctoral Research Fellow



Dr. Austin Mroz

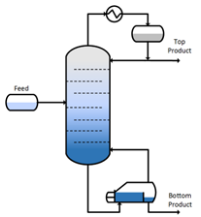
AI Postdoctoral Research Fellow

Data-driven Optimization

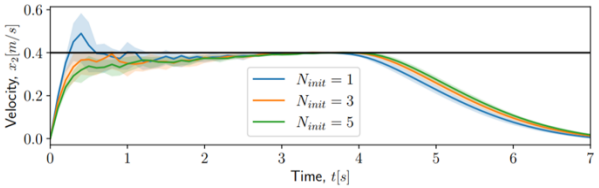


LLMs & Chem Eng

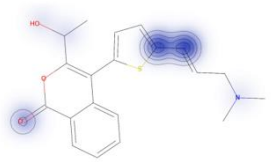


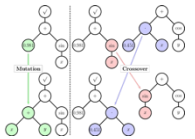


Control

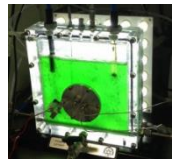


Automated model building

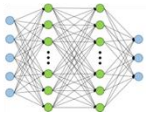




Biochemical applications



$$\frac{d}{dt}\mathbf{x} = \mathbf{f}(\mathbf{x}) +$$



Data-driven optimisation

Data-driven optimization – definitions

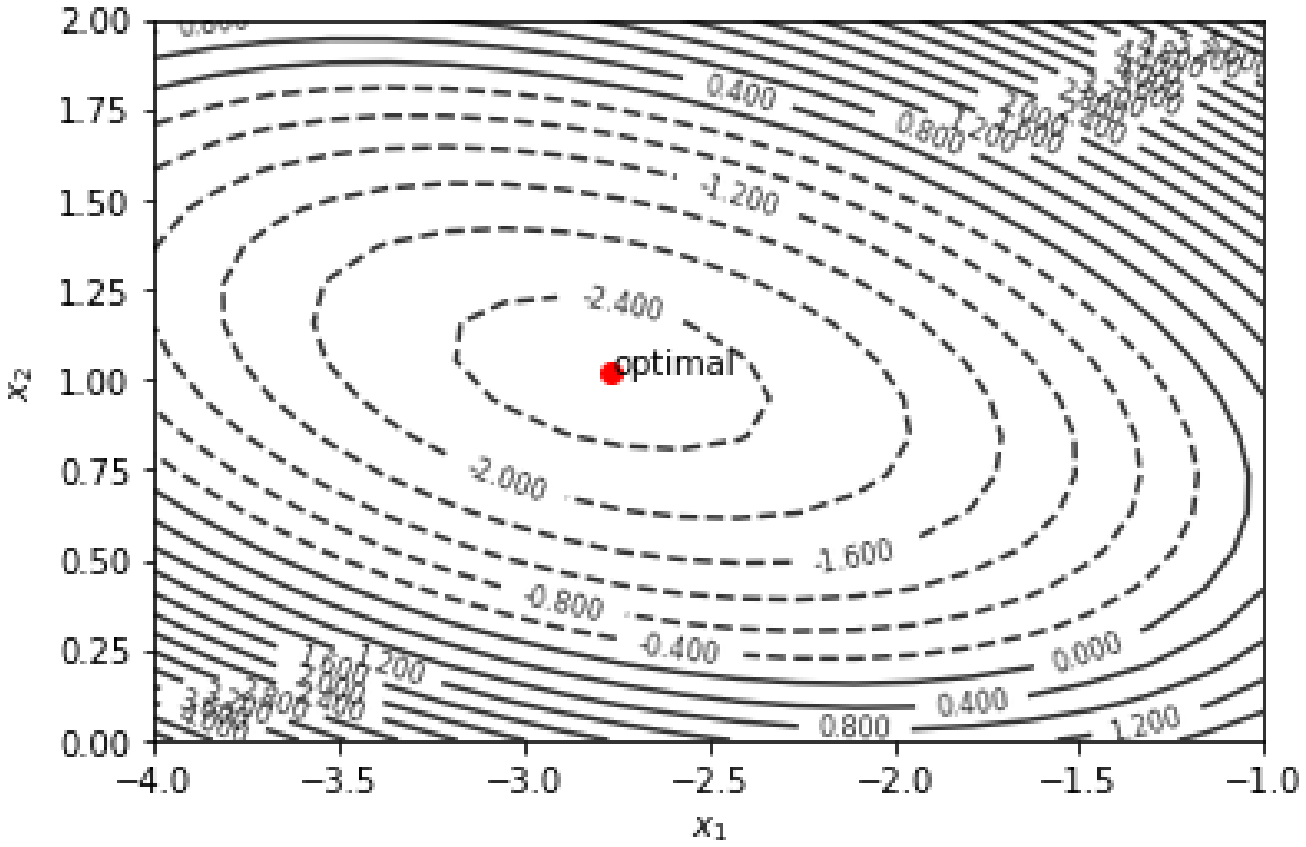
Many definitions:

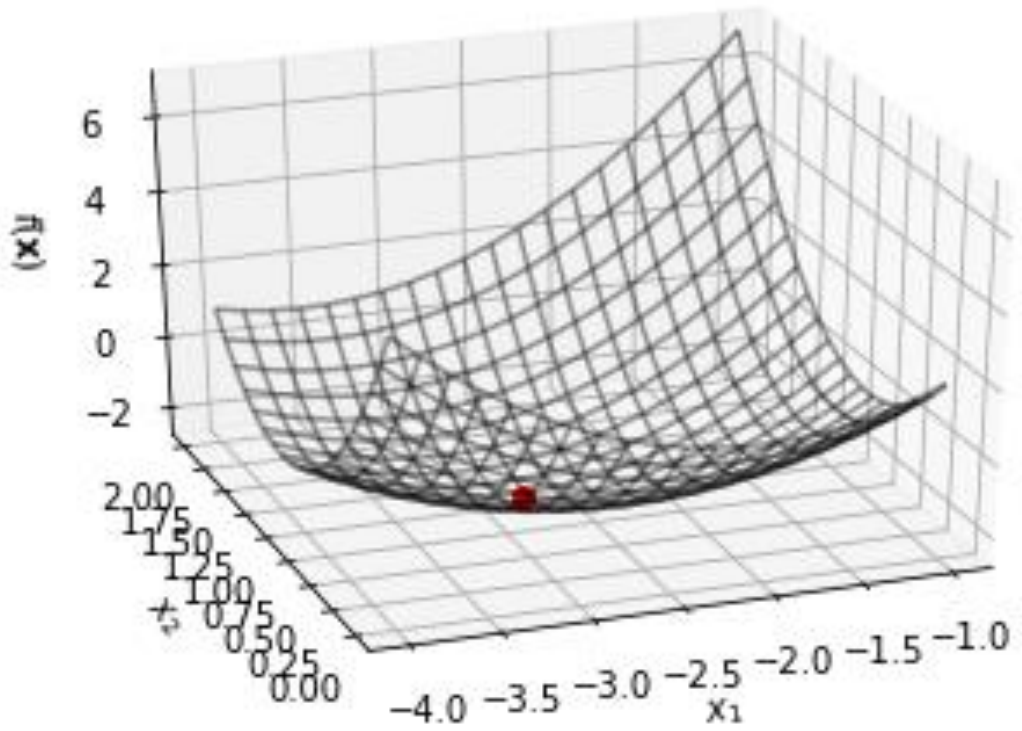
- Data-driven optimization uses observations as direct inputs to mathematical programming problems.
- Decisions driven by uncertain and online data, and how computational algorithms can be used to achieve solutions efficiency and near optimality.
- Algorithms and software tools that enable efficient optimization of systems that rely on input-output data streams.
- The process of collecting data intelligently and efficiently to reach some objective.

I will use data-driven, derivative free, gradient free, zeroth order, simulation-based, black-box **optimization** interchangeably.

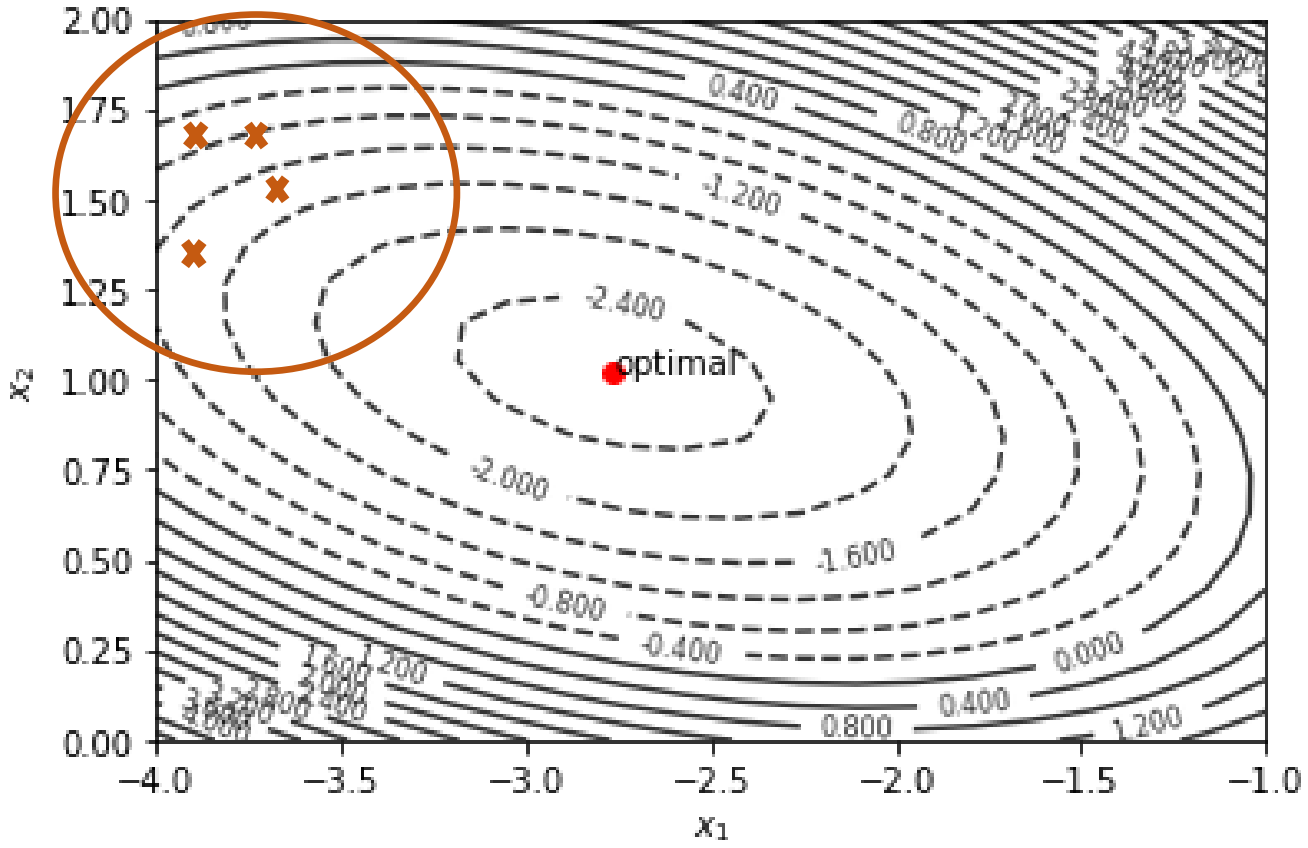
(A) Big picture

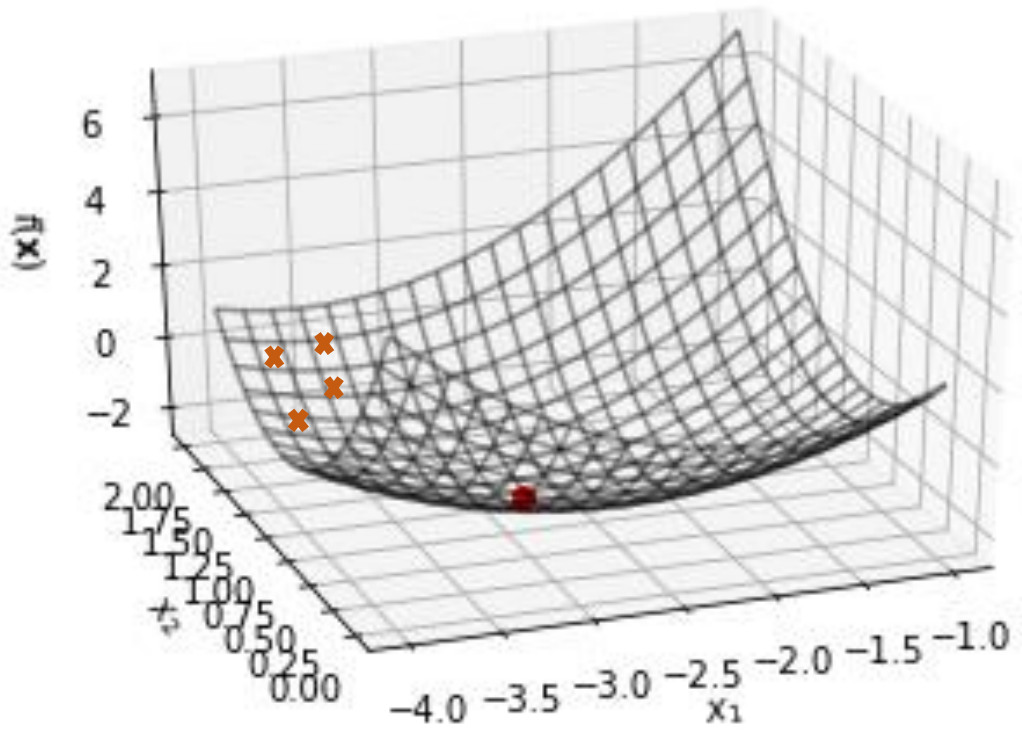
Data-driven optimization – Big picture





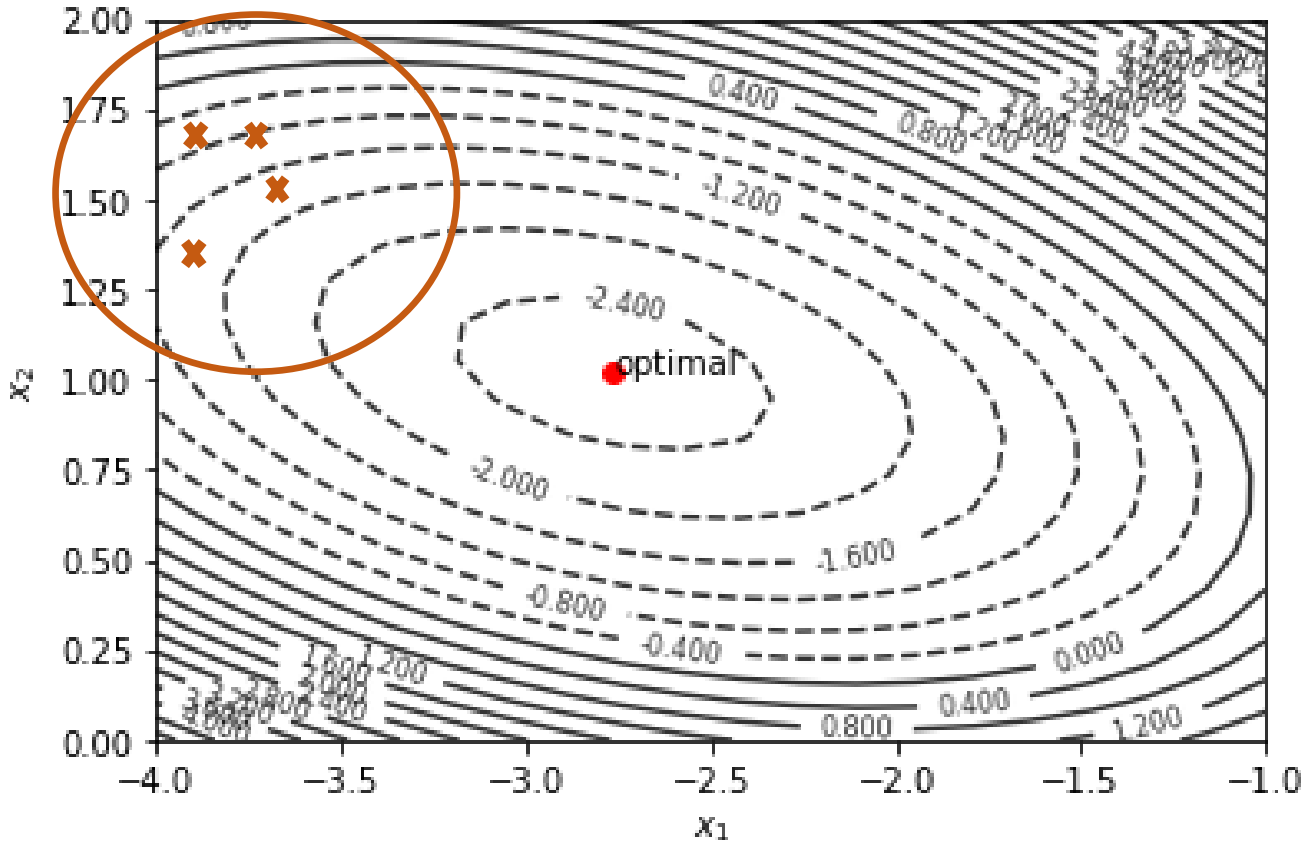
Data-driven optimization – Big picture

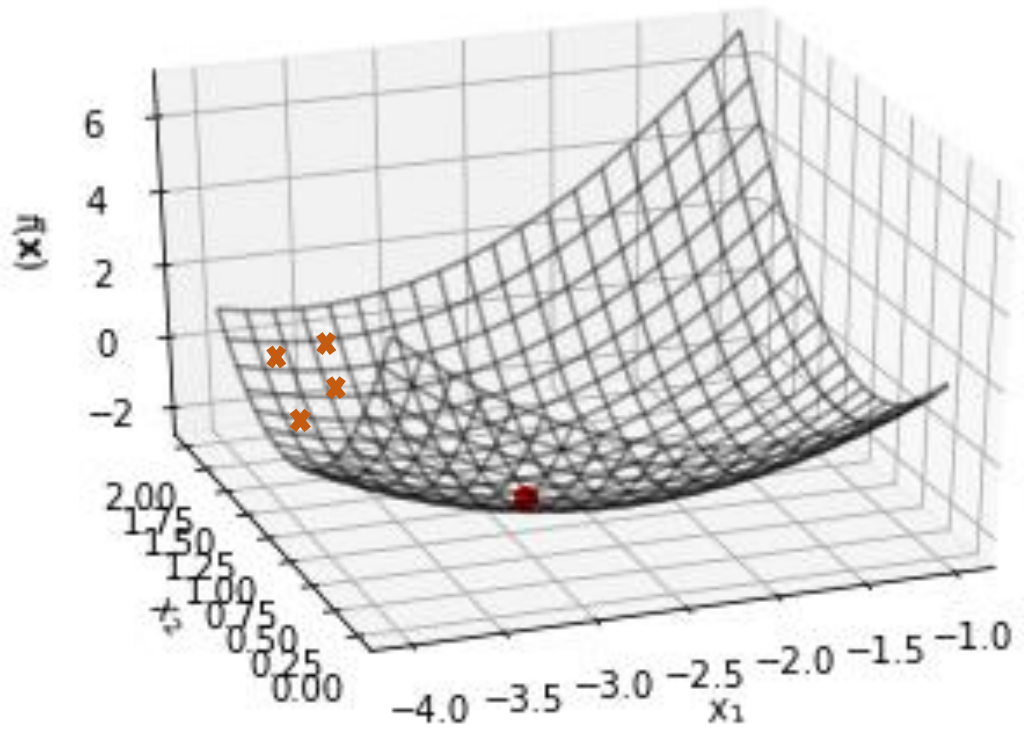




- We have some initial number of samples ✖. Our goal is to reach the optimal ●.

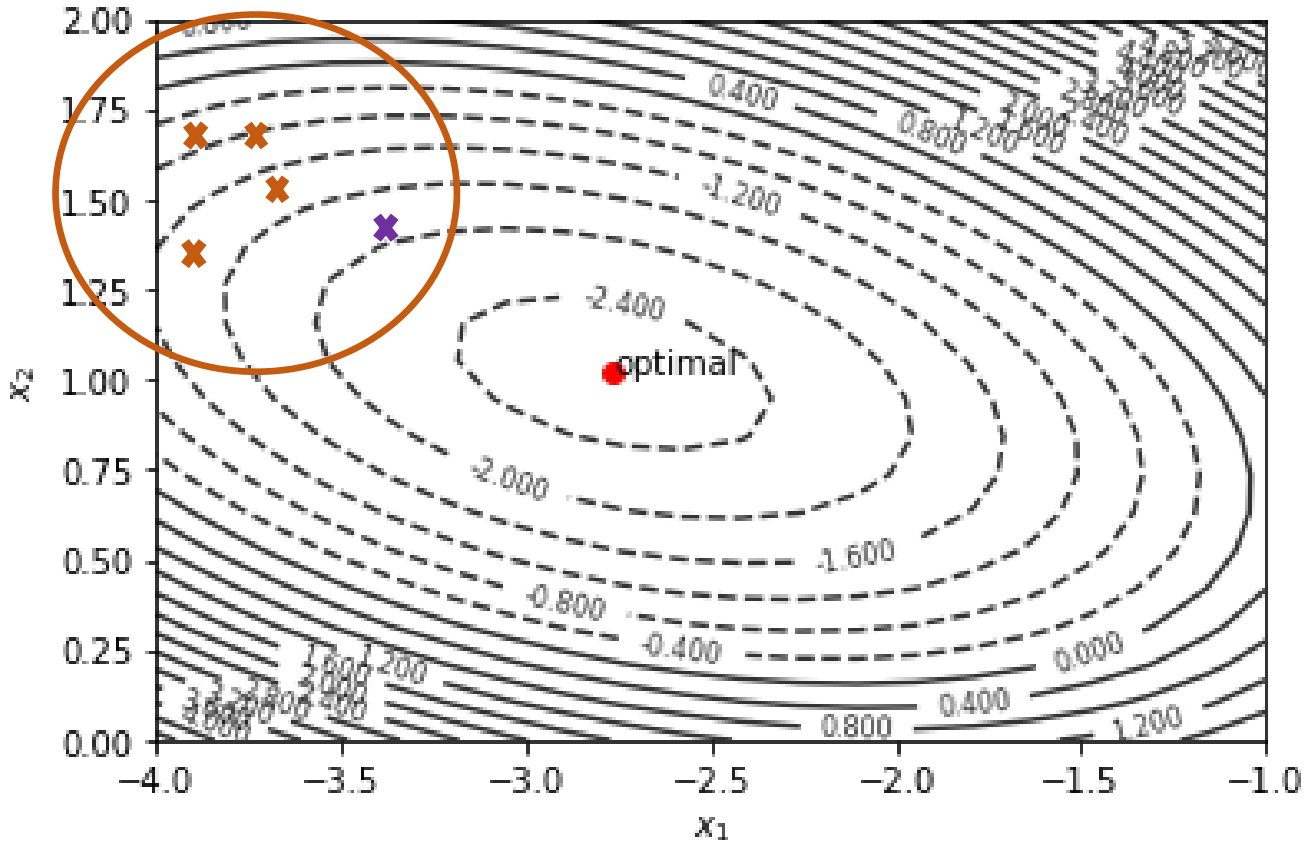
Data-driven optimization – Big picture





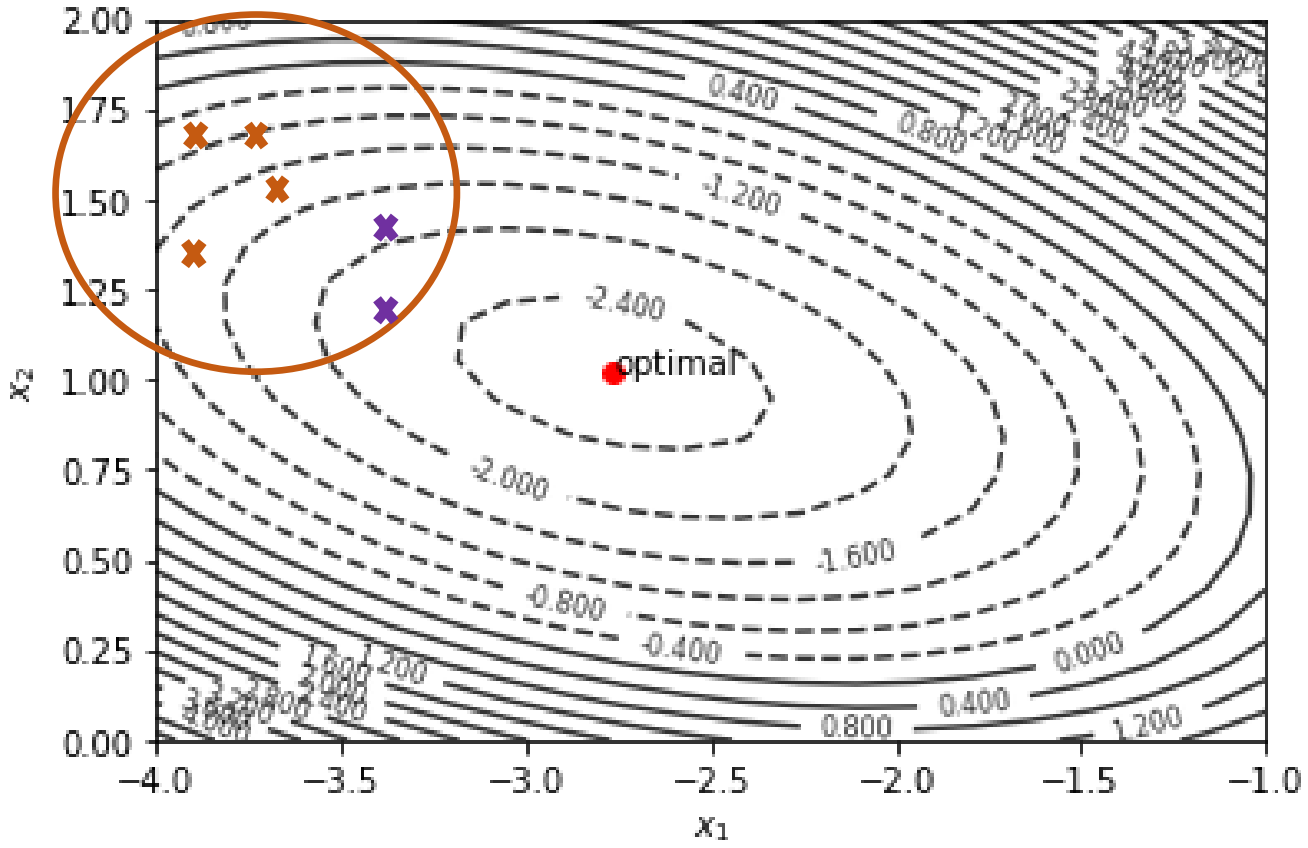
- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We **do not know the shape** of the function, no derivatives, we **can only sample**.
  - We only have the values of x_1, x_2 coordinates and their function value $f(x_1, x_2)$.

Data-driven optimization – Big picture



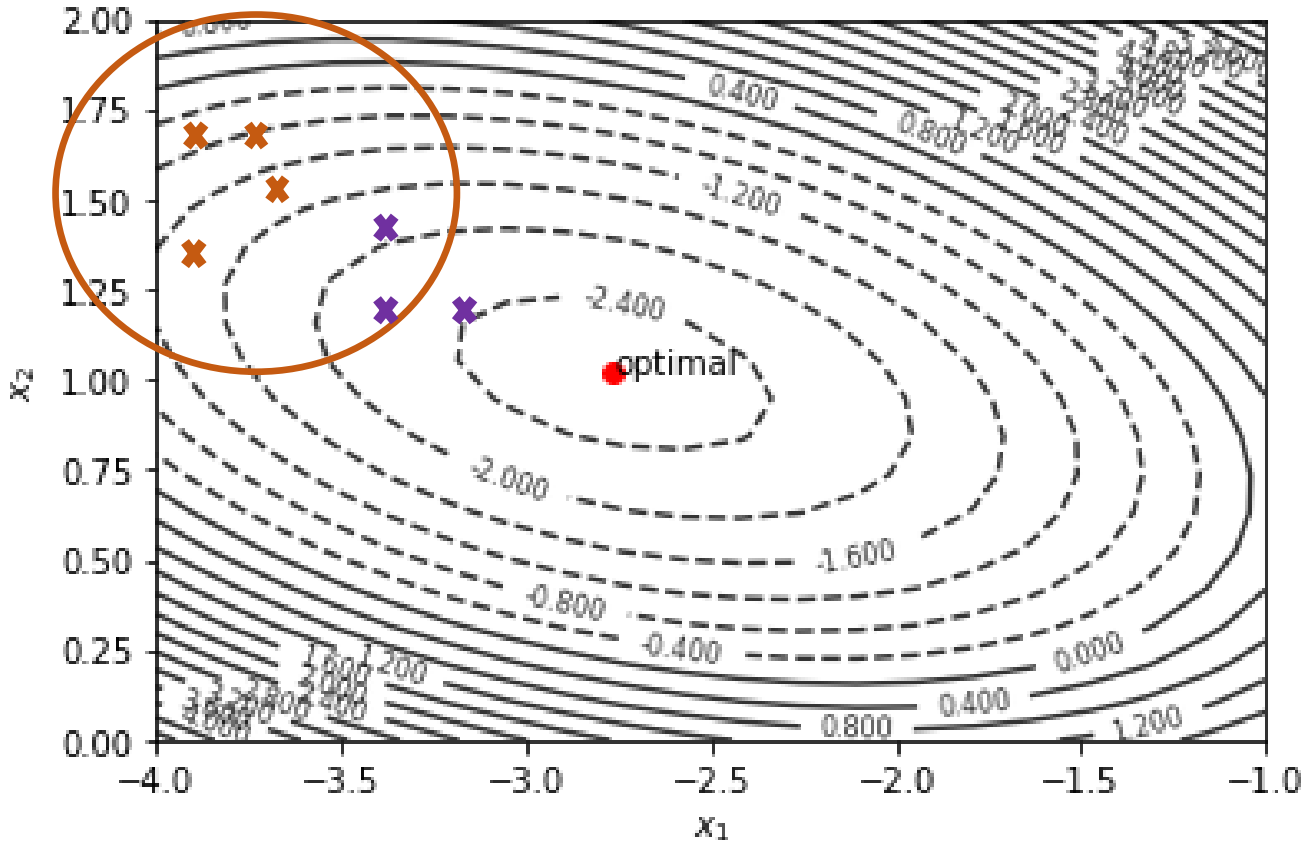
- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We wish to get as efficiently (time or iterations or other) to the optimum as possible.

Data-driven optimization – Big picture



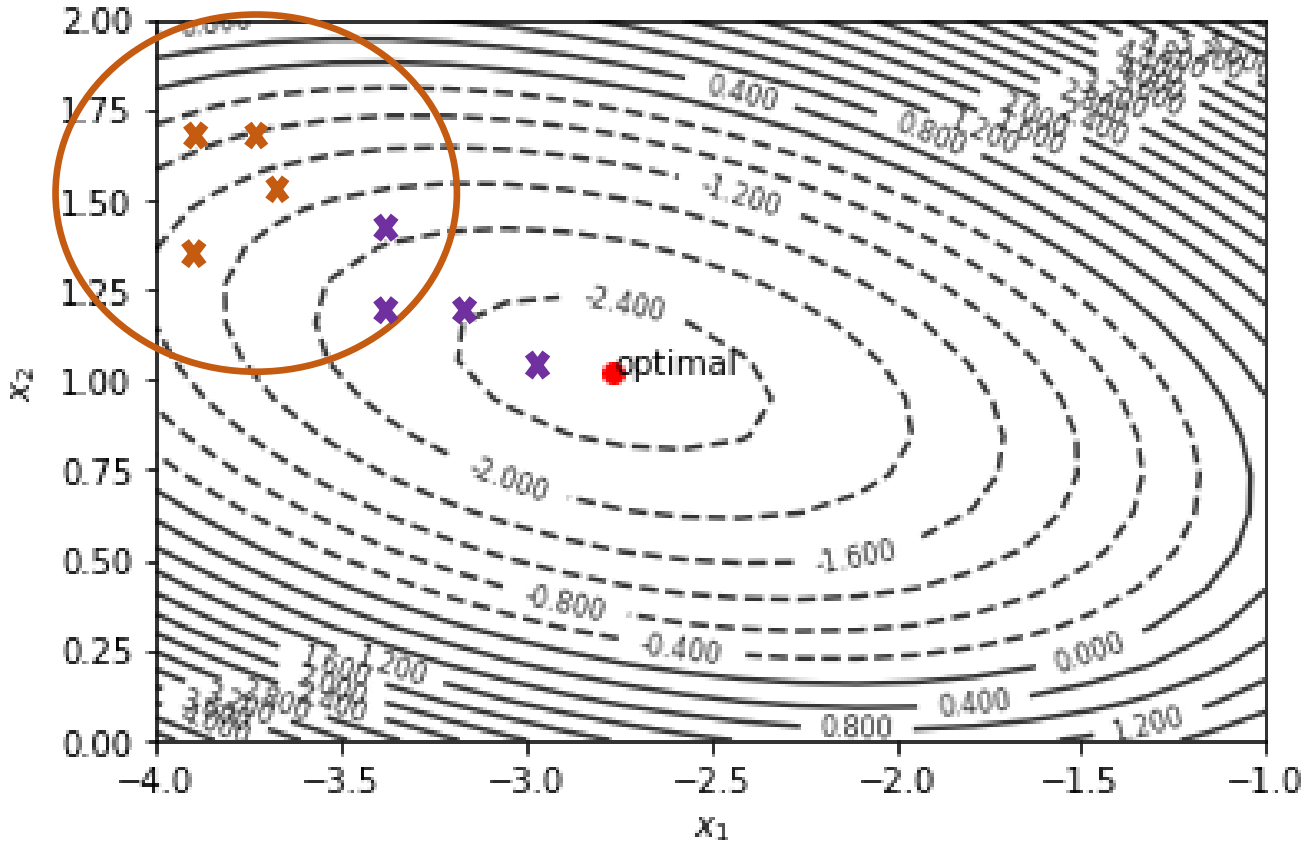
- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We wish to get as efficiently (time or iterations or other) to the optimum as possible.

Data-driven optimization – Big picture



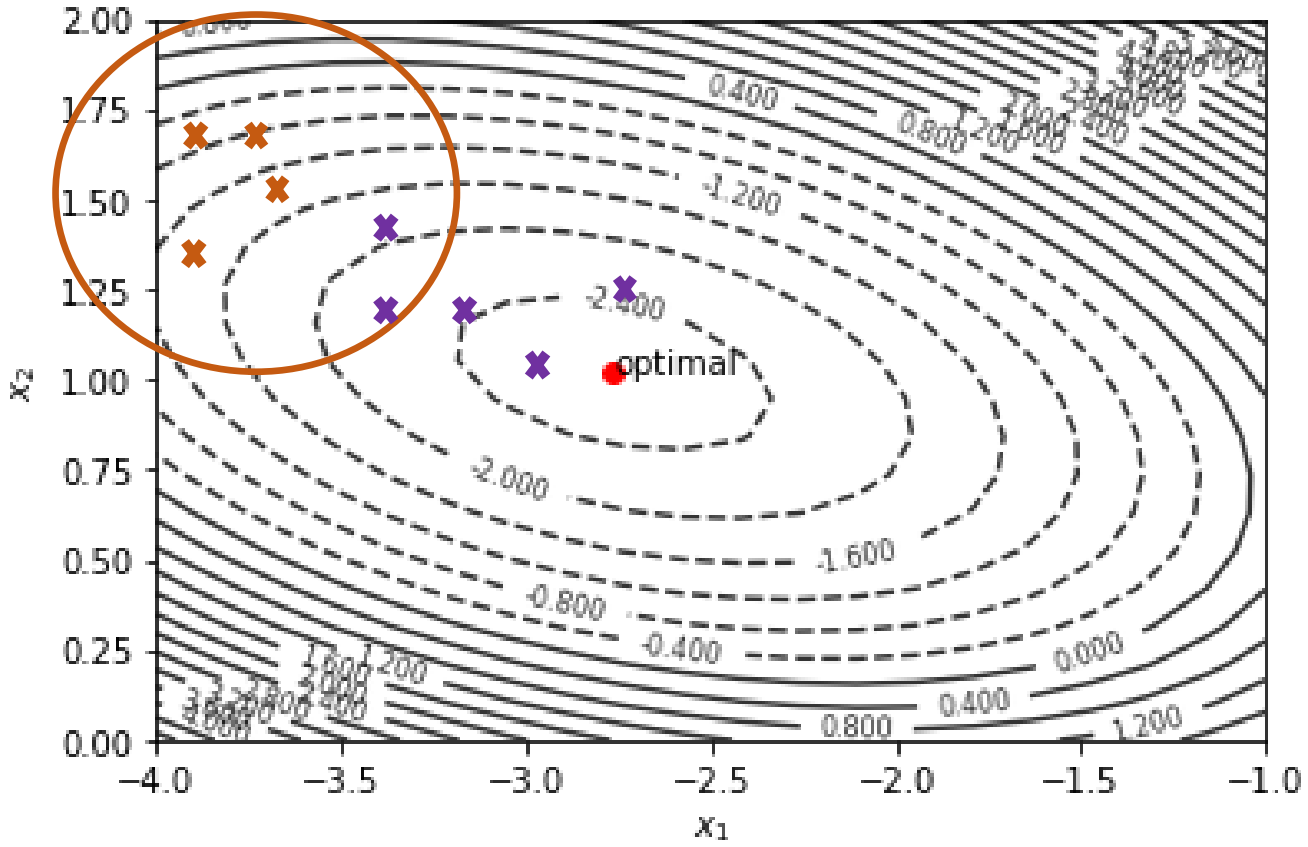
- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We wish to get as efficiently (time or iterations or other) to the optimum as possible.

Data-driven optimization – Big picture



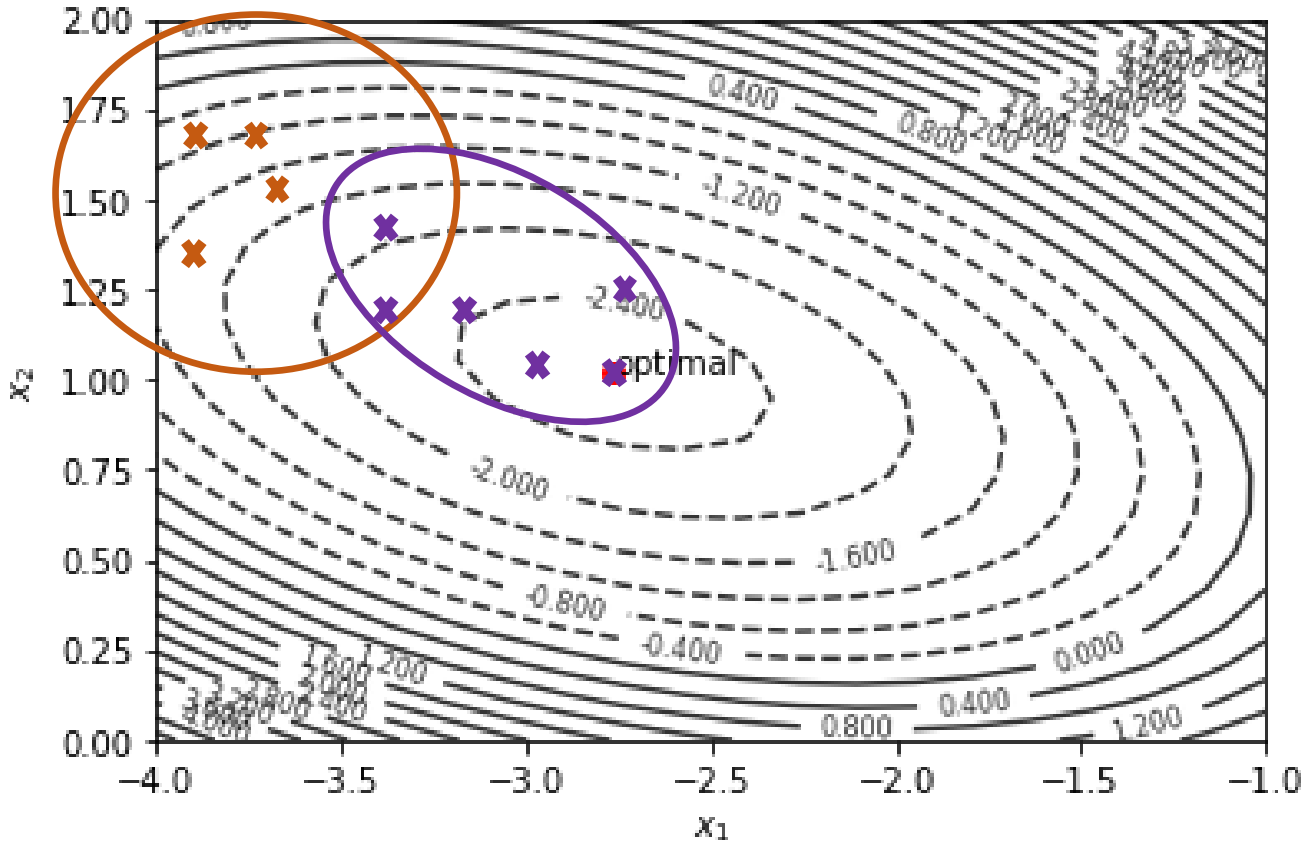
- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We wish to get as efficiently (time or iterations or other) to the optimum as possible.

Data-driven optimization – Big picture



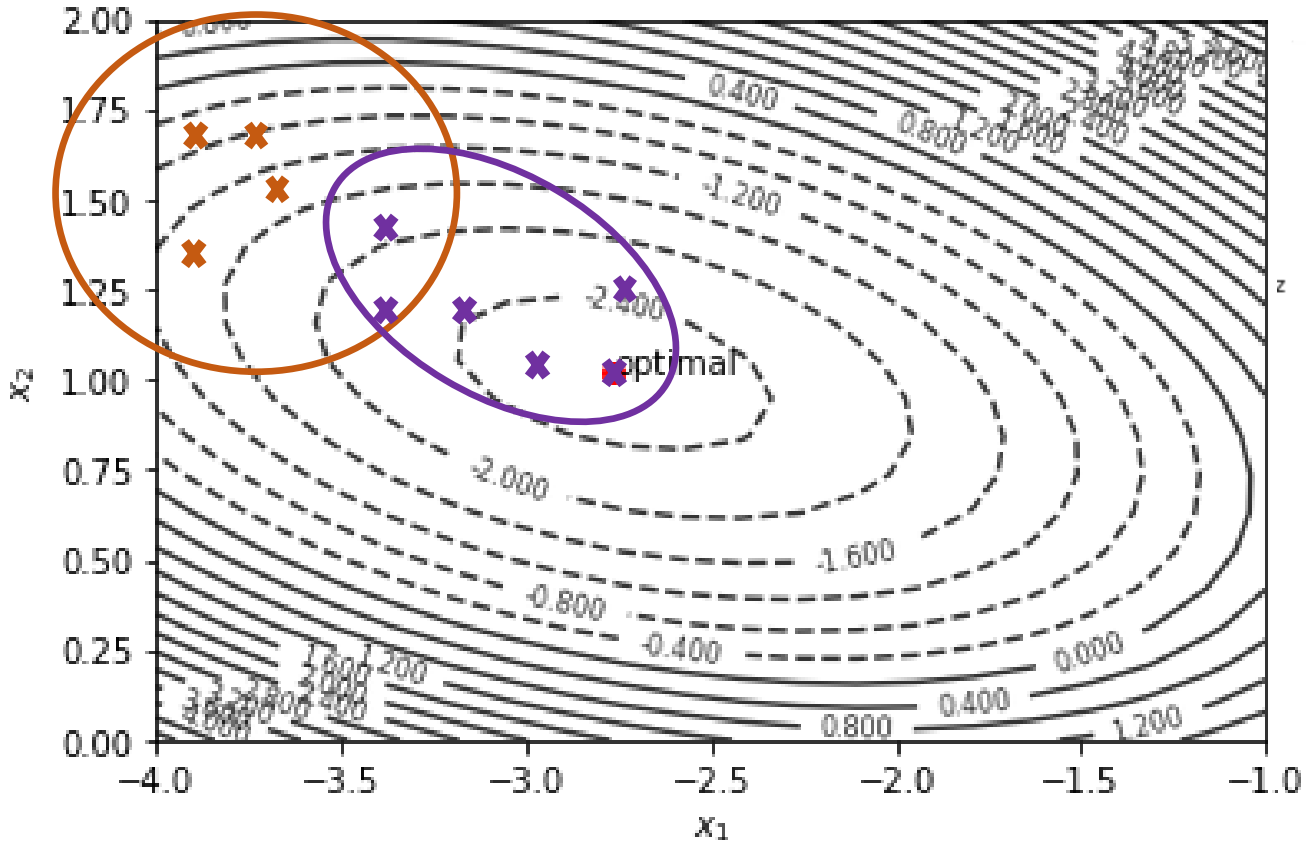
- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We wish to get as efficiently (time or iterations or other) to the optimum as possible.

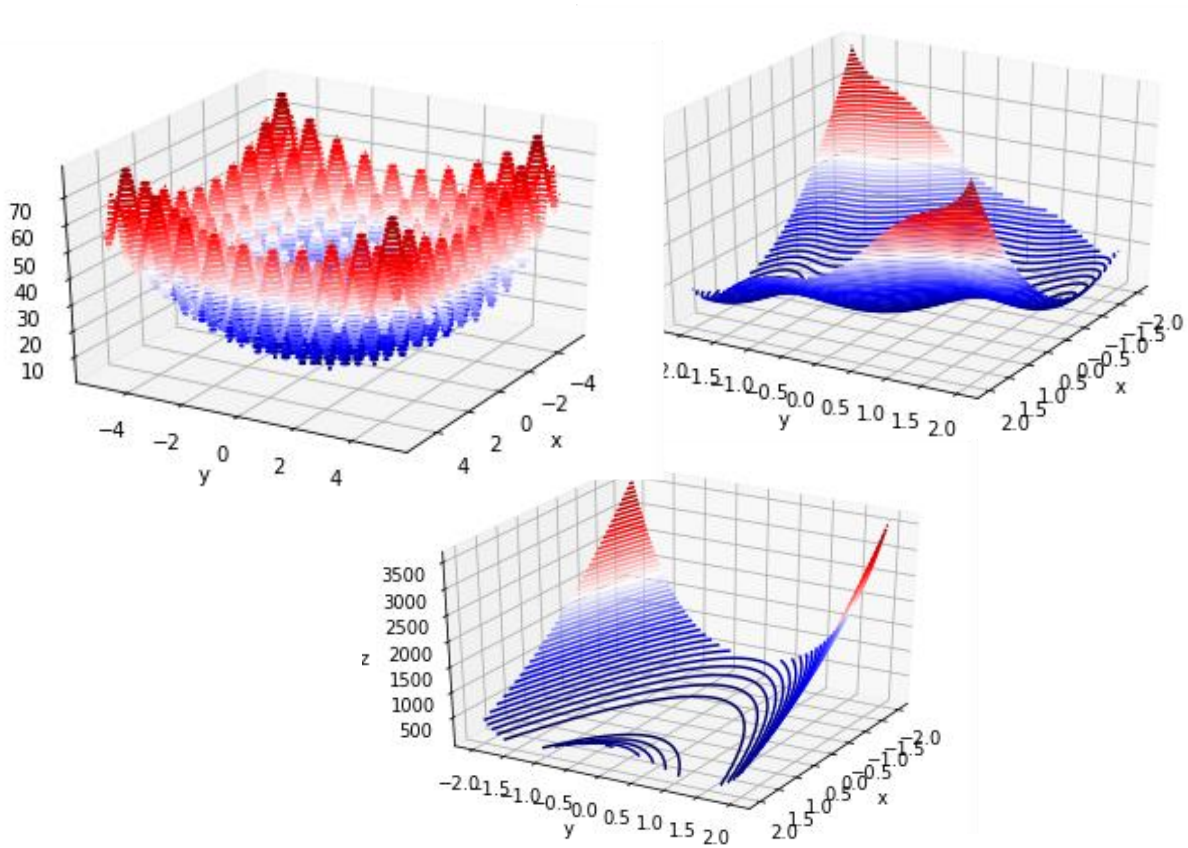
Data-driven optimization – Big picture



- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We wish to get as efficiently (time or iterations or other) to the optimum as possible.

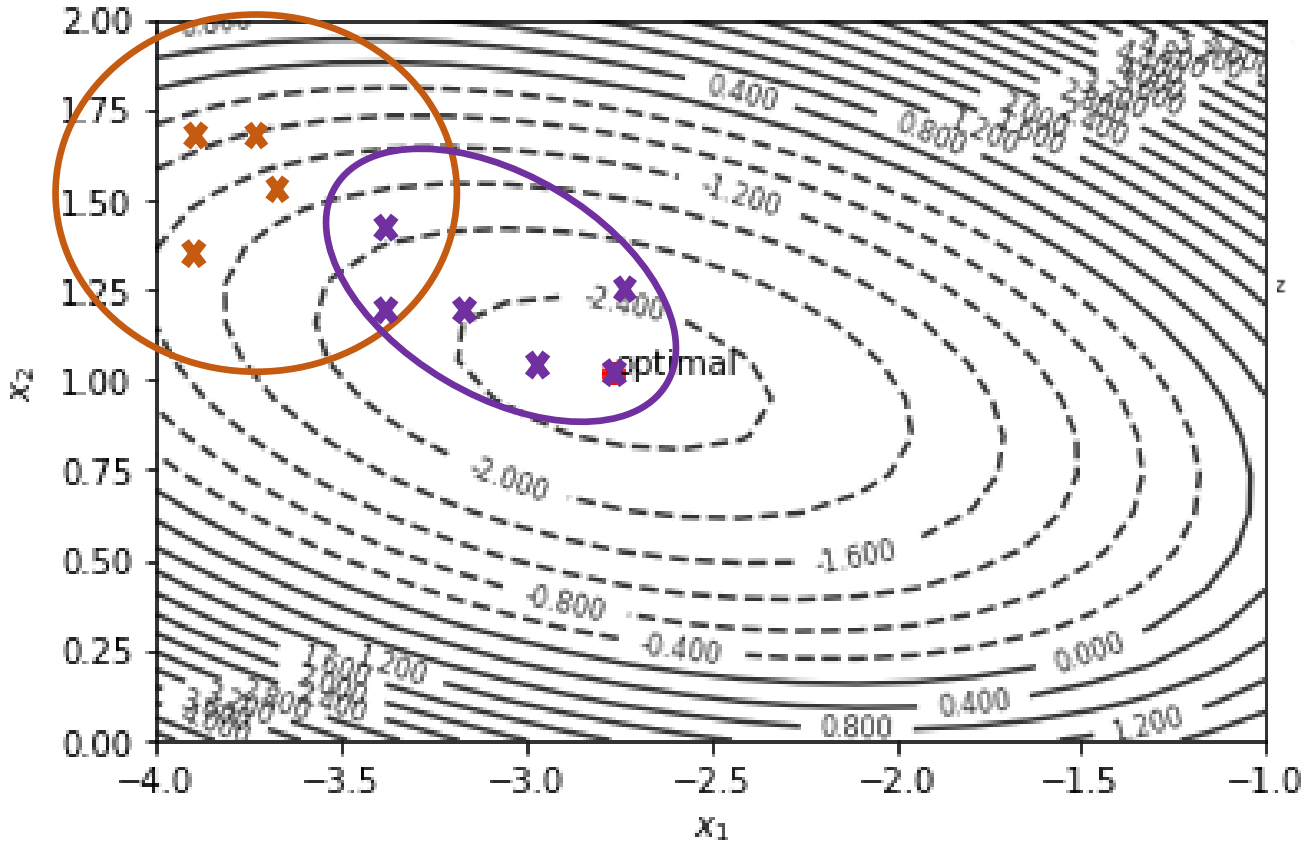
Data-driven optimization – Big picture

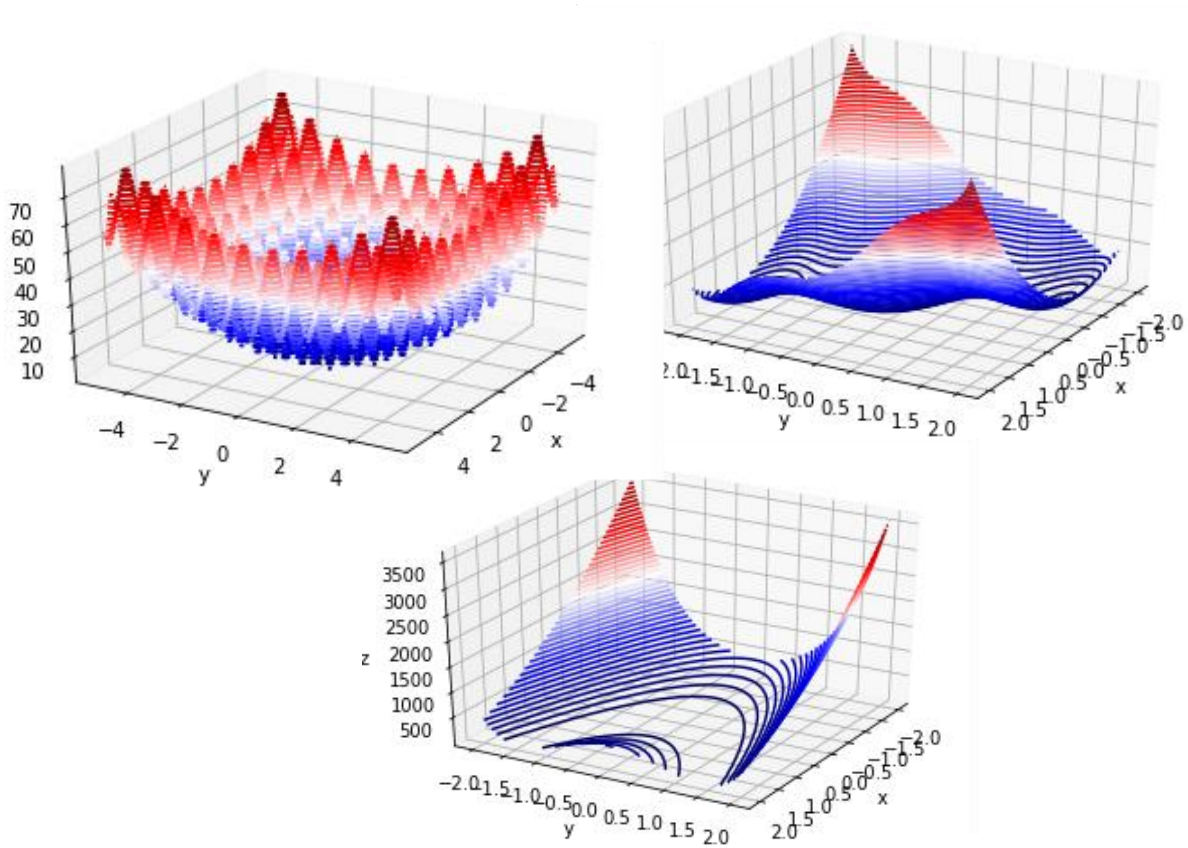




- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We wish to get as efficiently (time or iterations or other) to the optimum as possible.
  - We might also want to *explore* the function. Is our function multimodal?

Data-driven optimization – Big picture



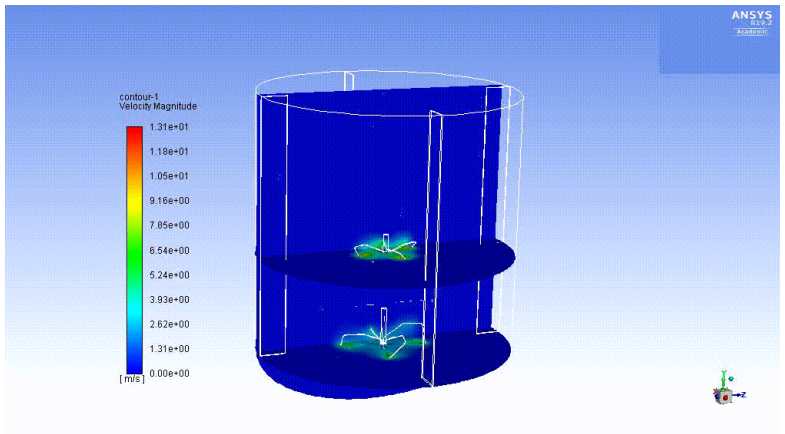


- We have some initial number of samples ✖. Our goal is to reach the optimal ●.
  - We wish to get as efficiently (time or iterations or other) to the optimum as possible.
  - We might also want to *explore* the function. Is our function multimodal?
  - Is our function stochastic (random)? Are there constraints? Can we sample safely?

Some examples problems

Derivative free optimization: **some examples**

Computational Fluid Dynamics Simulations



Molecular discovery and Molecular dynamics



Crystal Structures

→ Molecular Dynamics Simulations

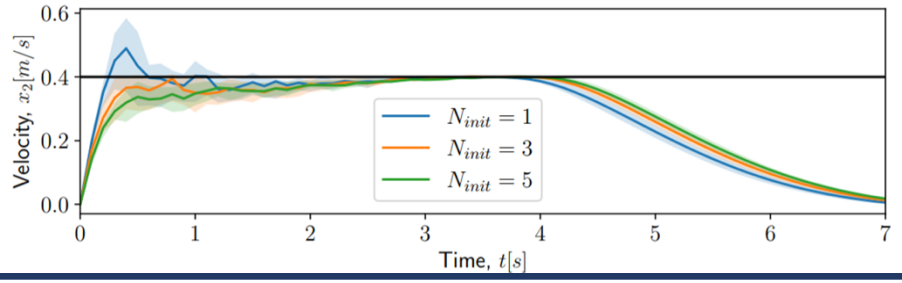
→ New Binding Sites

→ Free Energy

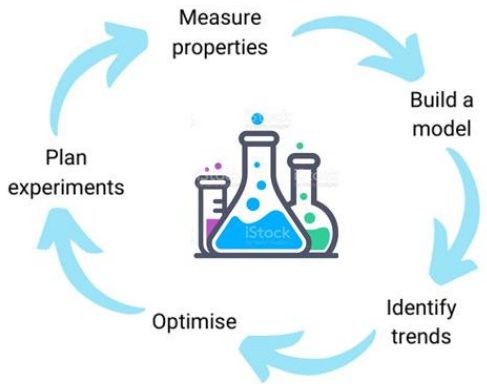
→ Dynamics

→ Drug Design

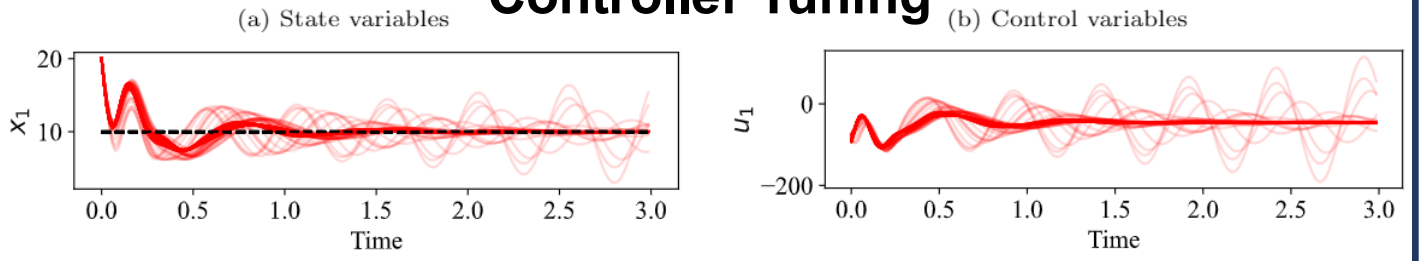
Data-driven control and reinforcement learning



Design of Experiments



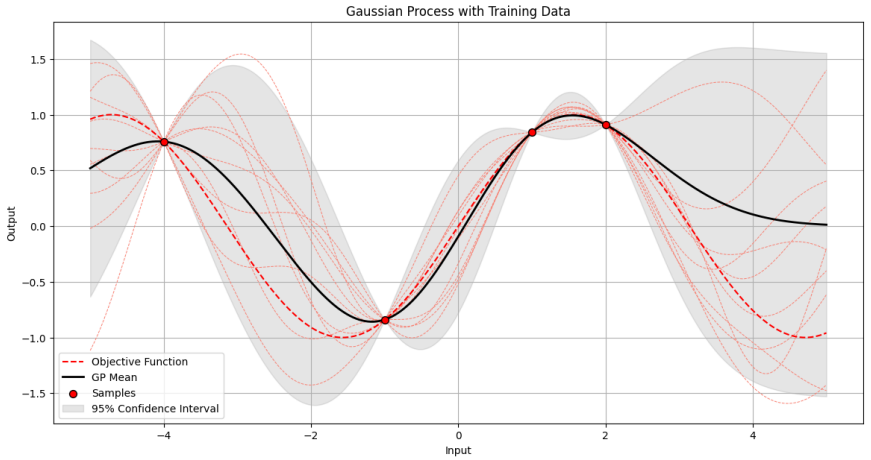
Controller Tuning



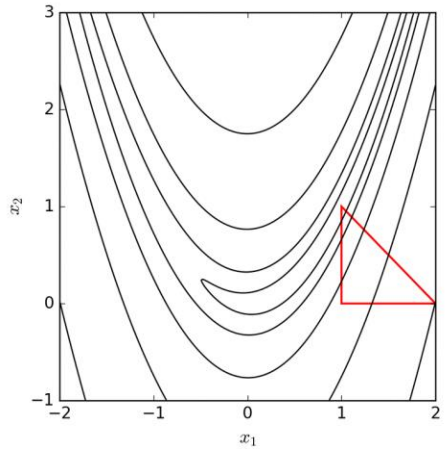
Type of algorithms

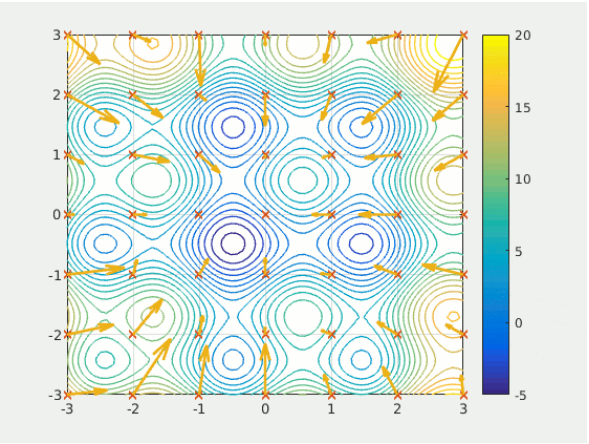
Types of derivative free optimization **algorithms**

Model-based derivative free methods (**Surrogate-based** optimization)

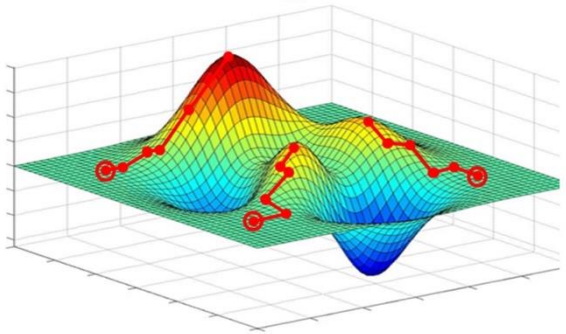


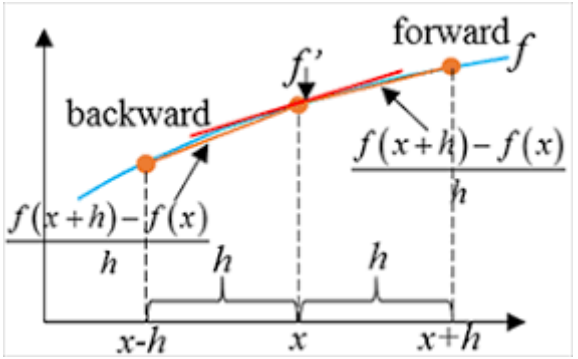
Direct derivative free methods





Finite difference methods





Direct derivative free methods

Direct derivative free methods

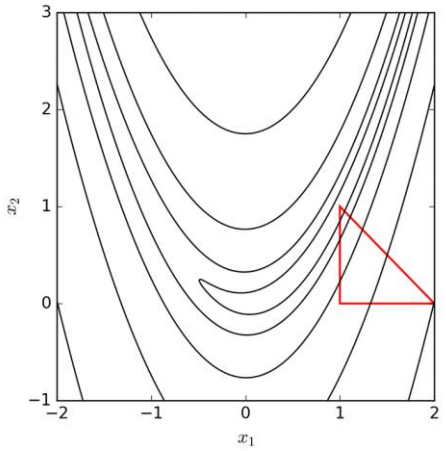
- Chooses the next sample based on the position and information of current samples.

- Compares objective function value of different samples.

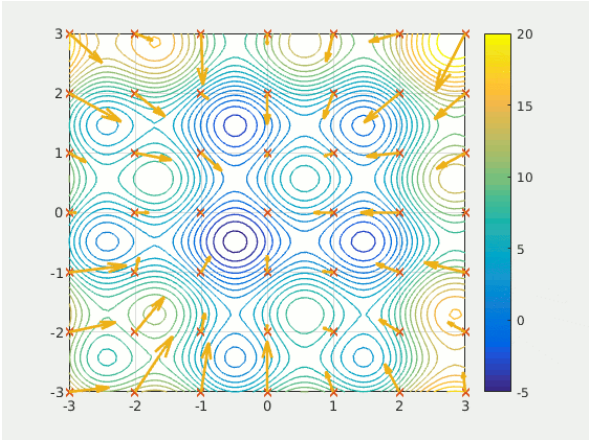
Direct derivative free methods

- Chooses the next sample based on the position and information of current samples.
- Compares objective function value of different samples.

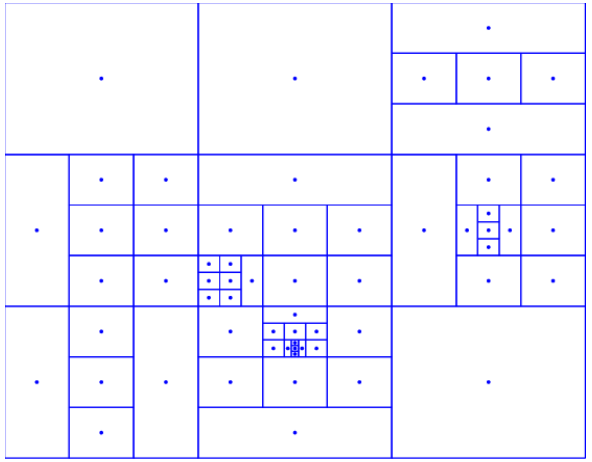
Simplex (Nelder–Mead)



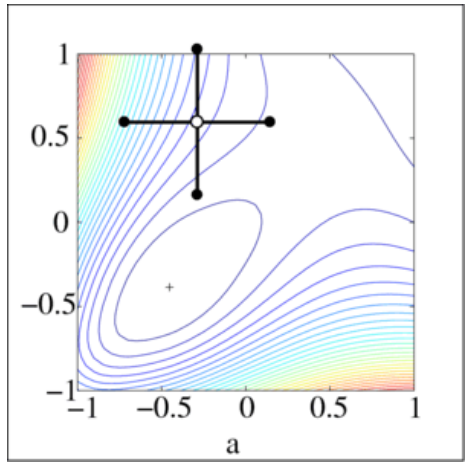
Evolutionary Algorithms / Particle Swarm



Direct Global Optimization Algorithm



Simulated Annealing, Pattern Search



Direct derivative free methods

Coded example

Direct derivative free methods

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

```python
def rosenbrock(x):
    return 100*(x[1]-x[0]**2)**2 + (1-x[0])**2
```

Let us try a simple algorithm: **Random Search**

1. **Randomly sample the function**
2. **Choose the best value**

Direct derivative free methods

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

```python
def rosenbrock(x):
    return 100*(x[1]-x[0]**2)**2 + (1-x[0])**2
```

Let us try a simple algorithm: **Random Search**

1. **Randomly sample the function**
2. **Choose the best value**

```python

x_bounds = np.array([[-4,-3],[3,4]])

```

```python
def Random_search(f, x_bounds, n_eval):

    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far
    x_range = x_bounds[1,:] - x_bounds[0,:] # range
    x_bias  = x_bounds[0,:]                 # lower bound

    for i in range(n_eval):
        x_ = np.random.uniform(low=.0, high=1., size=2) * x_range + x_bias
        f_ = f(x_)
        if f_ < f_best:
            f_best = f_
            x_best = x_

    return x_best, f_best

```

Direct derivative free methods

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

```python
def rosenbrock(x):
    return 100*(x[1]-x[0]**2)**2 + (1-x[0])**2
```

Let us try a simple algorithm: **Random Search**

1. **Randomly sample the function**
2. **Choose the best value**

```python

x_bounds = np.array([[-4,-3],[3,4]])

```

Keep track of best point

Define range and bias for random sampling

```python
def Random_search(f, x_bounds, n_eval):

    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far
    x_range = x_bounds[1,:] - x_bounds[0,:] # range
    x_bias  = x_bounds[0,:]                 # lower bound

    for i in range(n_eval):
        x_ = np.random.uniform(low=.0, high=1., size=2) * x_range + x_bias
        f_ = f(x_)
        if f_ < f_best:
            f_best = f_
            x_best = x_

    return x_best, f_best

```

Direct derivative free methods

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

```python
def rosenbrock(x):
    return 100*(x[1]-x[0]**2)**2 + (1-x[0])**2
```

Let us try a simple algorithm: **Random Search**

1. **Randomly sample the function**
2. **Choose the best value**

```python

x_bounds = np.array([[-4,-3],[3,4]])

```

```python
def Random_search(f, x_bounds, n_eval):

    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far
    x_range = x_bounds[1,:] - x_bounds[0,:] # range
    x_bias  = x_bounds[0,:]                 # lower bound

    for i in range(n_eval):
        x_ = np.random.uniform(low=.0, high=1., size=2) * x_range + x_bias
        f_ = f(x_)
        if f_ < f_best:
            f_best = f_
            x_best = x_

    return x_best, f_best

```

Keep track of best point

Sample uniformly from [0,1] then scale and shift for original variable space

Define range and bias for random sampling

Direct derivative free methods

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

```python
def rosenbrock(x):
    return 100*(x[1]-x[0]**2)**2 + (1-x[0])**2
```

Let us try a simple algorithm: **Random Search**

1. **Randomly sample the function**
2. **Choose the best value**

```python

x_bounds = np.array([[-4,-3],[3,4]])

```

```python
def Random_search(f, x_bounds, n_eval):

    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far
    x_range = x_bounds[1,:] - x_bounds[0,:] # range
    x_bias  = x_bounds[0,:]                 # lower bound

    for i in range(n_eval):
        x_ = np.random.uniform(low=.0, high=1., size=2) * x_range + x_bias
        f_ = f(x_)
        if f_ < f_best:
            f_best = f_
            x_best = x_

    return x_best, f_best

```

Keep track of best point

Define range and bias for random sampling

Sample uniformly from [0,1] then scale and shift for original variable space

Evaluate new sampled point

Direct derivative free methods

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

```python
def rosenbrock(x):
    return 100*(x[1]-x[0]**2)**2 + (1-x[0])**2
```

Let us try a simple algorithm: **Random Search**

1. **Randomly sample the function**
2. **Choose the best value**

```python

x_bounds = np.array([[-4,-3],[3,4]])

```

```python
def Random_search(f, x_bounds, n_eval):

    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far
    x_range = x_bounds[1,:] - x_bounds[0,:] # range
    x_bias  = x_bounds[0,:]                 # lower bound

    for i in range(n_eval):
        x_ = np.random.uniform(low=.0, high=1., size=2) * x_range + x_bias
        f_ = f(x_)
        if f_ < f_best:
            f_best = f_
            x_best = x_

    return x_best, f_best
```

Keep track of best point

Define range and bias for random sampling

Sample uniformly from [0,1] then scale and shift for original variable space

Evaluate new sampled point

Check if new point is better than the best known so far

Direct derivative free methods

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

```python
def rosenbrock(x):
    return 100*(x[1]-x[0]**2)**2 + (1-x[0])**2
```

Let us try a simple algorithm: **Random Search**

1. **Randomly sample the function**
2. **Choose the best value**

```python

x_bounds = np.array([[-4,-3],[3,4]])

```

```python
def Random_search(f, x_bounds, n_eval):

    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far
    x_range = x_bounds[1,:] - x_bounds[0,:] # range
    x_bias  = x_bounds[0,:]                 # lower bound

    for i in range(n_eval):
        x_ = np.random.uniform(low=.0, high=1., size=2) * x_range + x_bias
        f_ = f(x_)
        if f_ < f_best:
            f_best = f_
            x_best = x_

    return x_best, f_best

```

Keep track of best point

Define range and bias for random sampling

Sample uniformly from [0,1] then scale and shift for original variable space

Check if new point is better than the best known so far

Evaluate new sampled point

Return best point

Direct derivative free methods

```python
def Random_search(f, x_bounds, n_eval):
    '''Performs random search optimization

    Arguments:
        f: the objective function -> scalar
        x_bounds: bounds of the variables -> numpy array [[x_1lb,x_2lb],[x_1ub,x_2un]]
        n_eval: number of random evaluations -> scalar
    Returns:
        x_best: best point found -> numpy array
        f_best: best value found -> scalar

    '''
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far
    x_range = x_bounds[1,:] - x_bounds[0,:] # range
    x_bias  = x_bounds[0,:]                 # lower bound

    for i in range(n_eval):
        x_ = np.random.uniform(low=.0, high=1., size=2) * x_range + x_bias
        f_ = f(x_)
        if f_ < f_best:
            f_best = f_
            x_best = x_

    return x_best, f_best
```

Remember to annotate your functions

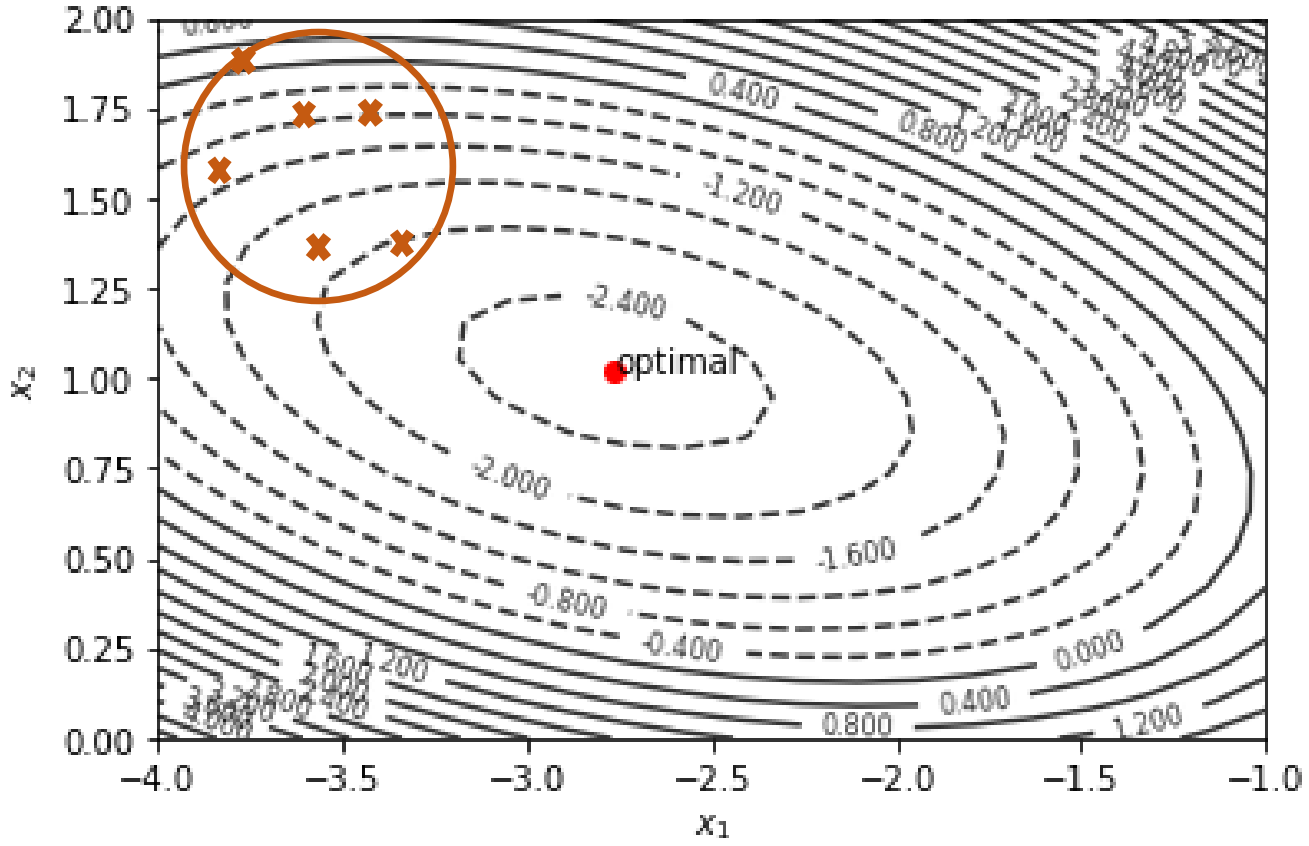
Direct derivative free methods

Coded example 2

Direct derivative free methods – Example 2

Random Local search

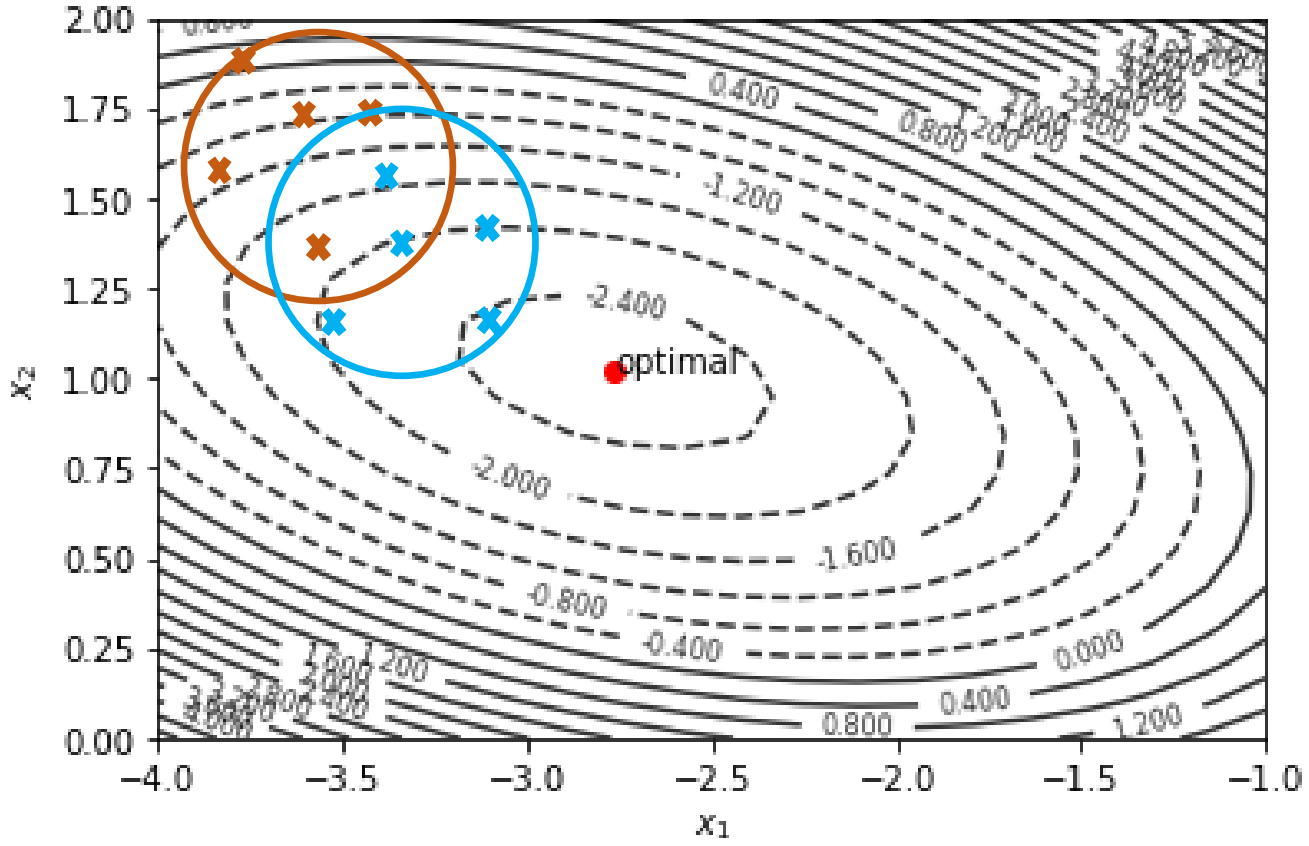
1. **Define** starting point and radius
2. **Sample** inside the radius
3. **Move** to the best point found



Direct derivative free methods – Example 2

Random Local search

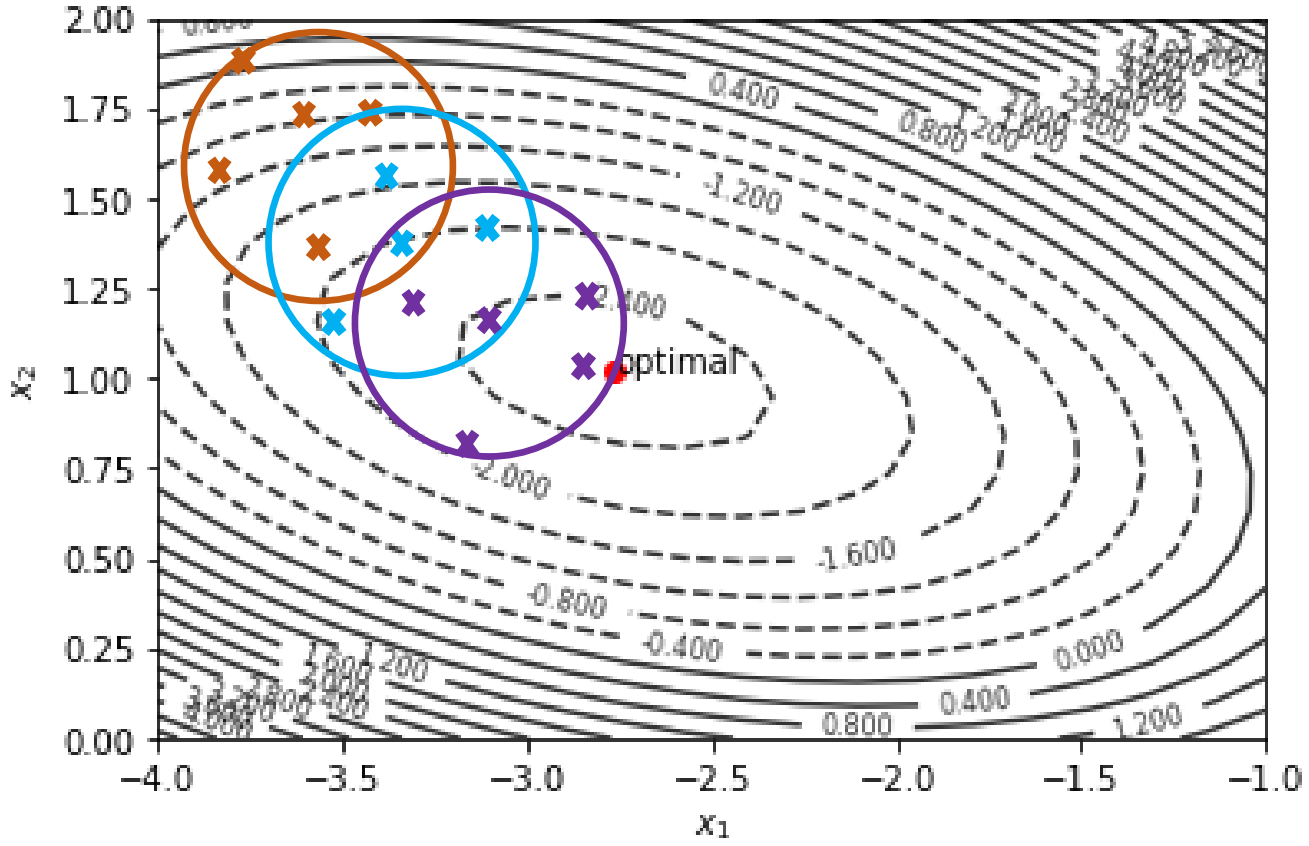
1. **Define** starting point and radius
2. **Sample** inside the radius
3. **Move** to the best point found



Direct derivative free methods – Example 2

Random Local search

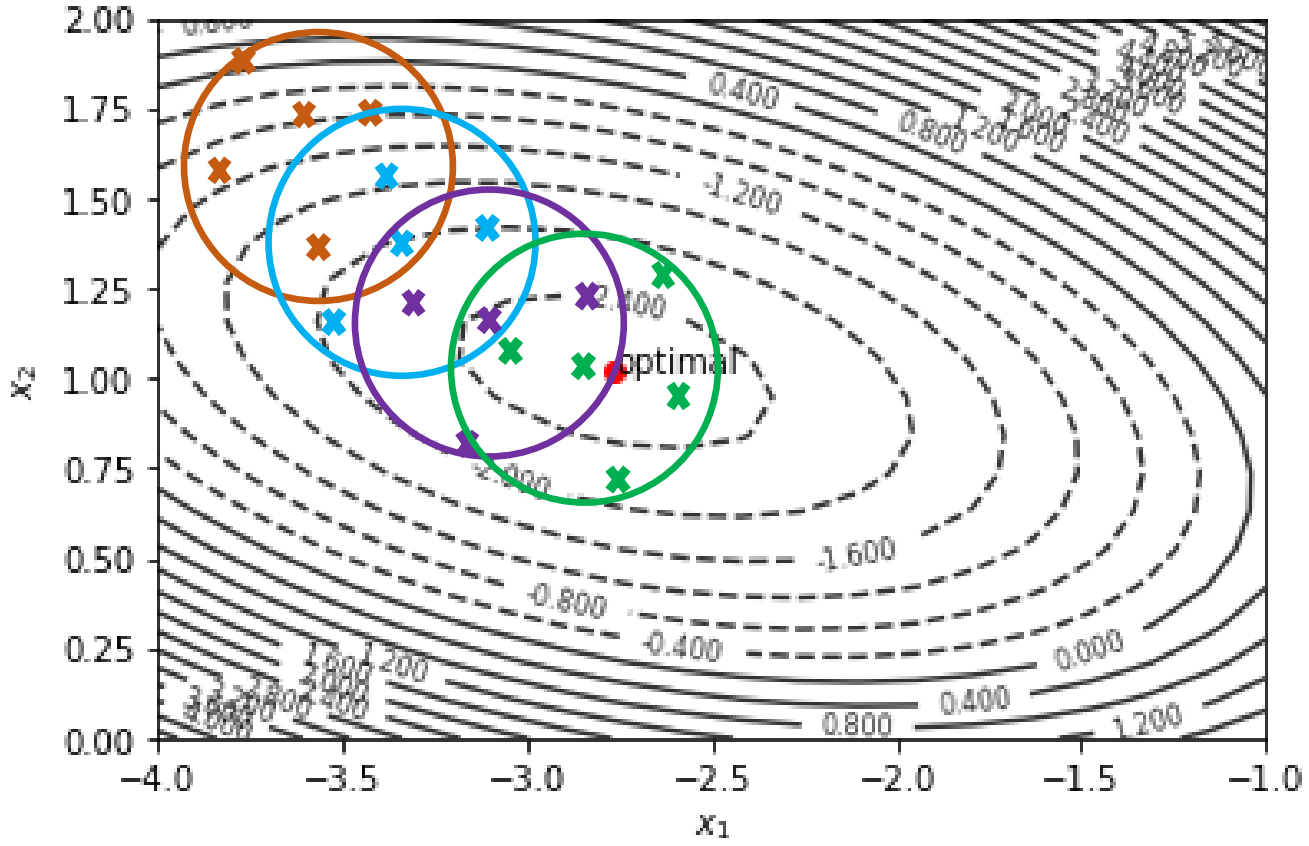
1. **Define** starting point and radius
2. **Sample** inside the radius
3. **Move** to the best point found



Direct derivative free methods – Example 2

Random Local search

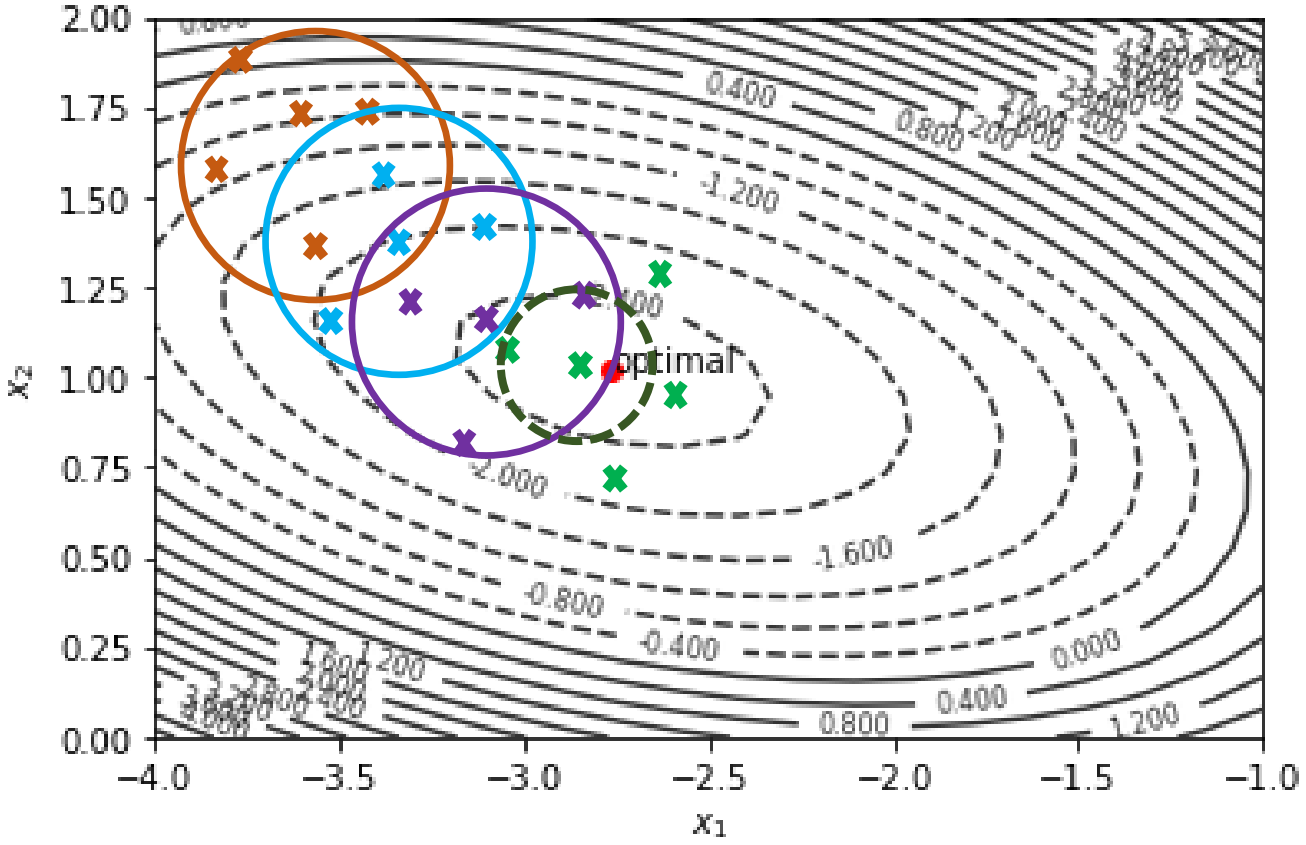
1. **Define** starting point and radius
2. **Sample** inside the radius
3. **Move** to the best point found



Direct derivative free methods – Example 2

Random Local search

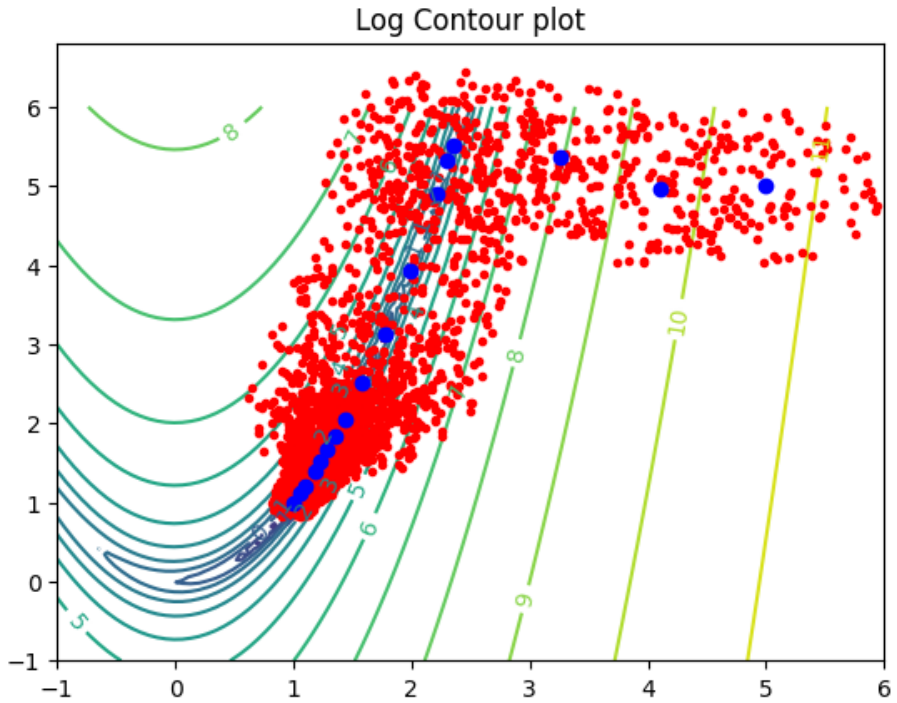
1. **Define** starting point and radius
2. **Sample** inside the radius
3. **Move** to the best point found
4. If no best point is found:
   a) Shrink radius

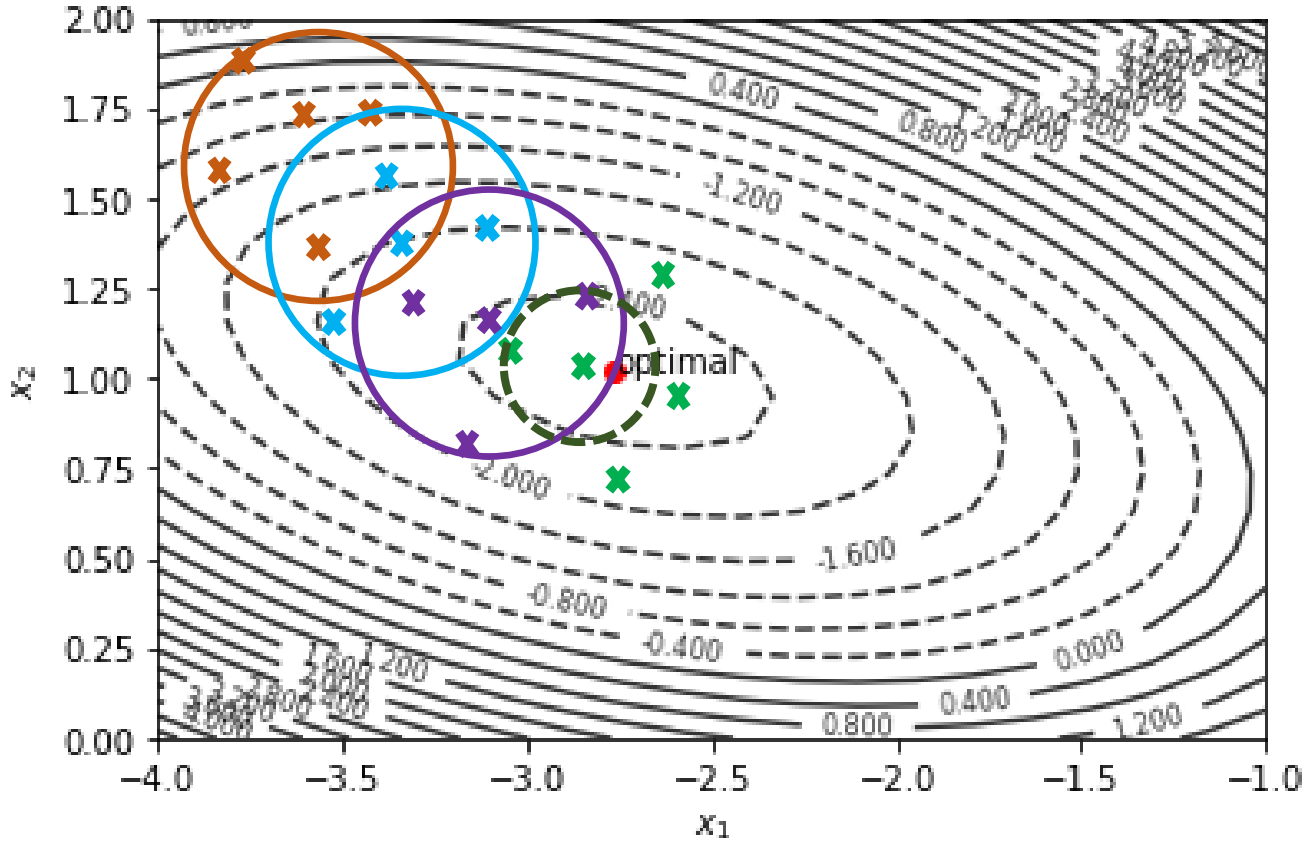


Direct derivative free methods – Example 2

Random Local search

1. **Define** starting point and radius
2. **Sample** inside the radius
3. **Move** to the best point found
4. **If** no best point is found:
   a) Shrink radius





Direct derivative free methods – Example 2

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Let's try a second algorithm: **Local random Search**

```python
def local_search(f, x_0, n_s, n_iter, Dx):
    '''Performs local random search optimization
    '''
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far

    for i in range(n_iter):
        x_    = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
        f_    = f(x_)
        f_min = np.min(f_)

        if f_min < f_best:
            f_best = f_min
            x_best = x_0 = x_[:,np.argmin(f_)]
        else:
            Dx = Dx*.5

    return x_best, f_best
```

Direct derivative free methods – Example 2

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Let's try a second algorithm: **Local random Search**

```python
def local_search(f, x_0, n_s, n_iter, Dx):
    '''Performs local random search optimization
    '''
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far

    for i in range(n_iter):
        x_    = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
        f_    = f(x_)
        f_min = np.min(f_)

        if f_min < f_best:
            f_best = f_min
            x_best = x_0 = x_[:,np.argmin(f_)]
        else:
            Dx = Dx*.5

    return x_best, f_best
```

Keep track of best point

Direct derivative free methods – Example 2

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Let's try a second algorithm: **Local random Search**

```
1  def local_search(f, x_0, n_s, n_iter, Dx):
2      '''Performs local random search optimization
3      '''
4      x_best  = None  # best point so far
5      f_best  = 10**8 # best value so far
6
7      for i in range(n_iter):
8          x_    = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
9          f_    = f(x_)
10         f_min = np.min(f_)
11
12         if f_min < f_best:
13             f_best = f_min
14             x_best = x_0 = x_[:,np.argmin(f_)]
15         else:
16             Dx = Dx*.5
17
18     return x_best, f_best
```

Keep track of best point

Sample around center point

Direct derivative free methods – Example 2

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Let's try a second algorithm: **Local random Search**

```python
def local_search(f, x_0, n_s, n_iter, Dx):
    '''Performs local random search optimization
    '''
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far

    for i in range(n_iter):
        x_    = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
        f_    = f(x_)
        f_min = np.min(f_)

        if f_min < f_best:
            f_best = f_min
            x_best = x_0 = x_[:,np.argmin(f_)]
        else:
            Dx = Dx*.5

    return x_best, f_best
```

Keep track of best point

Sample around center point

Note that we are sampling all `n_s` points at once

Direct derivative free methods – Example 2

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Let's try a second algorithm: **Local random Search**

```python
def local_search(f, x_0, n_s, n_iter, Dx):
    '''Performs local random search optimization
    '''
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far

    for i in range(n_iter):
        x_    = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
        f_    = f(x_)
        f_min = np.min(f_)

        if f_min < f_best:
            f_best = f_min
            x_best = x_0 = x_[:,np.argmin(f_)]
        else:
            Dx = Dx*.5

    return x_best, f_best
```

Keep track of best point

Sample around center point

We are using the efficient Rosenbrock function

Note that we are sampling all `n_s` points at once

Direct derivative free methods – Example 2

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Let's try a second algorithm: **Local random Search**

```python
def local_search(f, x_0, n_s, n_iter, Dx):
    '''Performs local random search optimization
    '''
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far

    for i in range(n_iter):
        x_     = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
        f_     = f(x_)
        f_min = np.min(f_)

        if f_min < f_best:
            f_best = f_min
            x_best = x_0 = x_[:,np.argmin(f_)]
        else:
            Dx = Dx*.5

    return x_best, f_best
```

Keep track of best point

Sample around center point

We are using the efficient Rosenbrock function

Note that we are sampling all `n_s` points at once

Checking if we found a better point

Direct derivative free methods – Example 2

$$\min_{\mathbf{x}} \; 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

Let's try a second algorithm: **Local random Search**

```python
def local_search(f, x_0, n_s, n_iter, Dx):
    '''Performs local random search optimization
    '''
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far

    for i in range(n_iter):
        x_     = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
        f_     = f(x_)
        f_min = np.min(f_)

        if f_min < f_best:
            f_best = f_min
            x_best = x_0 = x_[:,np.argmin(f_)]
        else:
            Dx = Dx*.5

    return x_best, f_best
```

Keep track of best point

Sample around center point

We are using the efficient Rosenbrock function

Note that we are sampling all `n_s` points at once

Checking if we found a better point

Reducing sampling domain if no better point is found

Direct derivative free methods – Example 2

```python
def rosenbrock_plot(x):
    return 100*(x[1,:]-x[0,:]**2)**2 + (1-x[0,:])**2
```

```python
def local_search(f, x_0, n_s, n_iter, Dx):
    '''Performs local random search optimization
    '''
    x_list = np.zeros((2, n_s*n_iter))
    f_list = np.zeros((n_s*n_iter))

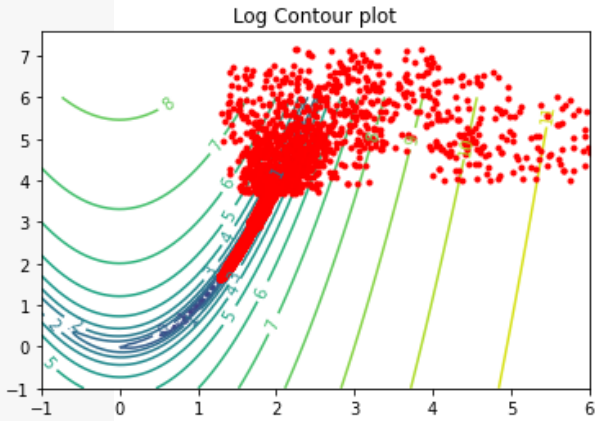
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far

    for i in range(n_iter):
        x_    = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
        f_    = f(x_)
        f_min = np.min(f_)

        #List keeping
        x_list[:,i*n_s:(i+1)*n_s] = x_
        f_list[i*n_s:(i+1)*n_s]   = f_

        if f_min < f_best:
            f_best = f_min
            x_best = x_0 = x_[:,np.argmin(f_)]
        else:
            Dx = Dx*.5

    print('best point found = ',x_best)
    print('best value found = ',f_best)
    return x_best, f_best, x_list, f_list
```




```python
x_0_ = np.array([5,5])

x_best_, f_best_, x_list_, f_list_ = local_search(rosenbrock_efficient, x_0_,
                                                  n_s=100, n_iter=100, Dx=1)
```

```python
# mesh
x_1      = np.linspace(-1, 6, n_points)
x_2      = np.linspace(-1, 6, n_points)
x        = [[x,y] for x in x_1 for y in x_2]
x        = np.array(x)
x        = x.T
# Contour plot
f        = rosenbrock_plot(x)
f_copy = f.reshape((n_points,n_points), order='F')
fig, ax = plt.subplots()
CS = ax.contour(x_1, x_2, np.log(f_copy), 15)
ax.plot(x_list_[0,:], x_list_[1,:], 'r.')
ax.clabel(CS, fontsize=10)
ax.set_title('Log Contour plot')
plt.show()
```

Direct derivative free methods – Example 2

```python
def rosenbrock_plot(x):
    return 100*(x[1,:]-x[0,:]**2)**2 + (1-x[0,:])**2
```

```python
def local_search(f, x_0, n_s, n_iter, Dx):
    '''Performs local random search optimization
    '''
    x_list = np.zeros((2, n_s*n_iter))
    f_list = np.zeros((n_s*n_iter))

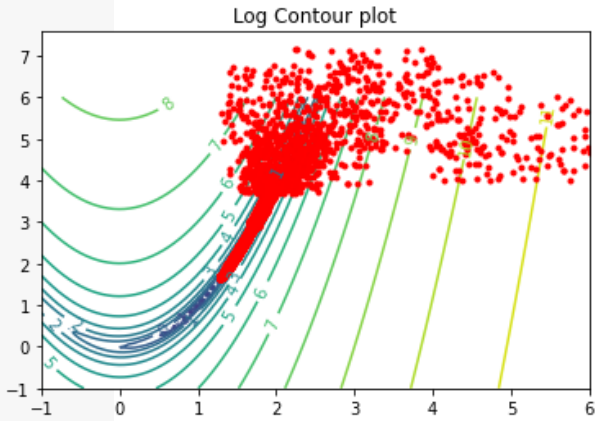
    x_best  = None  # best point so far
    f_best  = 10**8 # best value so far

    for i in range(n_iter):
        x_    = x_0.reshape(2,1) + np.random.uniform(low=-1., high=1., size=(2,n_s)) * Dx
        f_    = f(x_)
        f_min = np.min(f_)

        #List keeping
        x_list[:,i*n_s:(i+1)*n_s] = x_
        f_list[i*n_s:(i+1)*n_s]   = f_

        if f_min < f_best:
            f_best = f_min
            x_best = x_0 = x_[:,np.argmin(f_)]
        else:
            Dx = Dx*.5

    print('best point found = ',x_best)
    print('best value found = ',f_best)
    return x_best, f_best, x_list, f_list
```



Full code can be found here

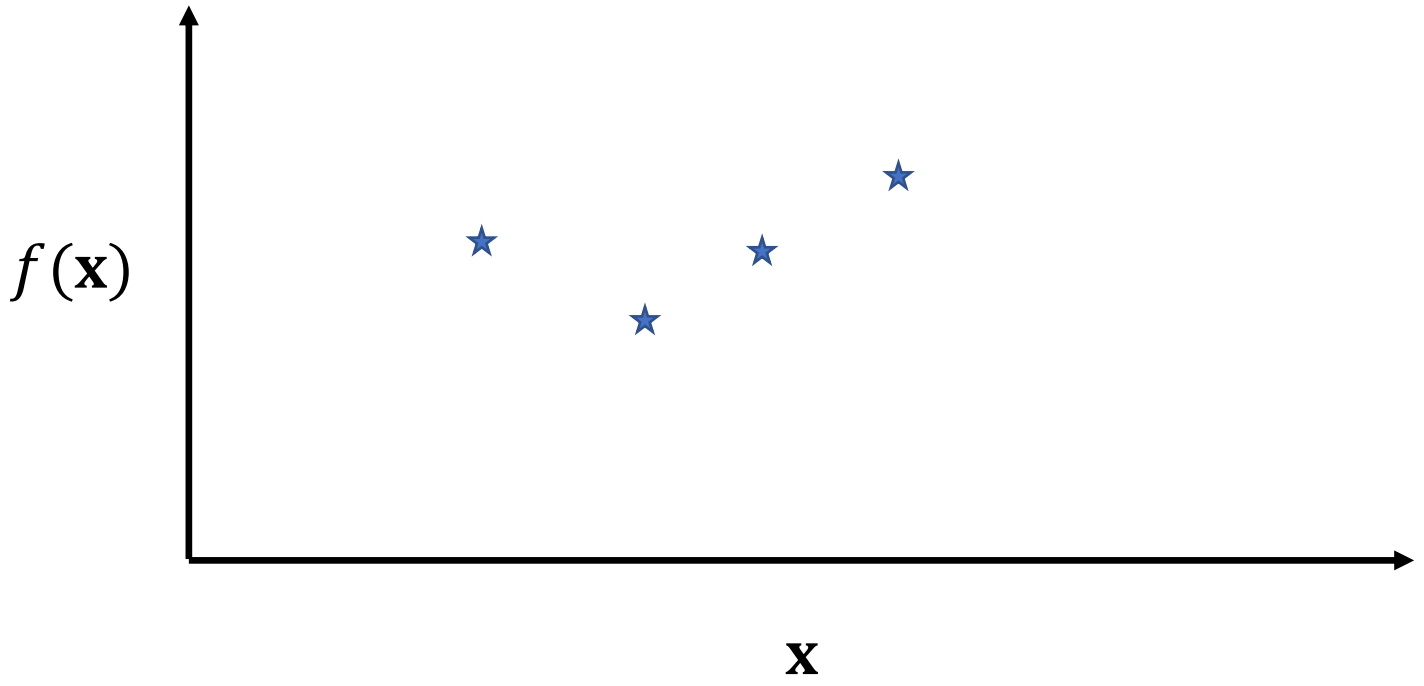
Model-based derivative free methods

(**Surrogate-based** optimization)

Model-based derivative free methods

Model-based derivative free methods

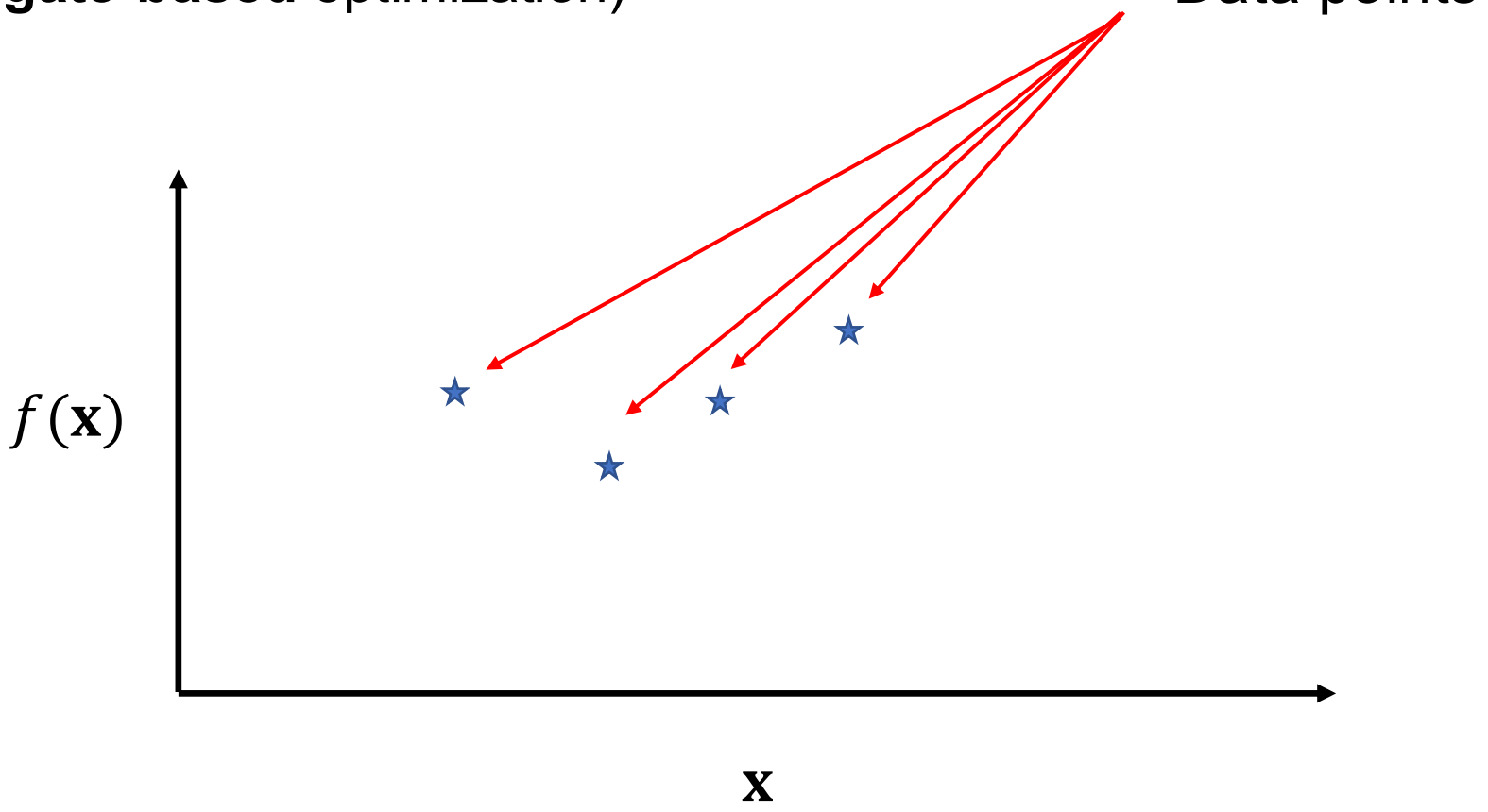
(**Surrogate-based** optimization)



Model-based derivative free methods

Model-based derivative free methods

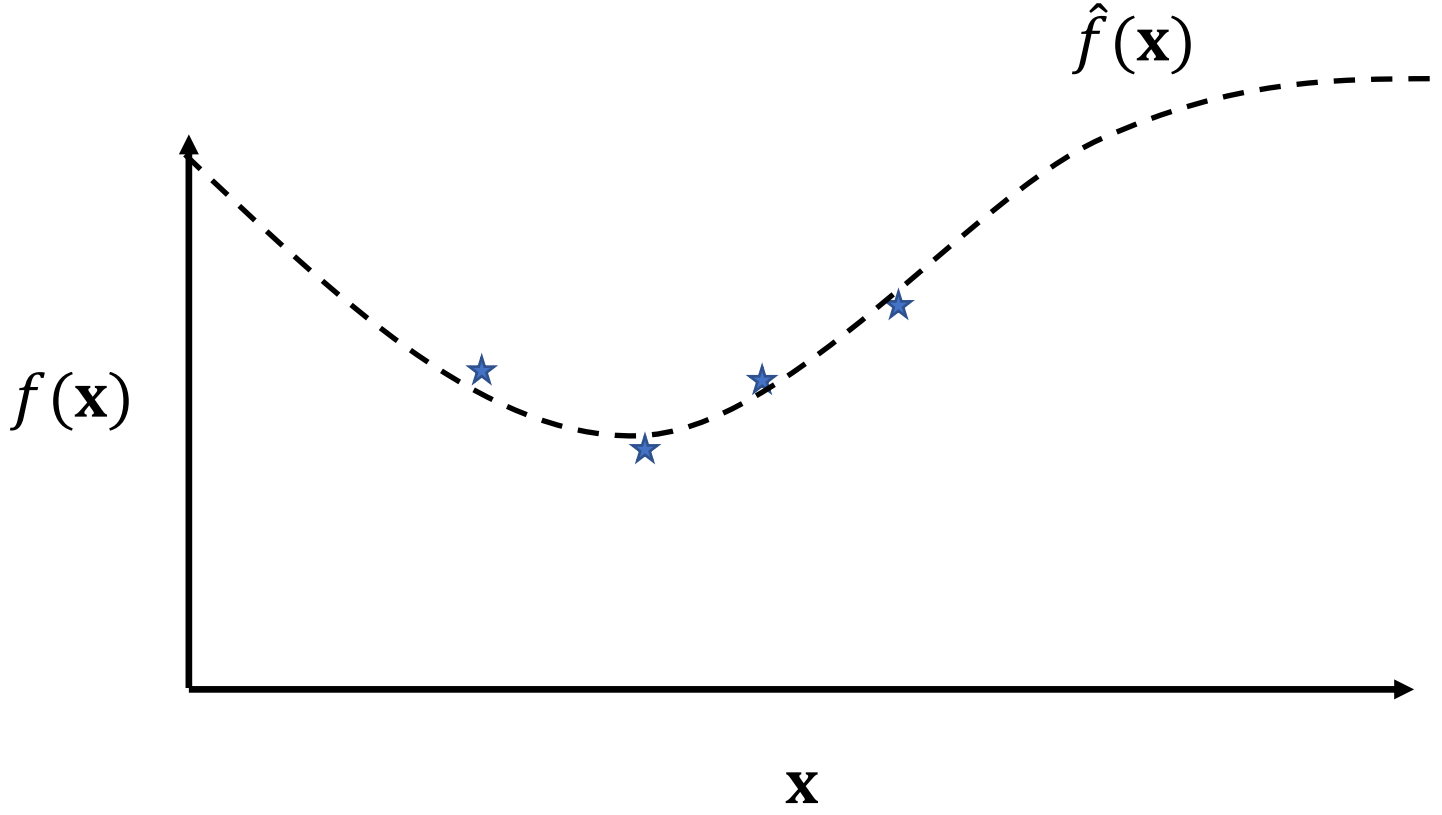
(**Surrogate-based** optimization)



Model-based derivative free methods

Model-based derivative free methods

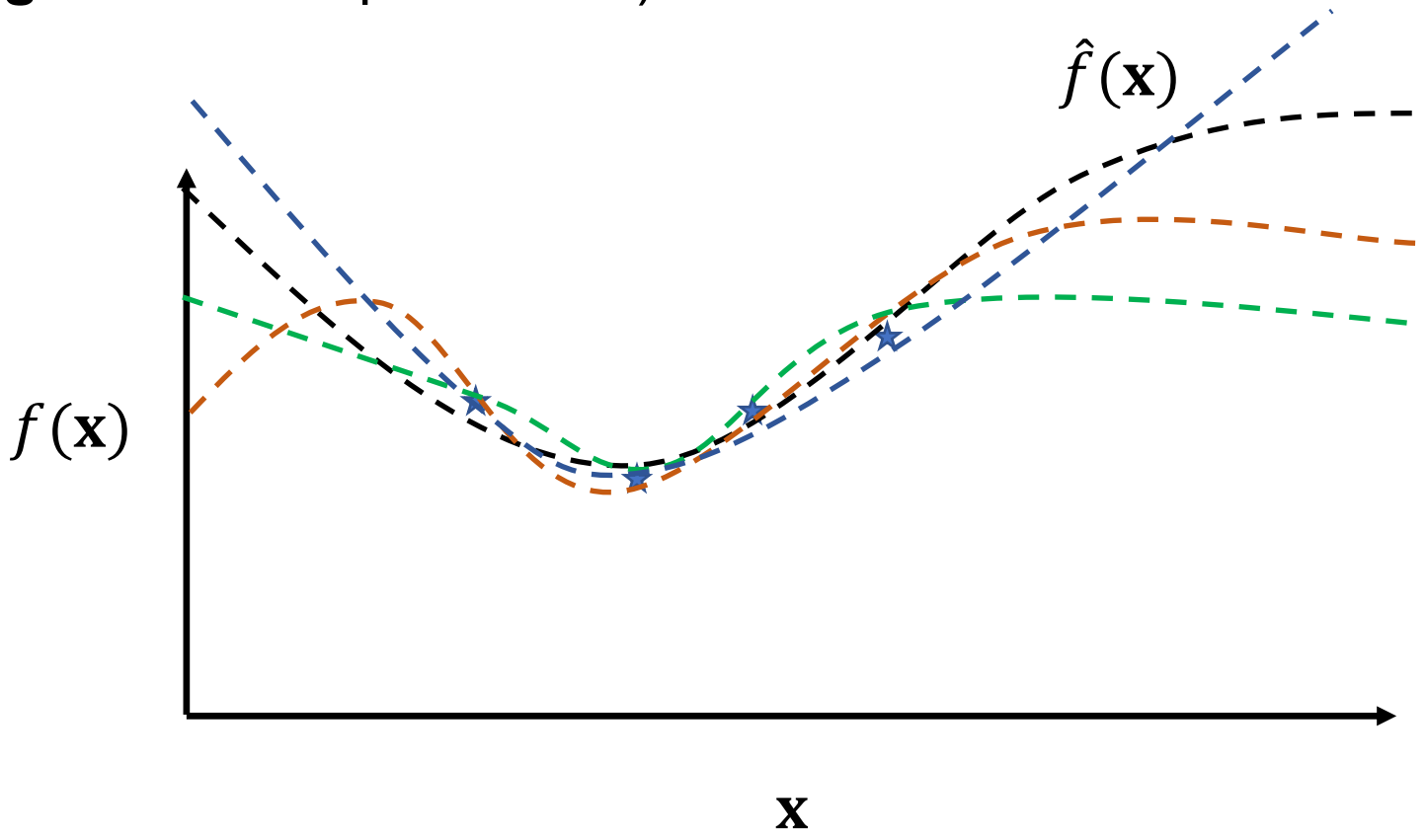
(**Surrogate-based** optimization)



Model-based derivative free methods

Model-based derivative free methods

(**Surrogate-based** optimization)



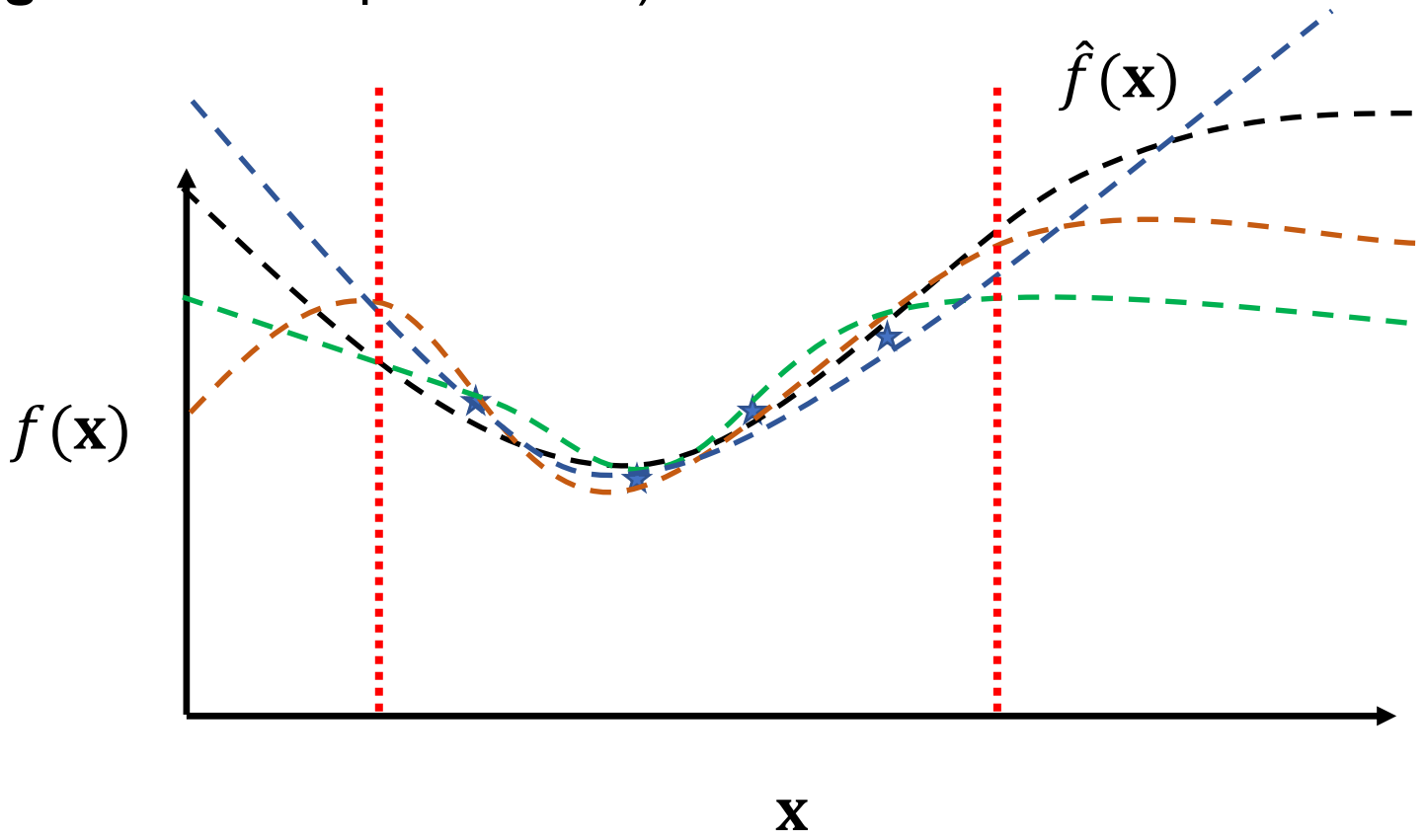
Key questions:

- How do we choose a function?
- Where do you trust your function? (trust region)

Model-based derivative free methods

Model-based derivative free methods

(**Surrogate-based** optimization)



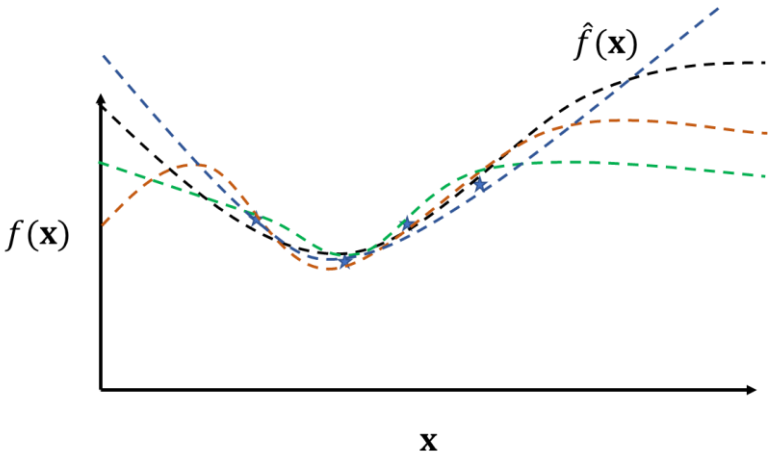
Key questions:

- How do we choose a function?
- Where do you trust your function? (trust region)

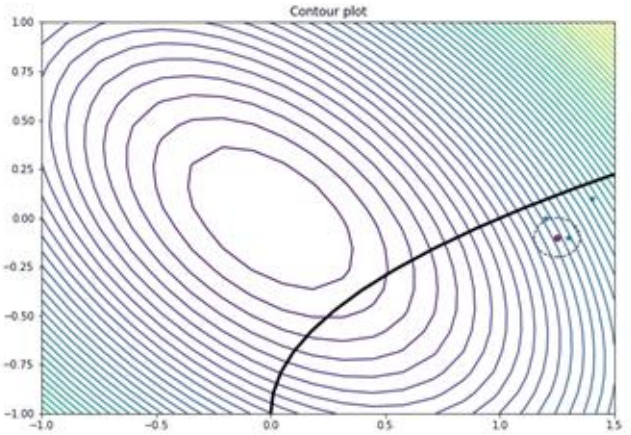
Model-based derivative free methods

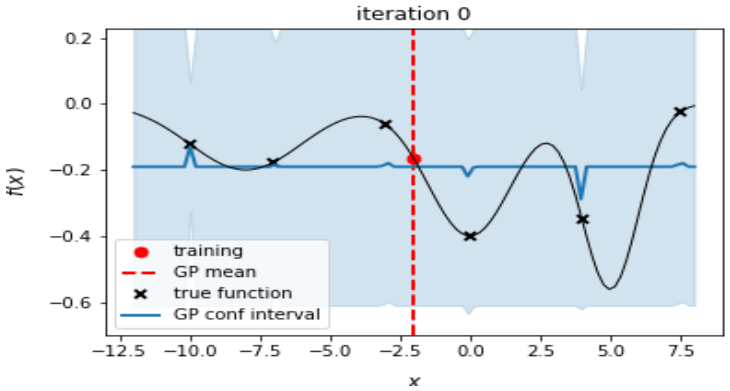
Model-based derivative free methods

(**Surrogate-based** optimization)

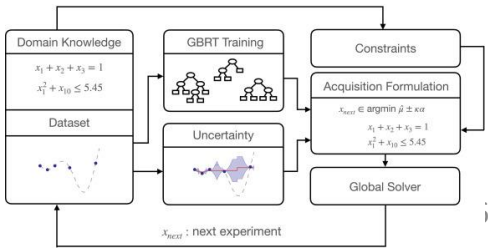


- Creates a model of the objective function (& constraints).
- Uses this model to choose the next best point.





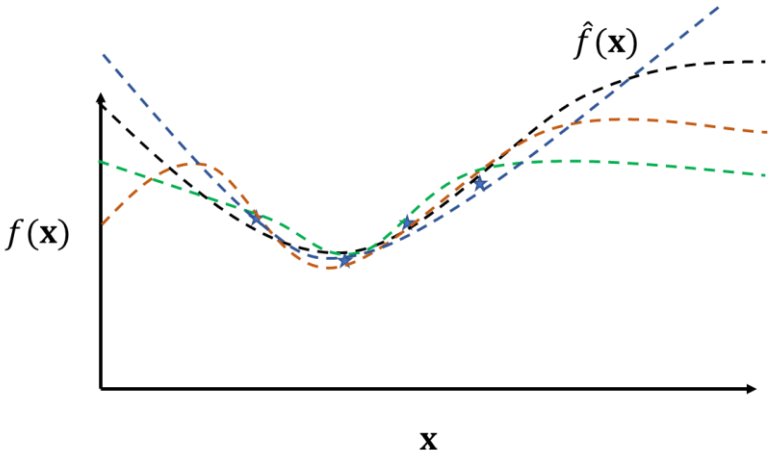
ENTMOOT

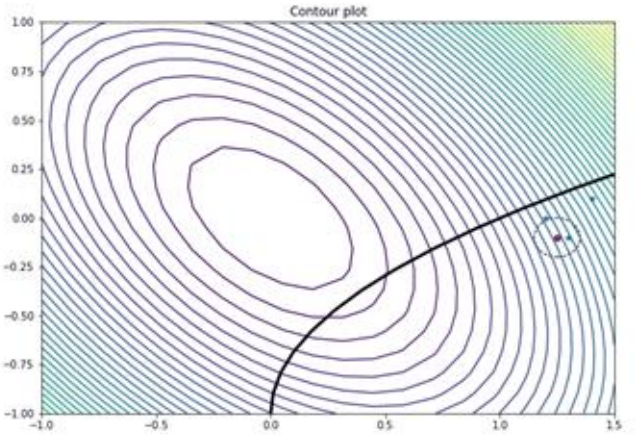


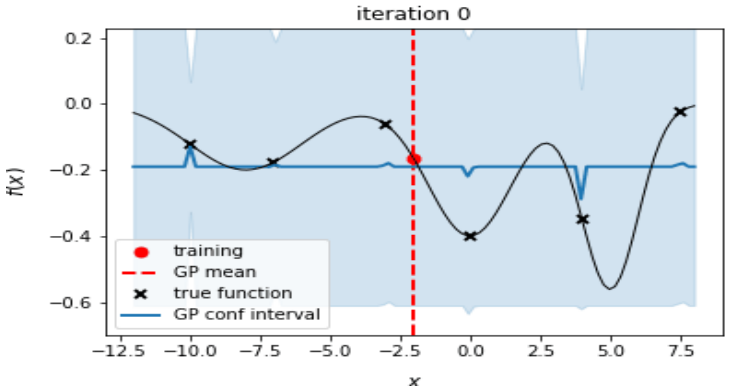
Model-based derivative free methods

Model-based derivative free methods

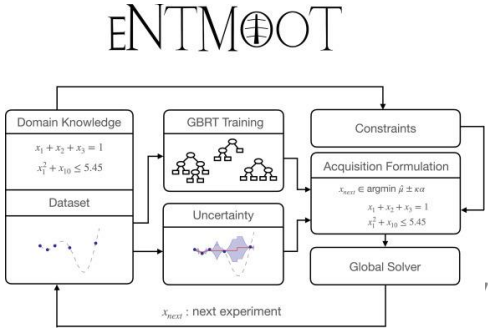
(**Surrogate-based** optimization)





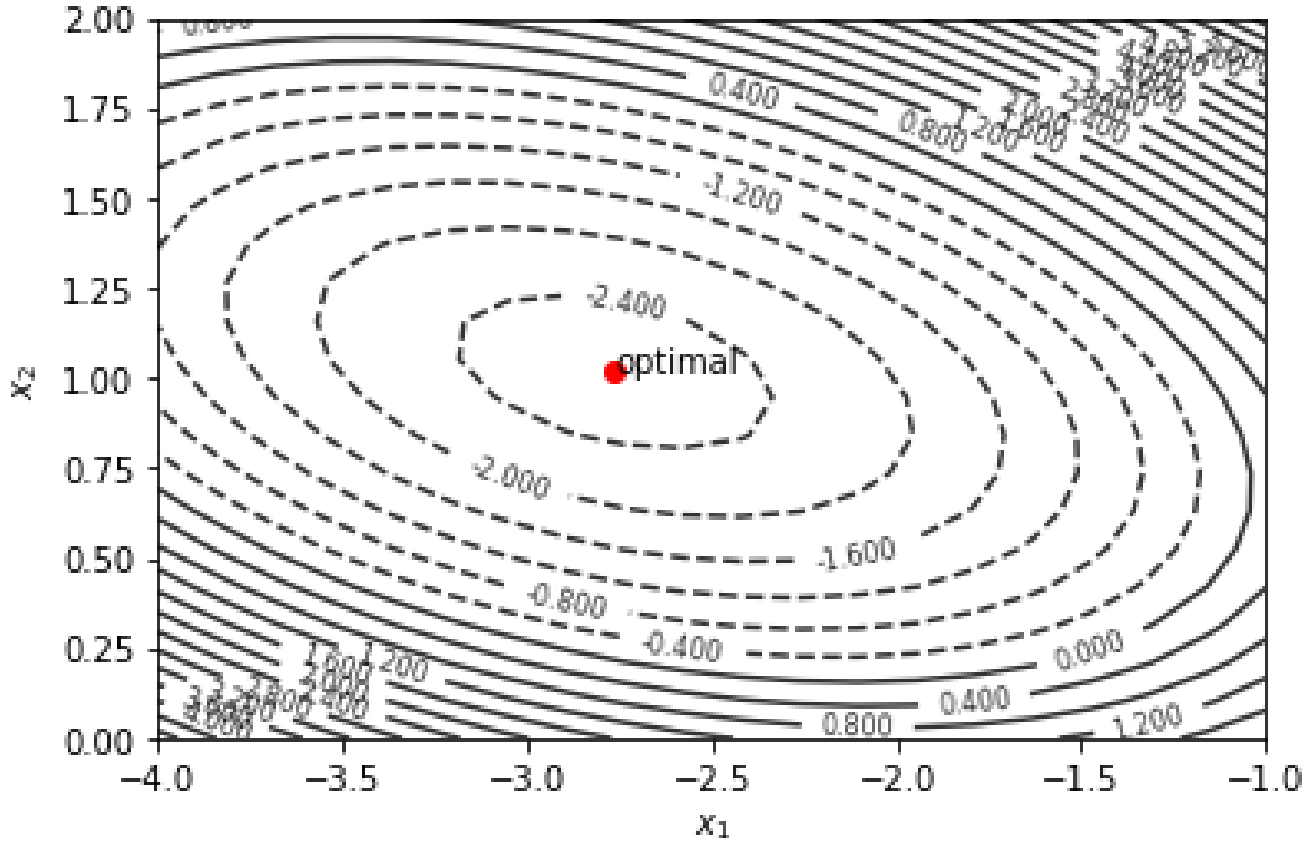


- Creates a model of the objective function (& constraints).
- Uses this model to choose the next best point.
- Many types of surrogates can be used:
  - **Quadratic surrogates**: COBYQA, CUATRO.
  - **Gaussian Process**: Bayesian Optimization
  - **Radial basis functions**: Shoemaker et. al.
  - **Decision Trees**: EntMoot



Local search with quadratic surrogate

Local search with quadratic surrogate

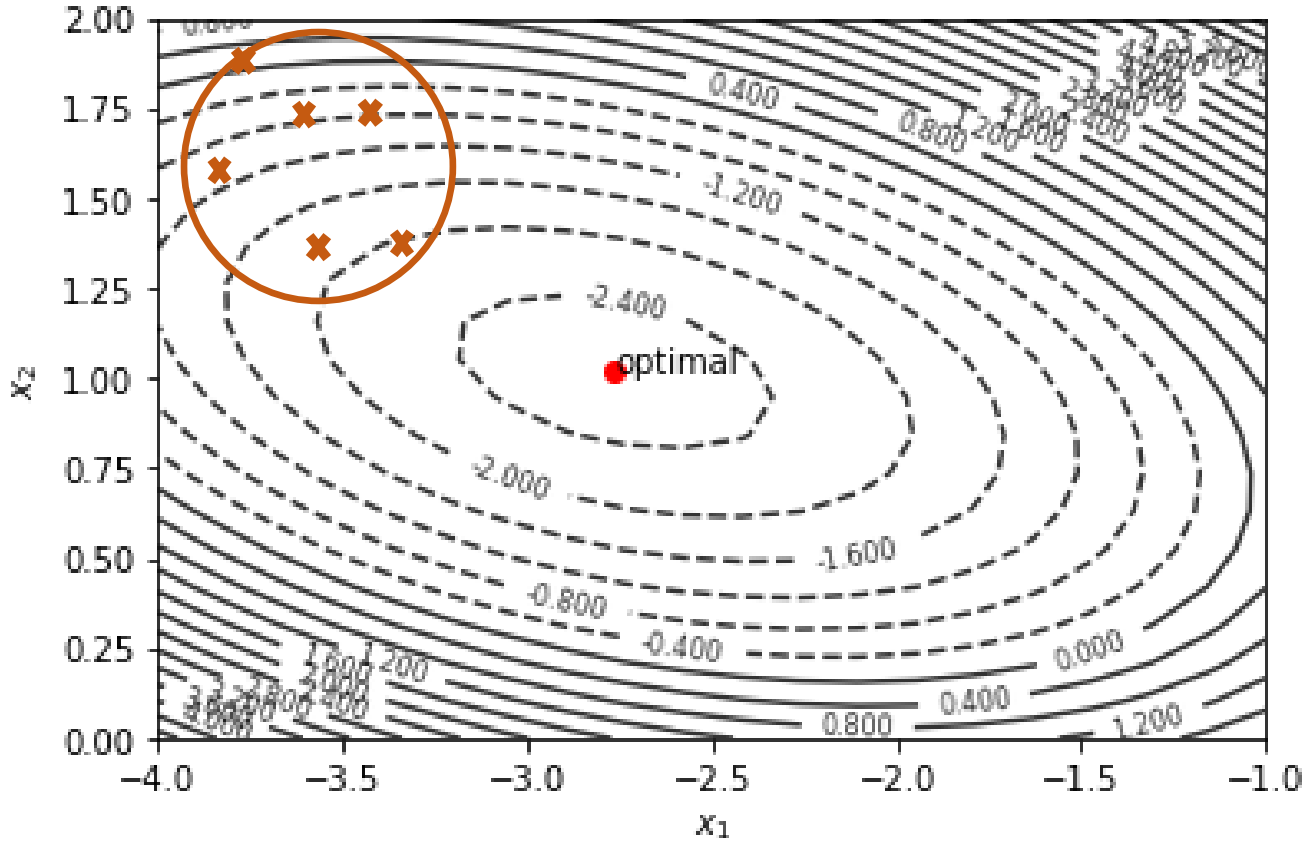


Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius

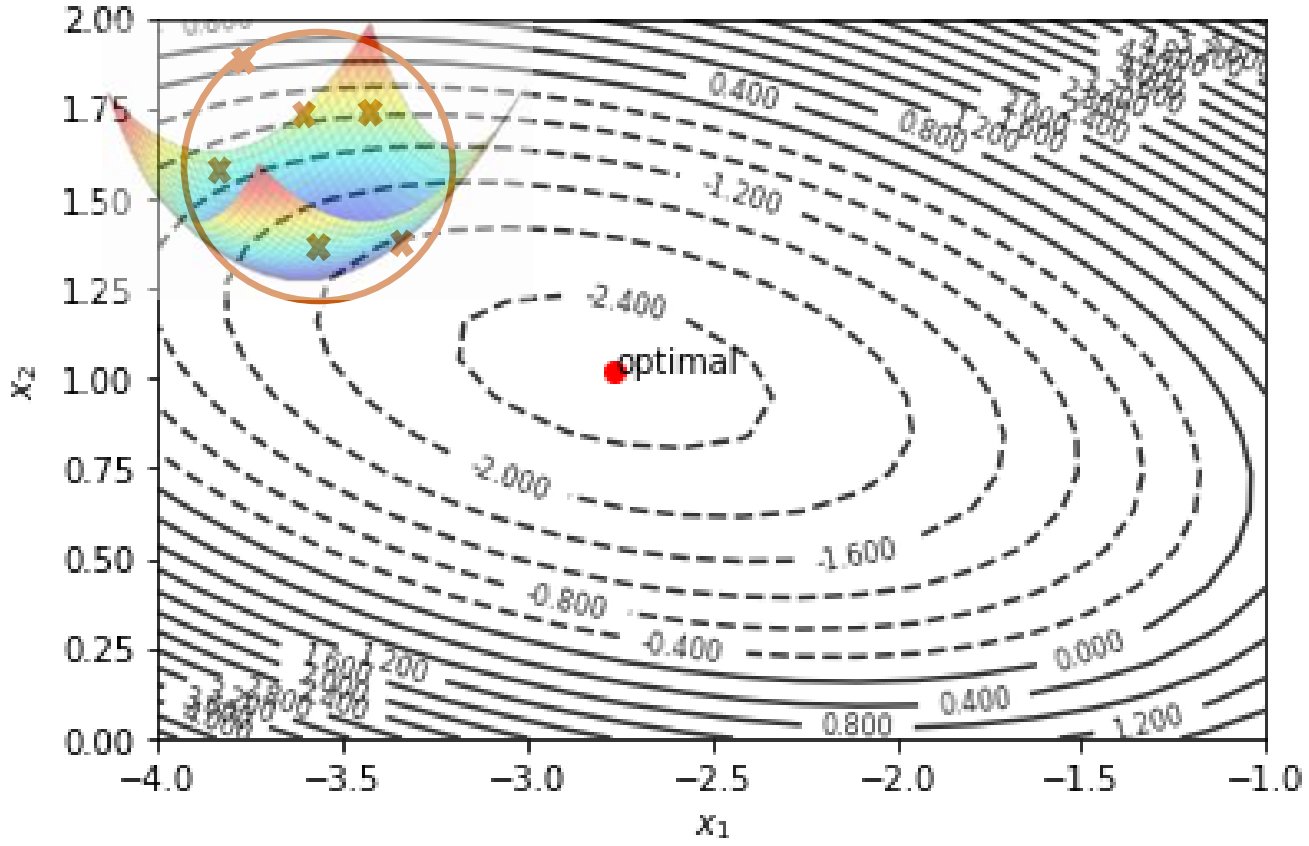


Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

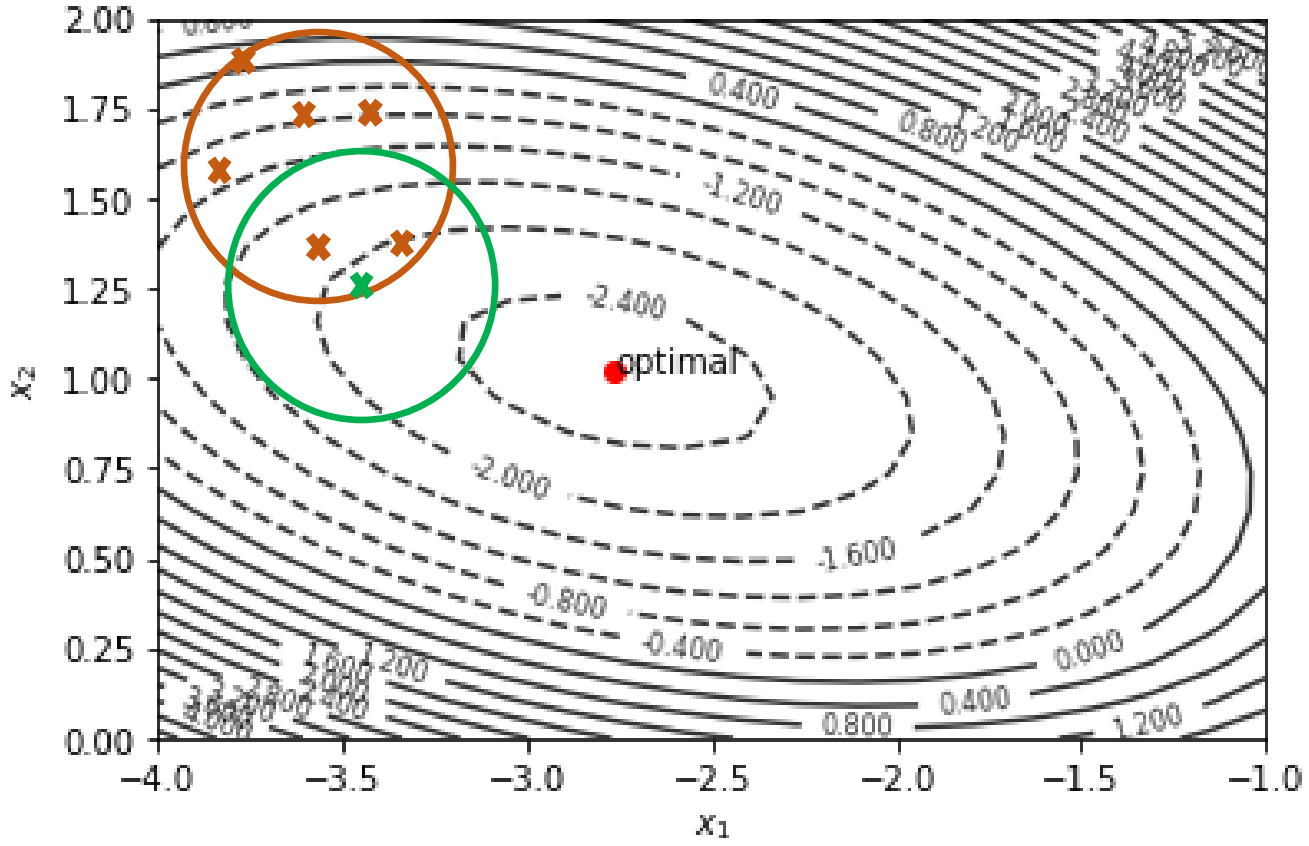


Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:
1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:
    1. **Optimise** surrogate
    2. **Move** to new point



Local search with quadratic surrogate

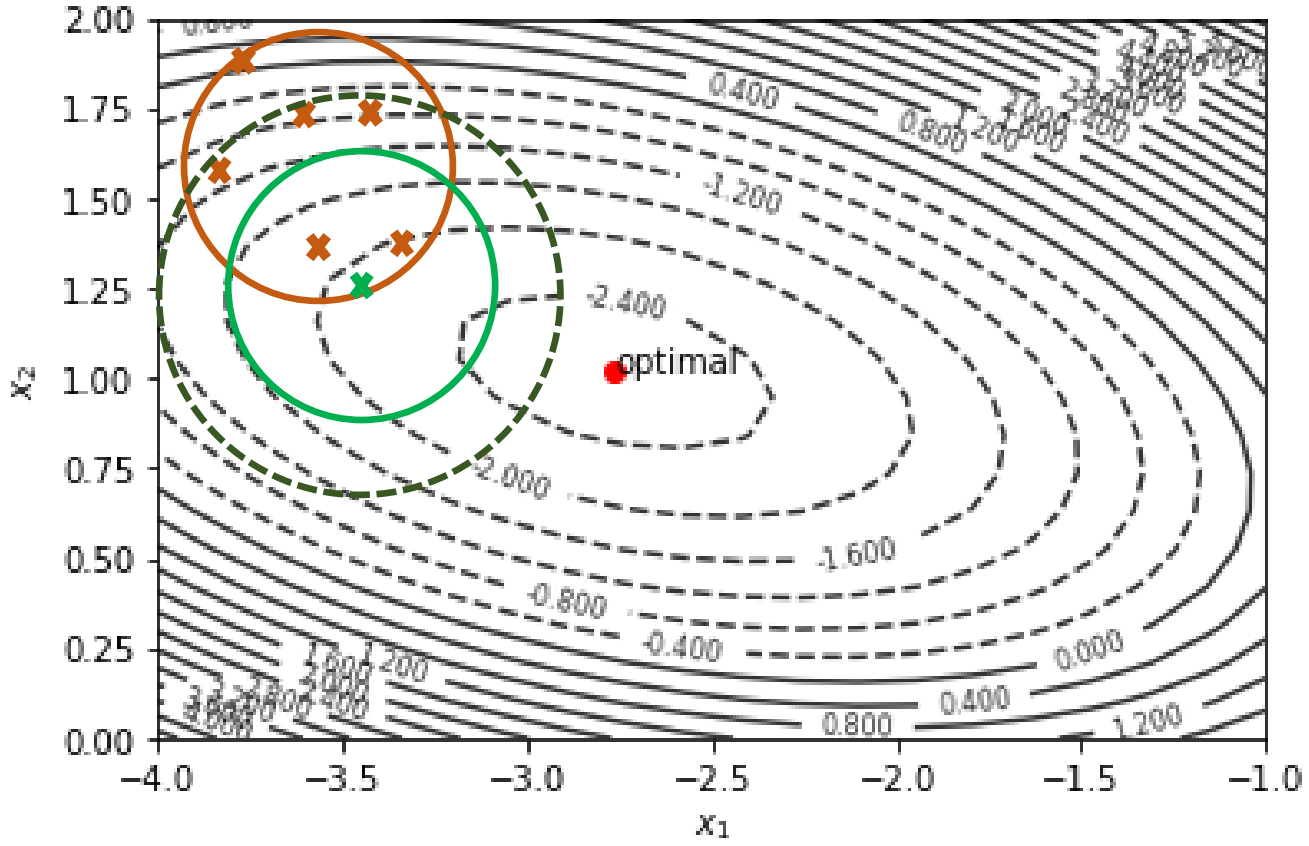
Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:

    1. **Optimise** surrogate
    2. **Move** to new point
    3. **Evaluate** how good the model is:
        If the model is good expand radius



Local search with quadratic surrogate

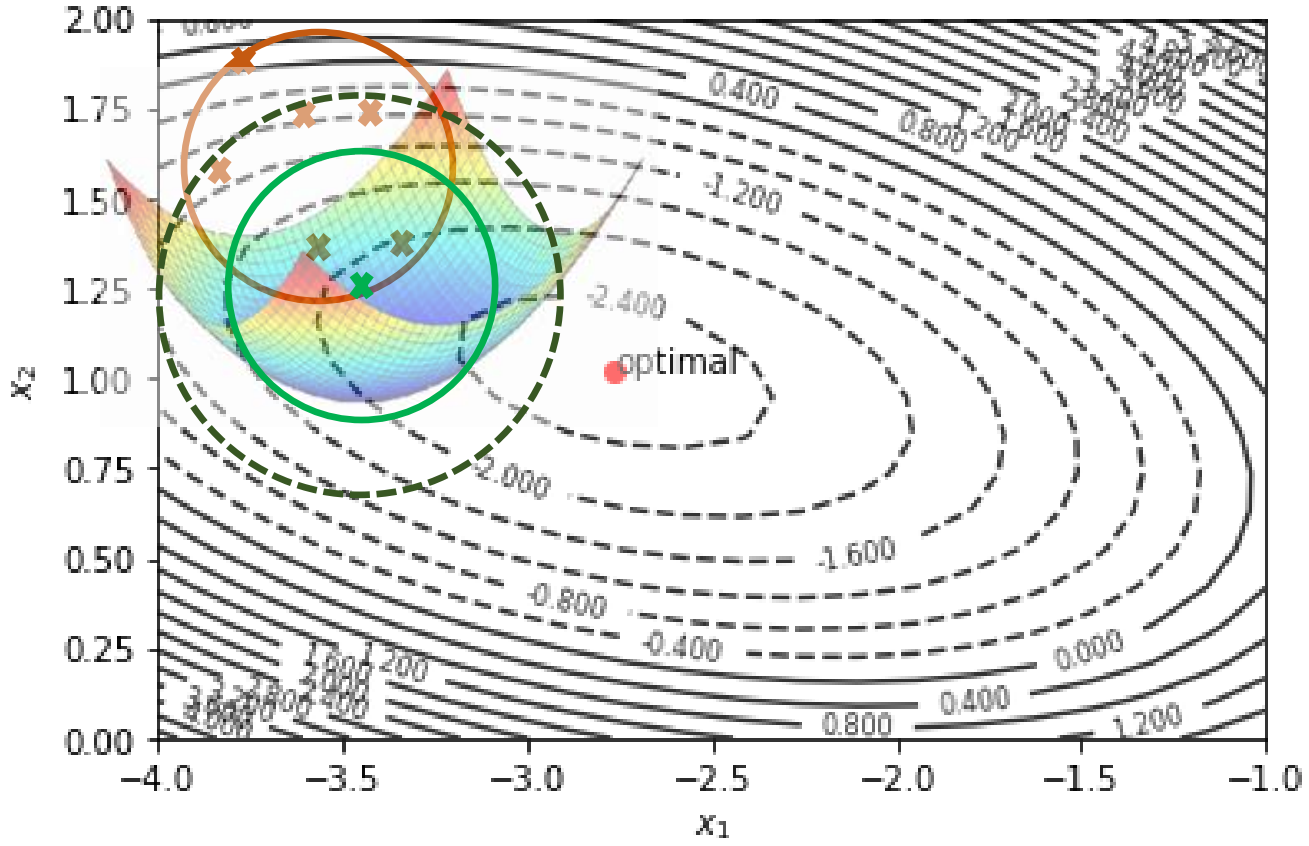
Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:

   1. **Optimise** surrogate
   2. **Move** to new point
   3. **Evaluate** how good the model is:
      If the model is good expand radius
   4. Re-train surrogate



Local search with quadratic surrogate

Local search with quadratic surrogate

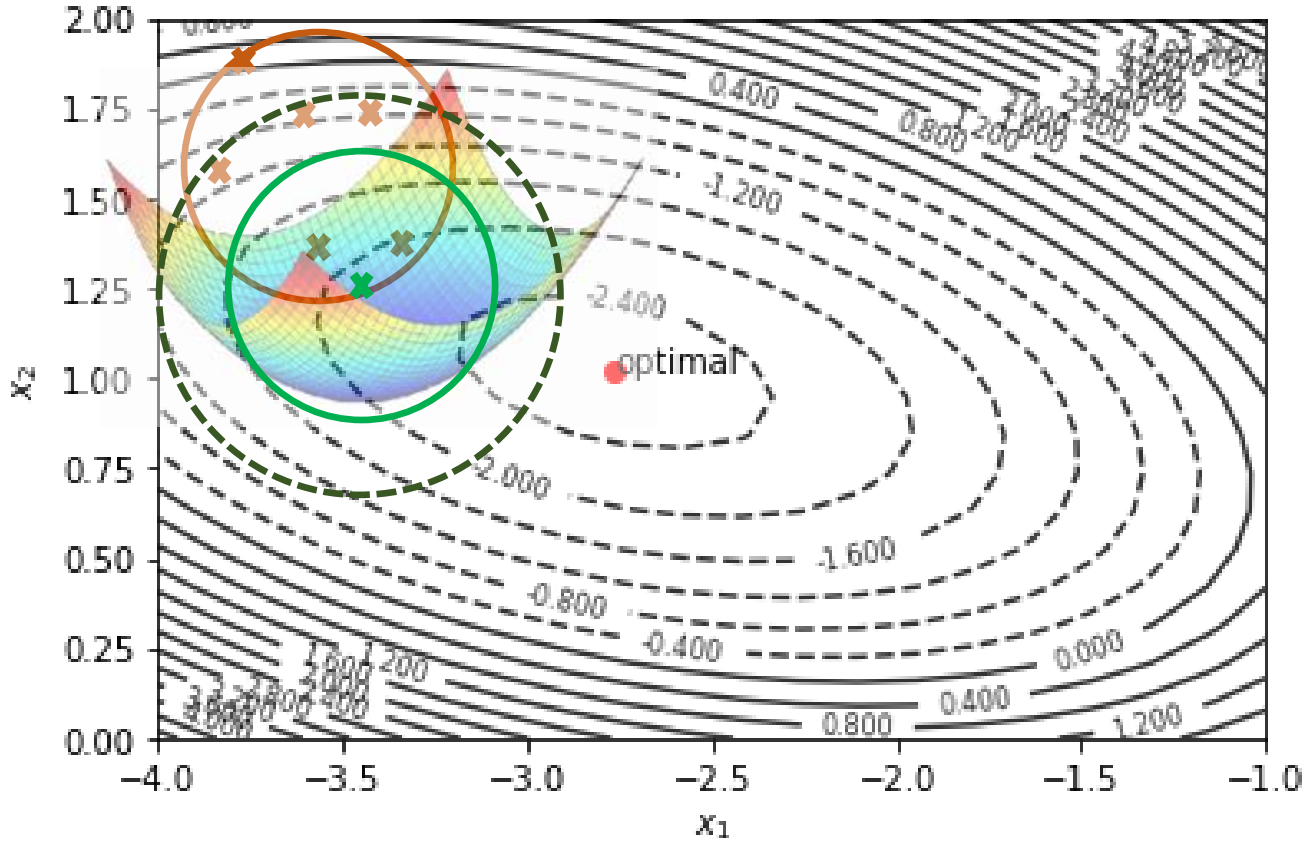
Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:

   1. **Optimise** surrogate
   2. **Move** to new point
   3. **Evaluate** how good the model is:
      If the model is good expand radius
   4. Re-train surrogate

Return best point



Local search with quadratic surrogate

Local search with quadratic surrogate

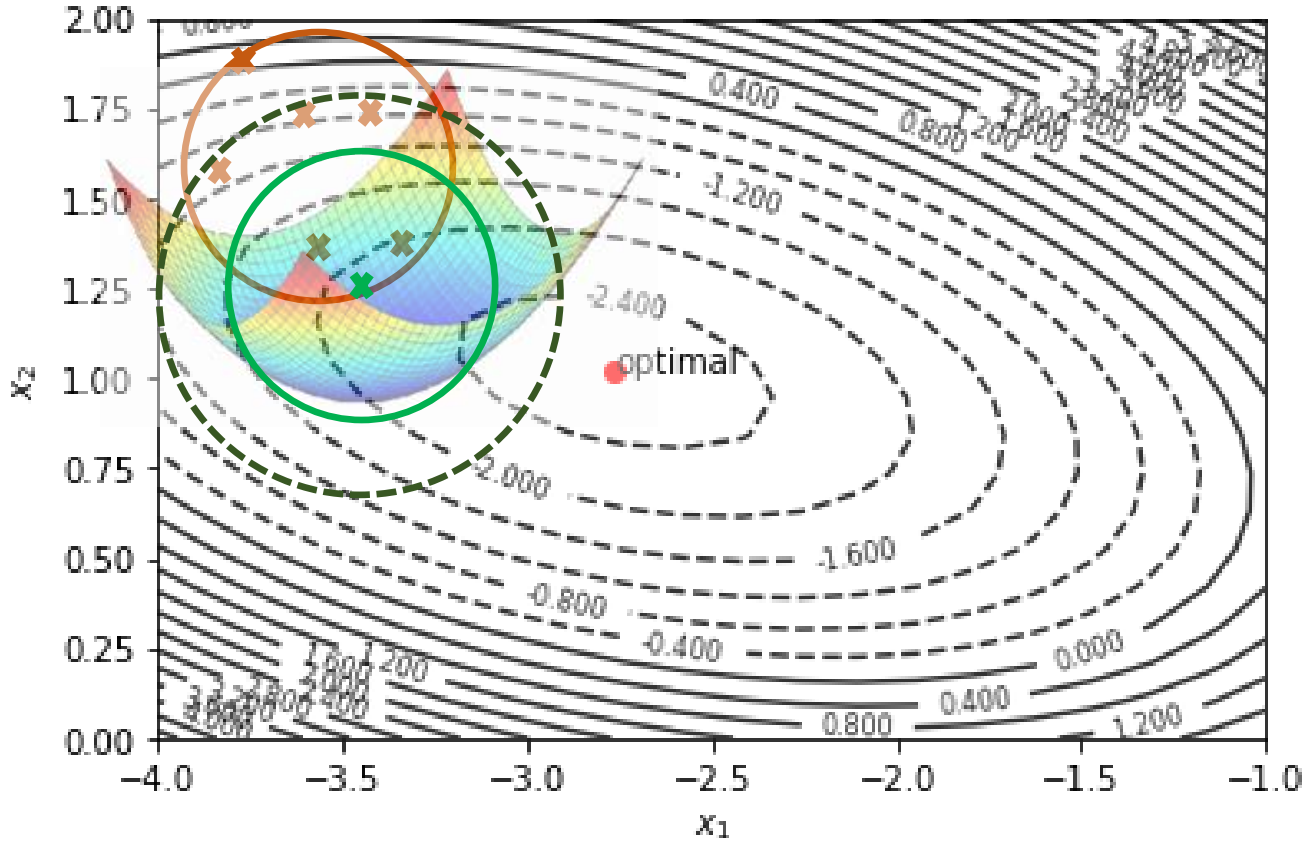
Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:

   1. **Optimise** surrogate
   2. **Move** to new point
   3. **Evaluate** how good the model is:
      If the model is good expand radius
   4. Re-train surrogate

Return best point



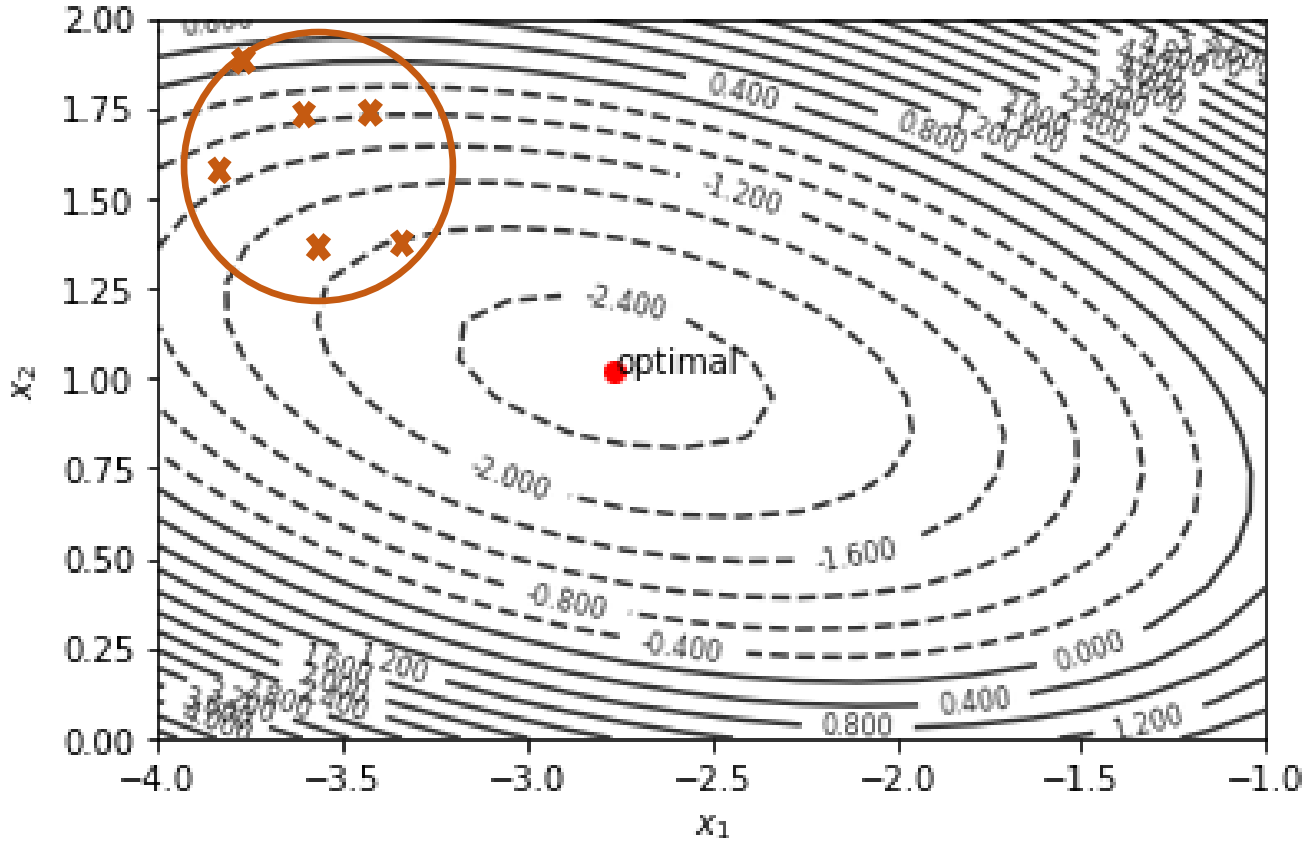
Let's look at how it would be implemented.

Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius

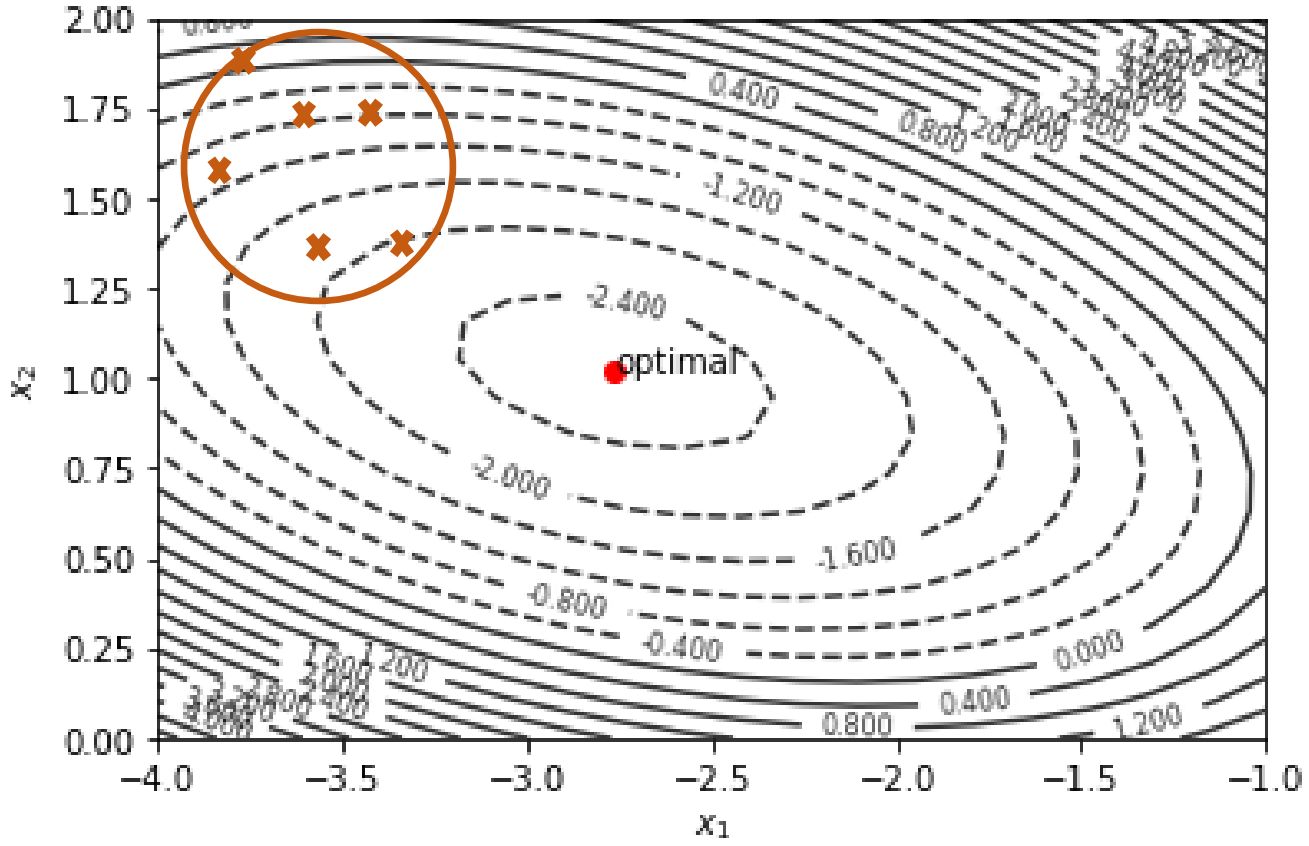


Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate



Local search with quadratic surrogate

Local search with quadratic surrogate

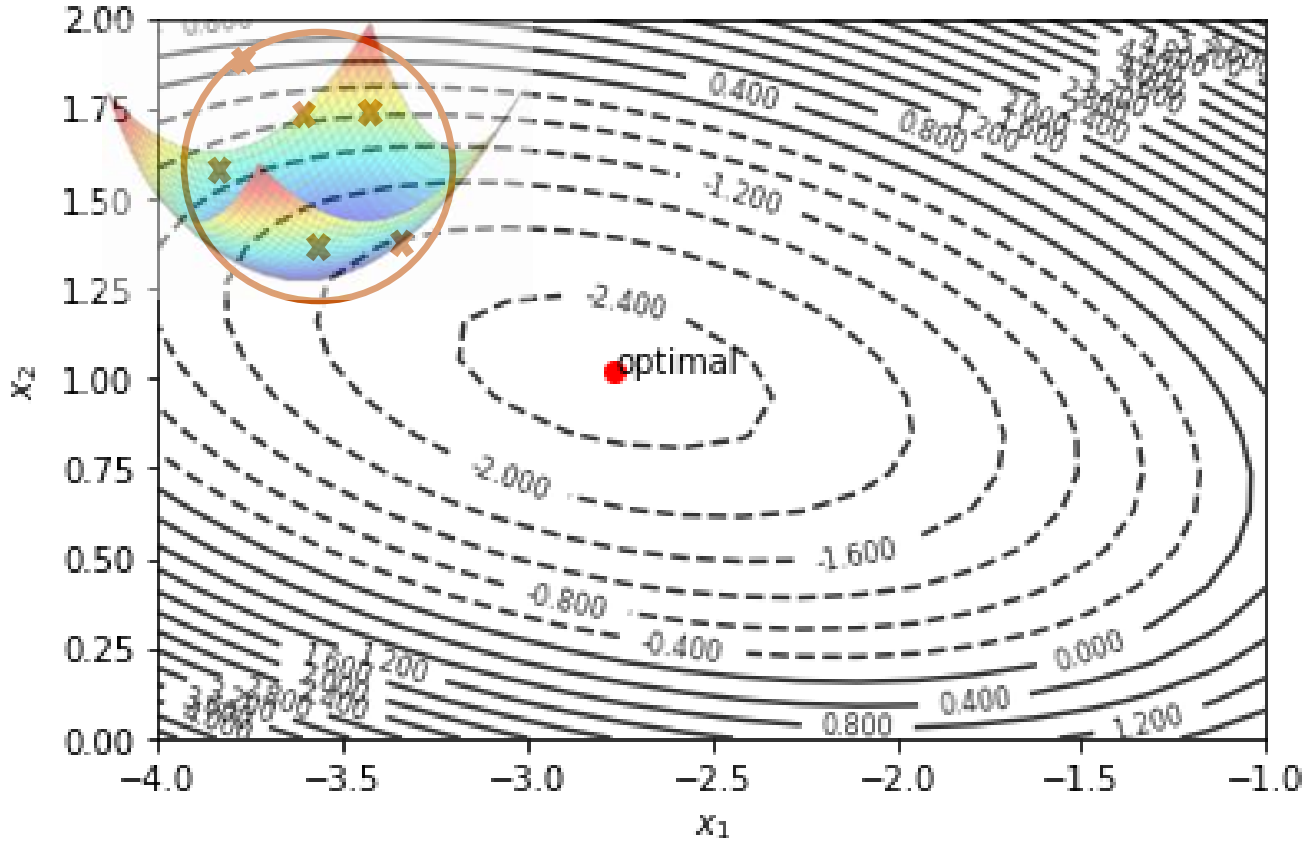
Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

$$\hat{f}(x_1, x_2) \approx f(x_1, x_2)$$

where

$$\hat{f}(x_1, x_2) = ax_1^2 + bx_2^2 + cx_1x_2$$

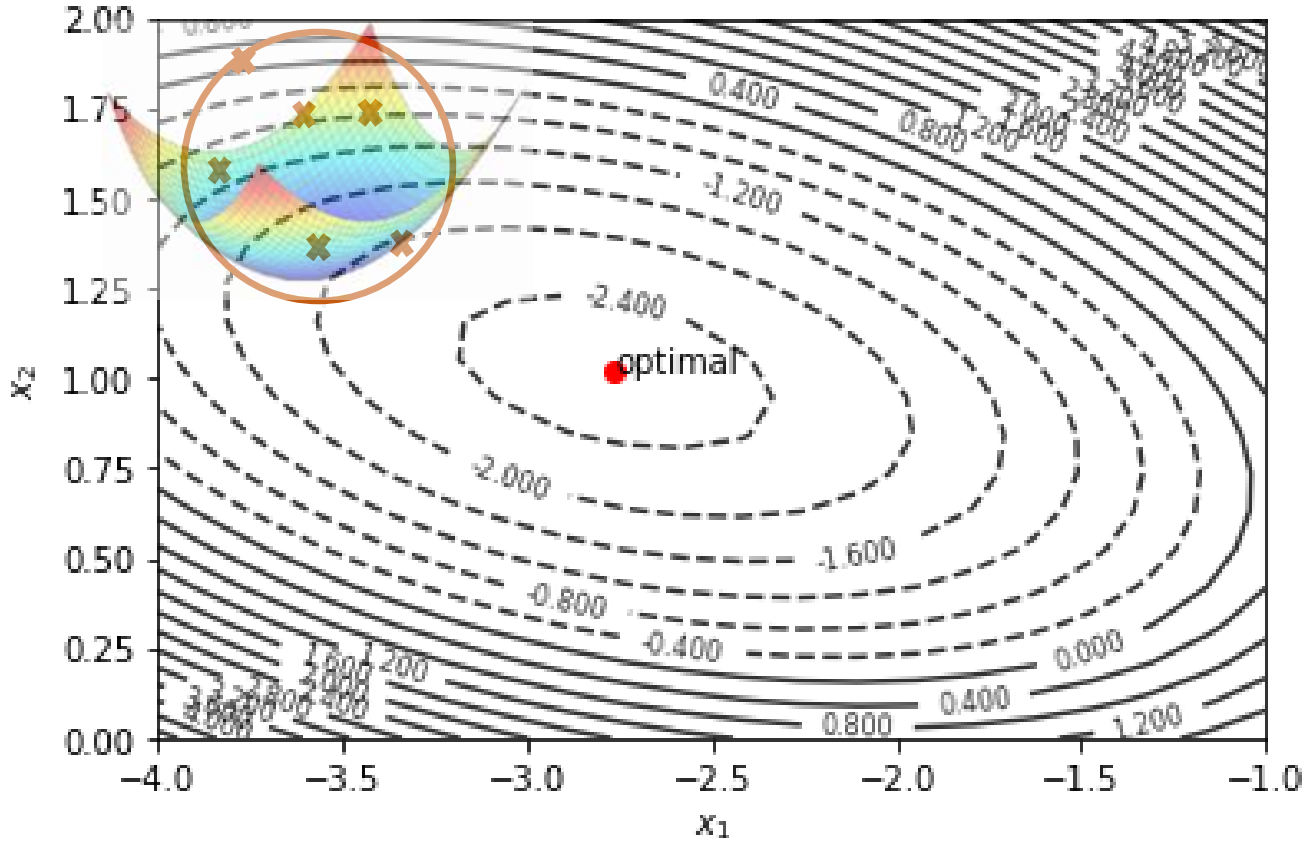


Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate



$$\hat{f}(x_1, x_2) \approx f(x_1, x_2)$$

where

$$\hat{f}(x_1, x_2) = ax_1^2 + bx_2^2 + cx_1x_2$$

Quadratic surrogate construction

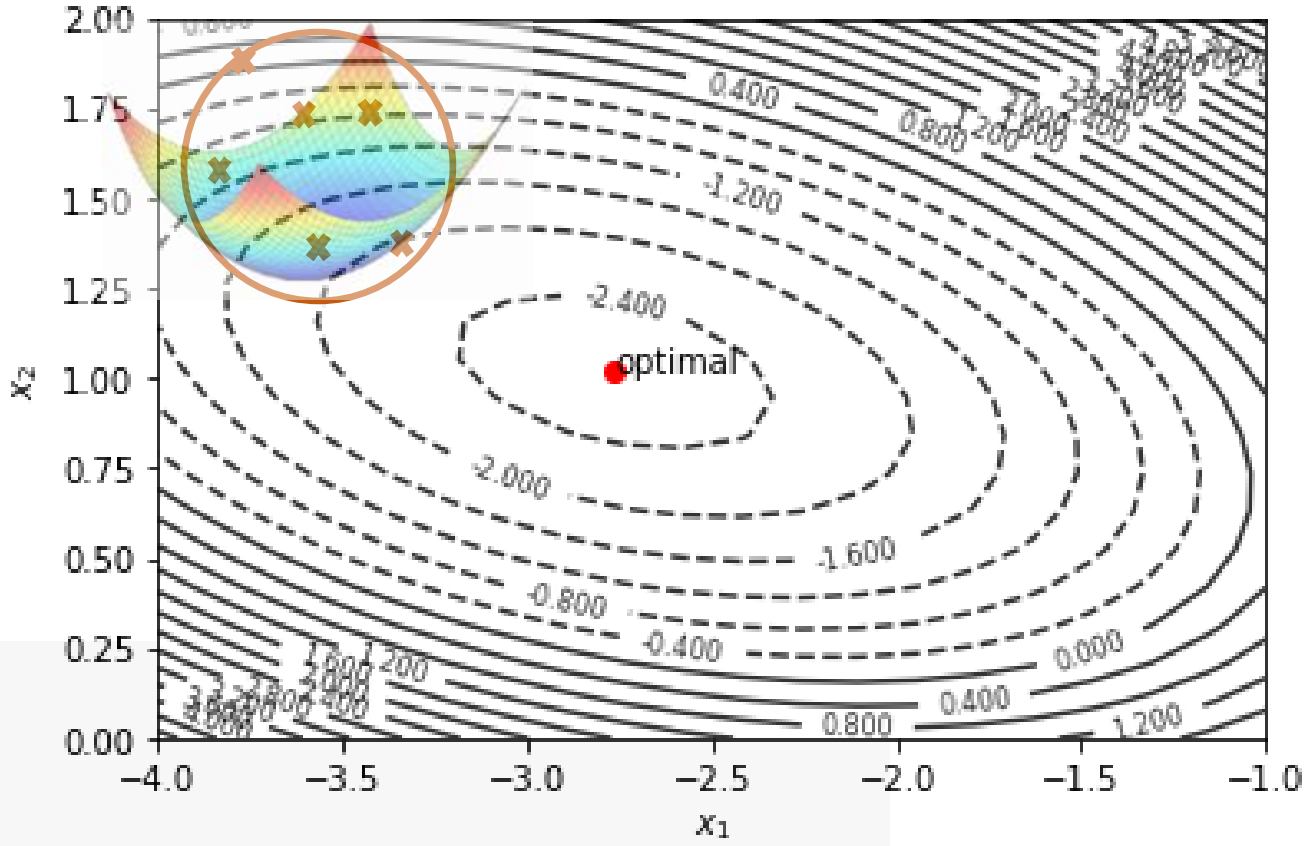
$$\min_{a,b,c} \sum_{i=1}^{n_d} \left(a\left(x_1^{(i)}\right)^2 + b\left(x_2^{(i)}\right)^2 + c\left(x_1^{(i)}\right)\left(x_2^{(i)}\right) - f\left(\left(x_1^{(i)}\right), \left(x_2^{(i)}\right)\right) \right)^2$$

Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate



Quadratic surrogate construction

```python
def LS(params, X, Y):
    '''
    X: matrix: [x^(1),...,x^(n_d)]
    Y: vecor:  [f(x^(1)),...,f(x^(n_d))]
    '''
    # renaming parameters for clarity (this can be avoided)
    a = params[0]; b = params[1]; c = params[2]
    # number of datapoints
    n_d = Y.shape[0]
    # weighted least squares
    LS = np.sum([(a*X[i][0]**2+b*X[i][1]**2+c*X[i][0]*X[i][1] - Y[i])**2 for i in range(n_d)])/n_d

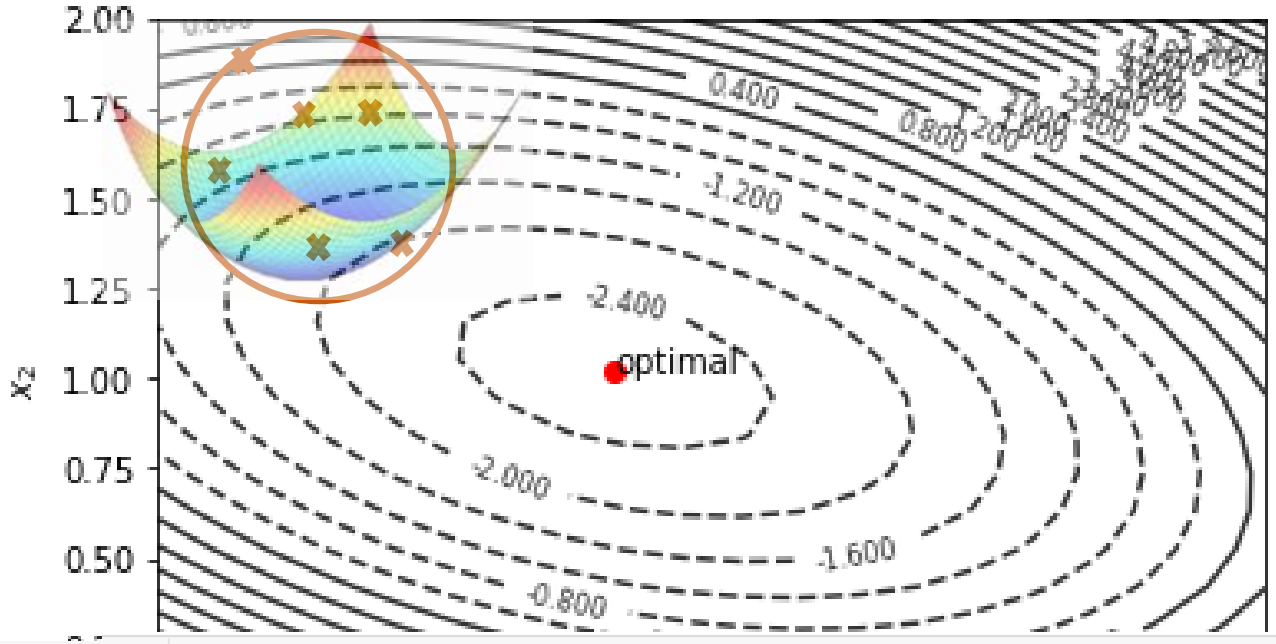
    return LS
```

Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate



Quadratic surrogate construction

```python
def LS(params, X, Y):
    '''
    X: matrix: [x^(1),...,x^(n_d)]
    Y: vecor:  [f(x^(1)),...,f(x^(n_d))]
    '''
    # renaming parameters for clarity (this can be avoided)
    a = params[0]; b = params[1]; c = params[2]
    # number of datapoints
    n_d = Y.shape[0]
    # weighted least squares
    LS = np.sum([(a*X[i][0]**2+b*X[i][1]**2+c*X[i][0]*X[i][
    return LS
```

```python
def quadratic_model_estimation(X, Y, p0):
    '''
    X: matrix: [x^(1),...,x^(n_d)]
    Y: vecor:  [f(x^(1)),...,f(x^(n_d))]
    p0: initial guess, preferably from last iteration
    '''
    # minimizing weighted least squares function with scipy
    res = minimize(LS, args=(X,Y), x0=p0, method='BFGS' )
    # obtaining solution
    params = res.x
    #f_val  = res.fun
    a = params[0]; b = params[1]; c = params[2]

    return a,b,c
```

Local search with quadratic surrogate

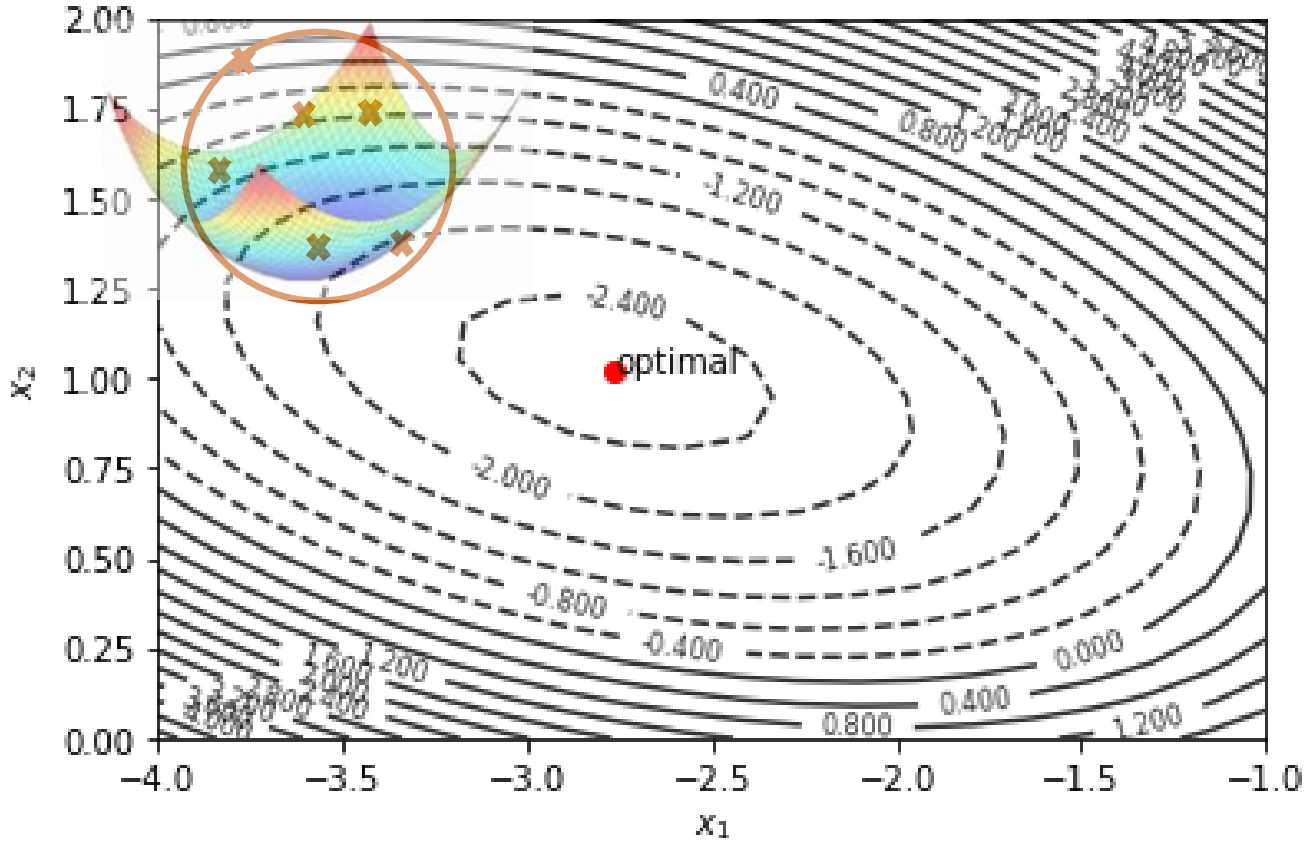
Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:

    1. **Optimise** surrogate



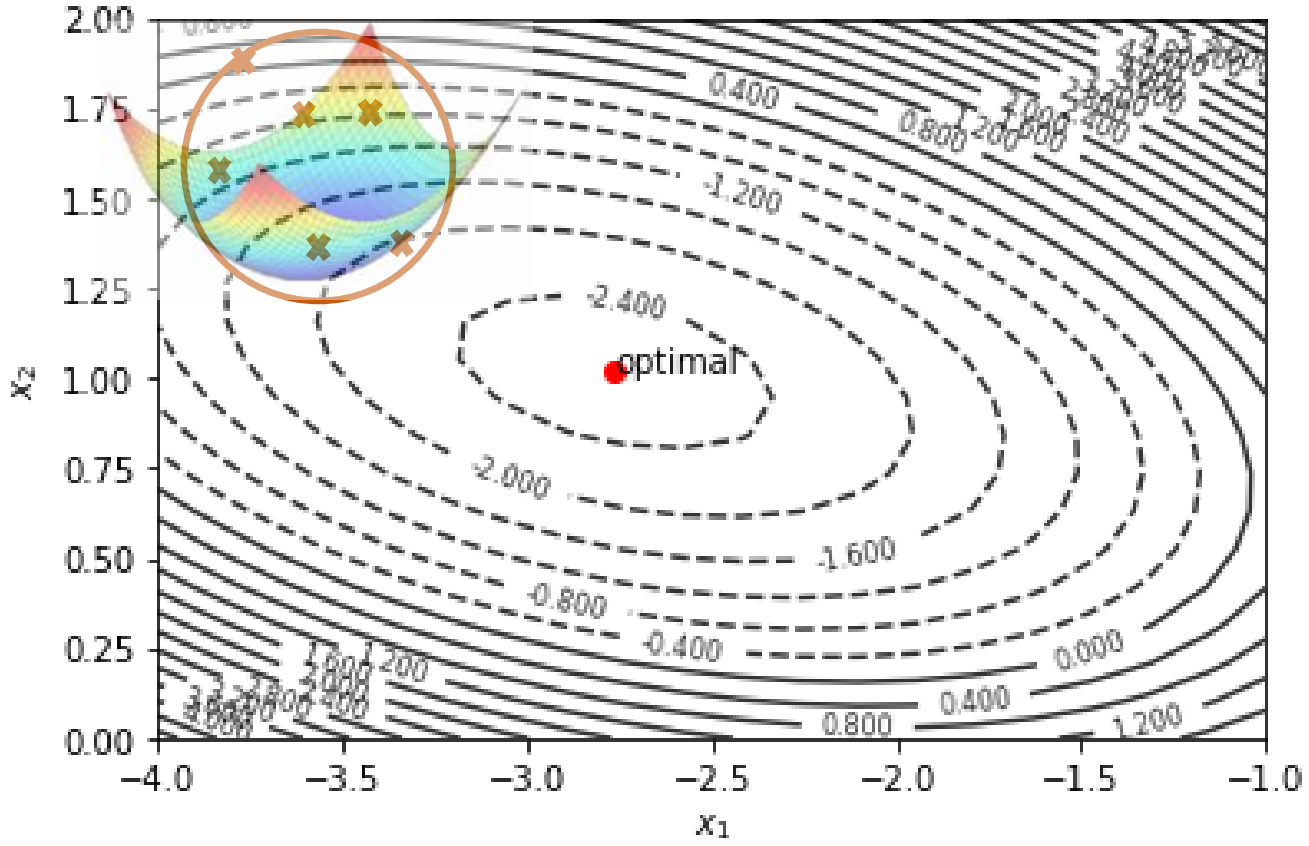
Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:
1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for i in **iterations:**
    1. **Optimise** surrogate

Optimise surrogate



$$\min_{x_1,x_2} \hat{f}(x_1, x_2) \coloneqq ax_1^2 + bx_2^2 + cx_1x_2$$

$$\text{subject to: } \sqrt{(\delta_1)^2 + (\delta_2)^2} \leq r$$

Where $x = x^0 + \delta$

Local search with quadratic surrogate

Optimise quadratic surrogate

$$\min_{x_1,x_2} \quad ax_1^2 + bx_2^2 + cx_1x_2$$
$$\text{subject to: } \sqrt{(\delta_1)^2 + (\delta_2)^2} \leq r$$

Where $x = x^0 + \delta$

$$\min_{\delta_1,\delta_2} \quad a(x_1^0 + \delta_1)^2 + b(x_2^0 + \delta_2)^2 + c(x_1^0 + \delta_1)(x_2^0 + \delta_2)$$
$$\text{subject to: } \sqrt{(\delta_1)^2 + (\delta_2)^2} \leq r$$

Where $\Delta x = x + \delta$

```
1  def quad_func(d, a,b,c,x0var):
2      '''
3      d: vector:  [d_1,d_2]: distance from centre in x,y coordinates
4      '''
5
6      return a*(x0var[0]+d[0])**2 + b*(x0var[1]+d[1])**2 + c*(x0var[0]+d[0])*(x0var[1]+d[1])
7
```

Local search with quadratic surrogate

Optimise quadratic surrogate

$$\min_{x_1, x_2} \quad ax_1^2 + bx_2^2 + cx_1x_2$$

$$\text{subject to: } \sqrt{(\delta_1)^2 + (\delta_2)^2} \leq r$$

Where $x = x^0 + \delta$

Local search with quadratic surrogate

Optimise quadratic surrogate

$$\min_{x_1,x_2} \quad ax_1^2 + bx_2^2 + cx_1x_2$$

$$\text{subject to: } \sqrt{(\delta_1)^2 + (\delta_2)^2} \leq r$$

Where $x = x^0 + \delta$

```python
from scipy.optimize import NonlinearConstraint
def opt_quadratic_model(a,b,c,x0var,r_t):
    '''
    a,b,c: parameters estimated for the quadratic model
    x0var: initial point: last
    '''
    # trust region constraint
    cons = ({'type': 'ineq', 'fun': lambda d:  r_t**2 - (d[0]**2 + d[1]**2) })
    #con = lambda x:  (x[0]**2 + x[1]**2)**2
    #nlc = NonlinearConstraint(con, -np.inf, r_t**2)

    # minimising quadratic model
    res = minimize(quad_func, args=(a,b,c,x0var), x0=np.array([0,0]),
                   method='SLSQP', constraints=cons, bounds=((-r_t,r_t),(-r_t,r_t)) )
    # Note: bounds are added: nonlinear trust region is handled poorly by SLSQP

    # retrieving solution
    d_sol = res.x
    #print('res = ',res) print status
    # returning solution
    return x0var + d_sol
```

```python
def quad_func(d, a,b,c,x0var):
    '''
    d: vector:  [d_1,d_2]: distance from centre in x,y coordinates
    '''

    return a*(x0var[0]+d[0])**2 + b*(x0var[1]+d[1])**2 + c*(x0var[0]+d[0])*(x0var[1]+d[1])
```

Local search with quadratic surrogate

Optimise quadratic surrogate

$$\min_{x_1,x_2} \quad ax_1^2 + bx_2^2 + cx_1x_2$$

$$\text{subject to: } \sqrt{(\delta_1)^2 + (\delta_2)^2} \leq r$$

$$\text{Where } x = x^0 + \delta$$

This is how you would include nonlinear constraints

```python
from scipy.optimize import NonlinearConstraint
def opt_quadratic_model(a,b,c,x0var,r_t):
    '''
    a,b,c: parameters estimated for the quadratic model
    x0var: initial point: last
    '''
    # trust region constraint
    cons = ({'type': 'ineq', 'fun': lambda d:  r_t**2 - (d[0]**2 + d[1]**2) })
    #con = lambda x:  (x[0]**2 + x[1]**2)**2
    #nlc = NonlinearConstraint(con, -np.inf, r_t**2)

    # minimising quadratic model
    res = minimize(quad_func, args=(a,b,c,x0var), x0=np.array([0,0]),
                   method='SLSQP', constraints=cons, bounds=((-r_t,r_t),(-r_t,r_t)) )
    # Note: bounds are added: nonlinear trust region is handled poorly by SLSQP

    # retrieving solution
    d_sol = res.x
    #print('res = ',res) print status
    # returning solution
    return x0var + d_sol
```

```python
def quad_func(d, a,b,c,x0var):
    '''
    d: vector:  [d_1,d_2]: distance from centre in x,y coordinates
    '''

    return a*(x0var[0]+d[0])**2 + b*(x0var[1]+d[1])**2 + c*(x0var[0]+d[0])*(x0var[1]+d[1])
```

Local search with quadratic surrogate

Optimise quadratic surrogate

$$\min_{x_1,x_2} \quad ax_1^2 + bx_2^2 + cx_1x_2$$

$$\text{subject to: } \sqrt{(\delta_1)^2 + (\delta_2)^2} \leq r$$

$$\text{Where } x = x^0 + \delta$$

```python
from scipy.optimize import NonlinearConstraint
def opt_quadratic_model(a,b,c,x0var,r_t):
    '''
    a,b,c: parameters estimated for the quadratic model
    x0var: initial point: last
    '''
    # trust region constraint
    cons = ({'type': 'ineq', 'fun': lambda d:  r_t**2 - (d[0]**2 + d[1]**2) })
    #con = lambda x:  (x[0]**2 + x[1]**2)**2
    #nlc = NonlinearConstraint(con, -np.inf, r_t**2)

    # minimising quadratic model
    res = minimize(quad_func, args=(a,b,c,x0var), x0=np.array([0,0]),
                   method='SLSQP', constraints=cons, bounds=((-r_t,r_t),(-r_t,r_t)) )
    # Note: bounds are added: nonlinear trust region is handled poorly by SLSQP

    # retrieving solution
    d_sol = res.x
    #print('res = ',res) print status
    # returning solution
    return x0var + d_sol
```

This is how you would include nonlinear constraints

Optimization of the quadratic function

```python
def quad_func(d, a,b,c,x0var):
    '''
    d: vector:  [d_1,d_2]: distance from centre in x,y coordinates
    '''

    return a*(x0var[0]+d[0])**2 + b*(x0var[1]+d[1])**2 + c*(x0var[0]+d[0])*(x0var[1]+d[1])
```

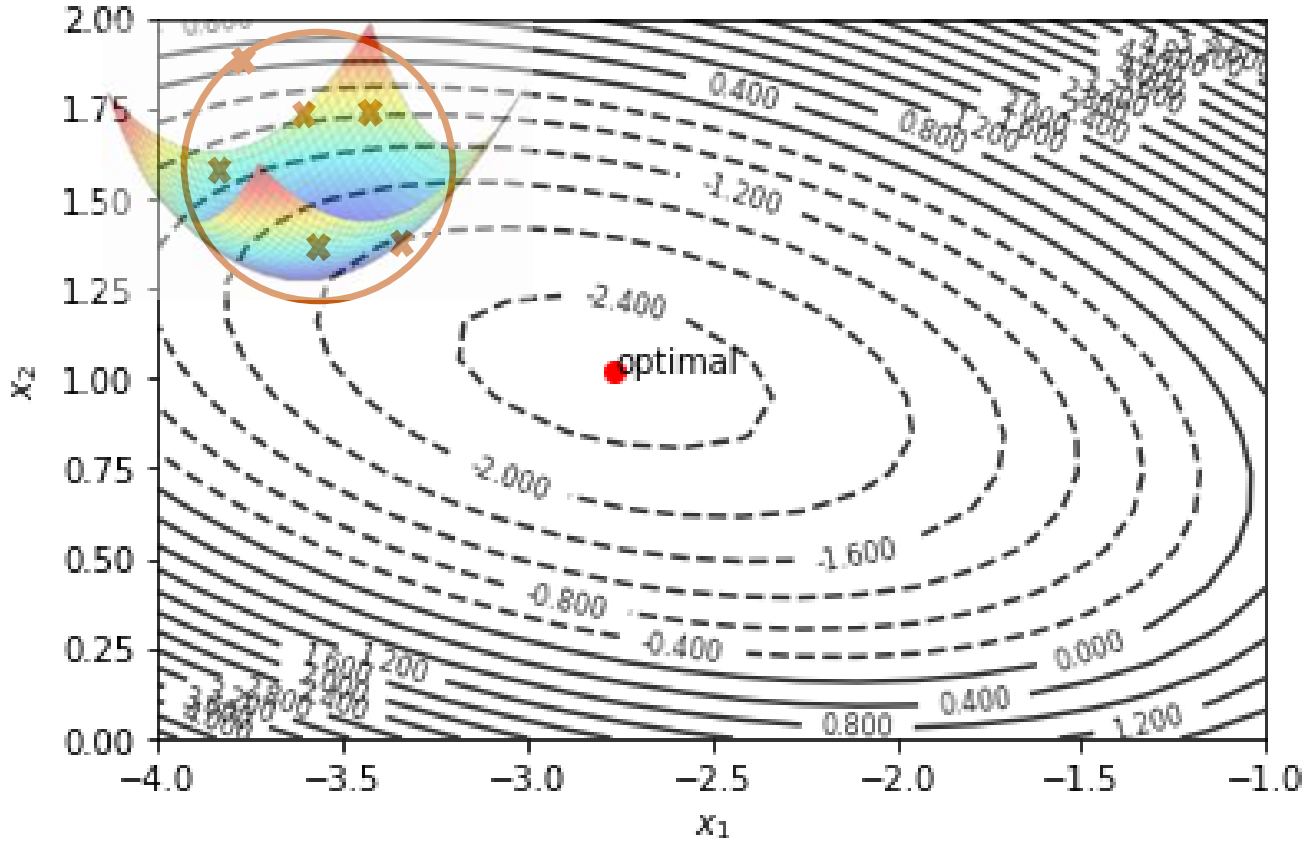
Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:
1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for i in **iterations**:
    1. **Optimise** surrogate

Optimise surrogate



$$\min_{x_1,x_2} \hat{f}(x_1, x_2) \coloneqq ax_1^2 + bx_2^2 + cx_1x_2$$

$$\text{subject to: } \sqrt{(\delta_1)^2 + (\delta_2)^2} \leq r$$

Where $x = x^0 + \delta$

Local search with quadratic surrogate

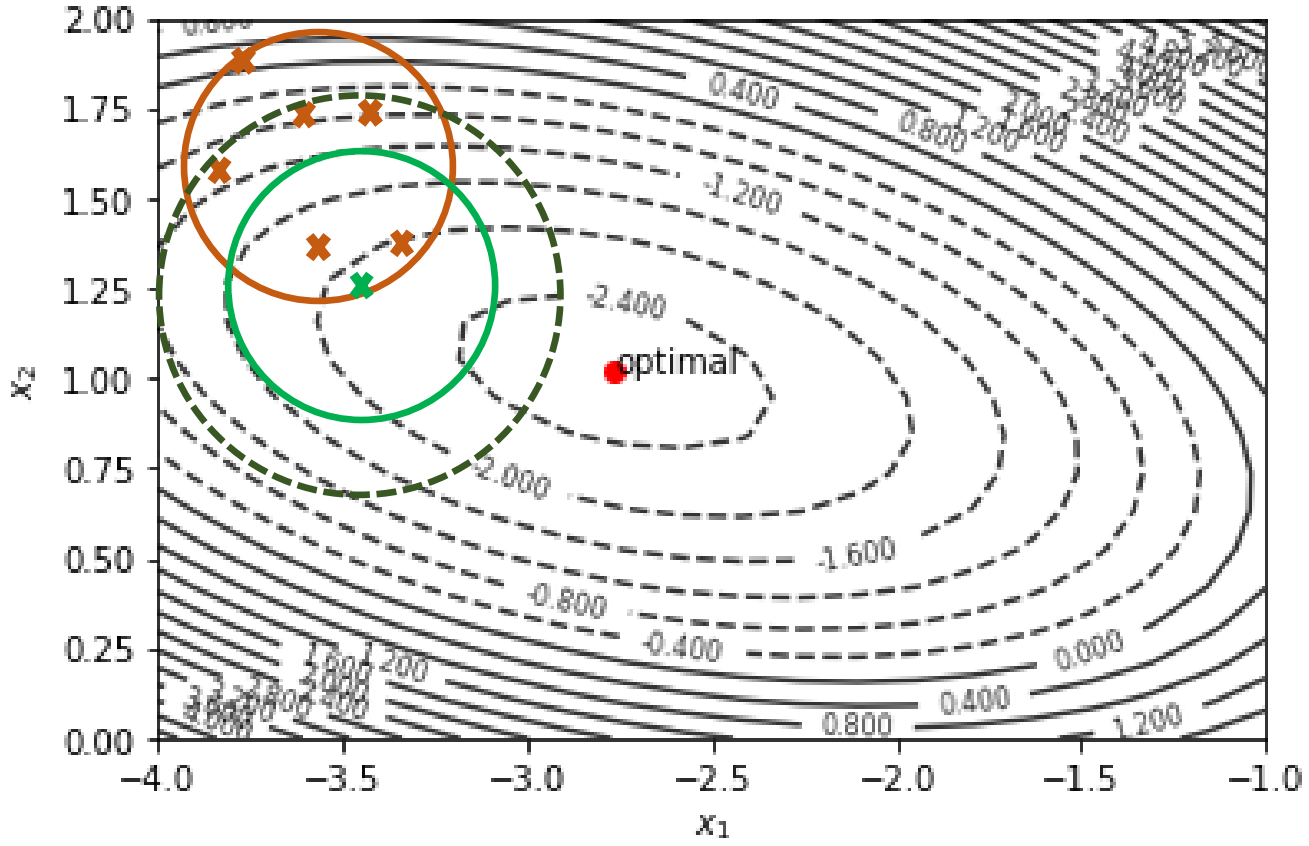
Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:

    1. **Optimise** surrogate
    2. **Move** to new point
    3. **Evaluate** how good the model is:
        If the model is good expand radius



Local search with quadratic surrogate

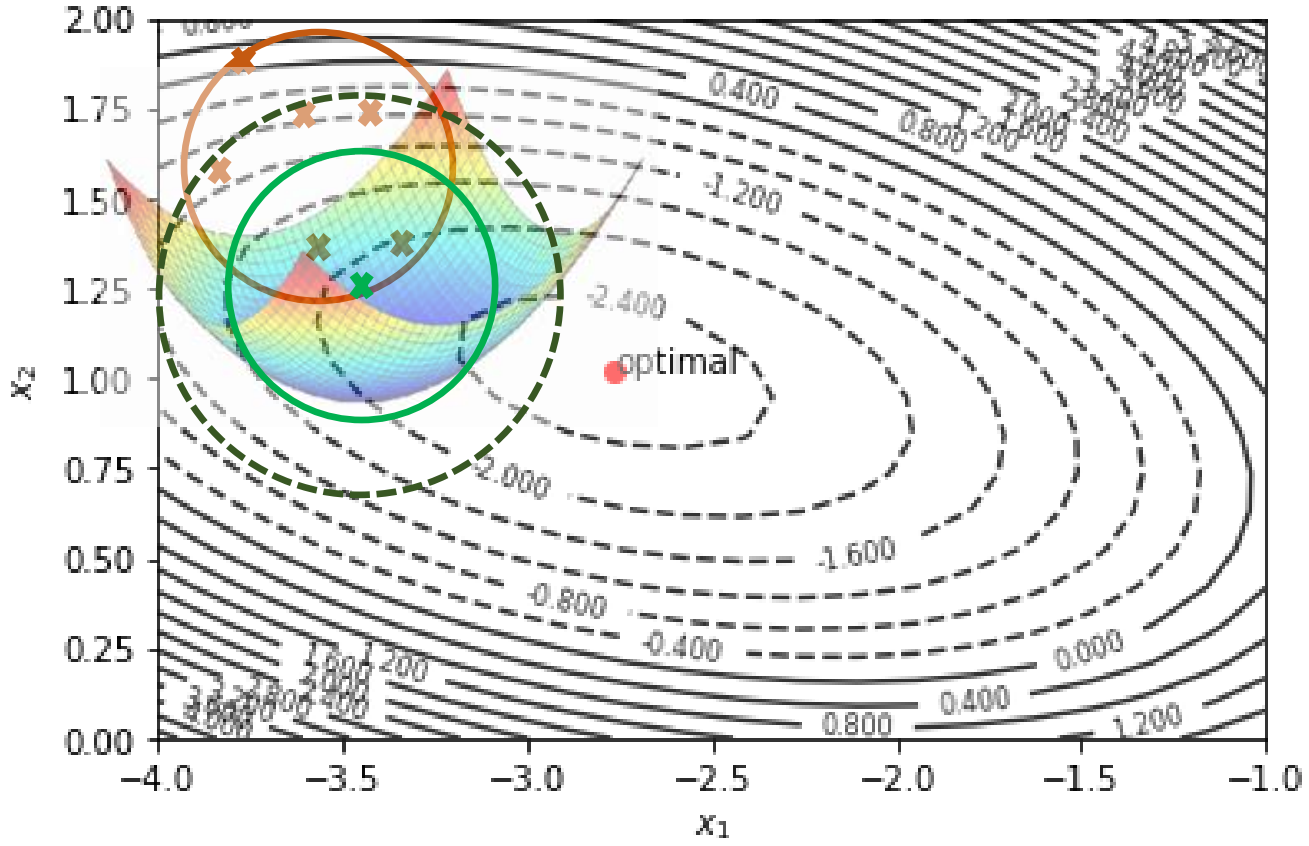
Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:

1. **Optimise** surrogate
2. **Move** to new point
3. **Evaluate** how good the model is:
   If the model is good expand radius
4. Re-train surrogate



Local search with quadratic surrogate

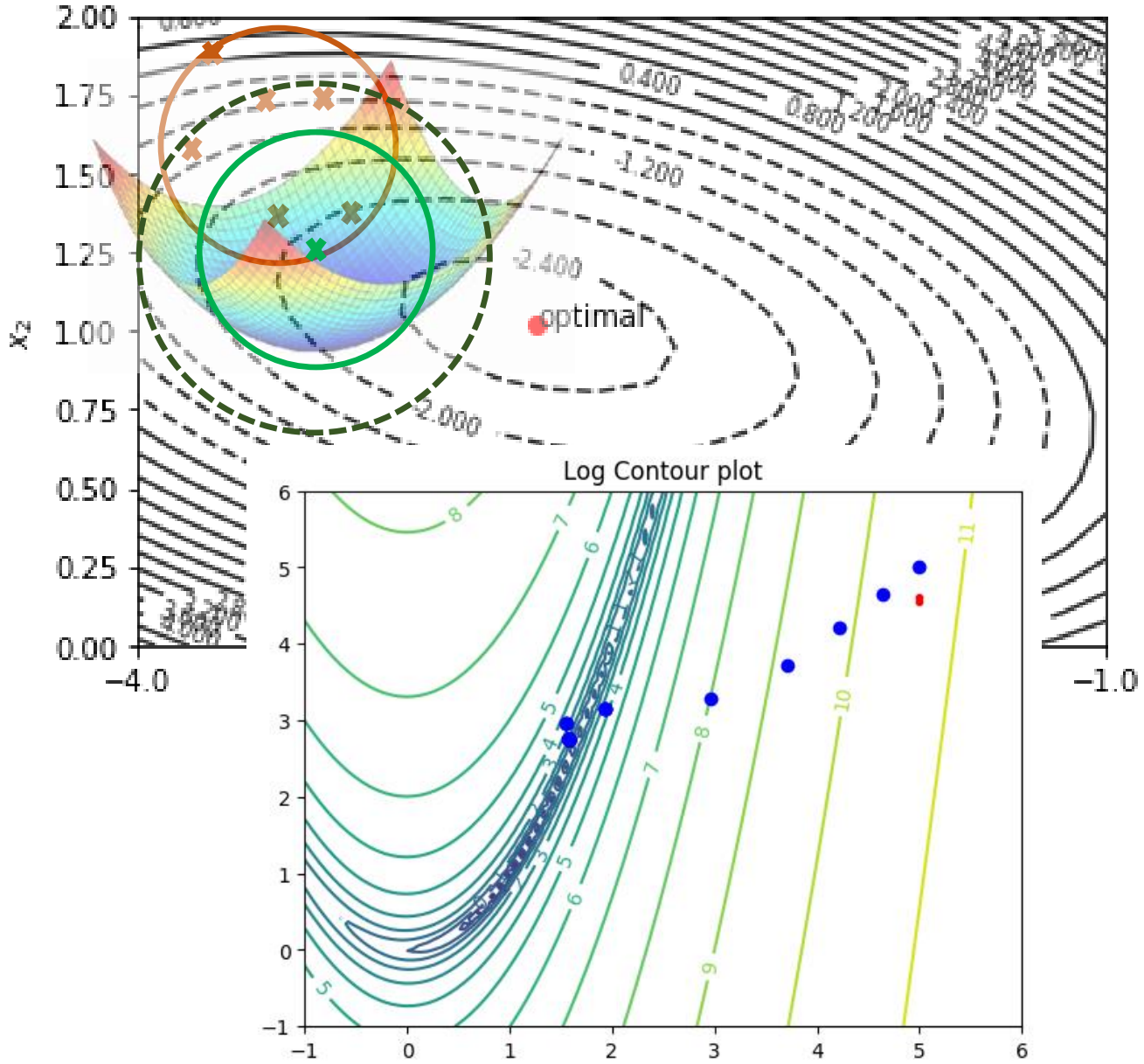
Local search with quadratic surrogate

Initialise:

1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:

1. **Optimise** surrogate
2. **Move** to new point
3. **Evaluate** how good the model is:
   If the model is good expand radius
4. Re-train surrogate

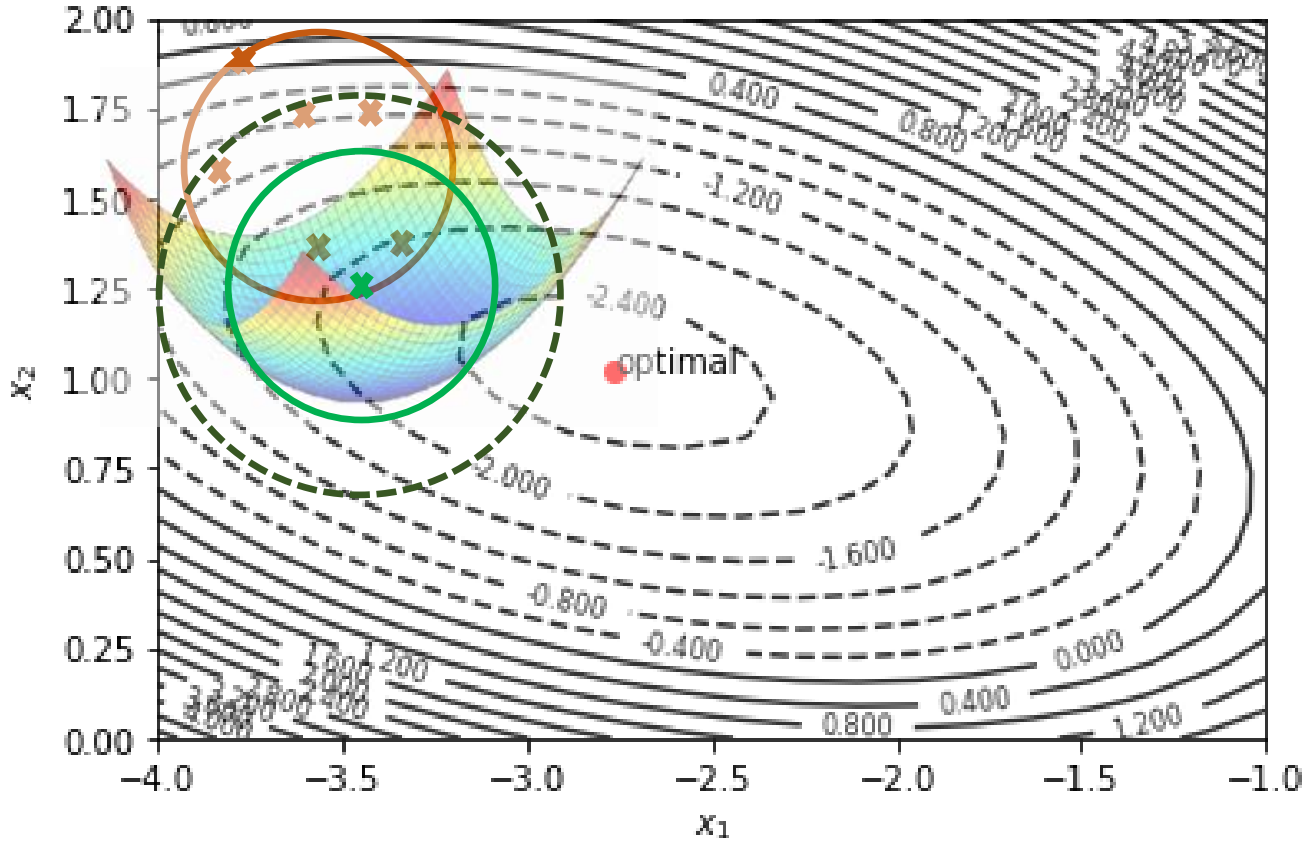


Local search with quadratic surrogate

Local search with quadratic surrogate

Initialise:
1. **Define** center point and radius
2. **Sample** inside the radius
3. **Construct** a quadratic surrogate

for *i* in **iterations**:
    1. **Optimise** surrogate
    2. **Move** to new point
    3. **Evaluate** how good the model is:
        If the model is good expand radius
    4. Re-train surrogate

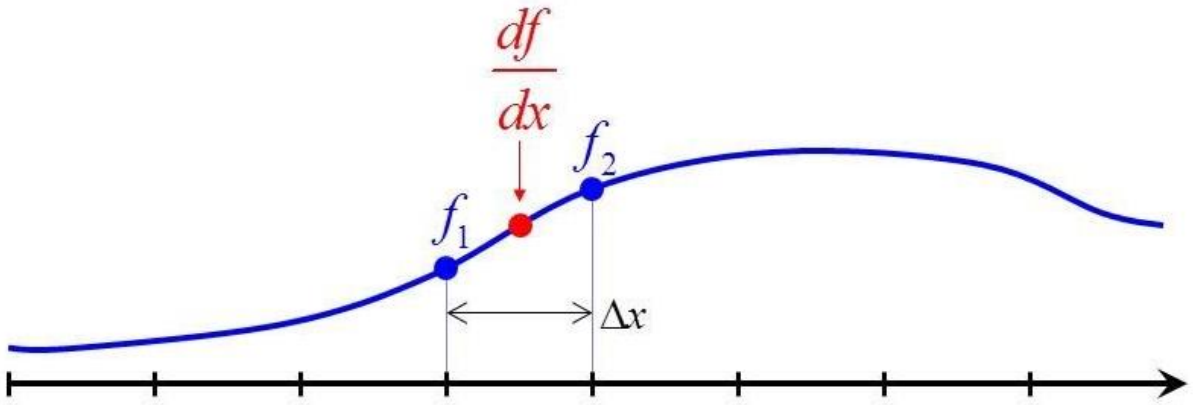


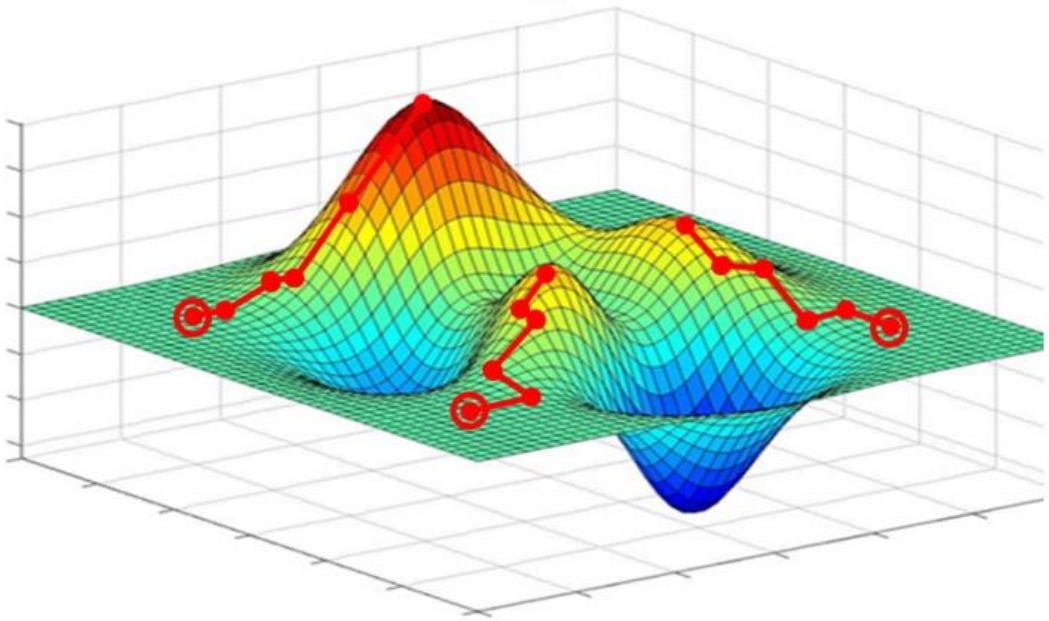
Some key points: how do we evaluate how good the model is? Do we forget points? What surrogate do we use? How do we build and optimise surrogates?

Finite difference methods

Finite difference methods

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$





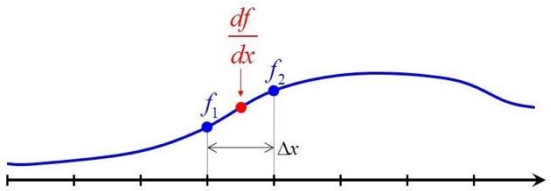
$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

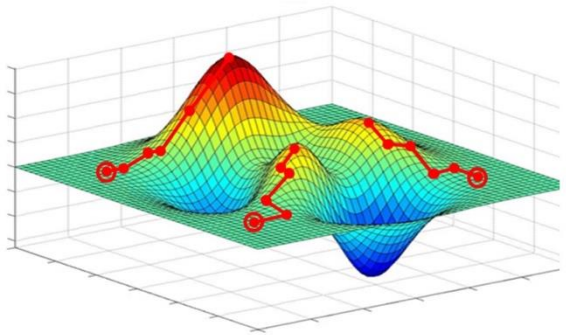
Finite difference methods

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate first-order derivative





```python
def central_finite_diff(f, x):
    '''Central finite differences approximation.

    Arguments:
        f  : Function
        x  : Position where to approximate the gradient
    Returns:
        grad: Approximation of the gradient of f at x
    '''
    dim  = x.shape[0]
    eps  = np.sqrt(np.finfo(float).eps)
    grad = np.zeros((1,dim))


    for i in range(dim):
        e           = np.zeros((1,dim))
        e[0,i]      = eps
        grad_approx = (f(x + e/2) - f(x - e/2))/eps
        grad[0,i]   = grad_approx

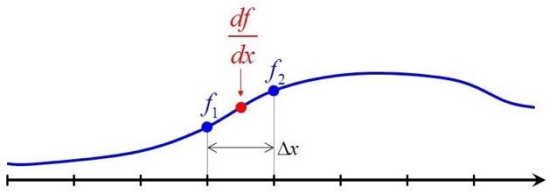
    return grad
```

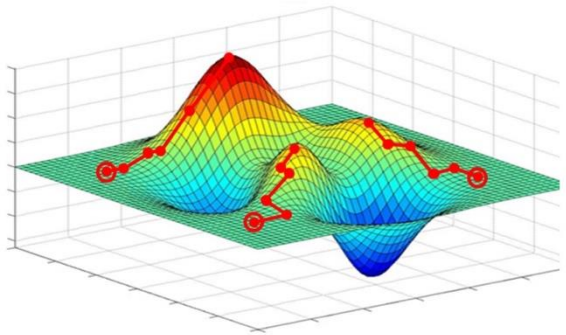
Finite difference methods

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate first-order derivative





perturbation *h*

```python
def central_finite_diff(f, x):
    '''Central finite differences approximation.

    Arguments:
        f  : Function
        x  : Position where to approximate the gradient
    Returns:
        grad: Approximation of the gradient of f at x
    '''
    dim  = x.shape[0]
    eps  = np.sqrt(np.finfo(float).eps)
    grad = np.zeros((1,dim))


    for i in range(dim):
        e           = np.zeros((1,dim))
        e[0,i]      = eps
        grad_approx = (f(x + e/2) - f(x - e/2))/eps
        grad[0,i]   = grad_approx

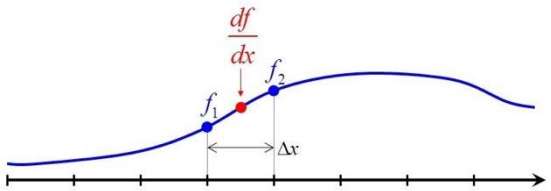
    return grad
```

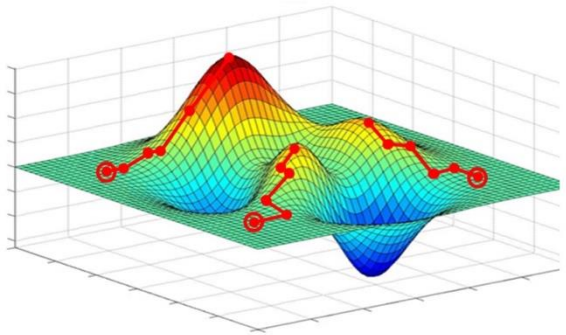
Finite difference methods

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate first-order derivative





perturbation *h*

```python
def central_finite_diff(f, x):
    '''Central finite differences approximation.

    Arguments:
        f  : Function
        x  : Position where to approximate the gradient
    Returns:
        grad: Approximation of the gradient of f at x
    '''
    dim  = x.shape[0]
    eps  = np.sqrt(np.finfo(float).eps)
    grad = np.zeros((1,dim))

    for i in range(dim):
        e           = np.zeros((1,dim))
        e[0,i]      = eps
        grad_approx = (f(x + e/2) - f(x - e/2))/eps
        grad[0,i]   = grad_approx

    return grad
```

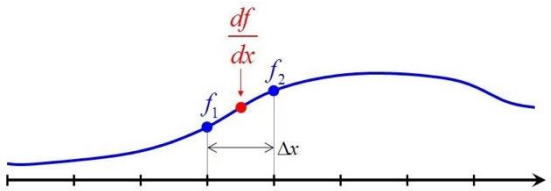
Initializing gradient

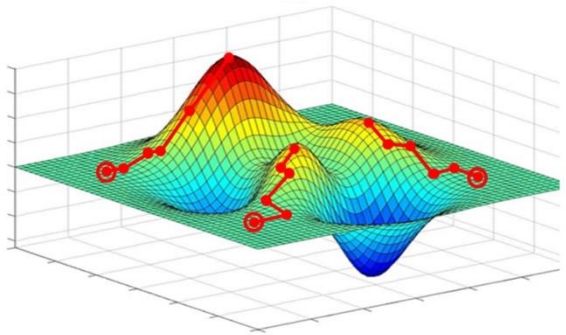
Finite difference methods

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate first-order derivative





perturbation *h*

Perturbing each dimension

Perturbation only in dimension *i*

```
1  def central_finite_diff(f, x):
2      '''Central finite differences approximation.
3
4      Arguments:
5          f  : Function
6          x  : Position where to approximate the gradient
7      Returns:
8          grad: Approximation of the gradient of f at x
9      '''
10     dim  = x.shape[0]
11     eps  = np.sqrt(np.finfo(float).eps)
12     grad = np.zeros((1,dim))
13
14
15     for i in range(dim):
16         e           = np.zeros((1,dim))
17         e[0,i]      = eps
18         grad_approx = (f(x + e/2) - f(x - e/2))/eps
19         grad[0,i]   = grad_approx
20
21     return grad
```

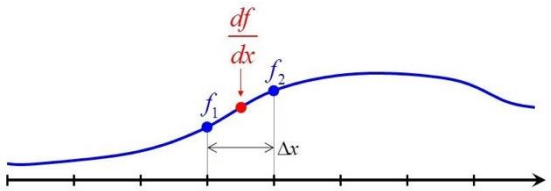
Initializing gradient

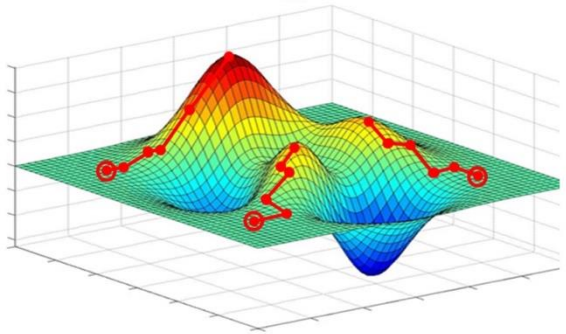
Finite difference methods

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate first-order derivative





perturbation *h*

Perturbing each dimension

Perturbation only in dimension *i*

```python
def central_finite_diff(f, x):
    '''Central finite differences approximation.

    Arguments:
        f  : Function
        x  : Position where to approximate the gradient
    Returns:
        grad: Approximation of the gradient of f at x
    '''
    dim  = x.shape[0]
    eps  = np.sqrt(np.finfo(float).eps)
    grad = np.zeros((1,dim))

    for i in range(dim):
        e           = np.zeros((1,dim))
        e[0,i]      = eps
        grad_approx = (f(x + e/2) - f(x - e/2))/eps
        grad[0,i]   = grad_approx

    return grad
```

Initializing gradient

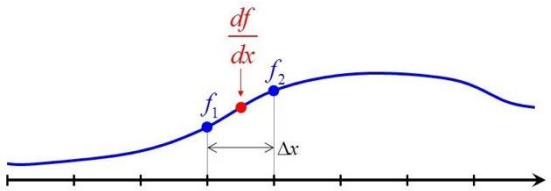
$$\frac{\partial f}{\partial x_i}$$

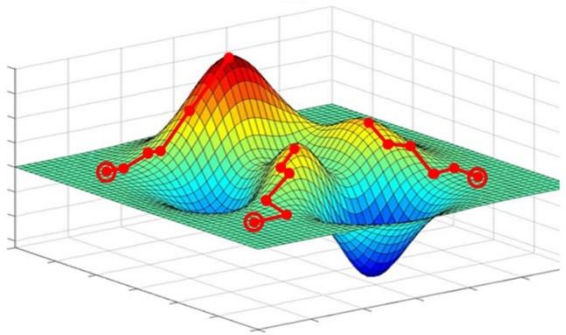
Finite difference methods

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate first-order derivative





perturbation *h*

Perturbing each dimension

Perturbation only in dimension *i*

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_{n_x}}\right]^T$$

```
1  def central_finite_diff(f, x):
2      '''Central finite differences approximation.
3
4      Arguments:
5          f  : Function
6          x  : Position where to approximate the gradient
7      Returns:
8          grad: Approximation of the gradient of f at x
9      '''
10     dim  = x.shape[0]
11     eps  = np.sqrt(np.finfo(float).eps)
12     grad = np.zeros((1,dim))
13
14
15     for i in range(dim):
16         e           = np.zeros((1,dim))
17         e[0,i]      = eps
18         grad_approx = (f(x + e/2) - f(x - e/2))/eps
19         grad[0,i]   = grad_approx
20
21     return grad
```

Initializing gradient

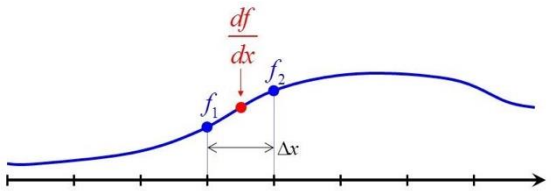
$\frac{\partial f}{\partial x_i}$

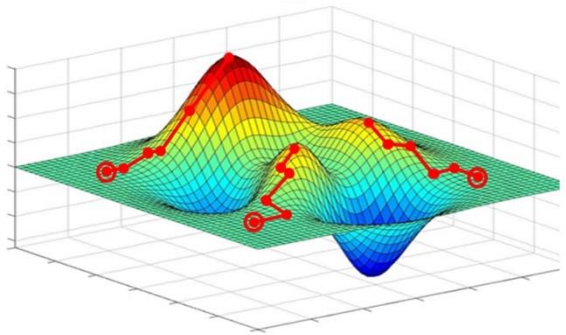
Finite difference methods

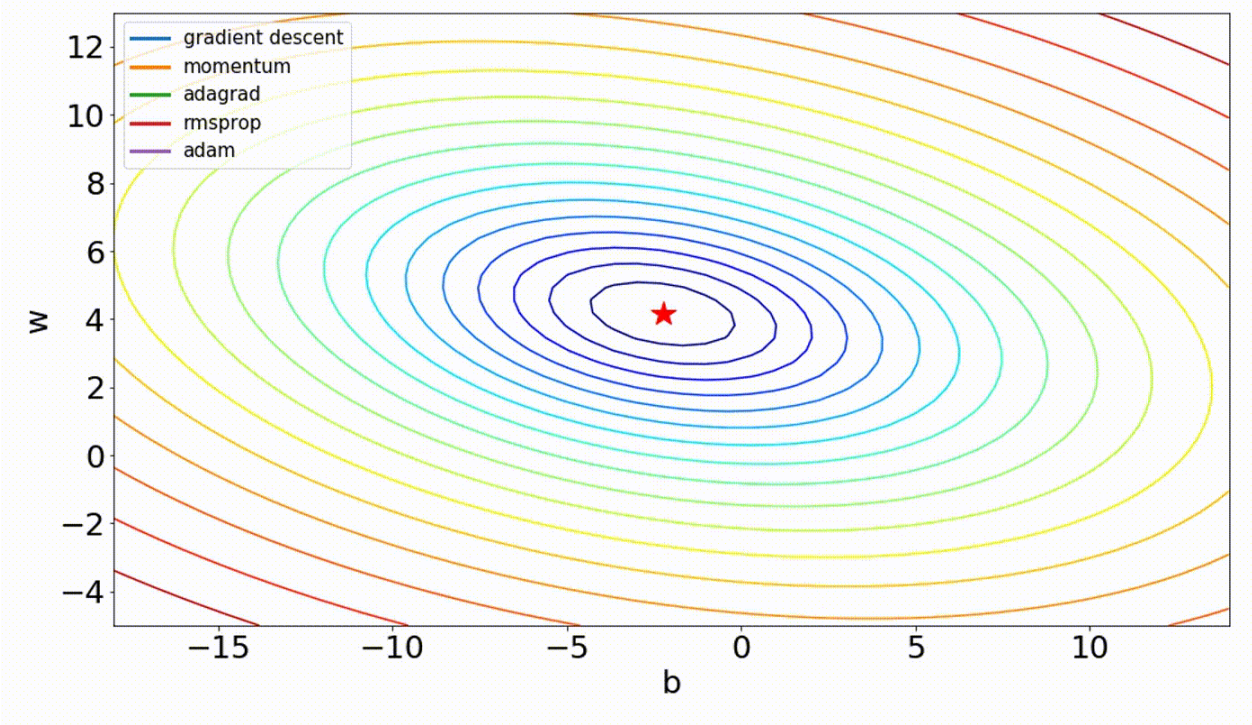
$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate first-order derivative







Algorithms:

- Gradient Descent
- +Momentum
- Adam
- Newton
- Quasi-Newton
  - (L)BFGS

Finite difference methods

```python
def gradient_descent(f, x0, grad_f, lr, max_iter=1e5, grad_tol=1e-4):
    '''Gradient Descent

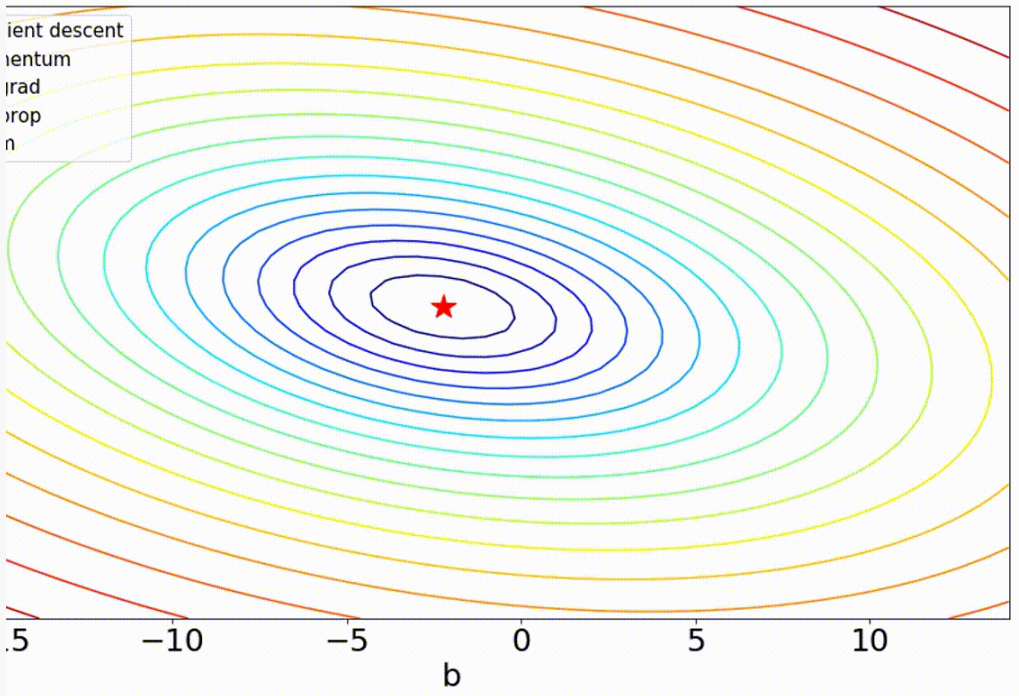
    Arguments:
        f        : Function
        x0       : Initial guess
        grad_f   : Gradient function
        lr       : Learning rate
        max_iter : Maximum number of iterations
        grad_tol : Tolerance for gradient approximation
    Returns:
        x        : Optimal point
    '''

    # initialize problem
    x      = np.copy(x0)
    grad_i = grad_tol + 1

    # optimization loop
    while np.sum(np.abs(grad_i)) > grad_tol:

        # gradient calculation
        grad_i  = grad_f(f,x)
        # gradient step
        x       = x - lr*grad_i

    return x
```



Algorithms:

- Gradient Descent
- +Momentum
- Adam
- Newton
- Quasi-Newton
  - (L)BFGS

Finite difference methods

```python
def gradient_descent(f, x0, grad_f, lr, max_iter=1e5, grad_tol=1e-4):
    '''Gradient Descent

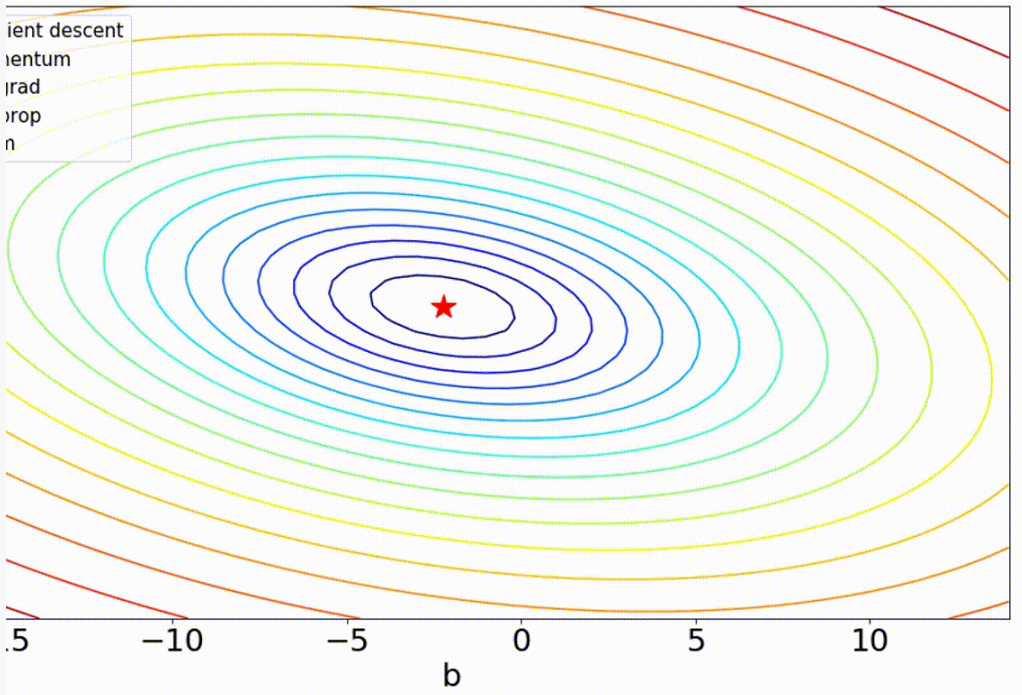
    Arguments:
        f        : Function
        x0       : Initial guess
        grad_f   : Gradient function
        lr       : Learning rate
        max_iter : Maximum number of iterations
        grad_tol : Tolerance for gradient approximation
    Returns:
        x        : Optimal point
    '''

    # initialize problem
    x      = np.copy(x0)
    grad_i = grad_tol + 1

    # optimization loop
    while np.sum(np.abs(grad_i)) > grad_tol:

        # gradient calculation
        grad_i  = grad_f(f,x)
        # gradient step
        x       = x - lr*grad_i

    return x
```



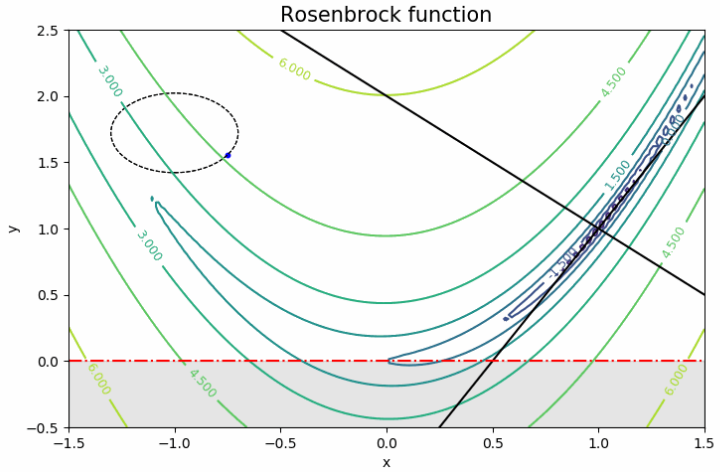
Algorithms:

- Gradient Descent
- +Momentum
- Adam
- Newton
- Quasi-Newton
  - (L)BFGS

More details in optimization workbook

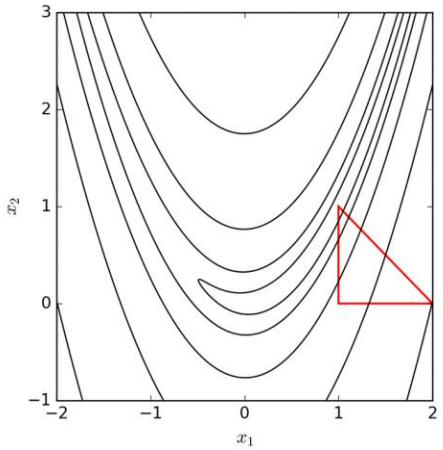
Types of derivative free optimization **algorithms**

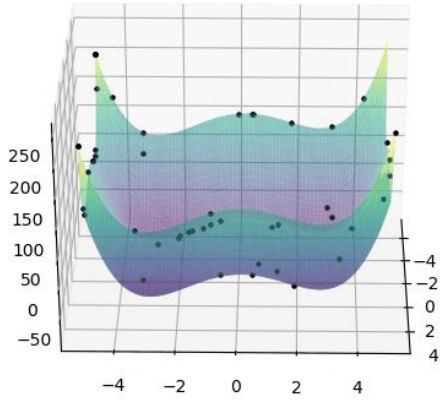
Model-based derivative free methods
(**Surrogate-based** optimization)



*https://github.com/edgarsmdn

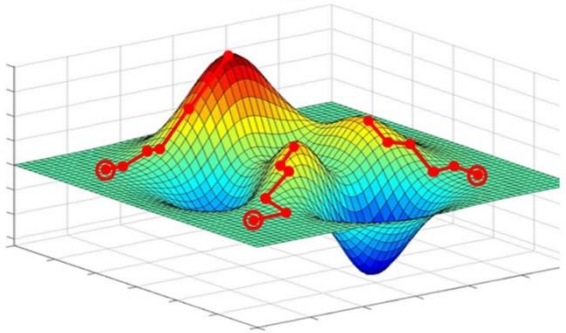
Direct derivative free methods





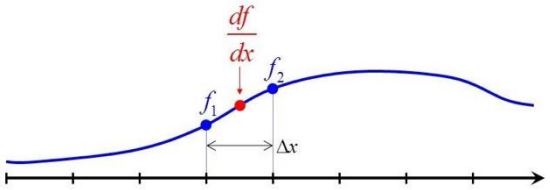
* https://tom-savage.co.uk/

Finite difference methods
(sub-class of Direct(?))



$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate
first-order derivative

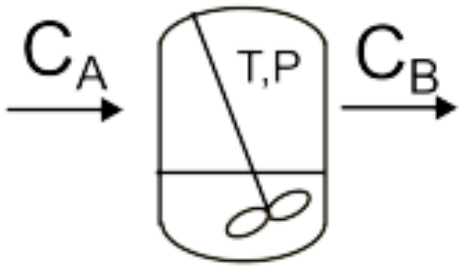


A closer look at some algorithms

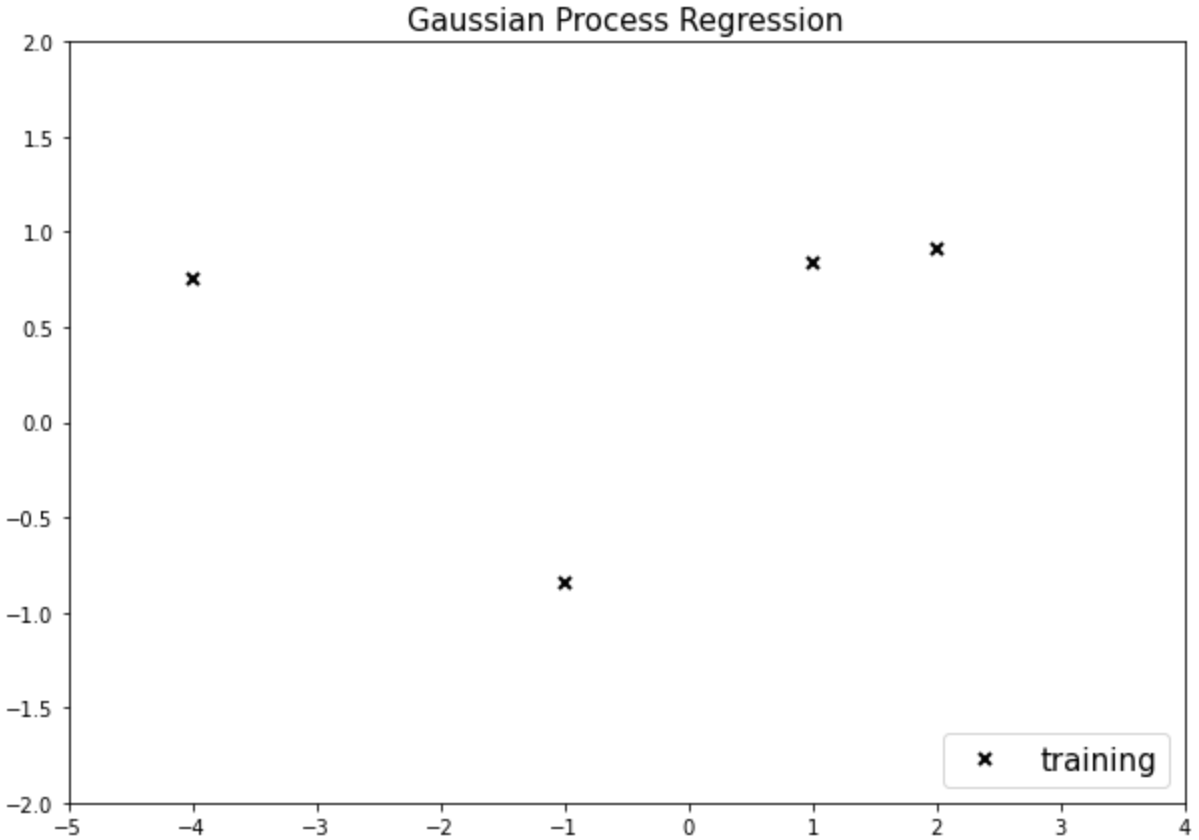
Model Based:
Bayesian Optimisation

Gaussian processes

Bayesian Optimization – Gaussian processes

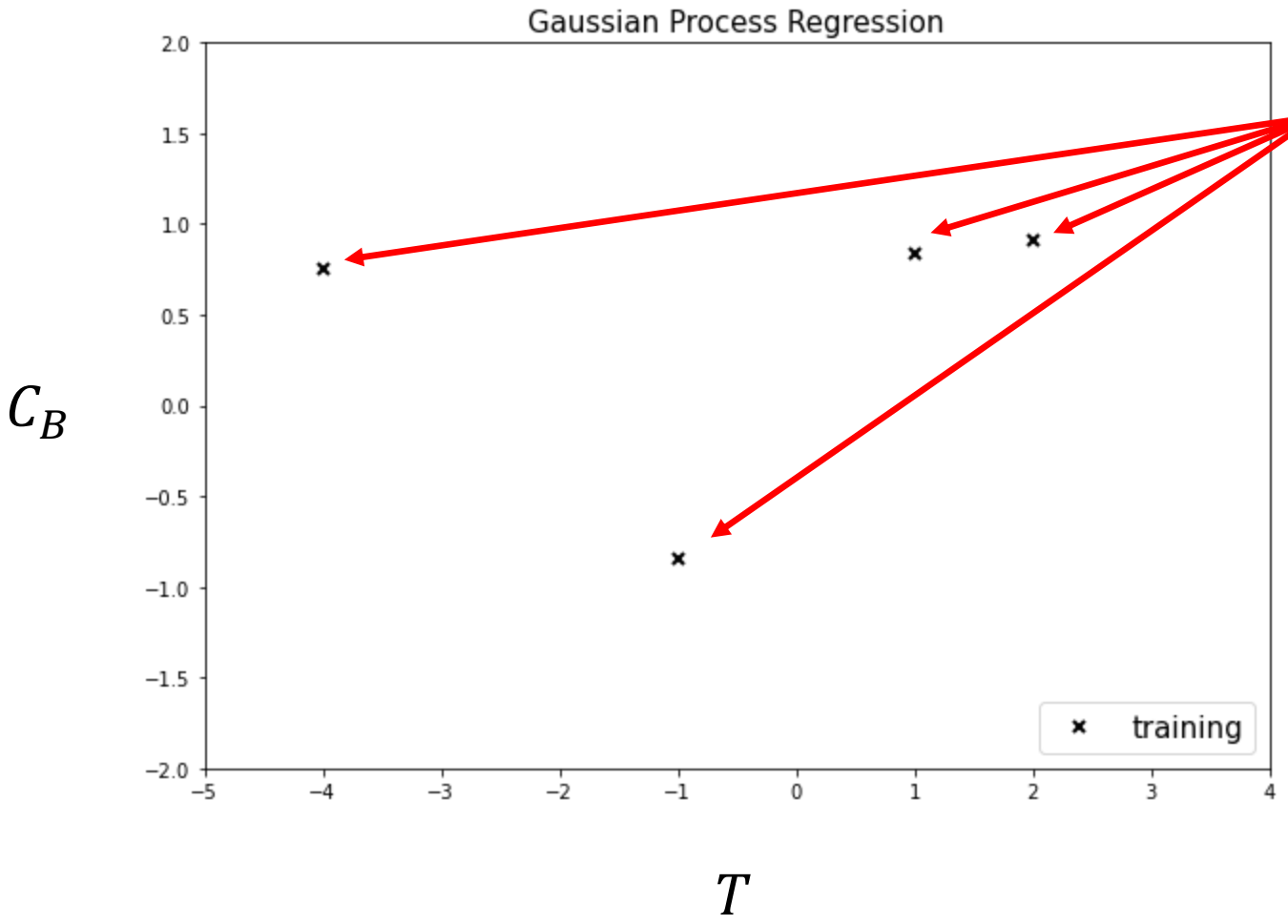


C_B



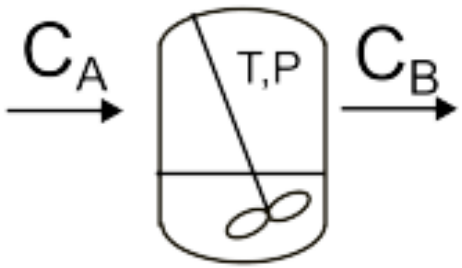
T

Bayesian Optimization – Gaussian processes

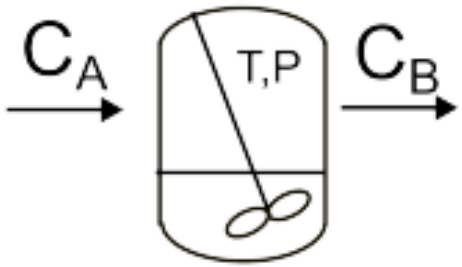


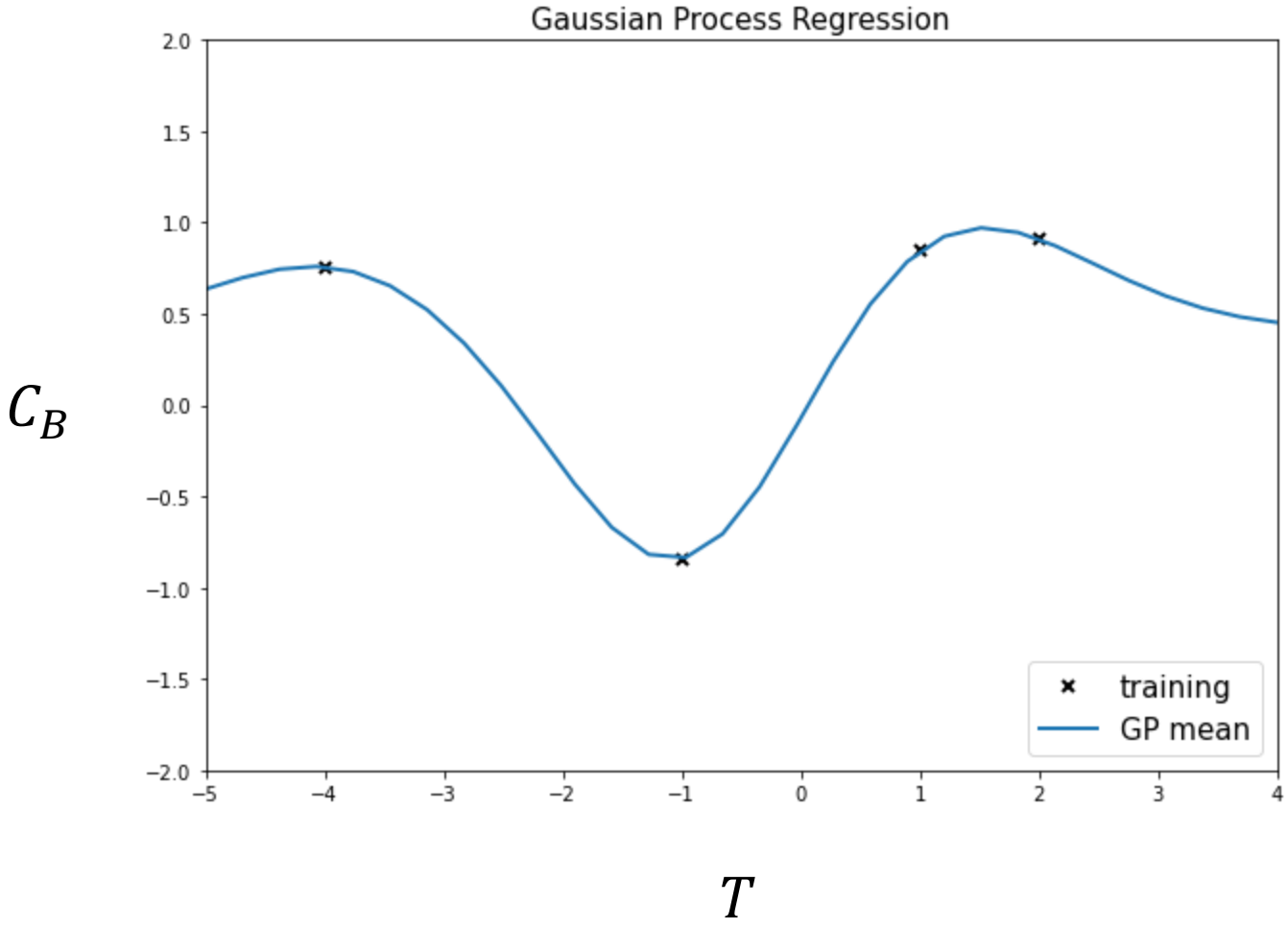
Datapoints

$x, y = T, C_B$

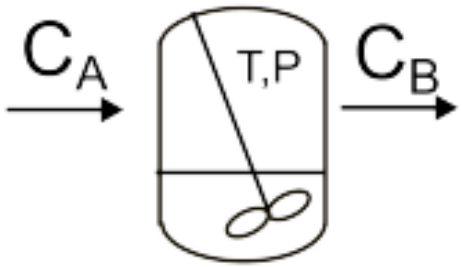


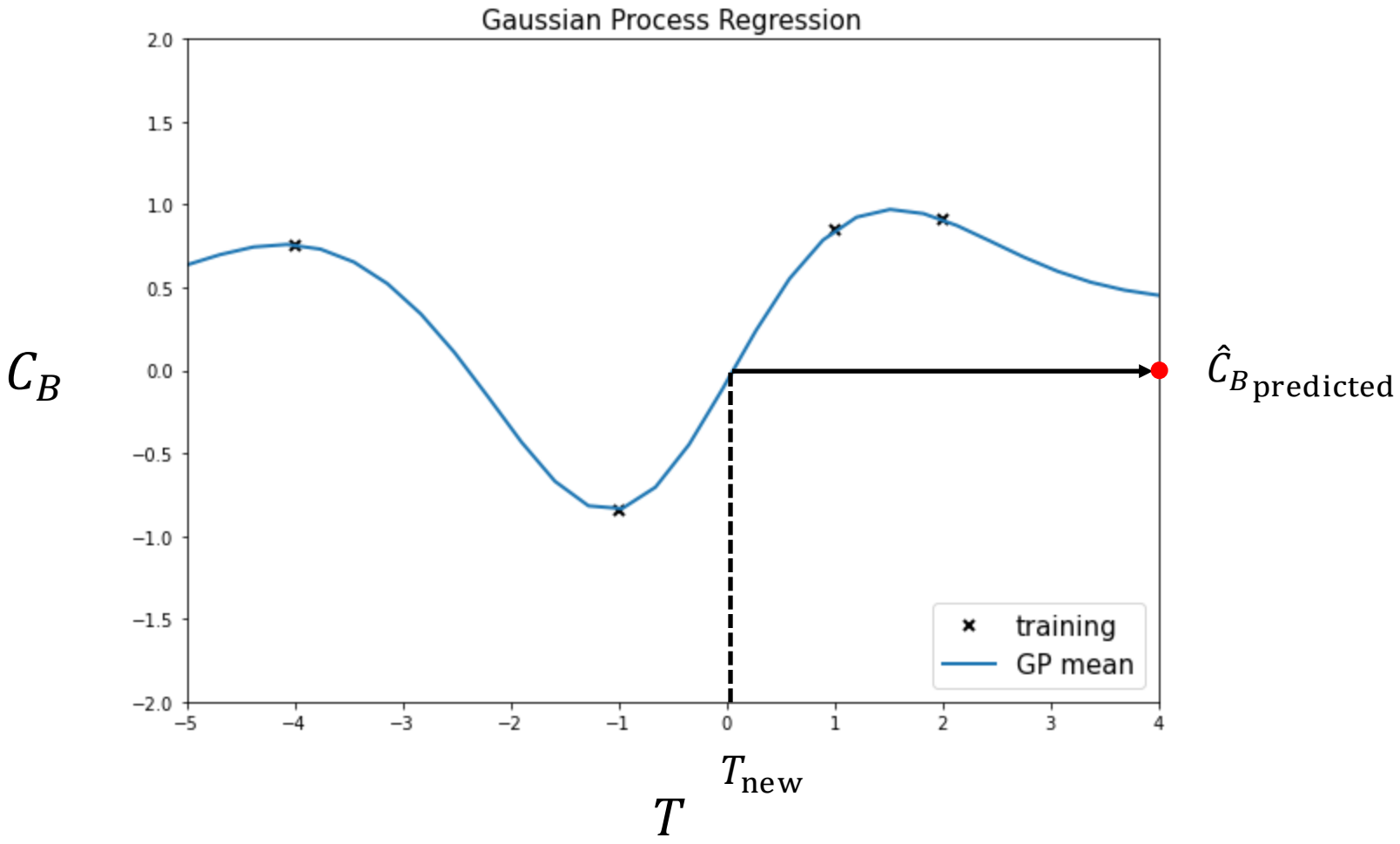
Bayesian Optimization – Gaussian processes



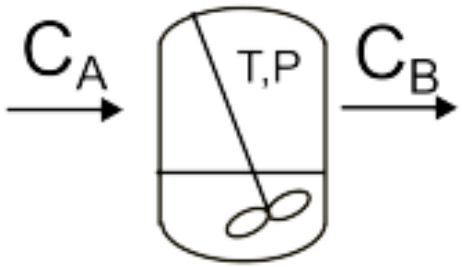


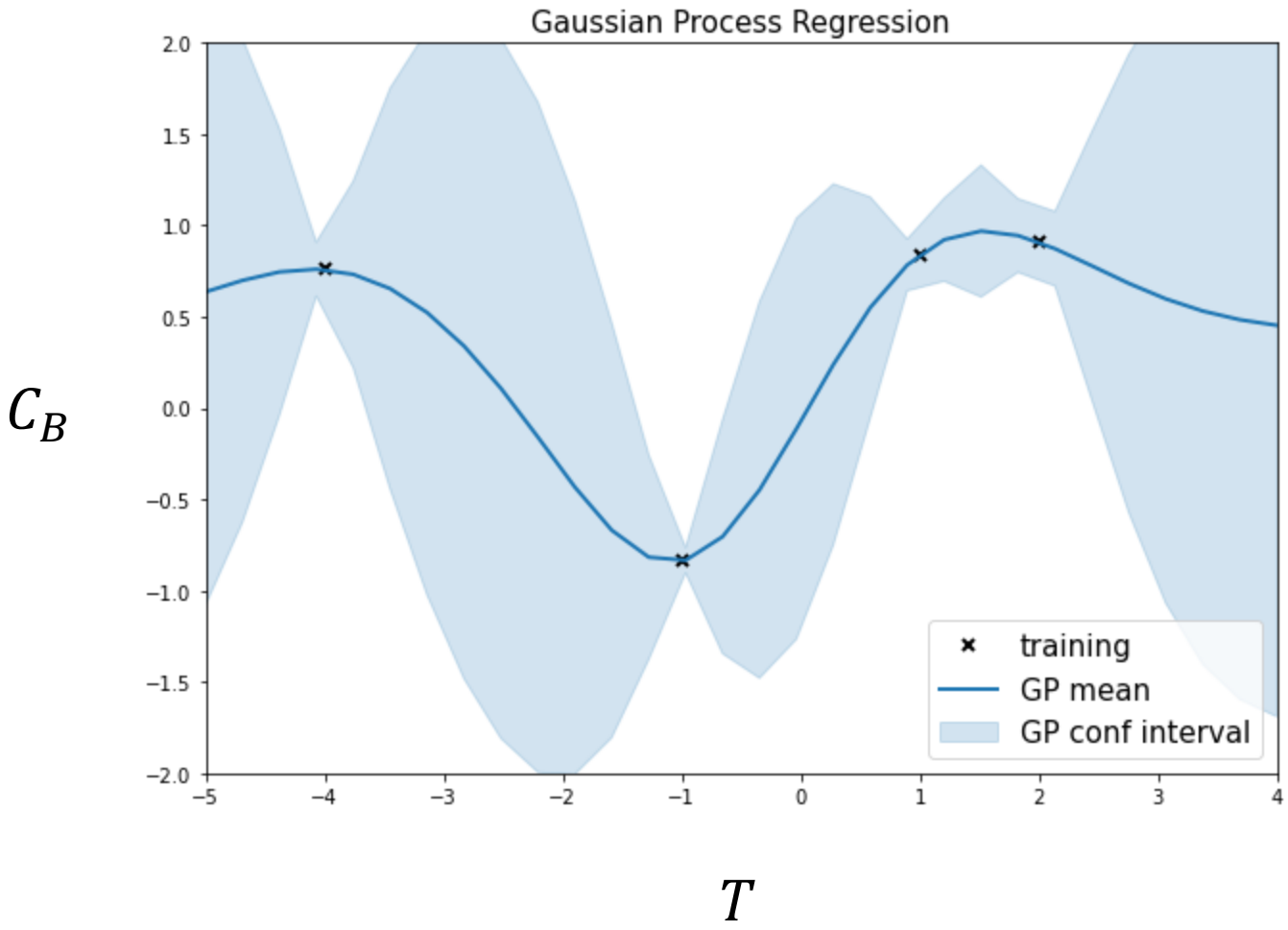
Bayesian Optimization – Gaussian processes





Bayesian Optimization – Gaussian processes

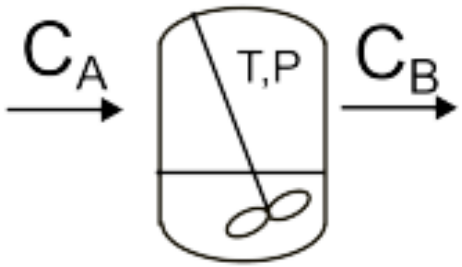


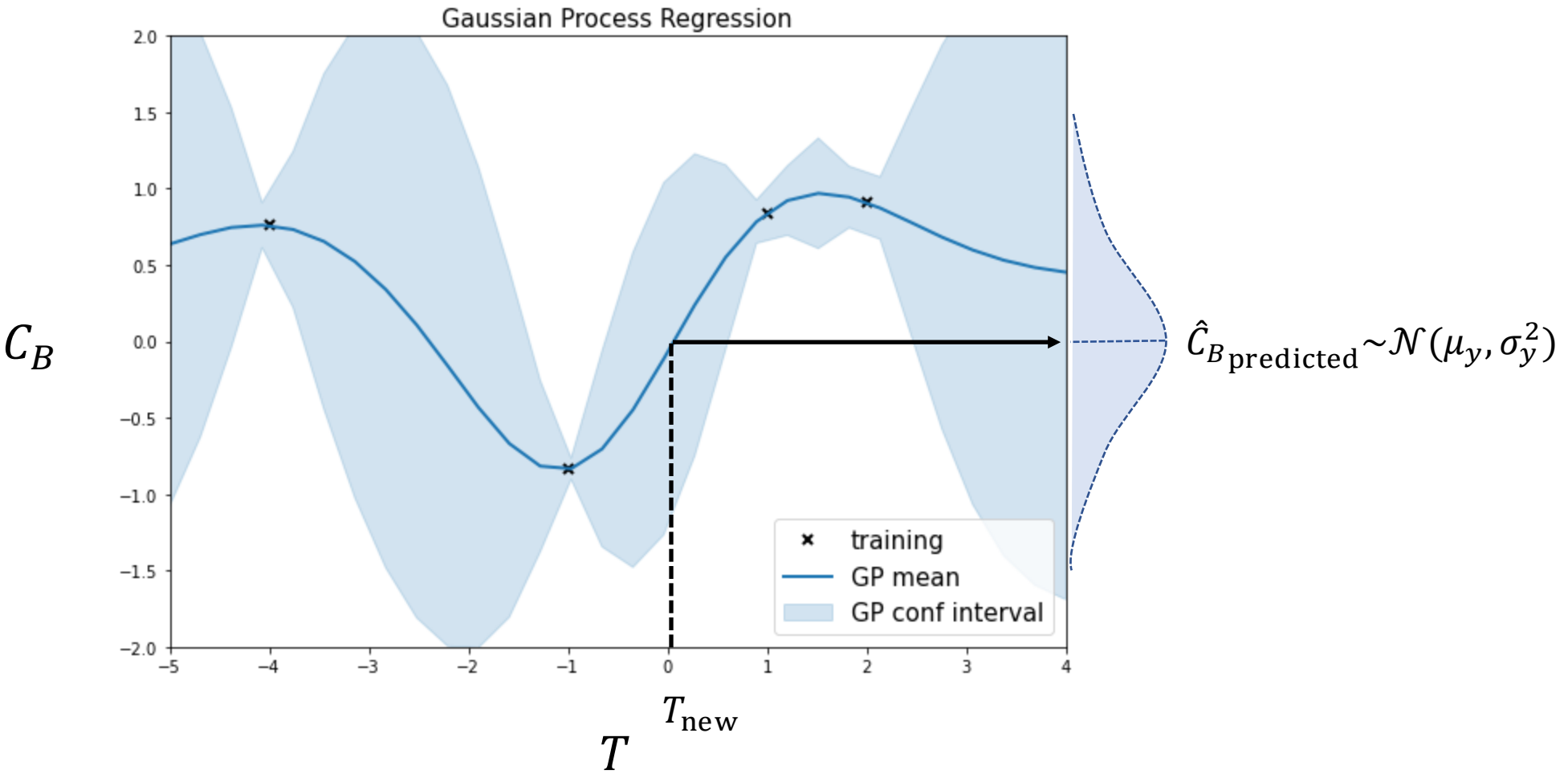


C_B

T

Bayesian Optimization – Gaussian processes



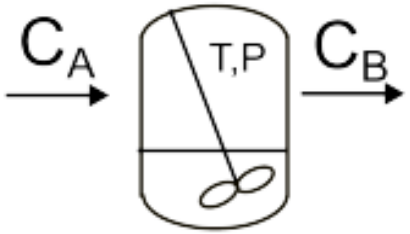


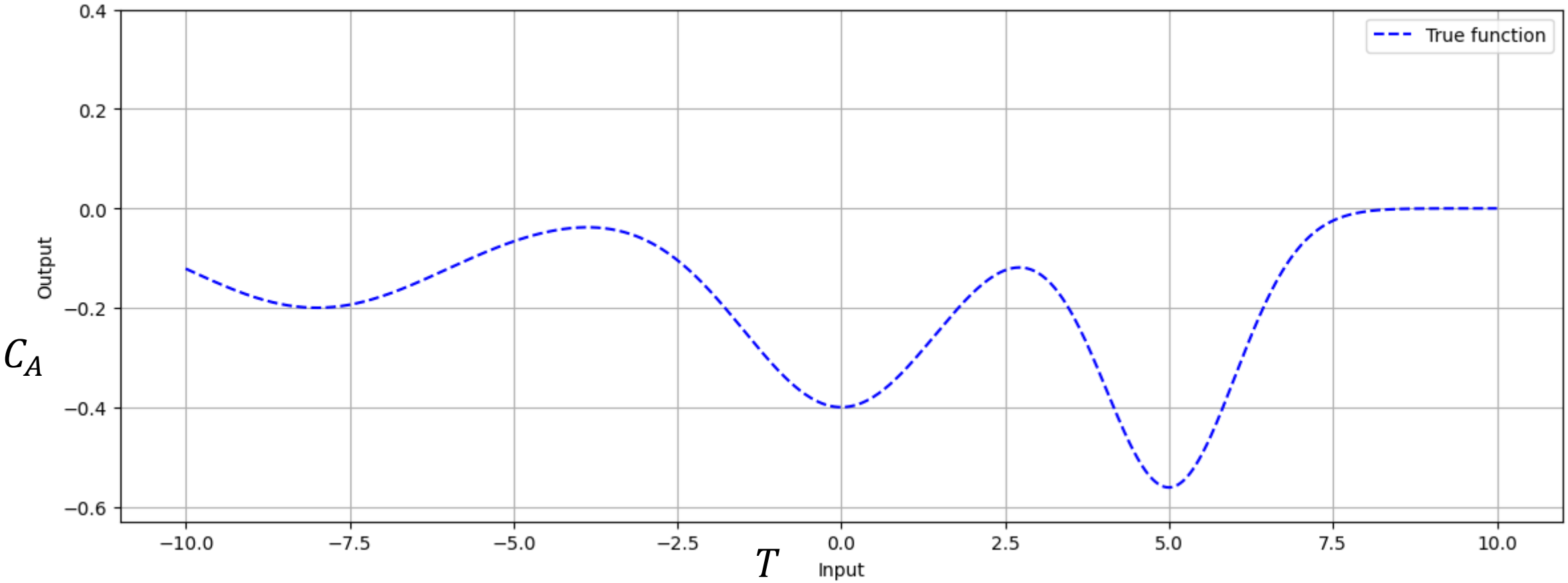
Bayesian Optimisation

Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

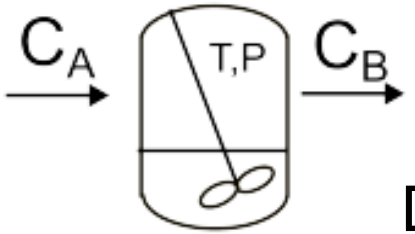




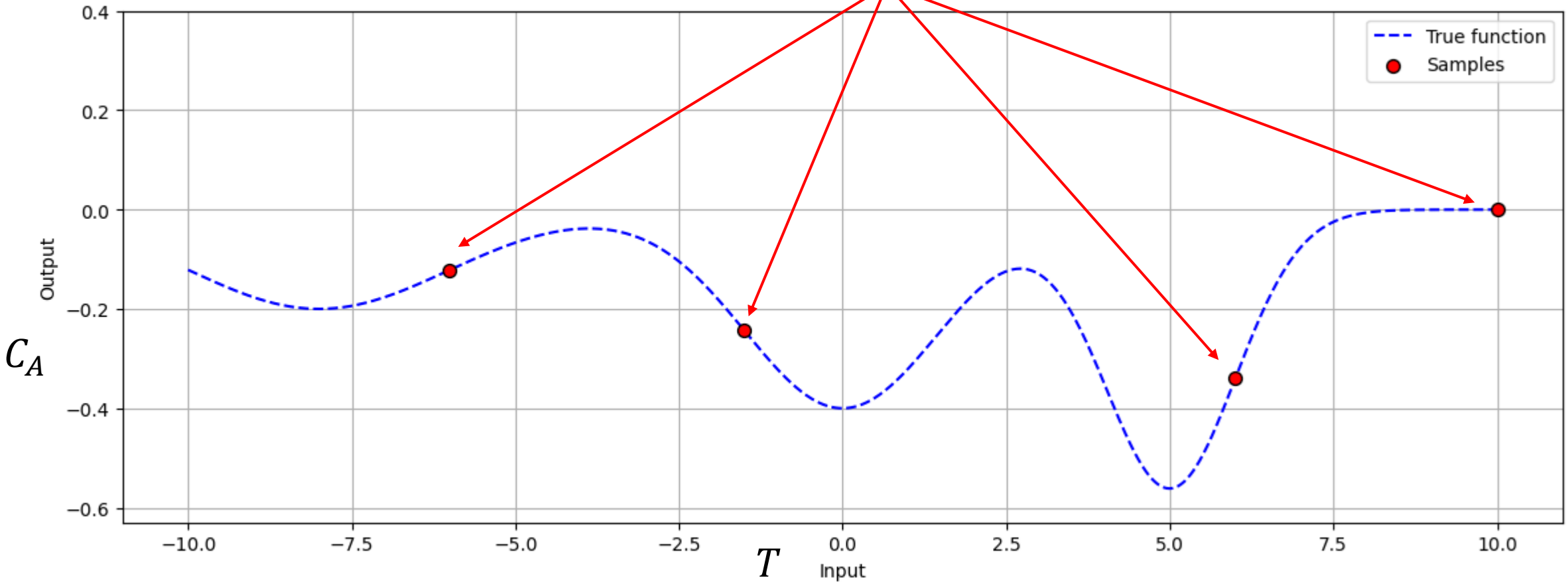
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$



Data points



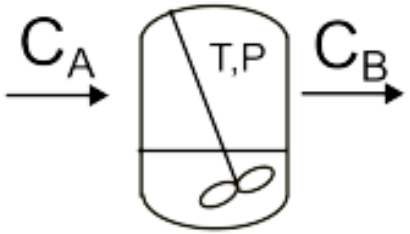
C_A

T

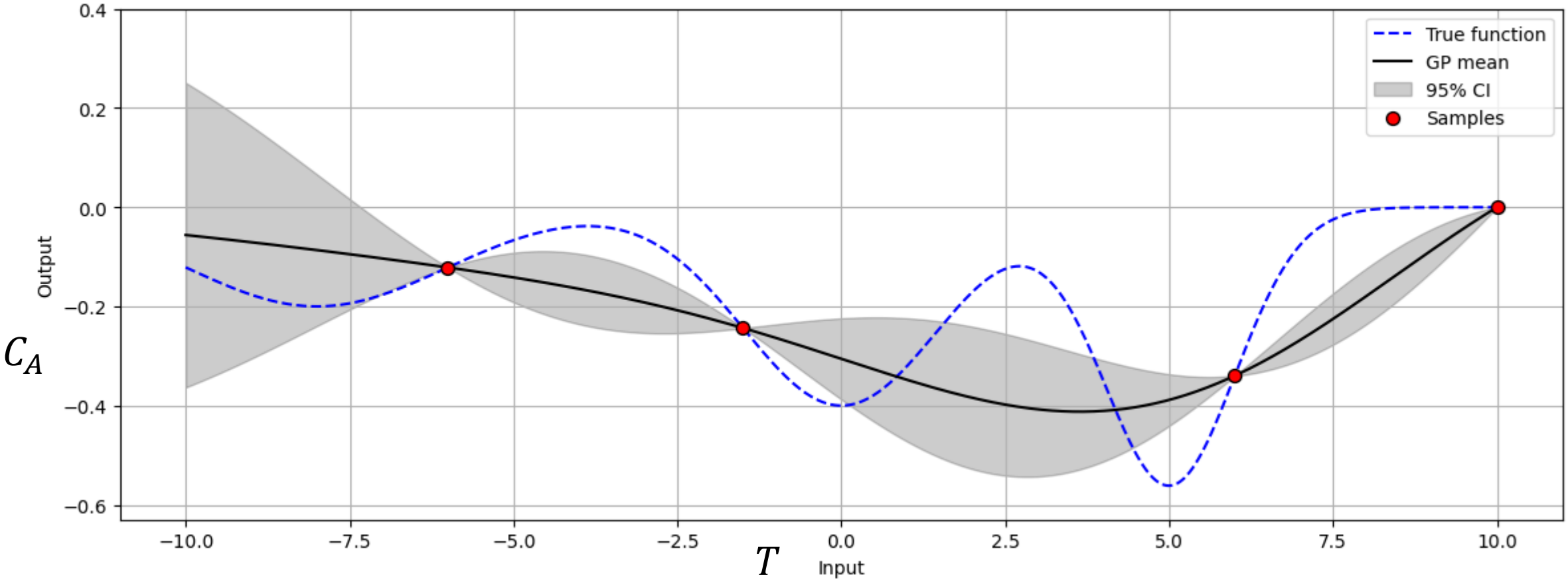
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$



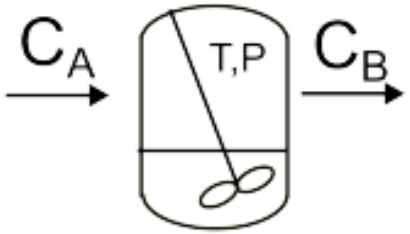
Key idea: model the objective function with a Gaussian process



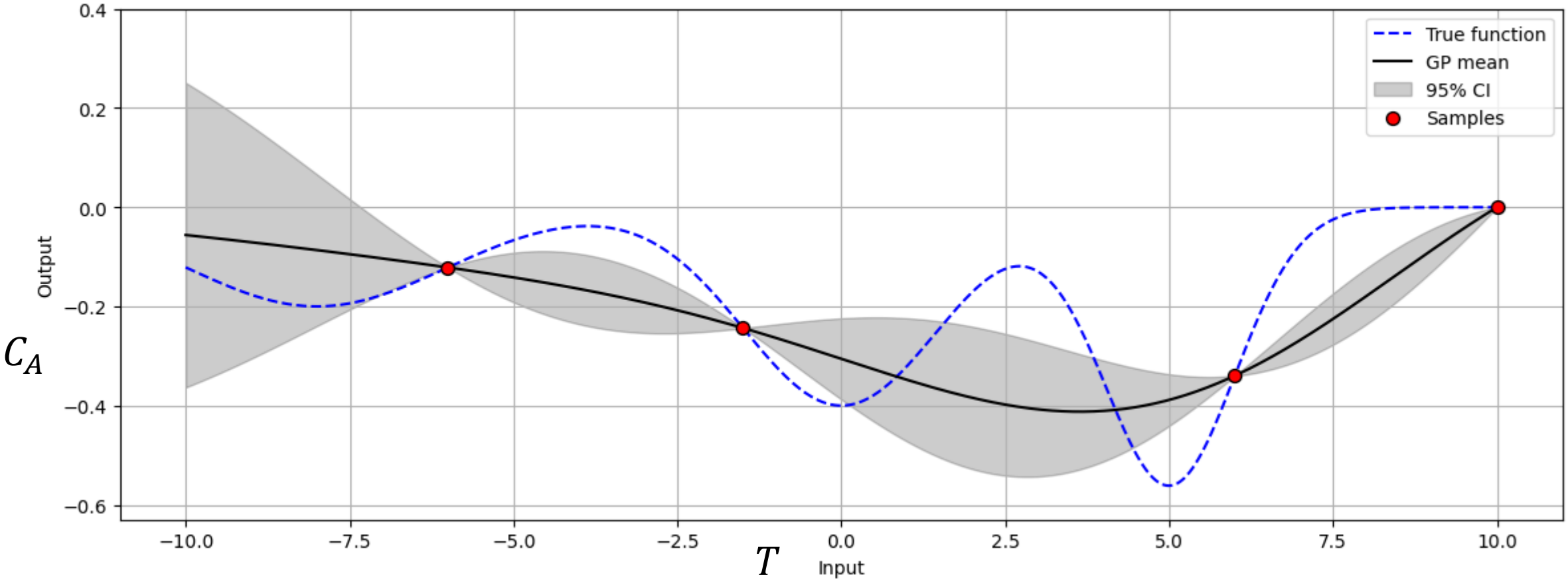
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$



$$\hat{f}(\mathbf{x}) \sim \mathcal{GP}\left(\mu_{\hat{f}}(\mathbf{x}), \sigma_{\hat{f}}(\mathbf{x})\right)$$



C_A

T

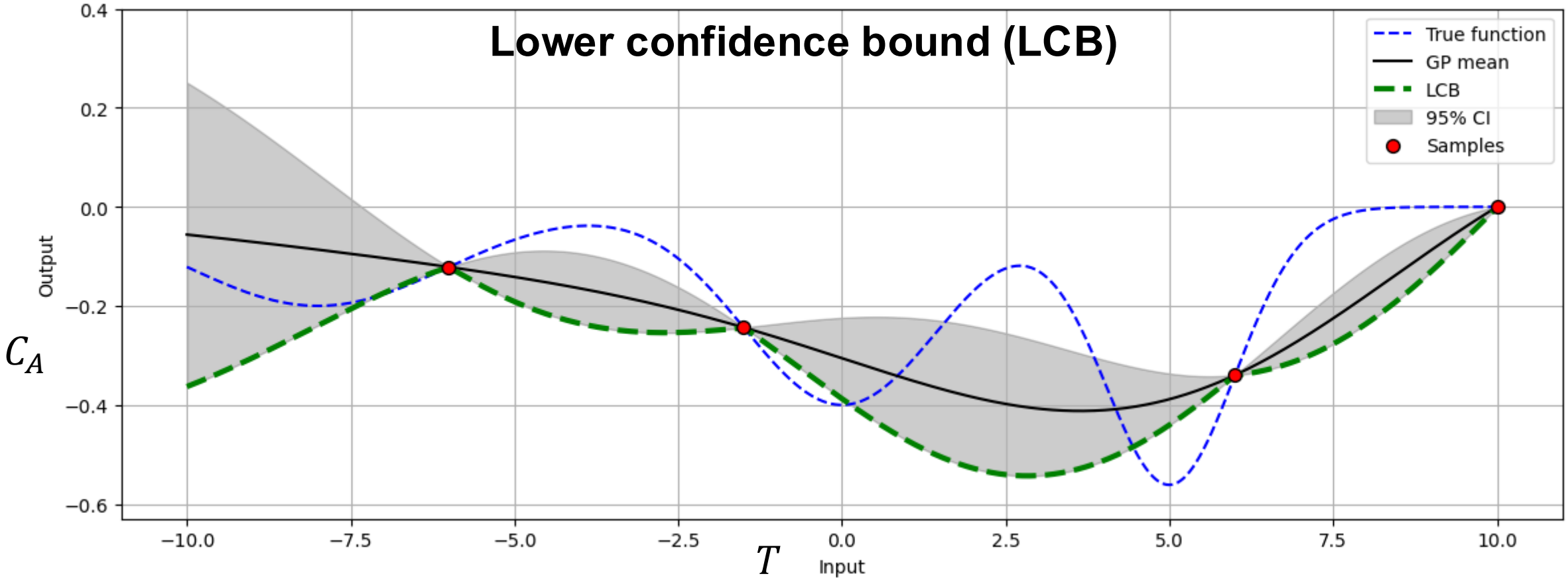
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



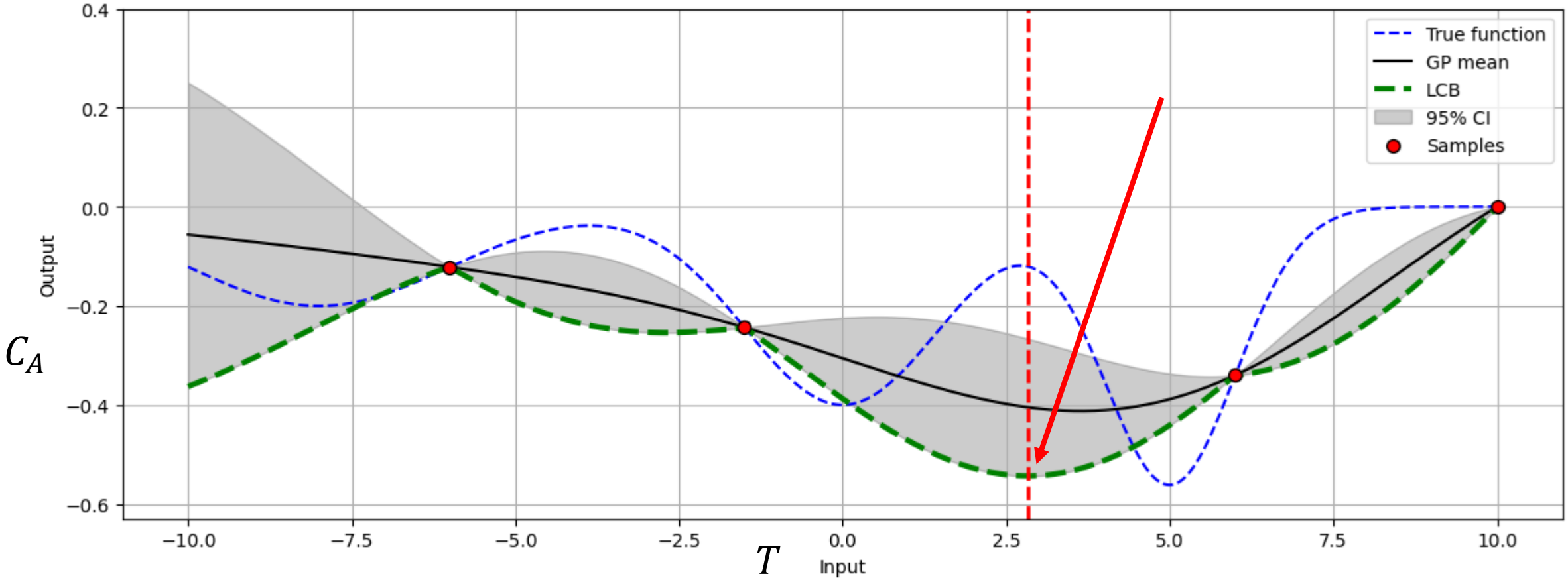
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



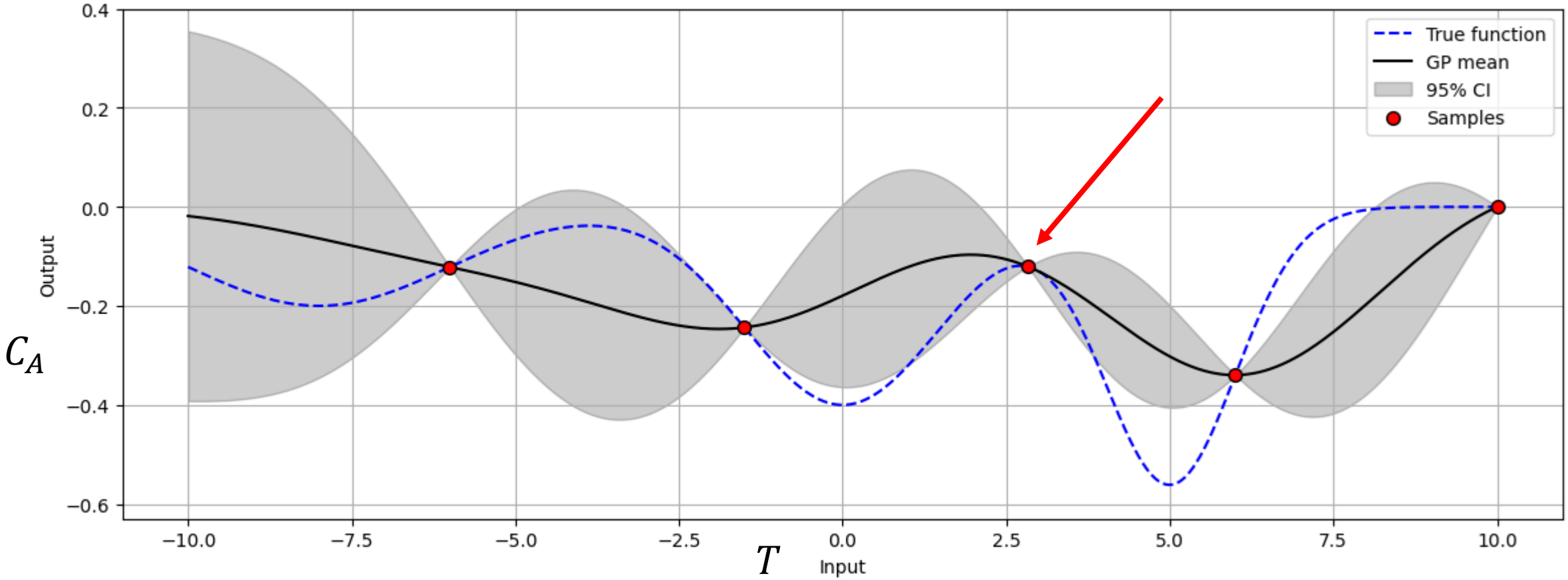
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



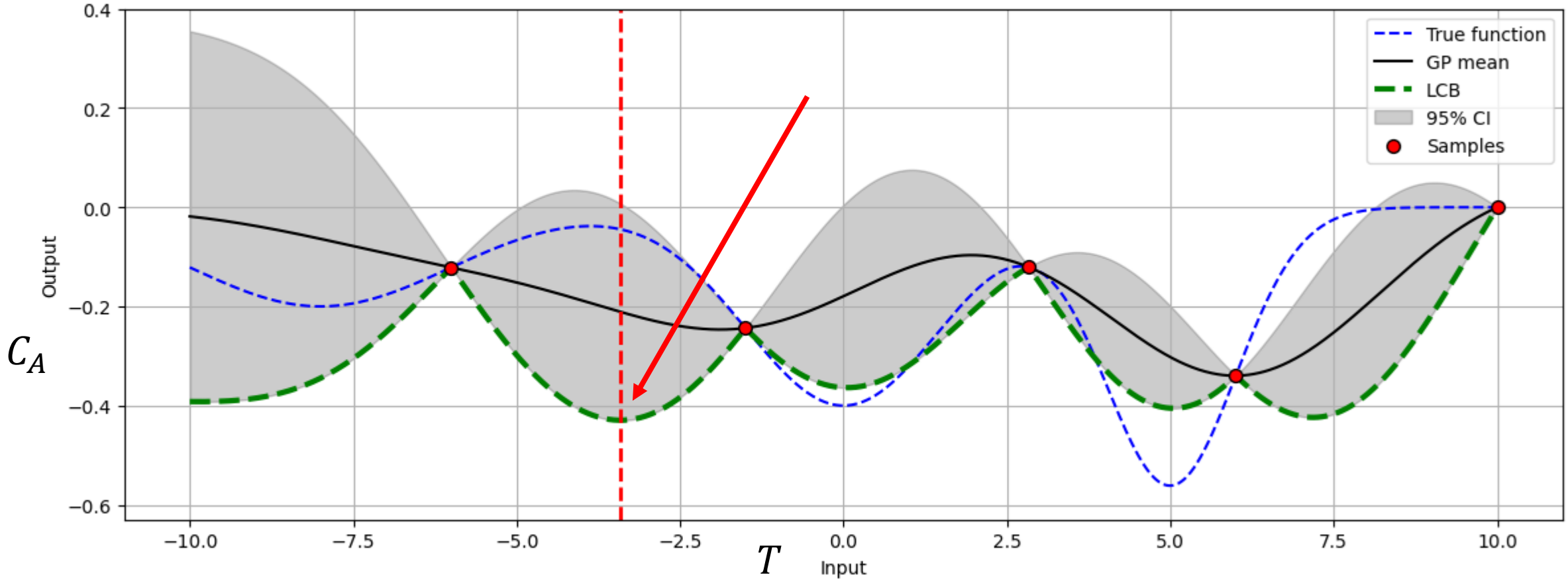
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



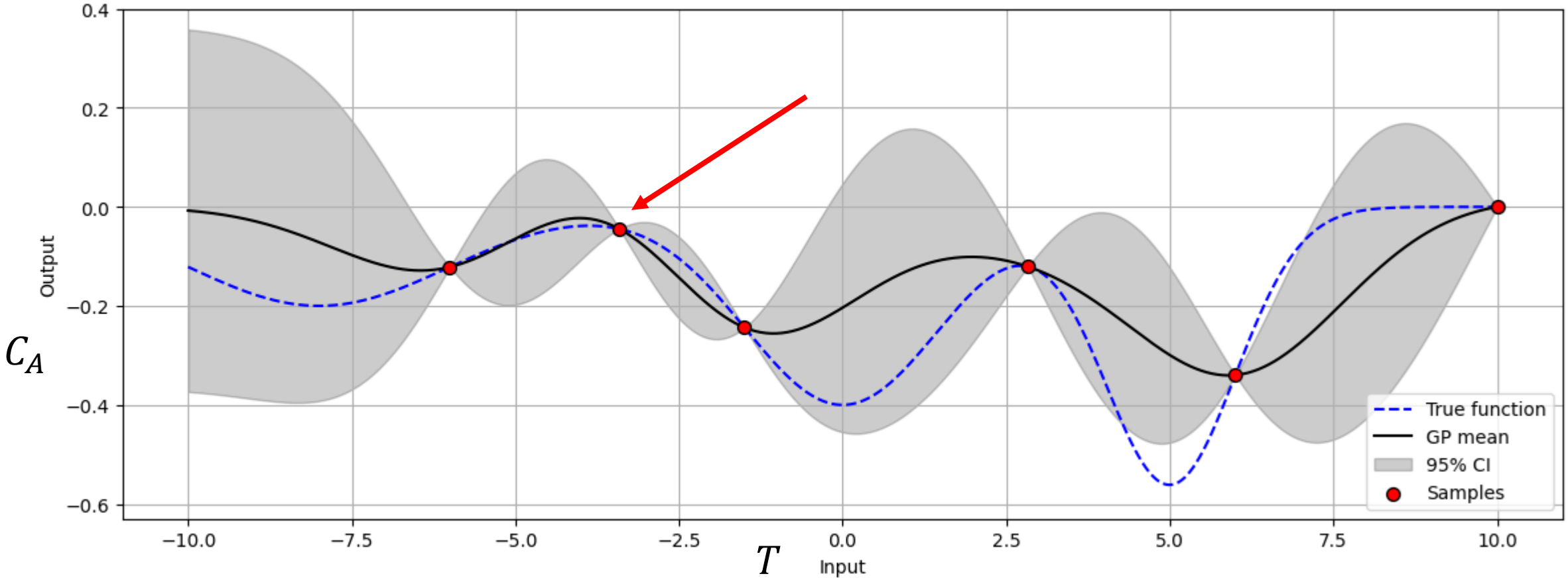
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



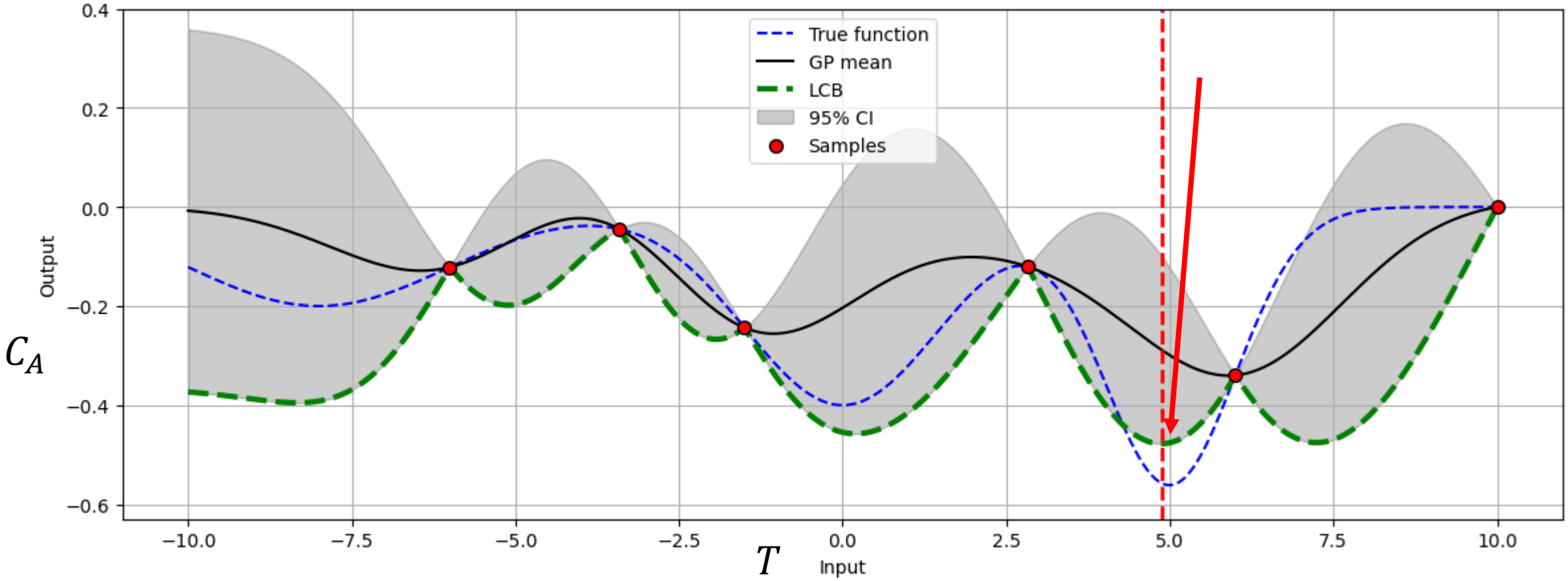
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



C_A

T

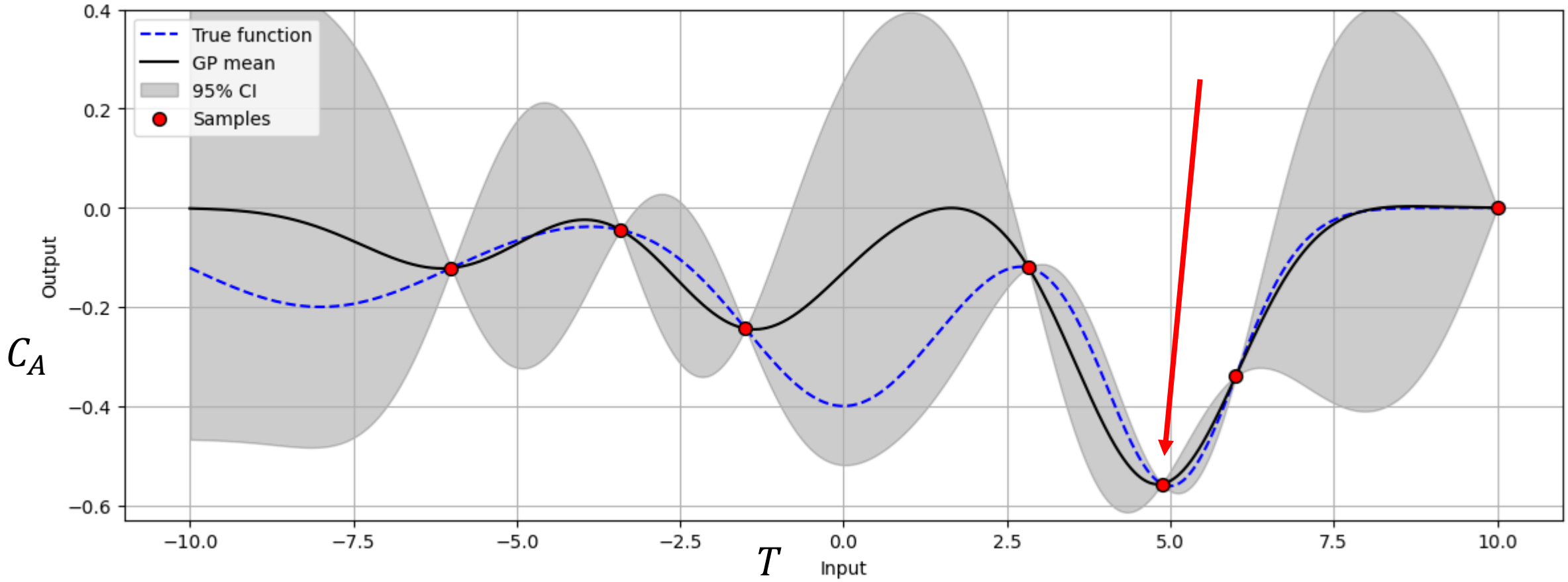
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



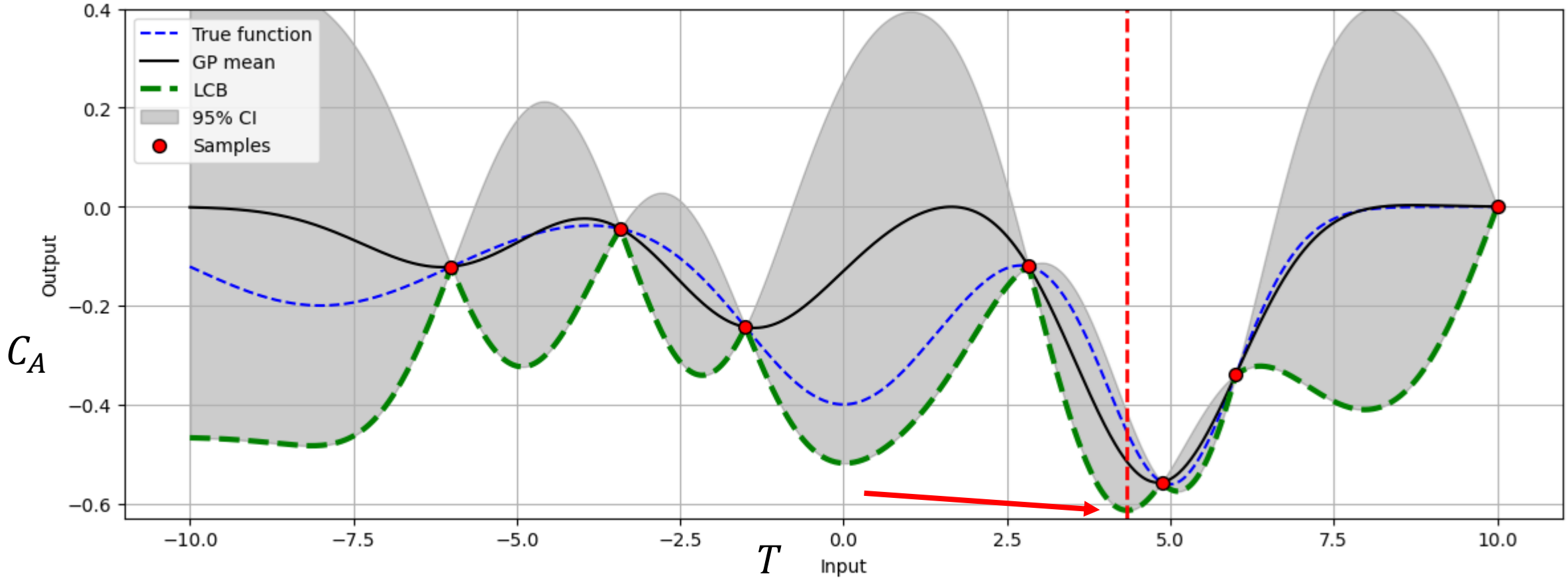
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



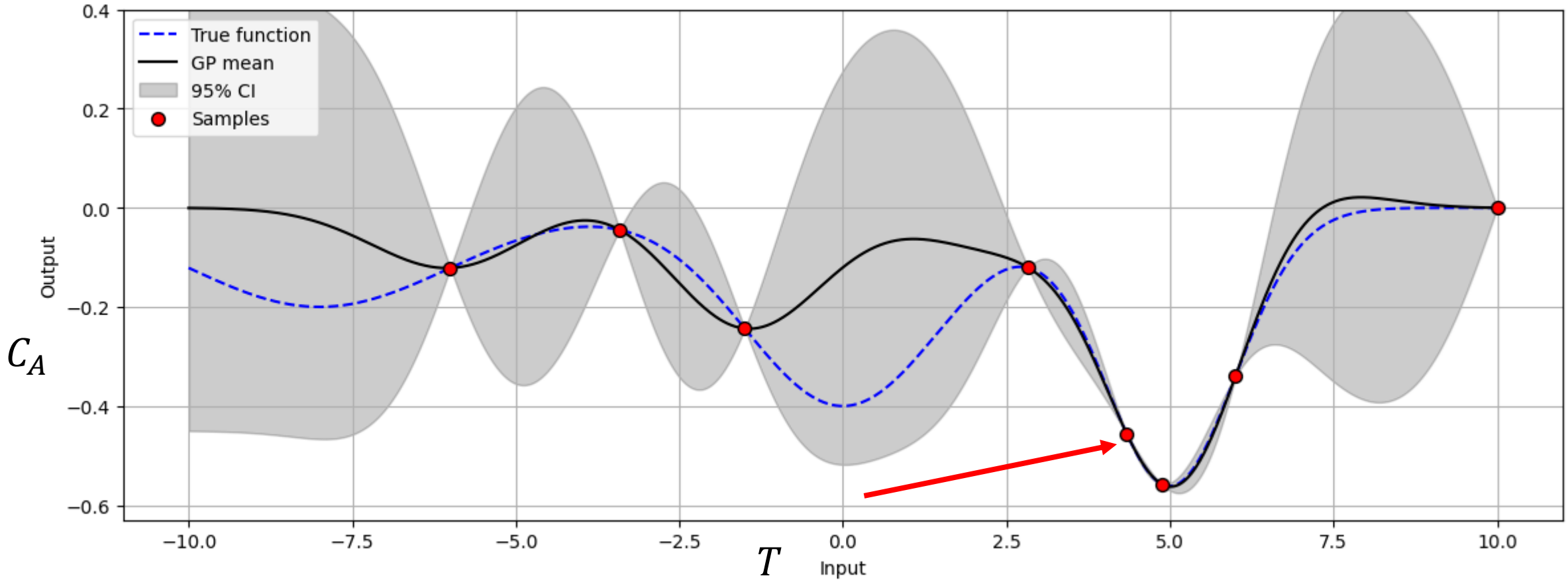
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



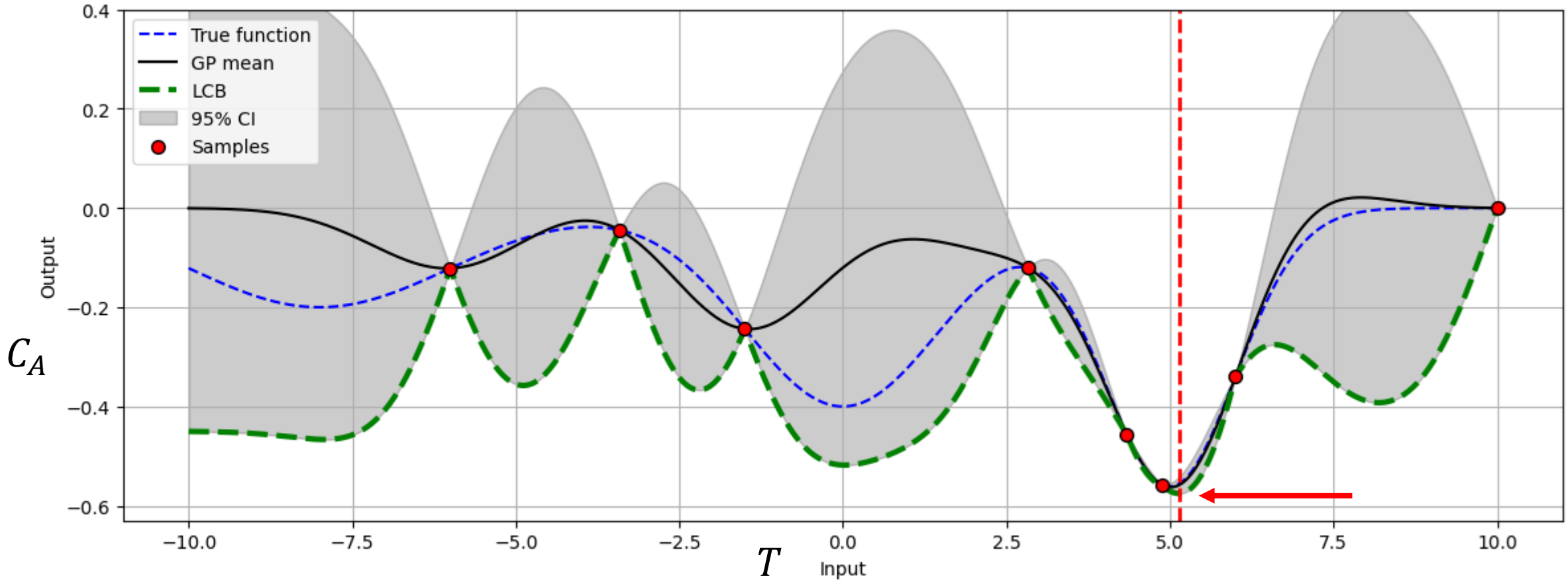
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



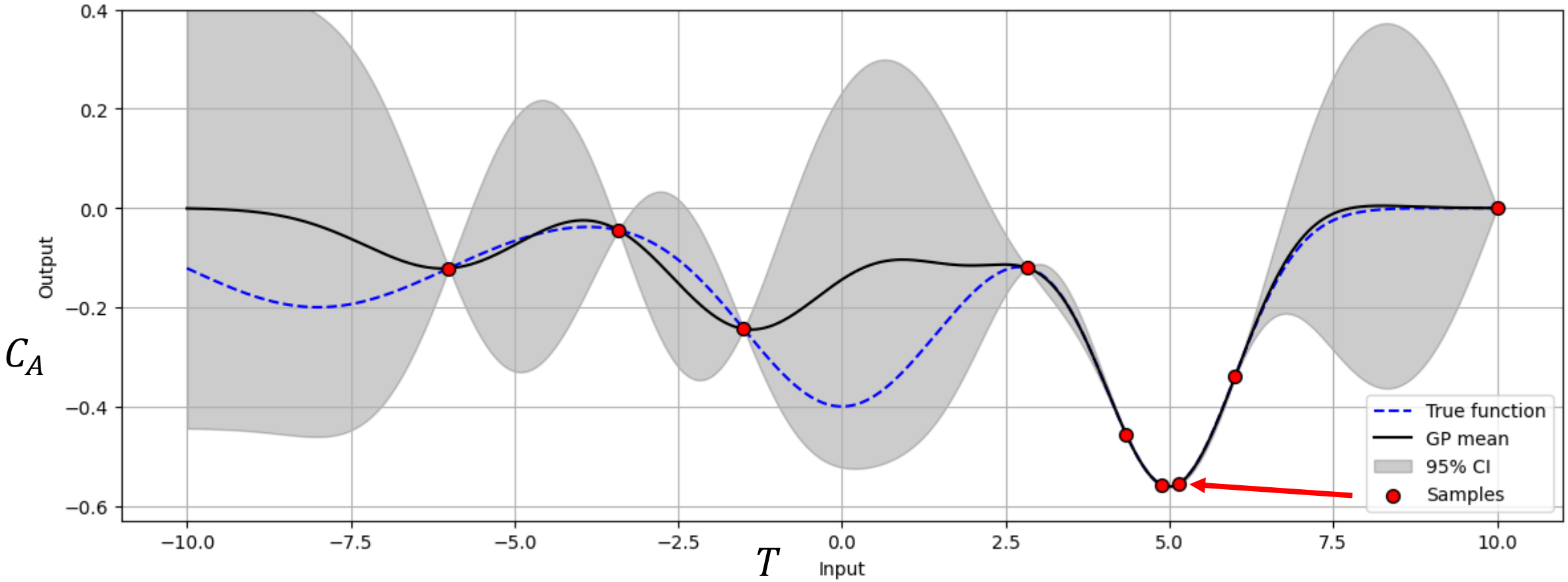
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



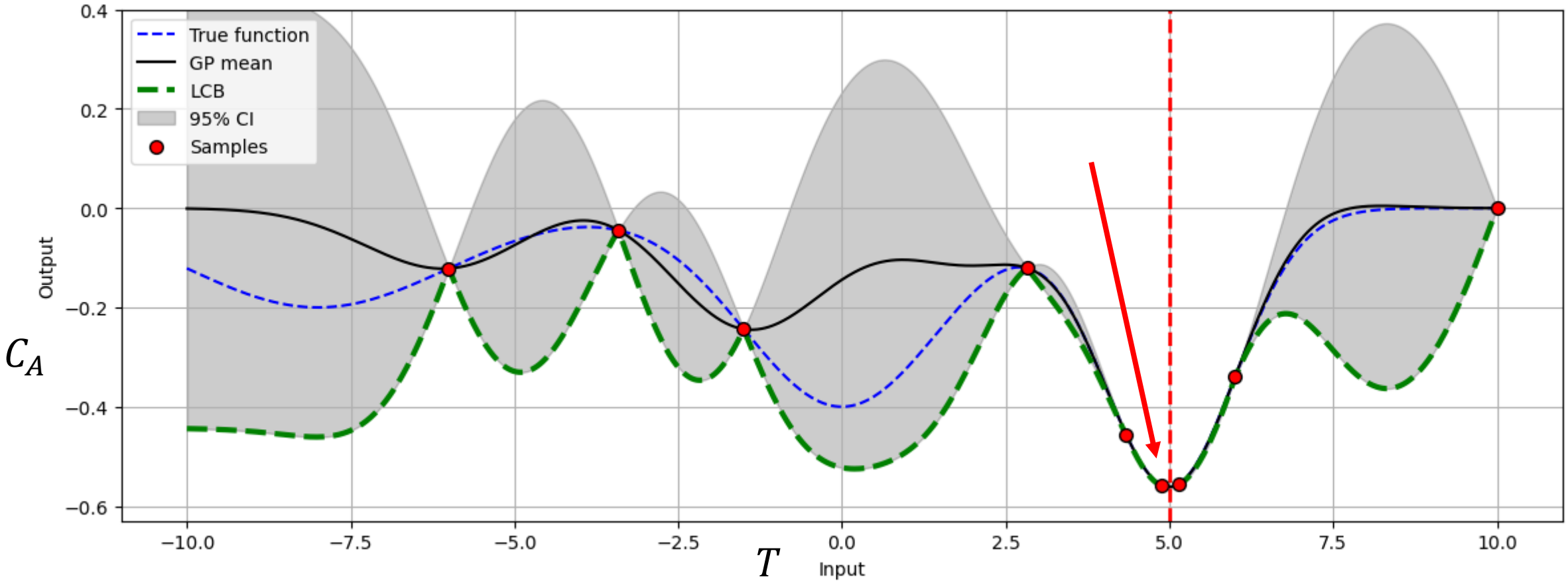
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$



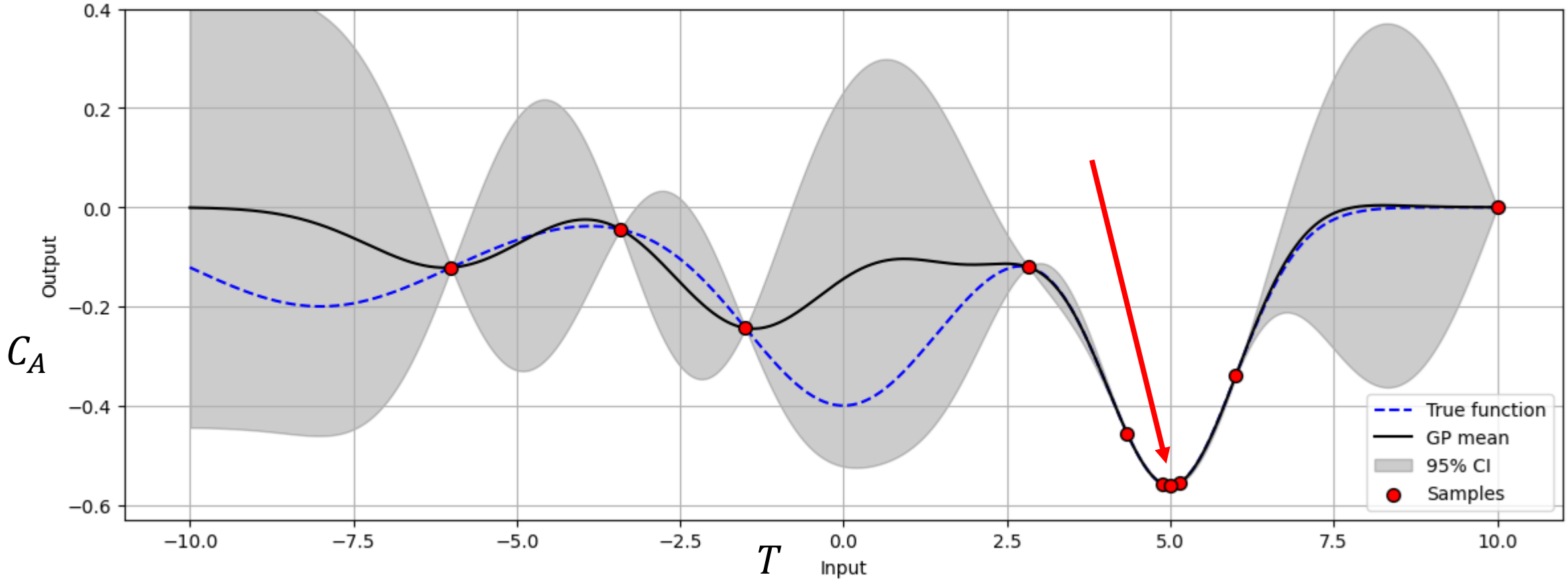
Bayesian Optimization – Big Picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$

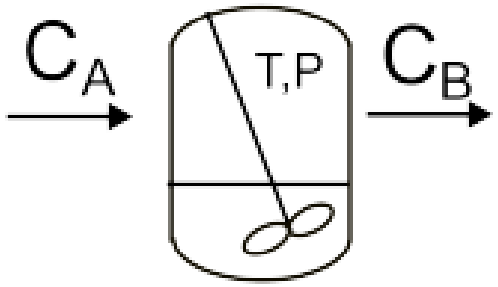


Bayesian Optimization – Big Picture

Problem statement

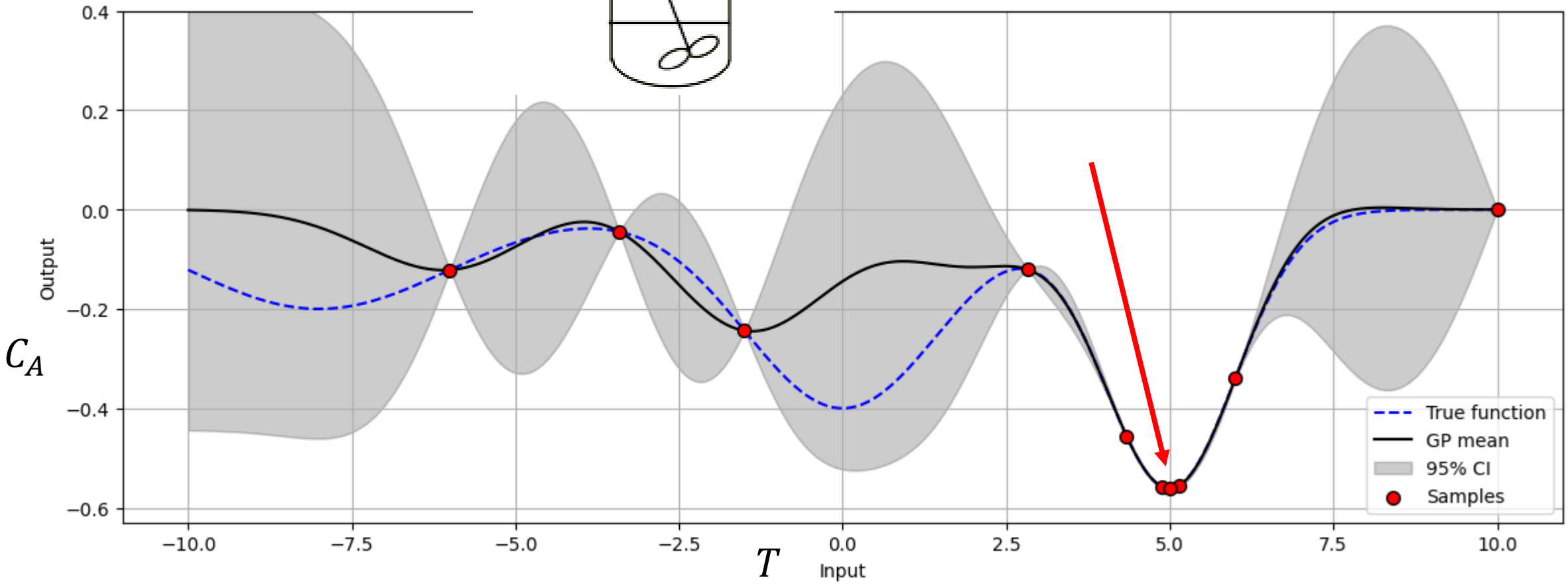
$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

T to **maximize** C_B production!



Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$

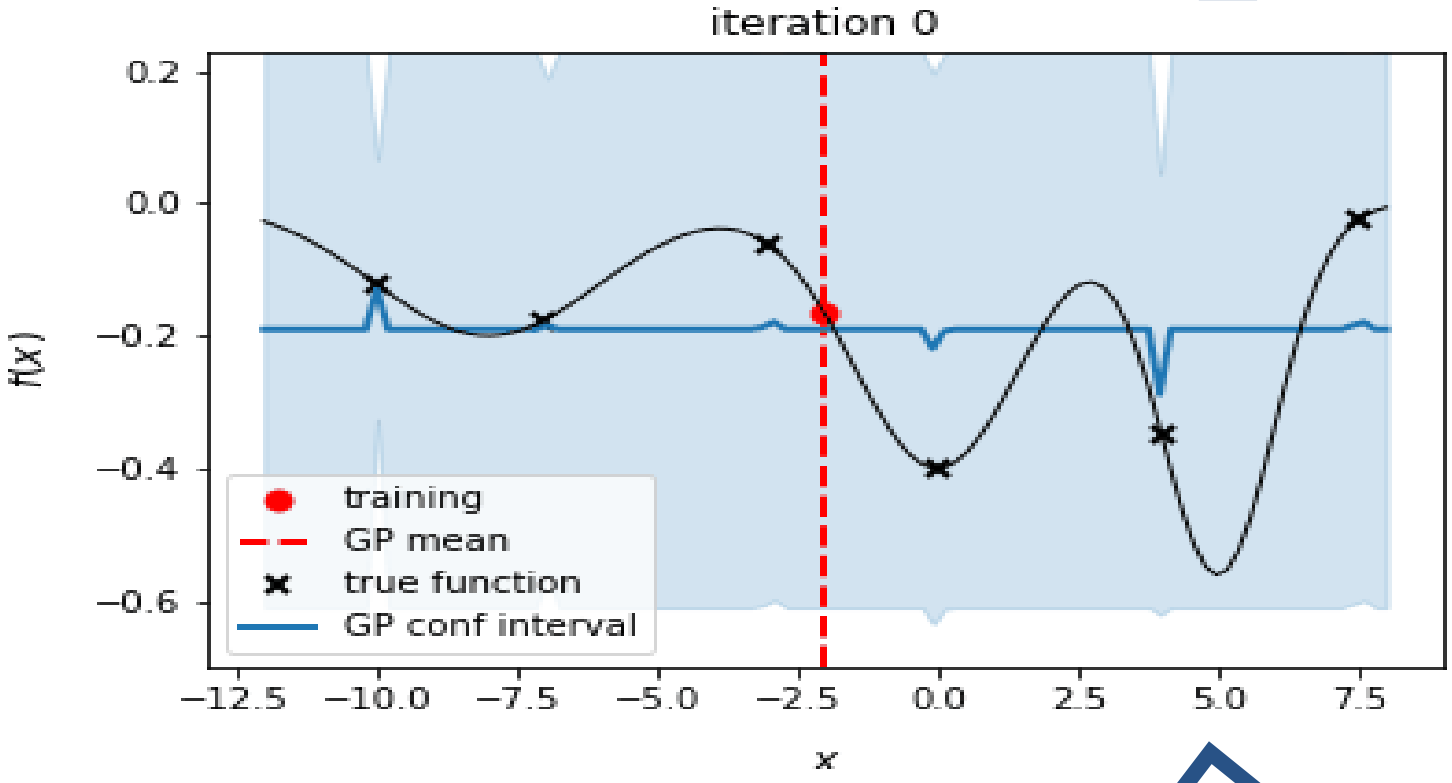


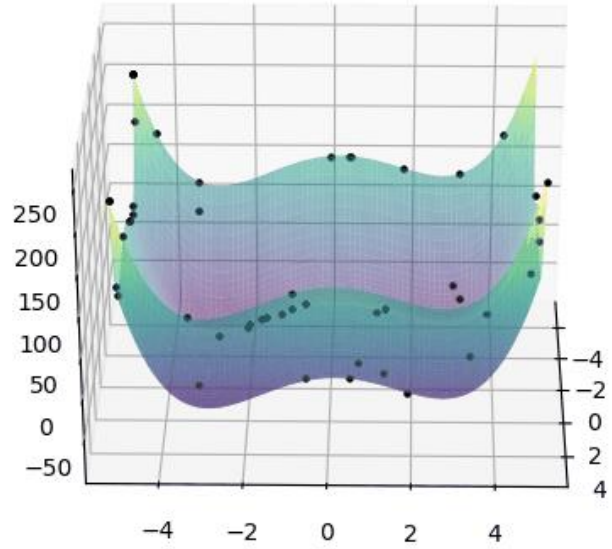
C_A

T

Bayesian Optimization – Big picture

Bayesian Optimization





optimization by evolutionary
or
Multi-start algorithm

Bayesian Optimization – Big picture

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x})$$

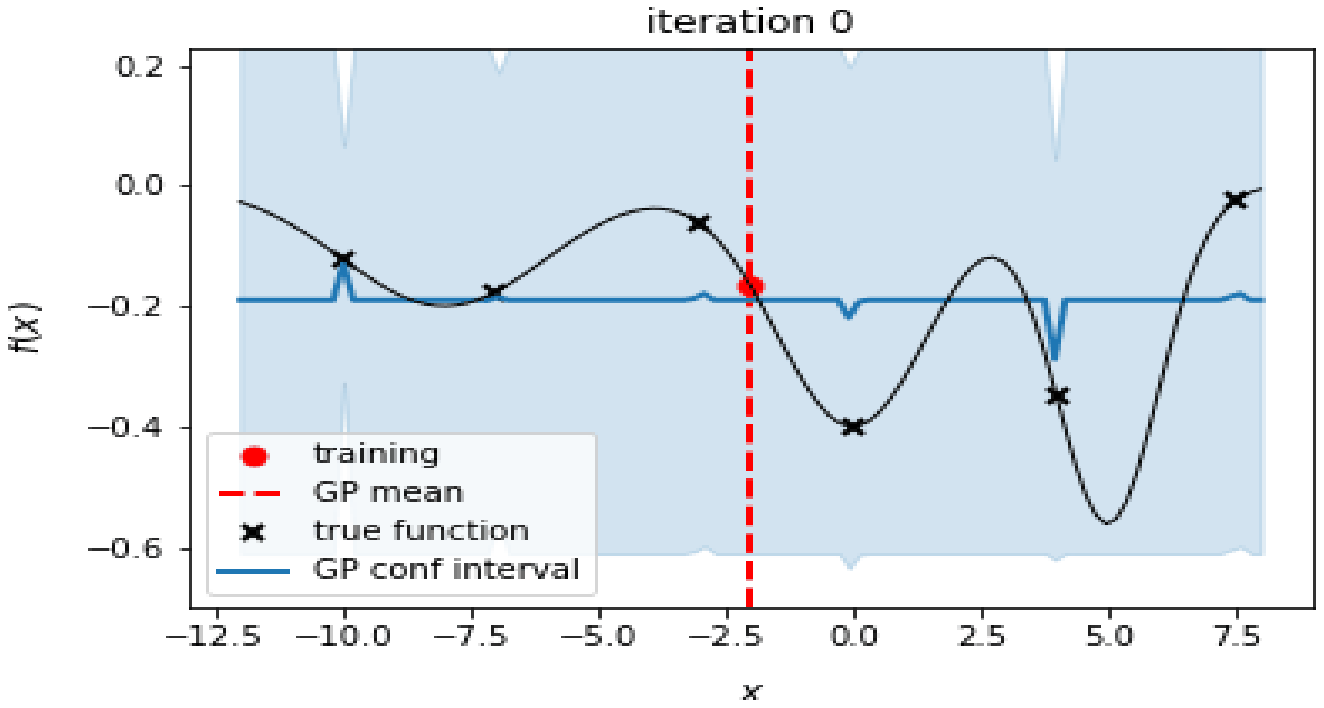
Lower confidence bound

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad \mu_{\hat{f}}(\mathbf{x}) - \beta\sigma_{\hat{f}}(\mathbf{x})$$

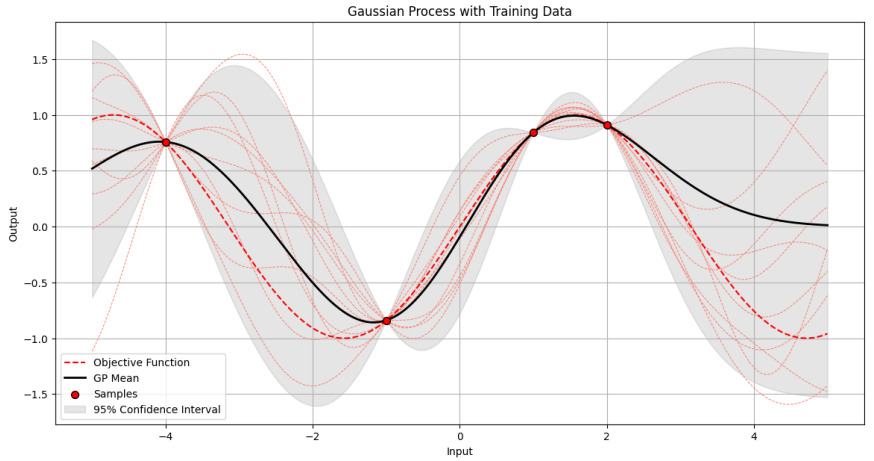
$$\hat{f}(\mathbf{x}) \sim \mathcal{GP}\left(\mu_{\hat{f}}(\mathbf{x}), \sigma_{\hat{f}}(\mathbf{x})\right)$$

Takeaways

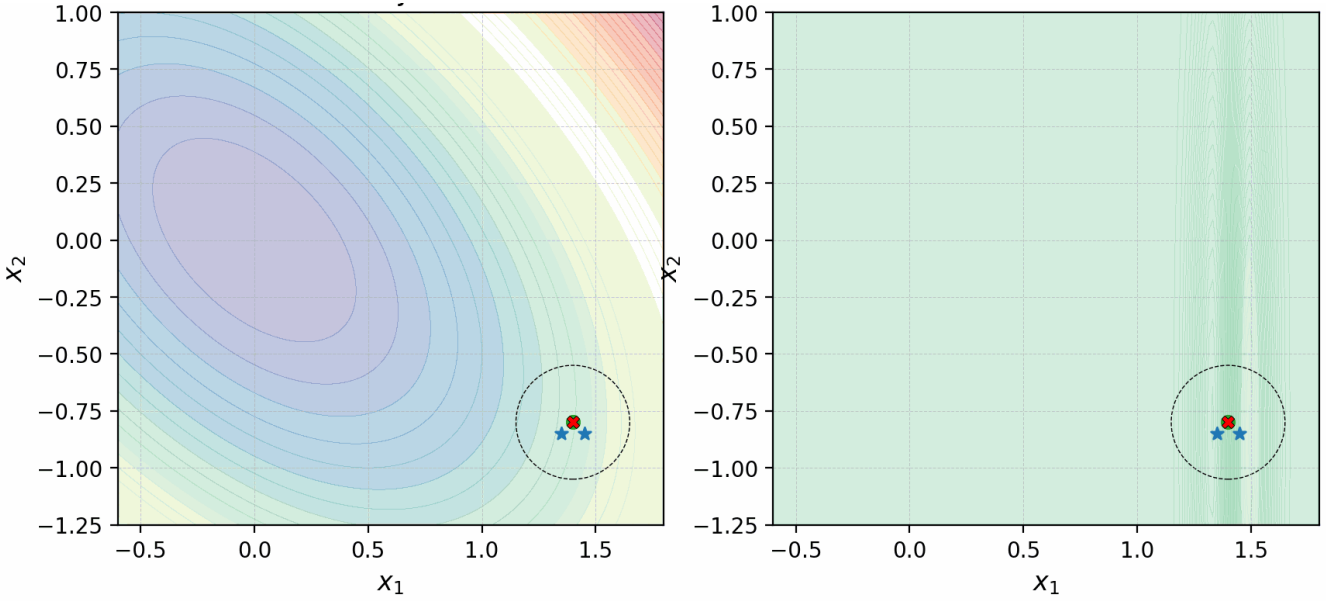
- Bayesian optimization is efficient for expensive black box problems.
- Optimizing Gaussian Processes is challenging.



A closer look at some **Model Based** algorithms



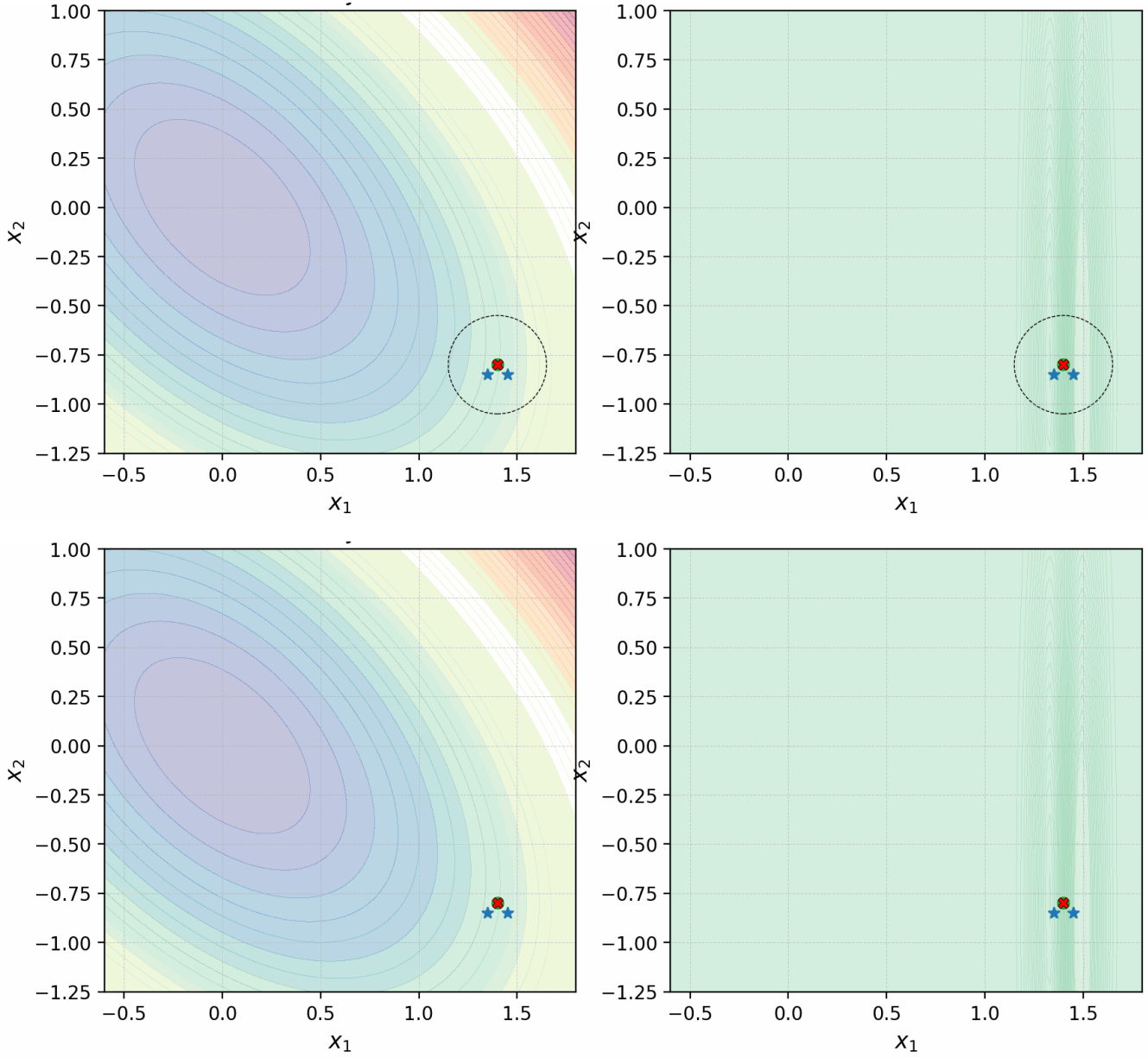
Local versus global methods - Model Based



<u>Local</u> model-based methods

Initial
Best path
Sampled locations
Best so far

Local versus global methods - Model Based



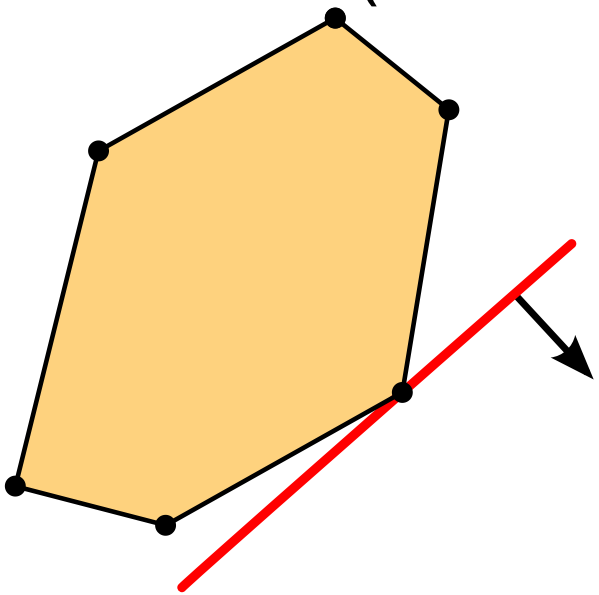
<u>Local</u> model-based methods

- Initial
- Best path
- Sampled locations
- Best so far

<u>Global</u> model-based methods

Model Based derivative free methods

COBYLA (Constrained Optimization BY Linear Approximation)



Linear surrogate for objective

$$m_k(x) = f_k + g_k^\top (x - x_k)$$

Linear surrogate for constraints

$$a_{i,k}^\top (x - x_k) + c_{i,k} \leq 0$$

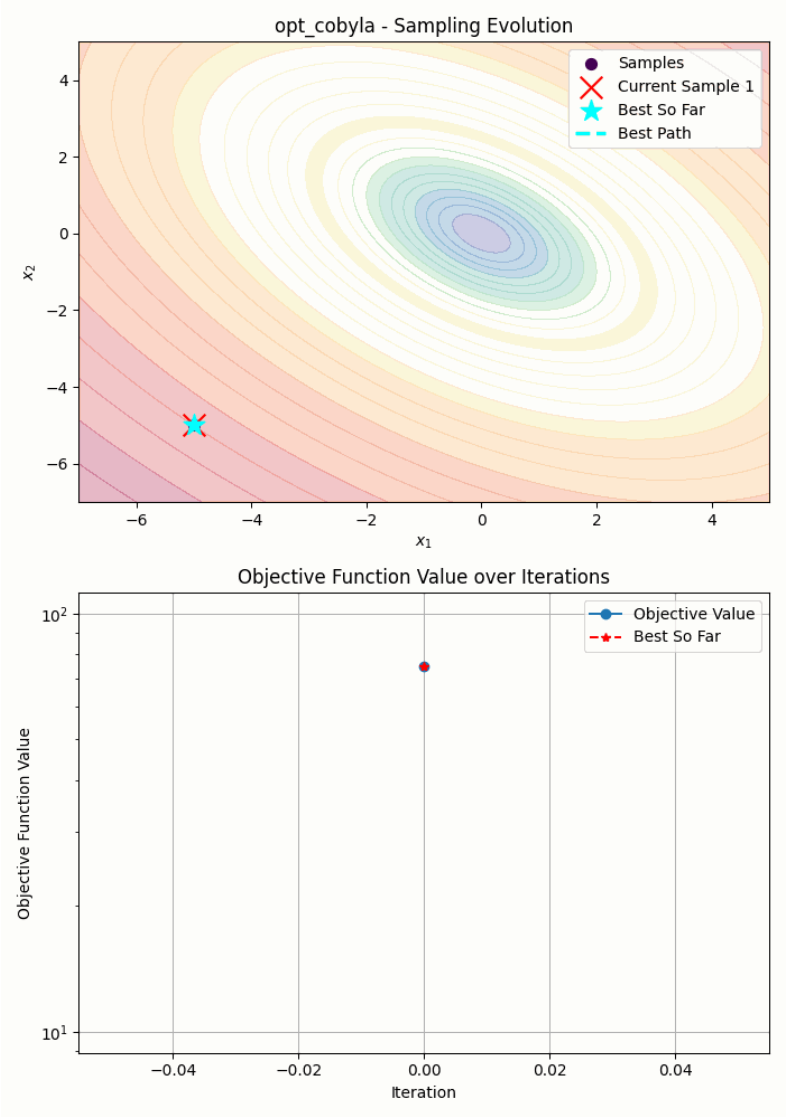
Creates a linear surrogate of the objective.

- Uses a trust region.
- Optimizes the surrogate via quadratically-constrained linear program (QCLP).
- Very fast (for model-based).

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & g_k^\top (x - x_k) \\ \text{subject to} \quad & a_{i,k}^\top (x - x_k) + c_{i,k} \leq 0, \\ & \|x - x_k\|_2 \leq \rho_k \end{aligned}$$

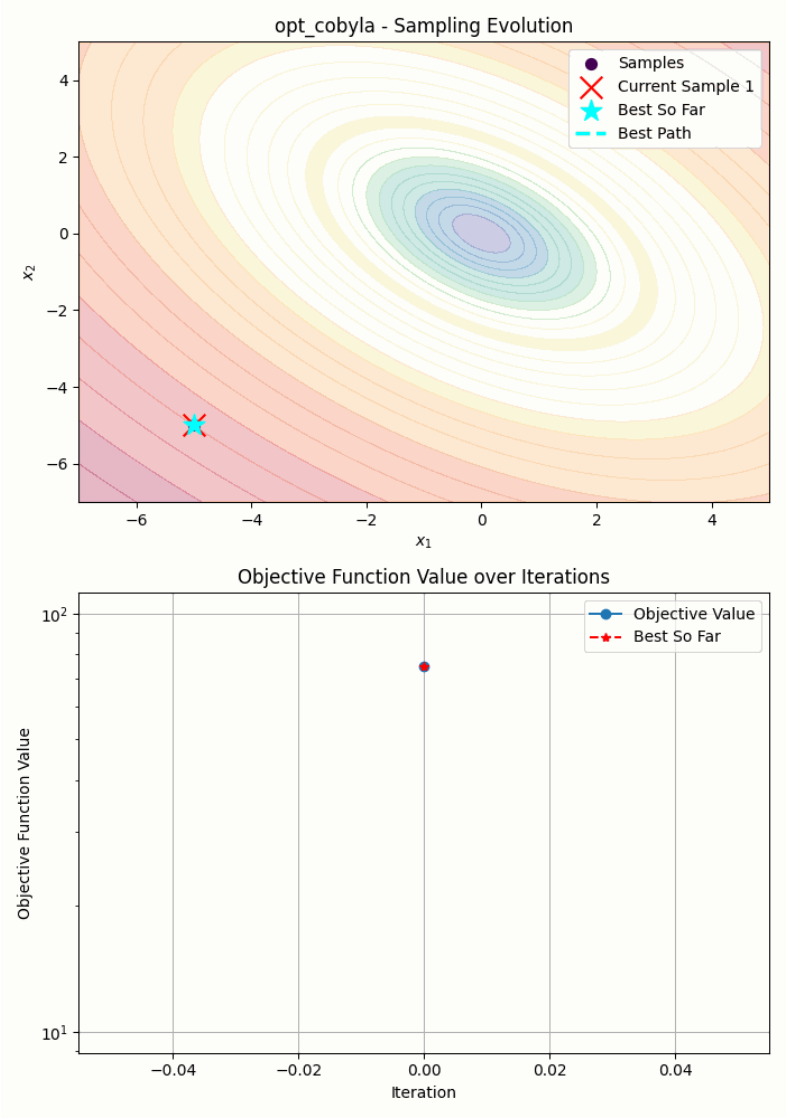
Model Based derivative free methods

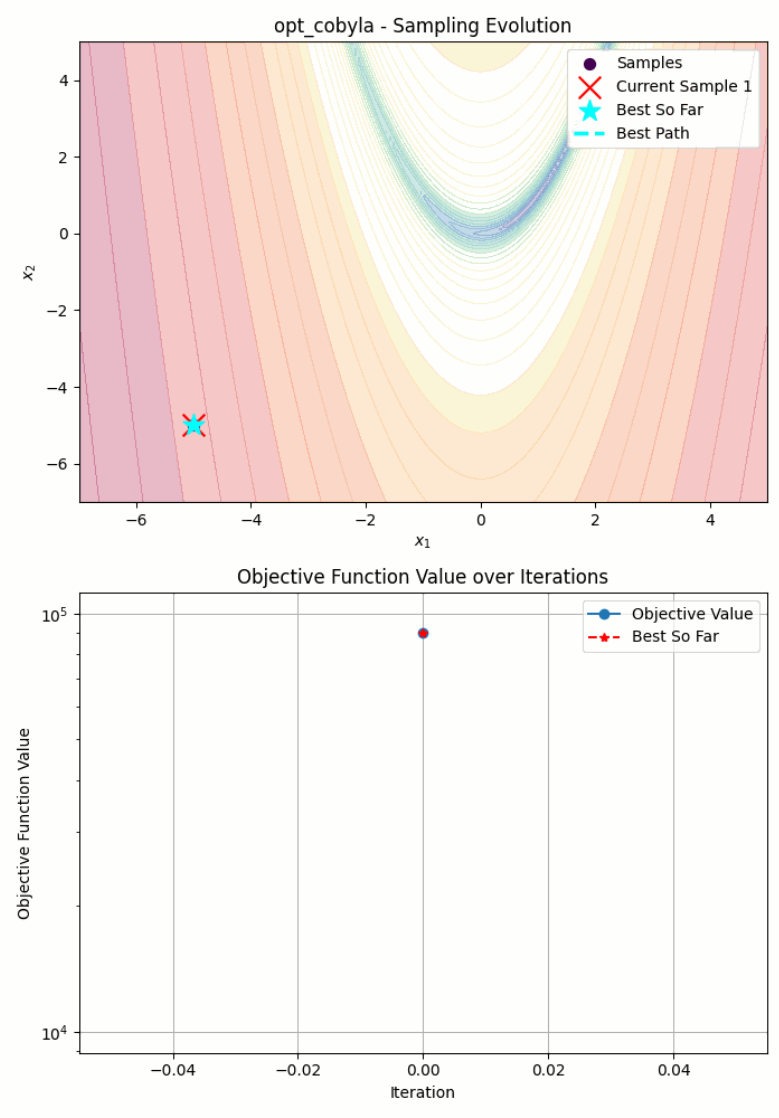
COBYLA (Constrained Optimization BY Linear Approximation)



Model Based derivative free methods

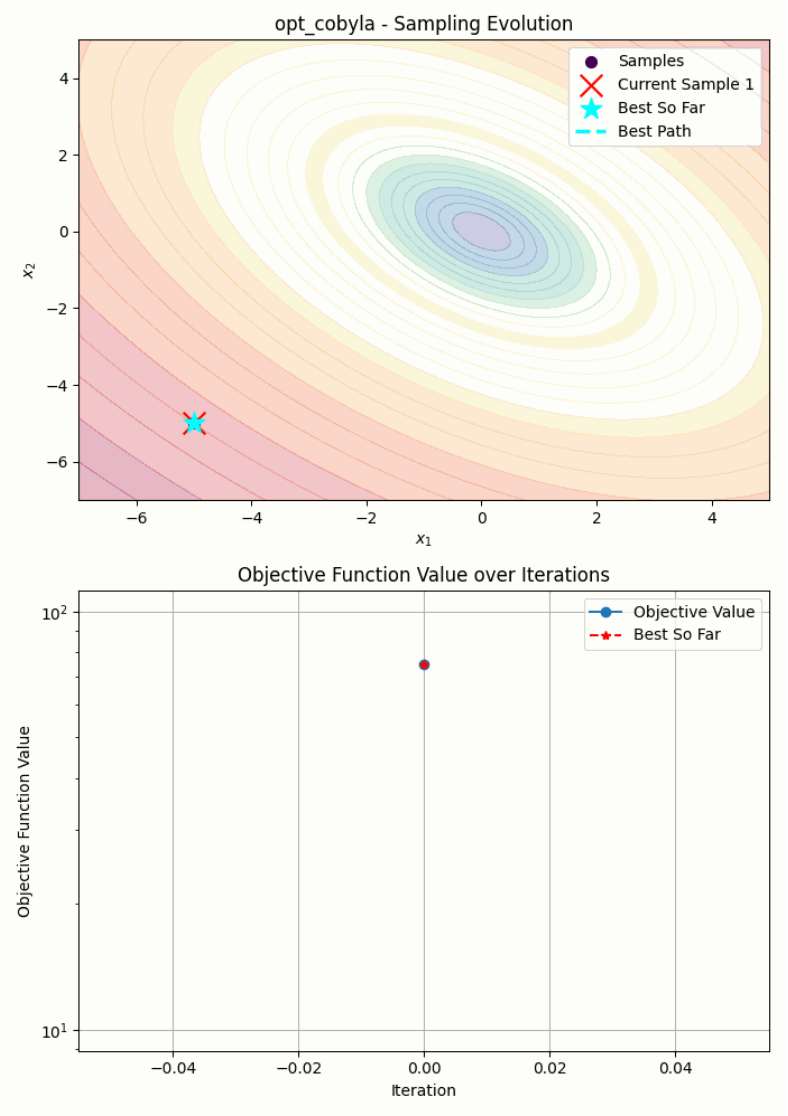
COBYLA (Constrained Optimization BY Linear Approximation)

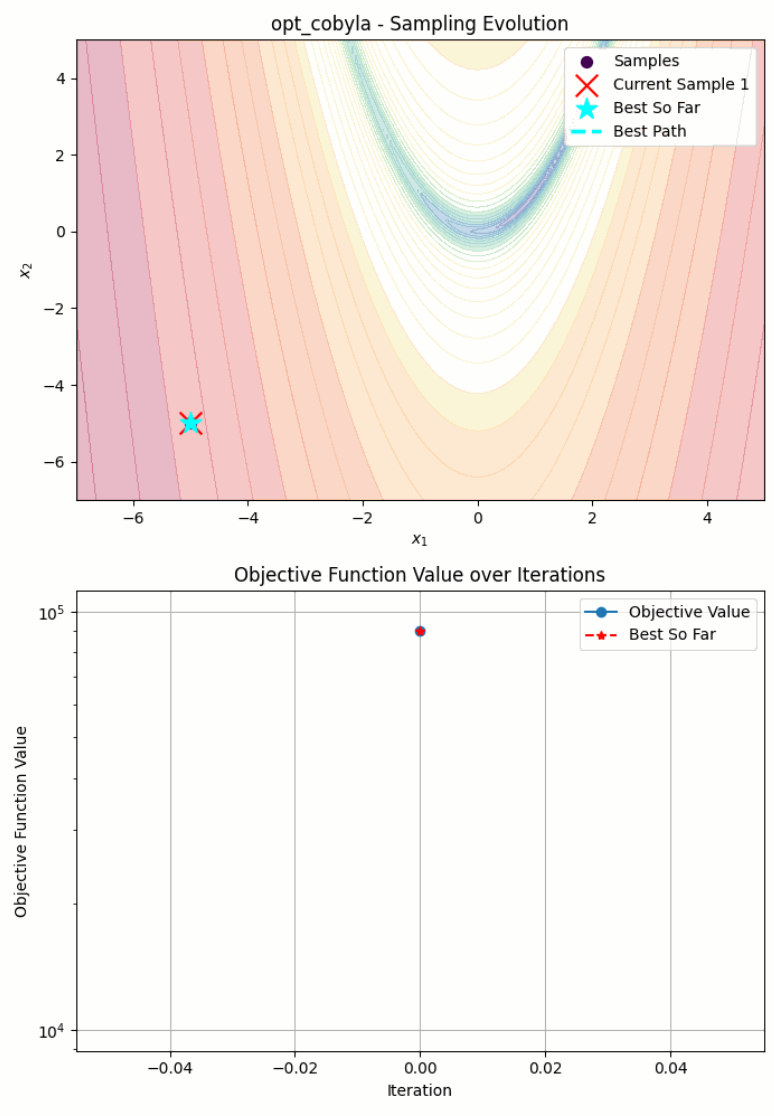


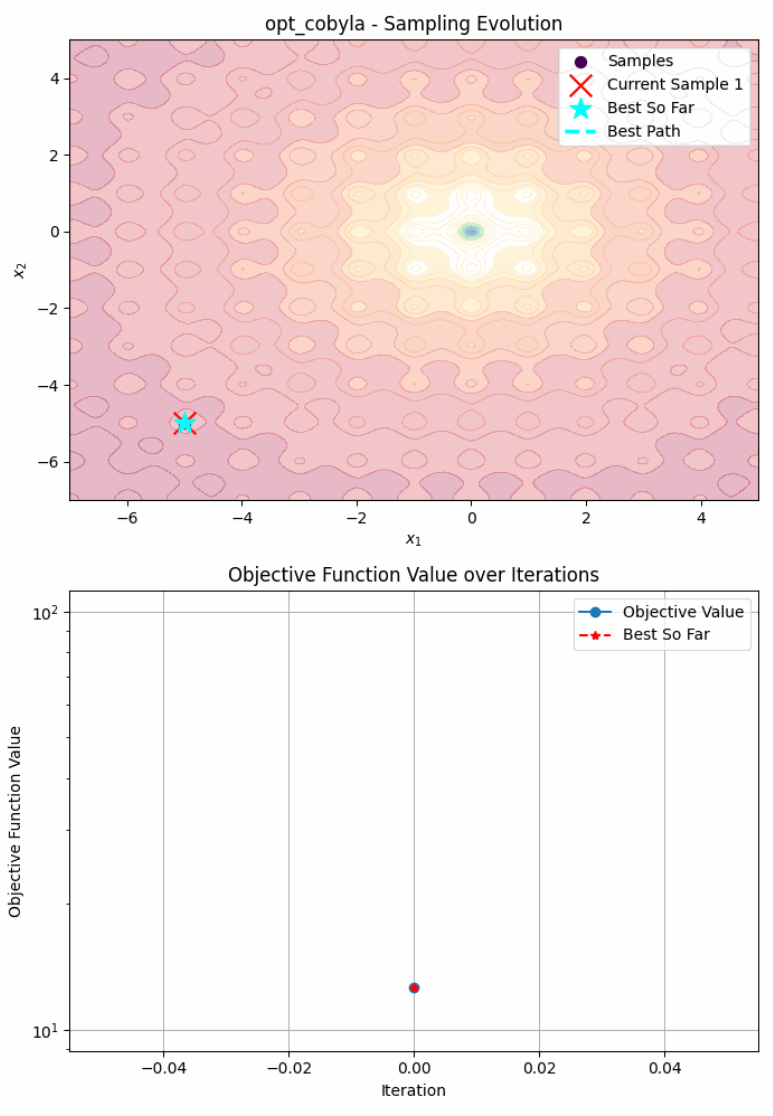


Model Based derivative free methods

COBYLA (Constrained Optimization BY Linear Approximation)

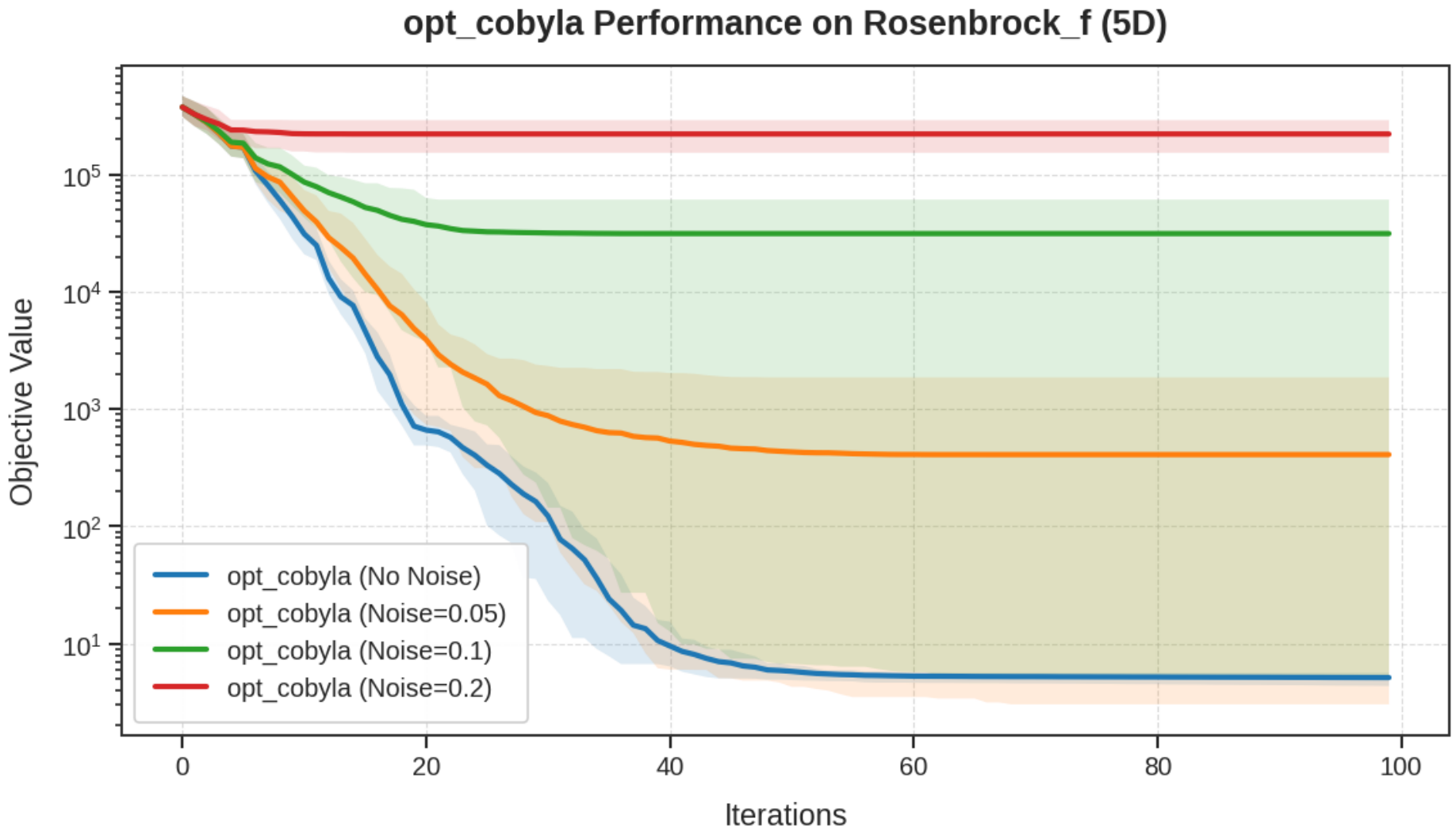






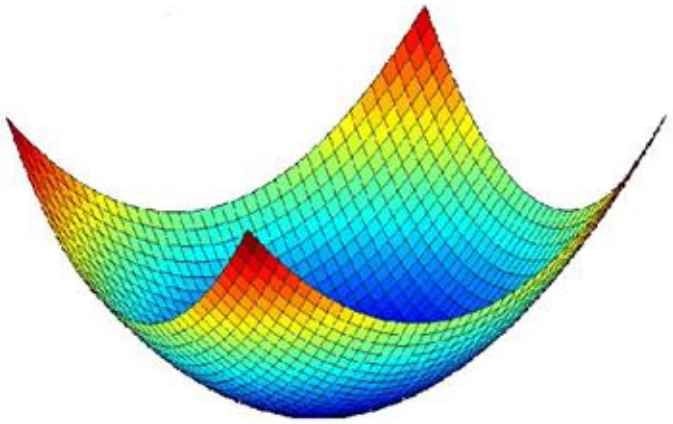
Model Based derivative free methods

COBYLA (Constrained Optimization BY Linear Approximation)



Model Based derivative free methods

BOBYQA (Bound Optimization BY Quadratic Approximation)



Quadratic surrogate for objective

$$m_k(x) = f_k + g_k^\top (x - x_k) + \frac{1}{2}(x - x_k)^\top H_k (x - x_k)$$

Optimization of surrogate

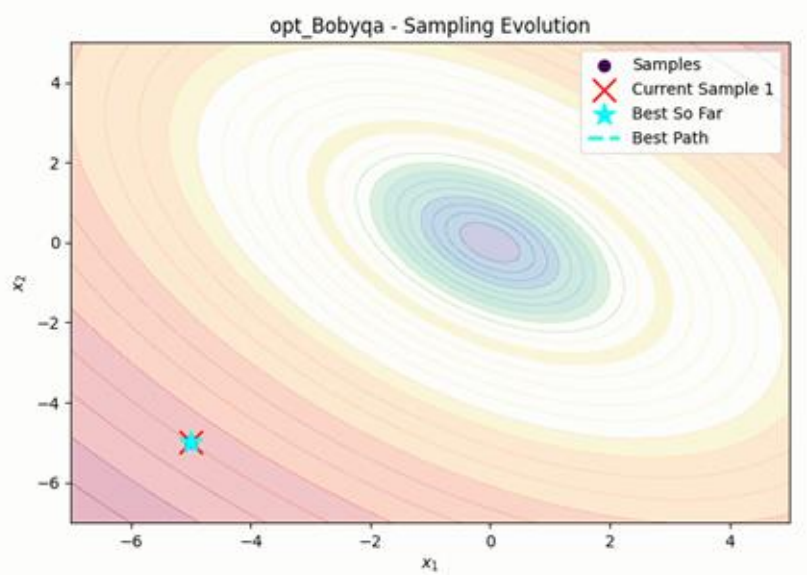
$$\min_x \quad m_k(x) \quad \text{subject to} \quad x^{(L)} \leq x \leq x^{(U)}, \quad \|x - x_k\| \leq \rho_k$$

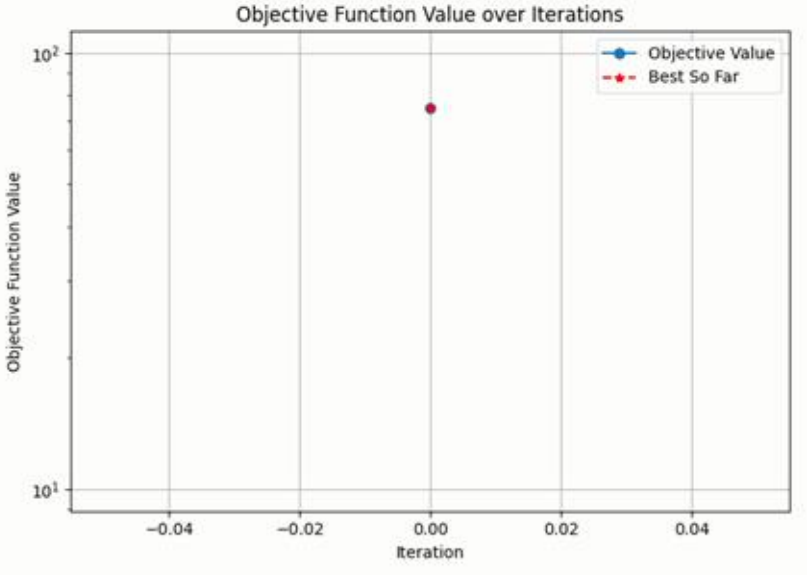
Creates a quadratic surrogate of the objective.

- Uses a trust region.
- Optimizes via Quadratic program.
- Very fast (for model-based).
- COBYQA variant allows for constraints.

Model Based derivative free methods

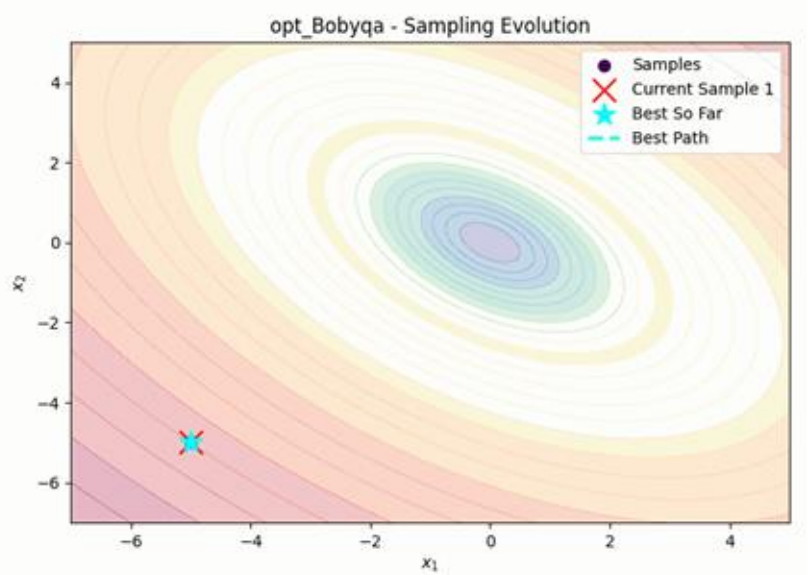
BOBYQA (Bound Optimization BY Quadratic Approximation)

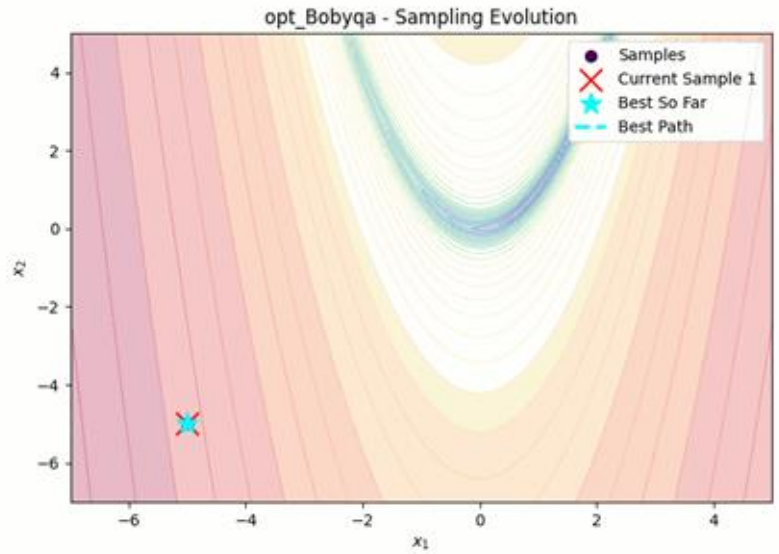


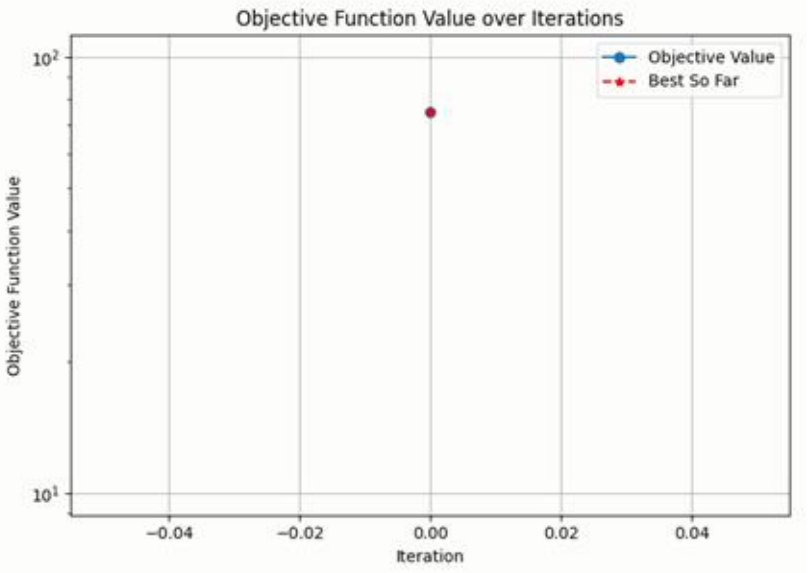


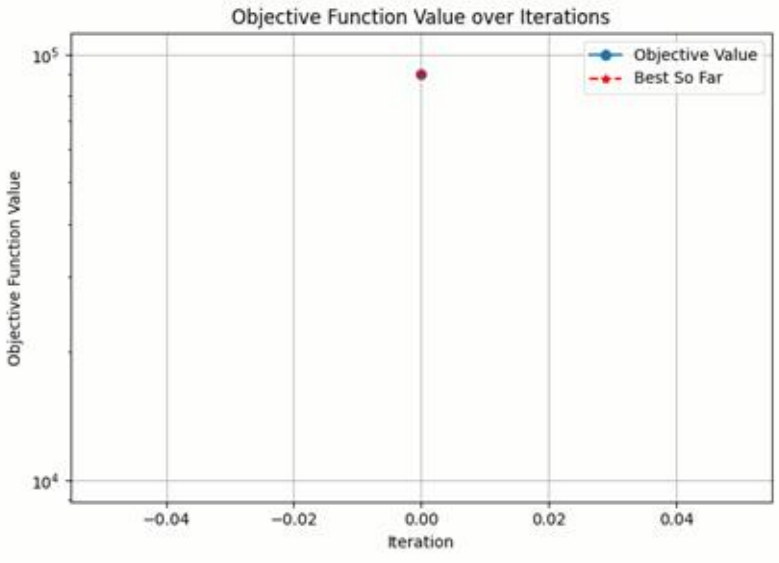
Model Based derivative free methods

BOBYQA (Bound Optimization BY Quadratic Approximation)



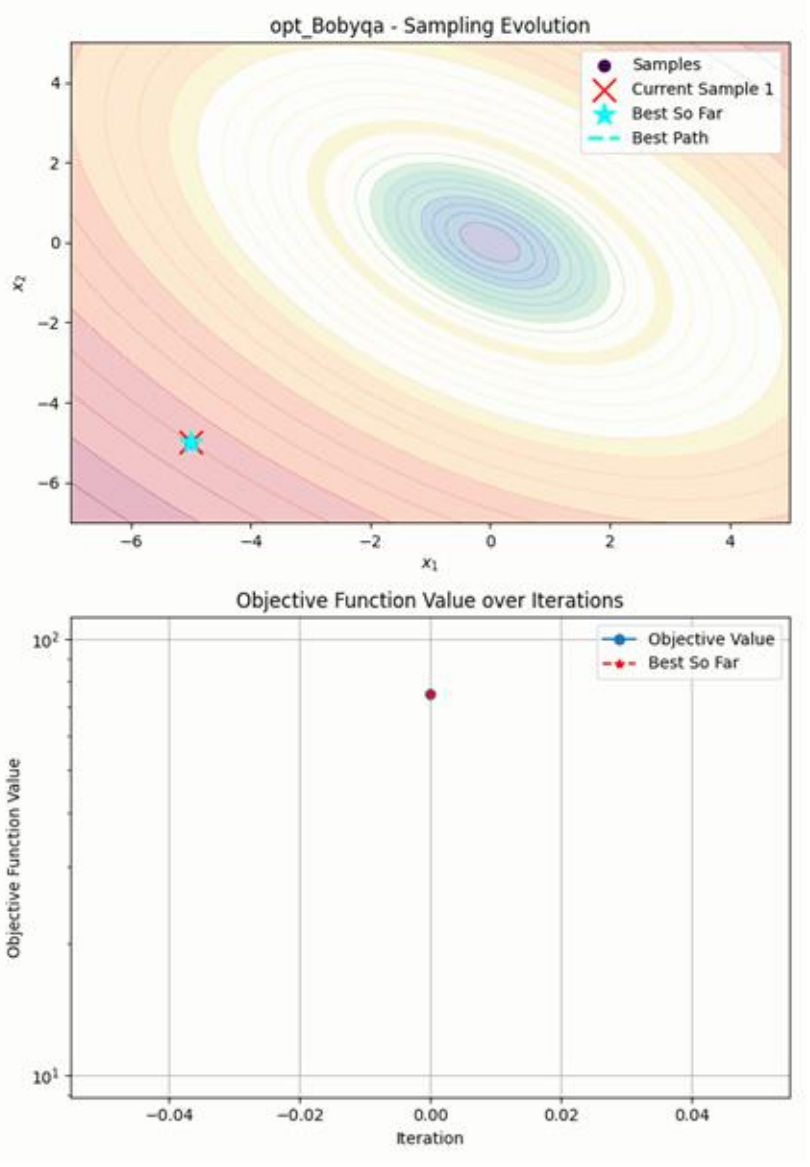


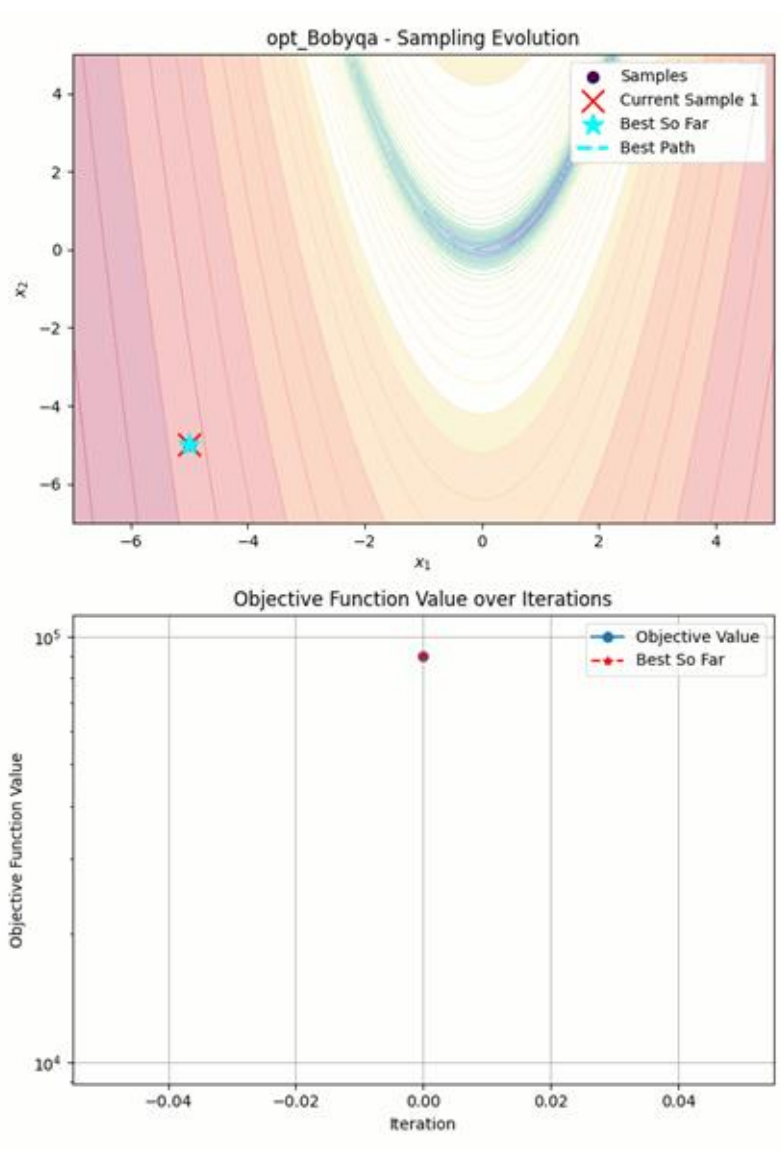


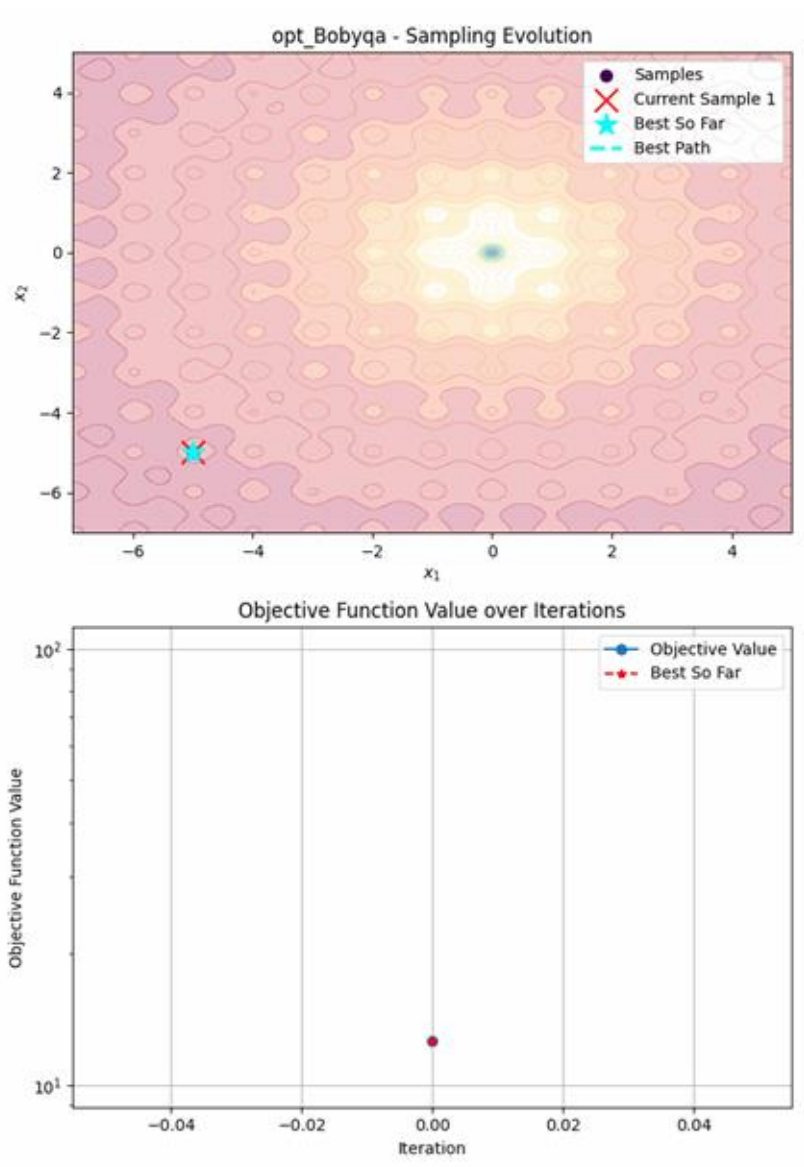


Model Based derivative free methods

BOBYQA (Bound Optimization BY Quadratic Approximation)

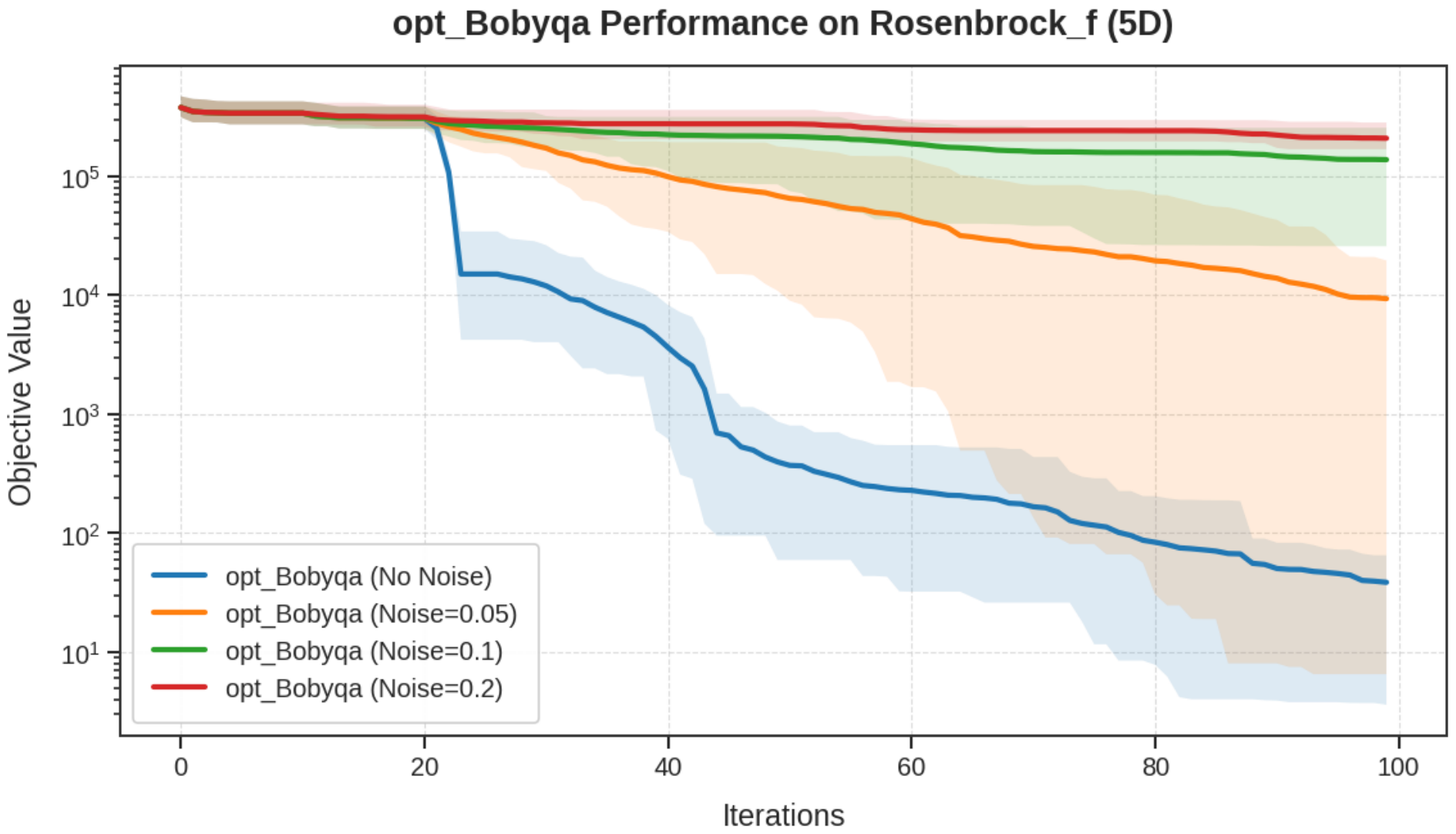






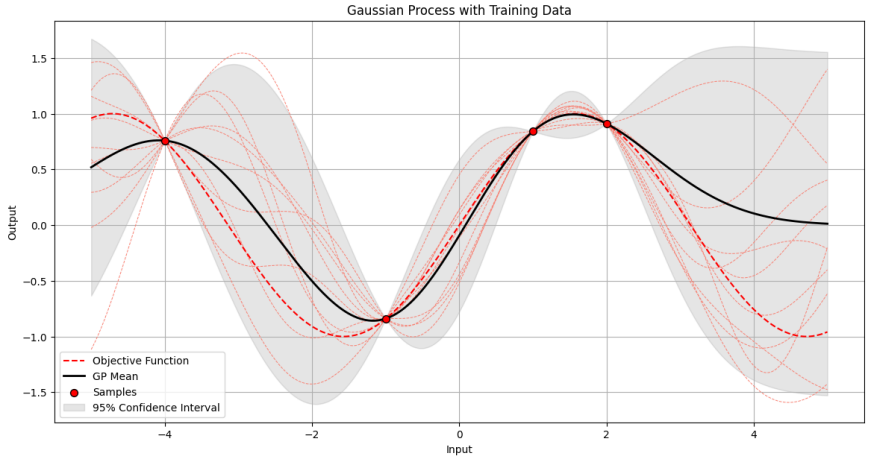
Model Based derivative free methods

BOBYQA (Bound Optimization BY Quadratic Approximation)



Model Based derivative free methods

Bayesian optimization



Gaussian process to approximate the objective

$$f(x) \sim \mathcal{GP}(\mu_k(x), \sigma_k^2(x))$$

Use of acquisition function

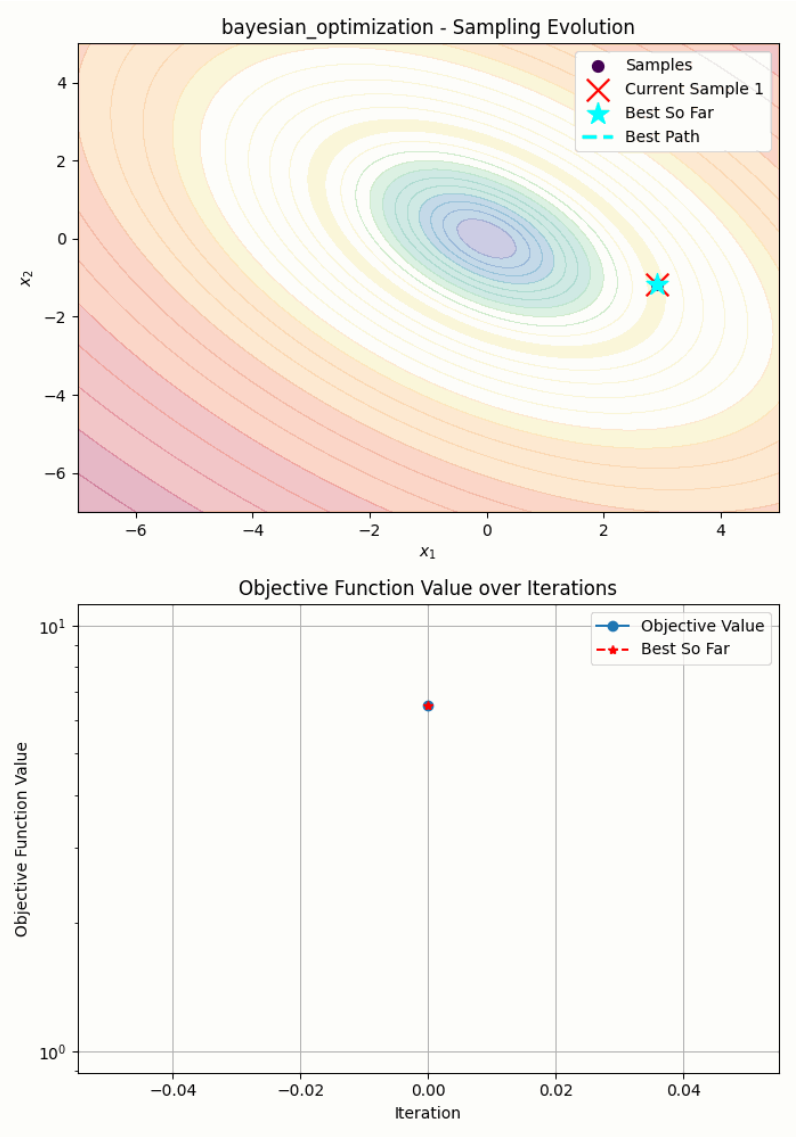
$$x_{k+1} = \arg\min_{x \in \mathcal{X}} \left[\mu_k(x) - \kappa \cdot \sigma_k(x)\right]$$

Creates a quadratic surrogate of the objective.

- Naturally balances exploration and exploitation.
- Needs to solve two nonconvex problems (training and optimization).
- One of the slowest per-iteration methods.
- Many variants, additions and modifications.

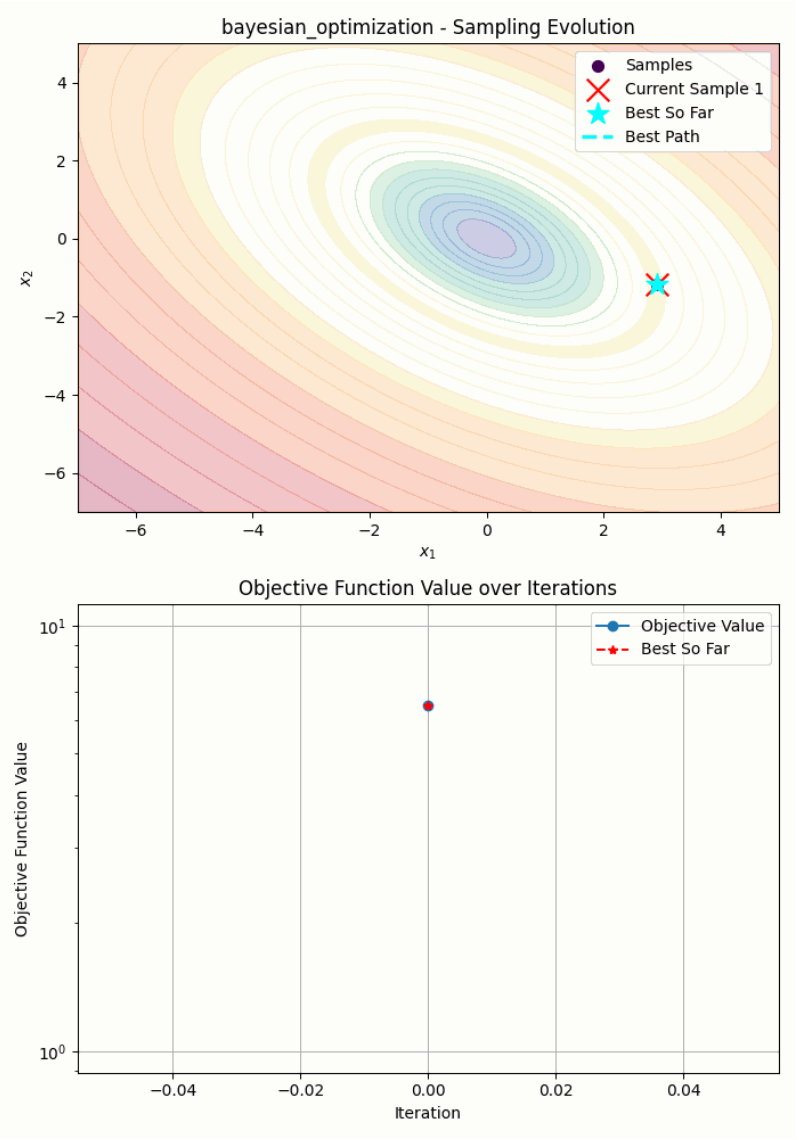
Model Based derivative free methods

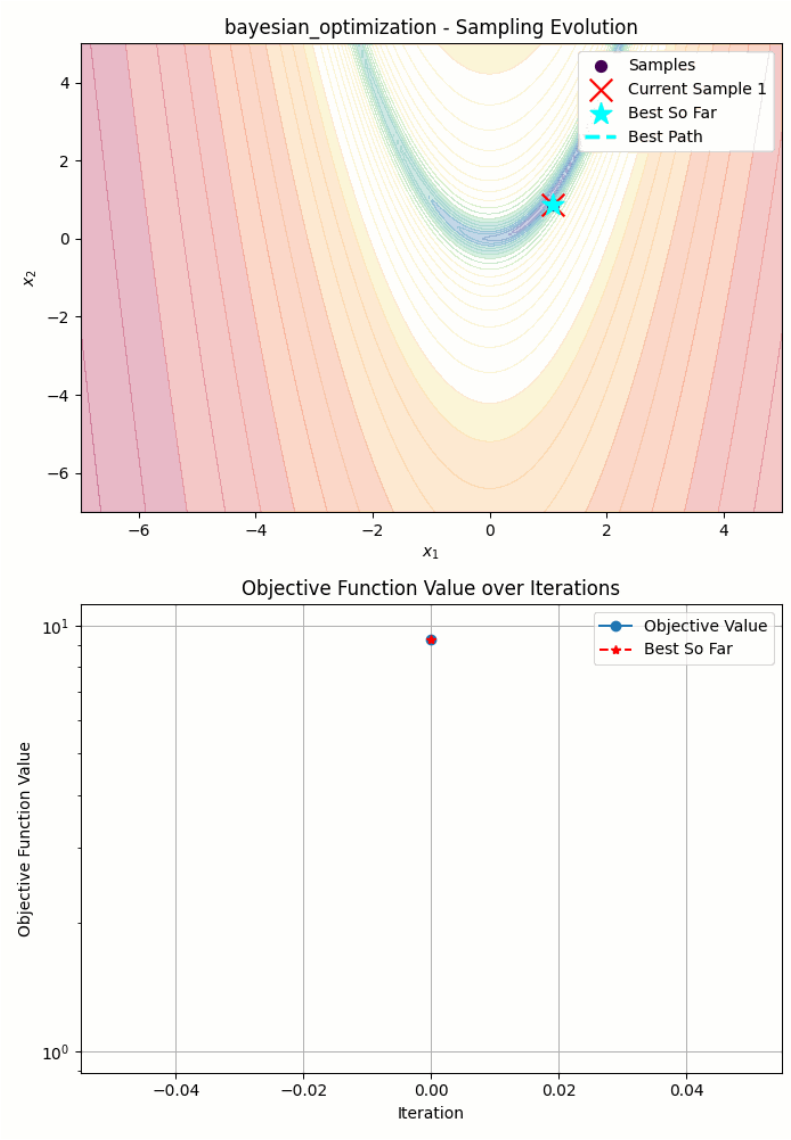
Bayesian optimization



Model Based derivative free methods

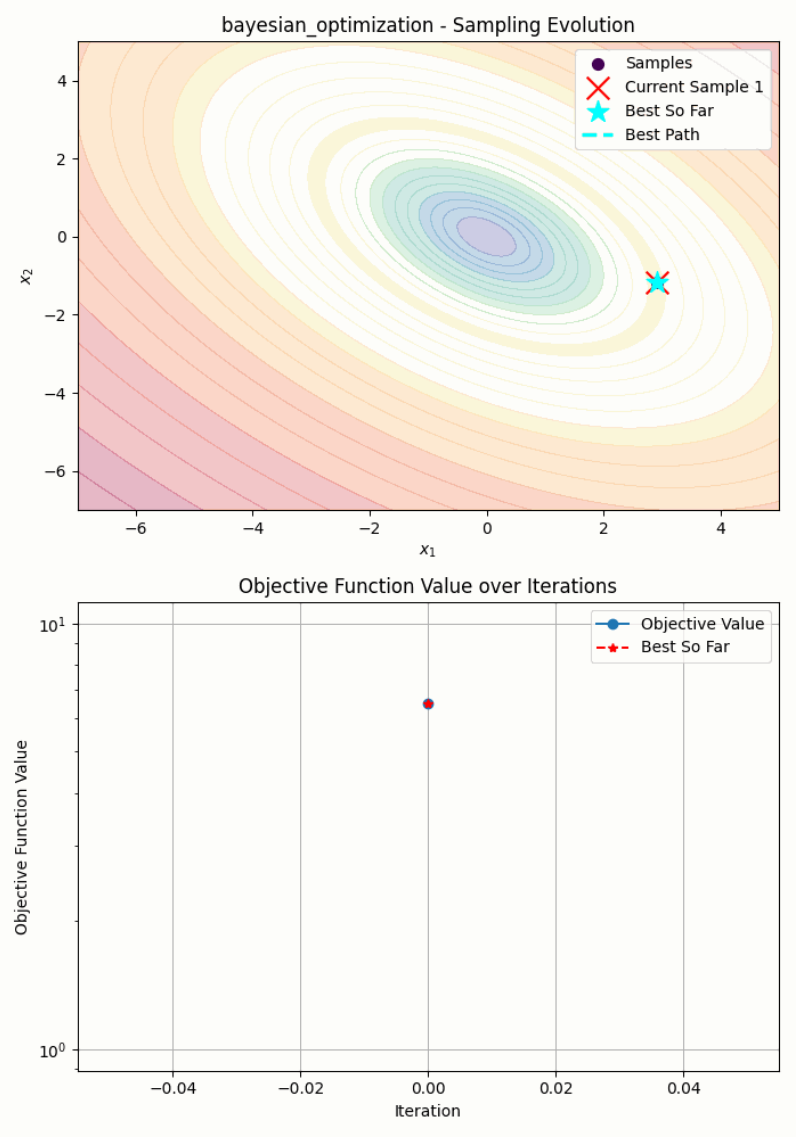
Bayesian optimization

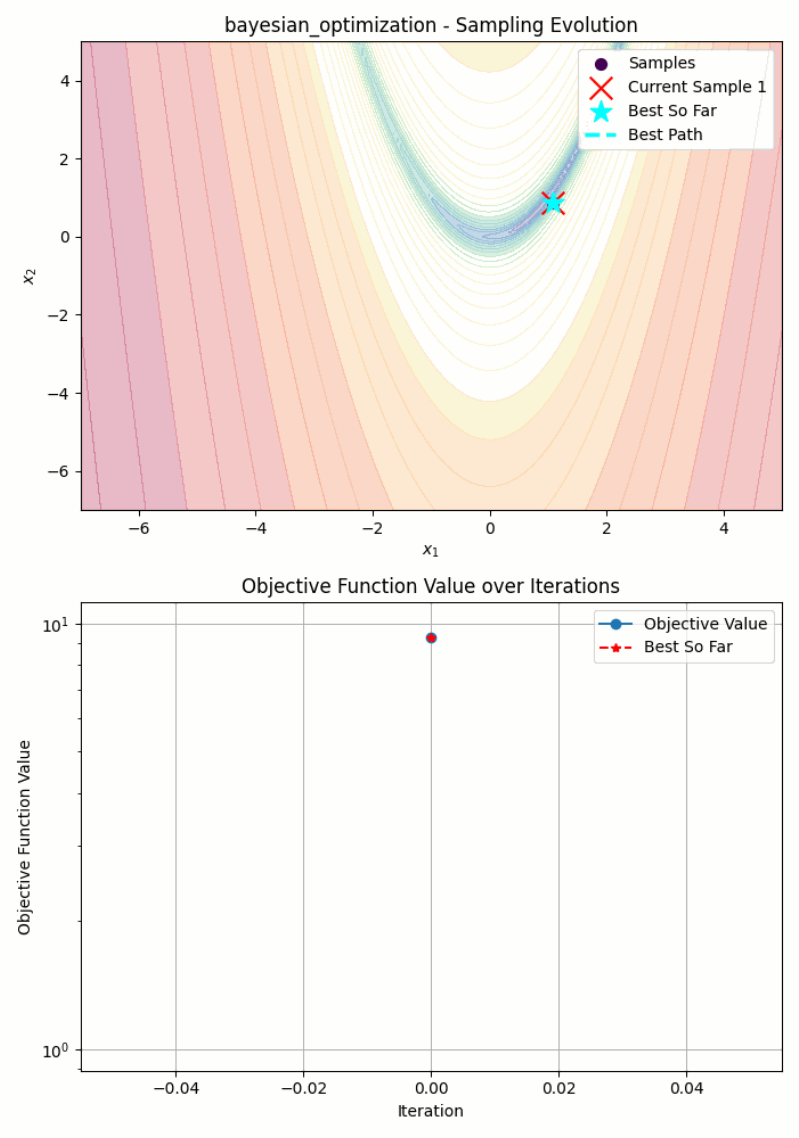


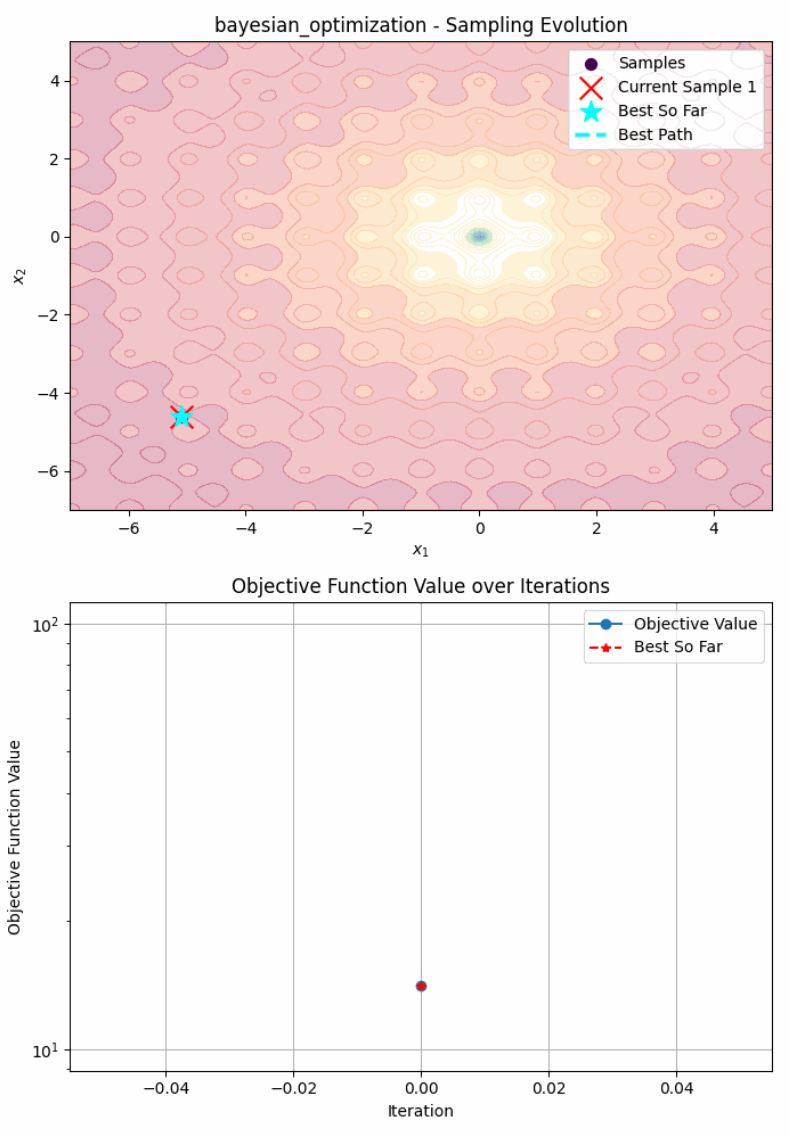


Model Based derivative free methods

Bayesian optimization

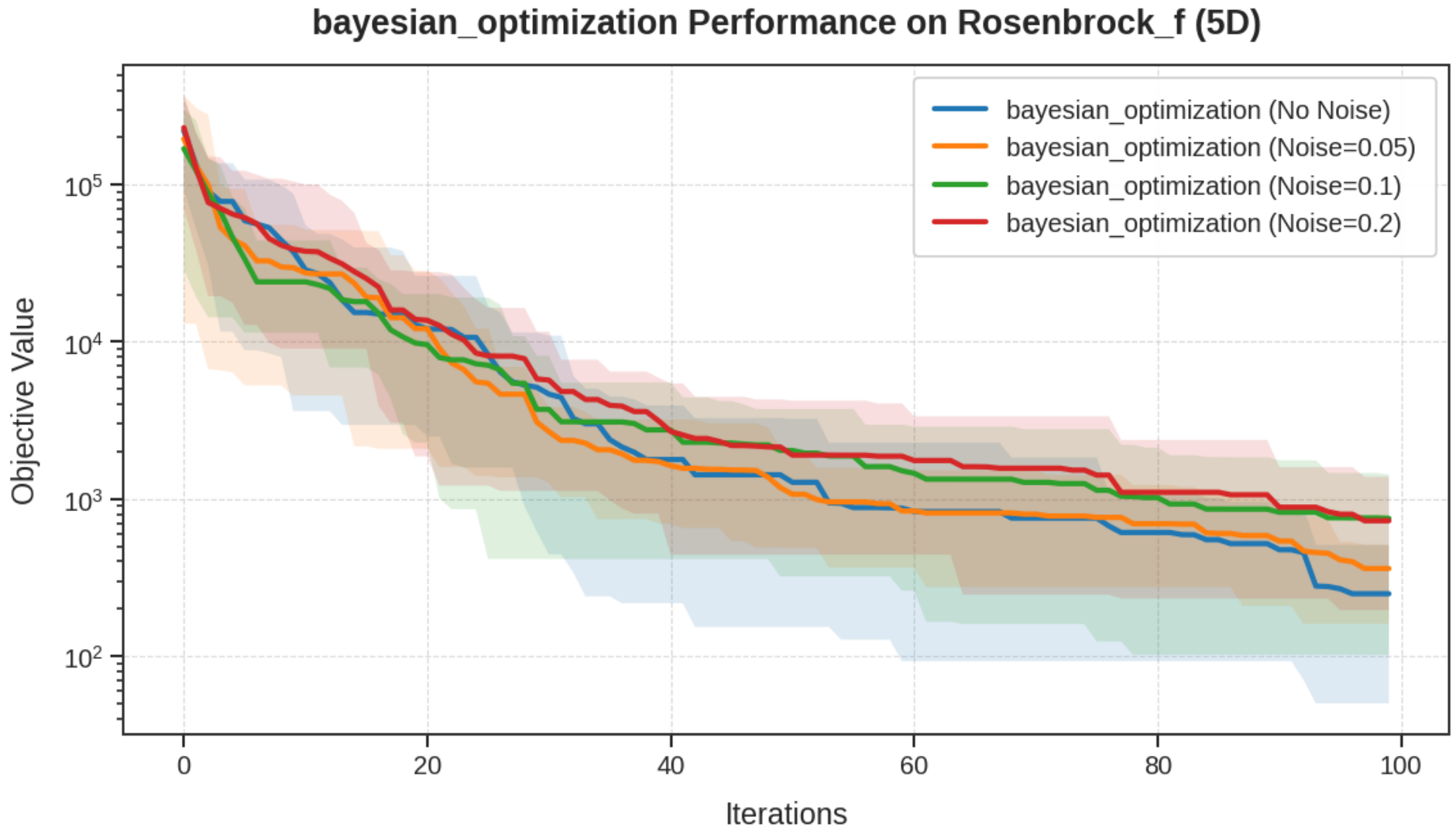






Model Based derivative free methods

Bayesian optimization



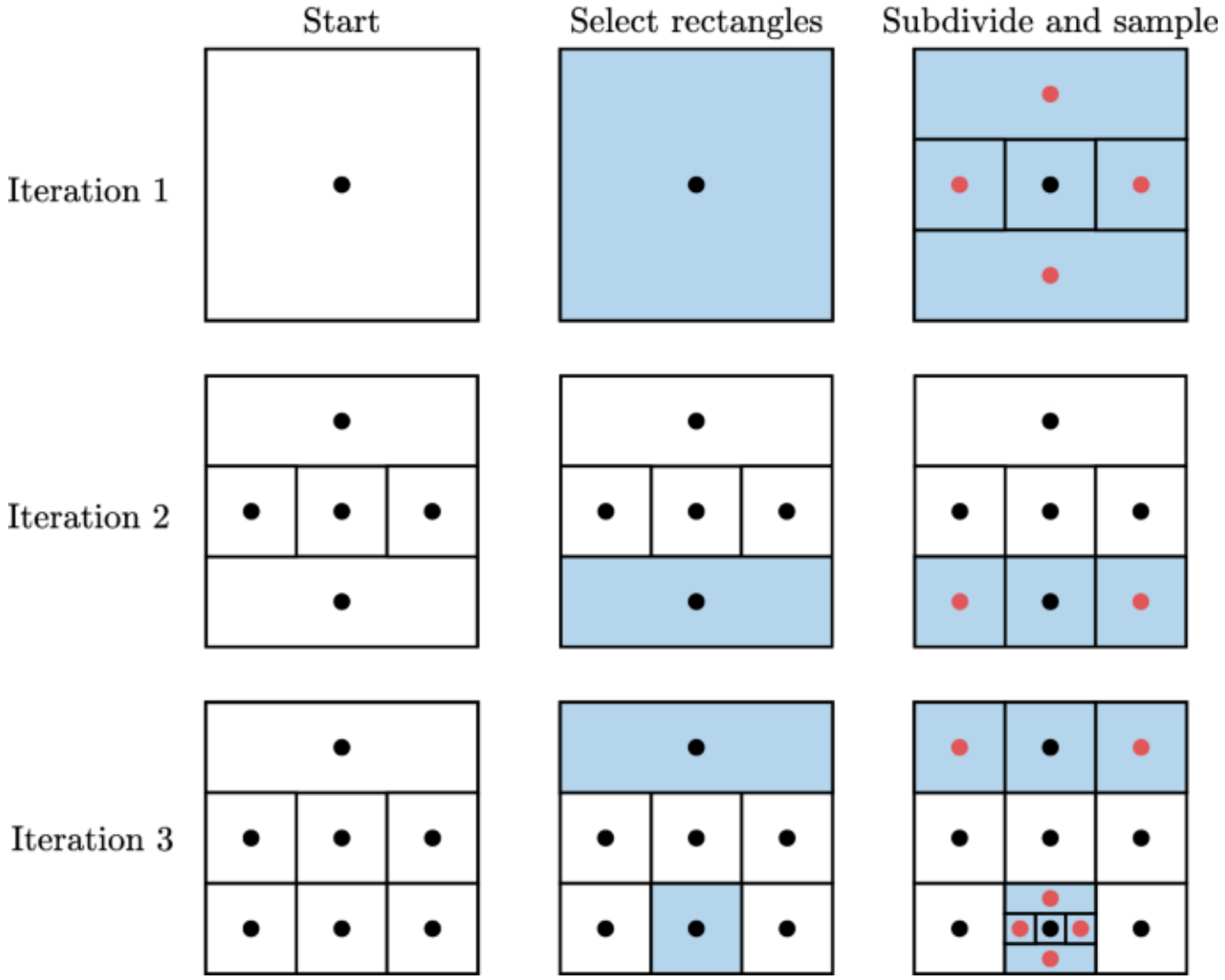
A closer look at some **Direct** algorithms

Direct derivative free methods

DIRECT global optimization algorithm

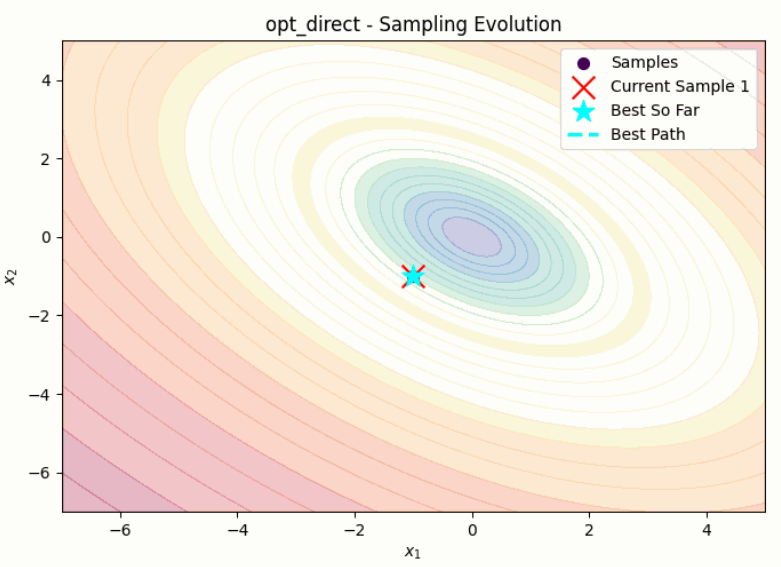
Works by partitioning the unit hypercube into subrectangles.

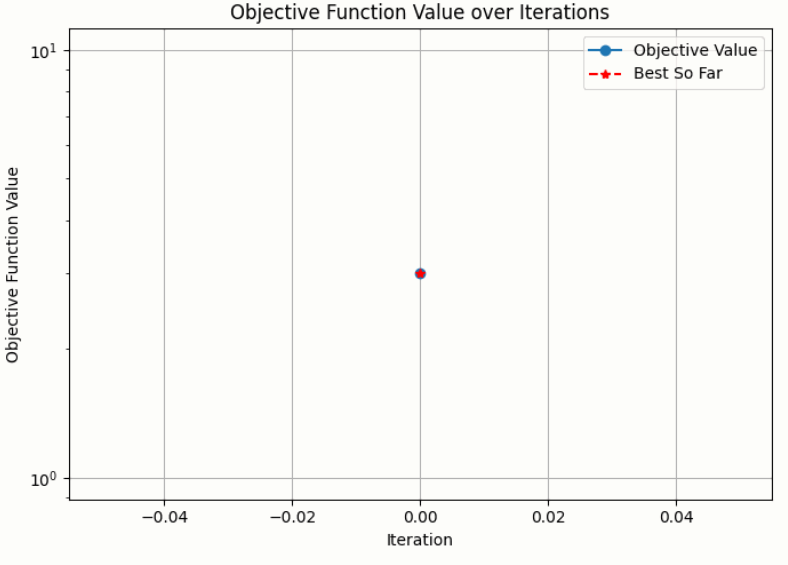
- Deterministic.
- Easy to implement.
- Balanced local and global search.
- Motivated by Lipschitzian global optimization.



Direct derivative free methods

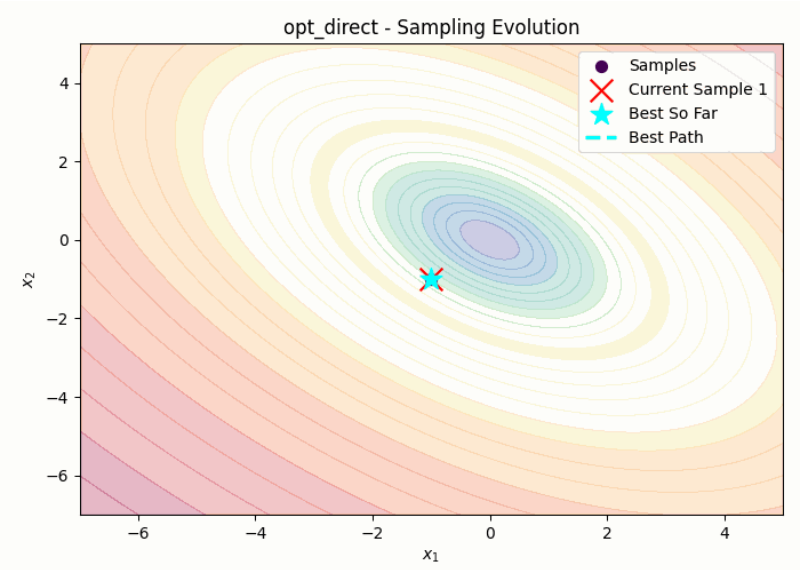
DIRECT global optimization algorithm

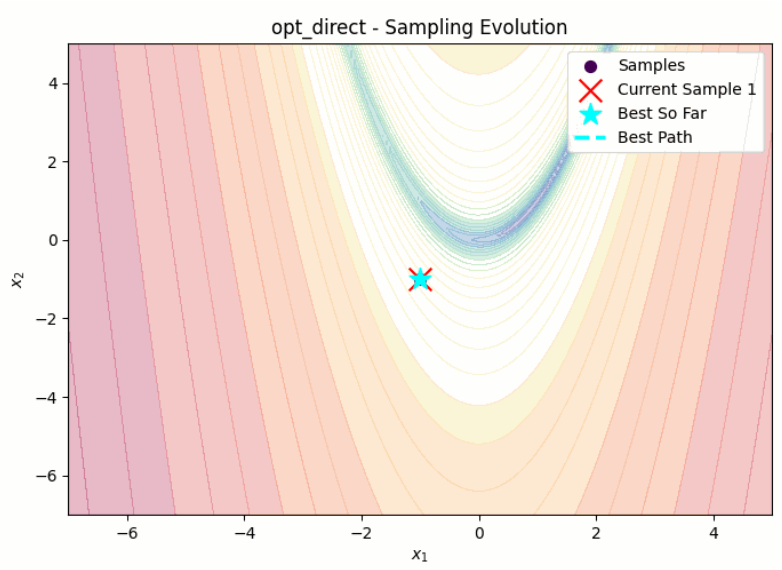


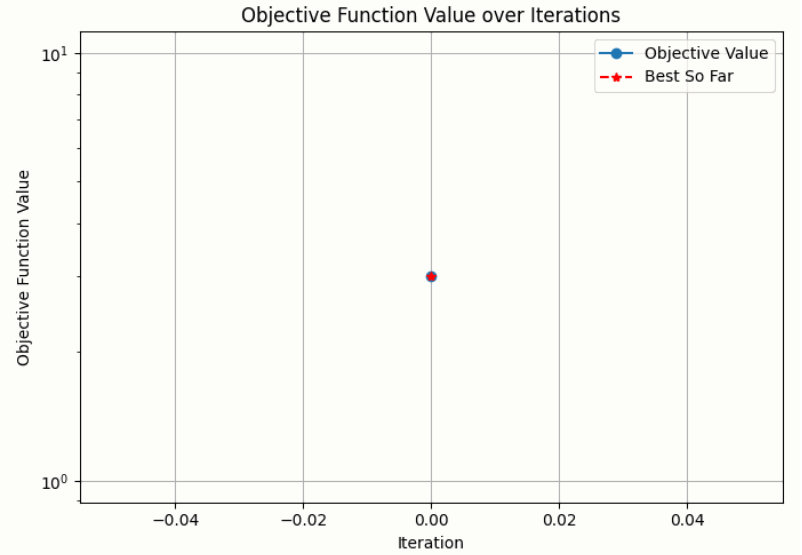


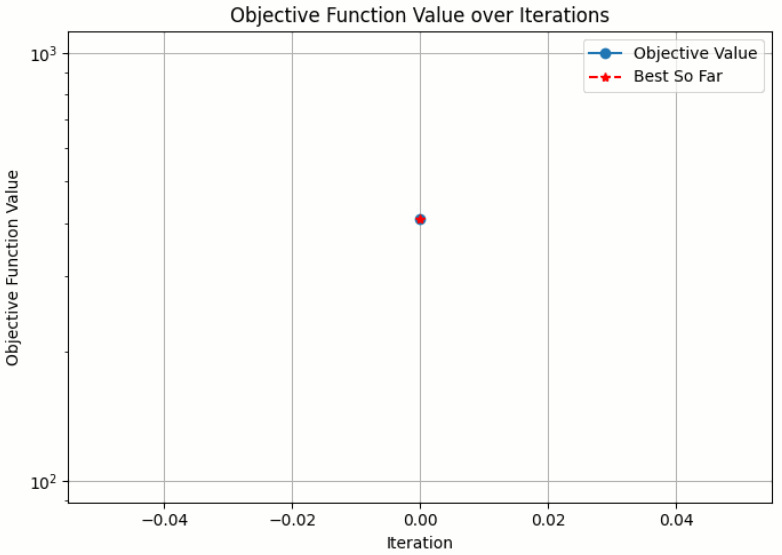
Direct derivative free methods

DIRECT global optimization algorithm



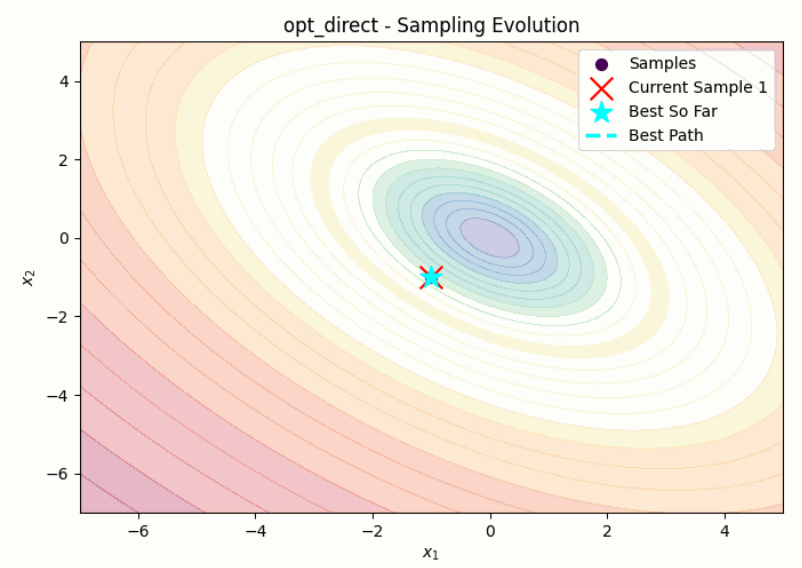


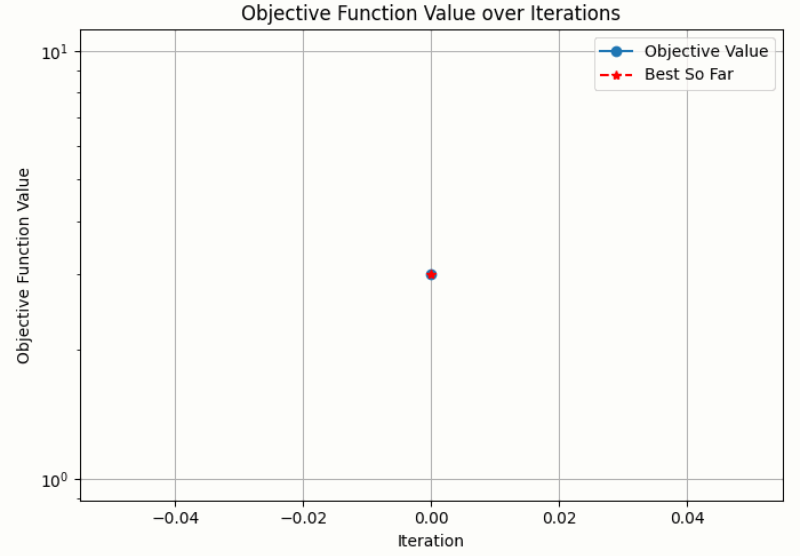


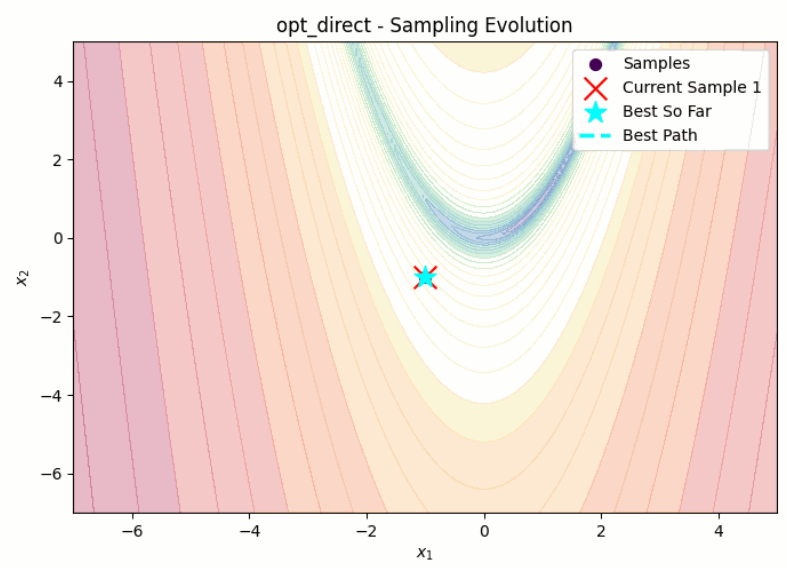


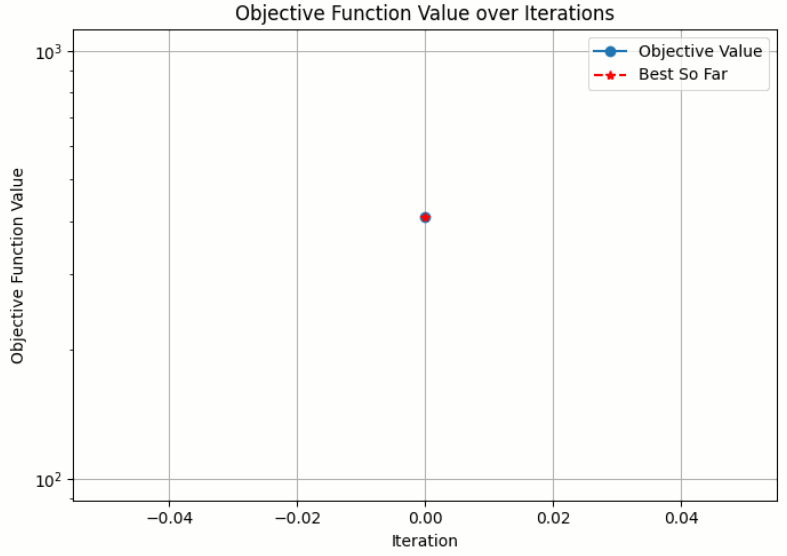
Direct derivative free methods

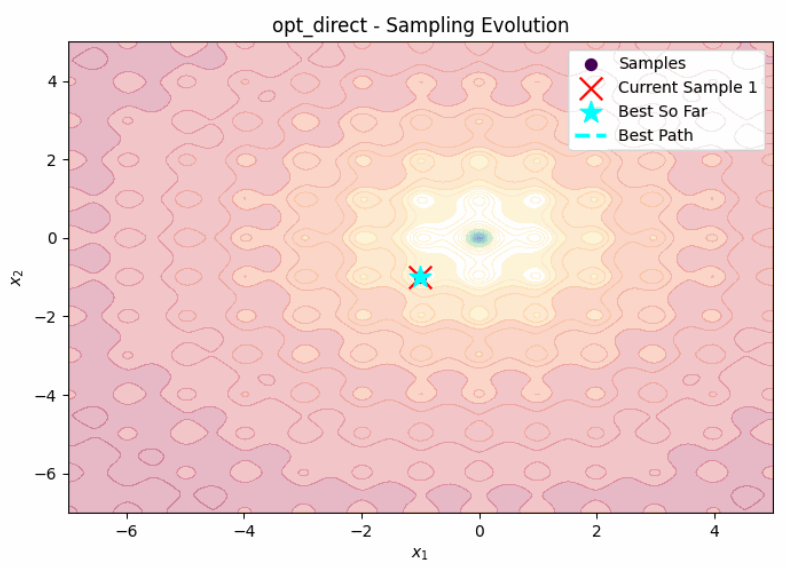
DIRECT global optimization algorithm

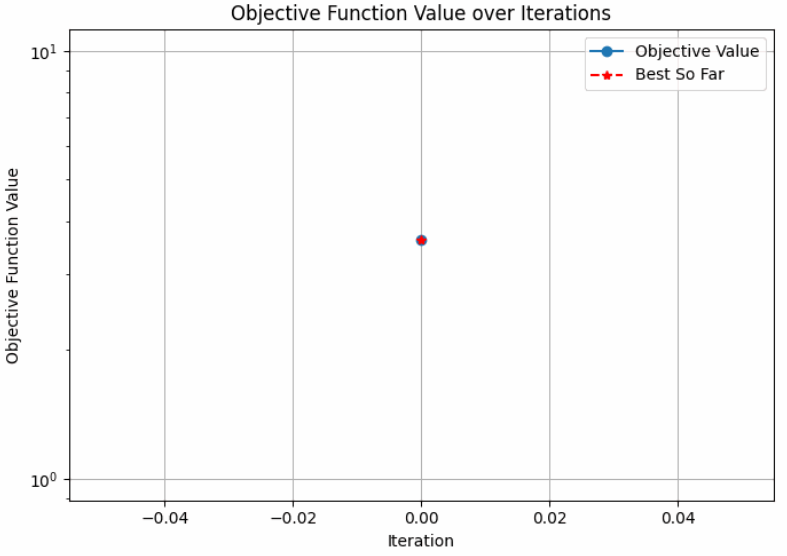






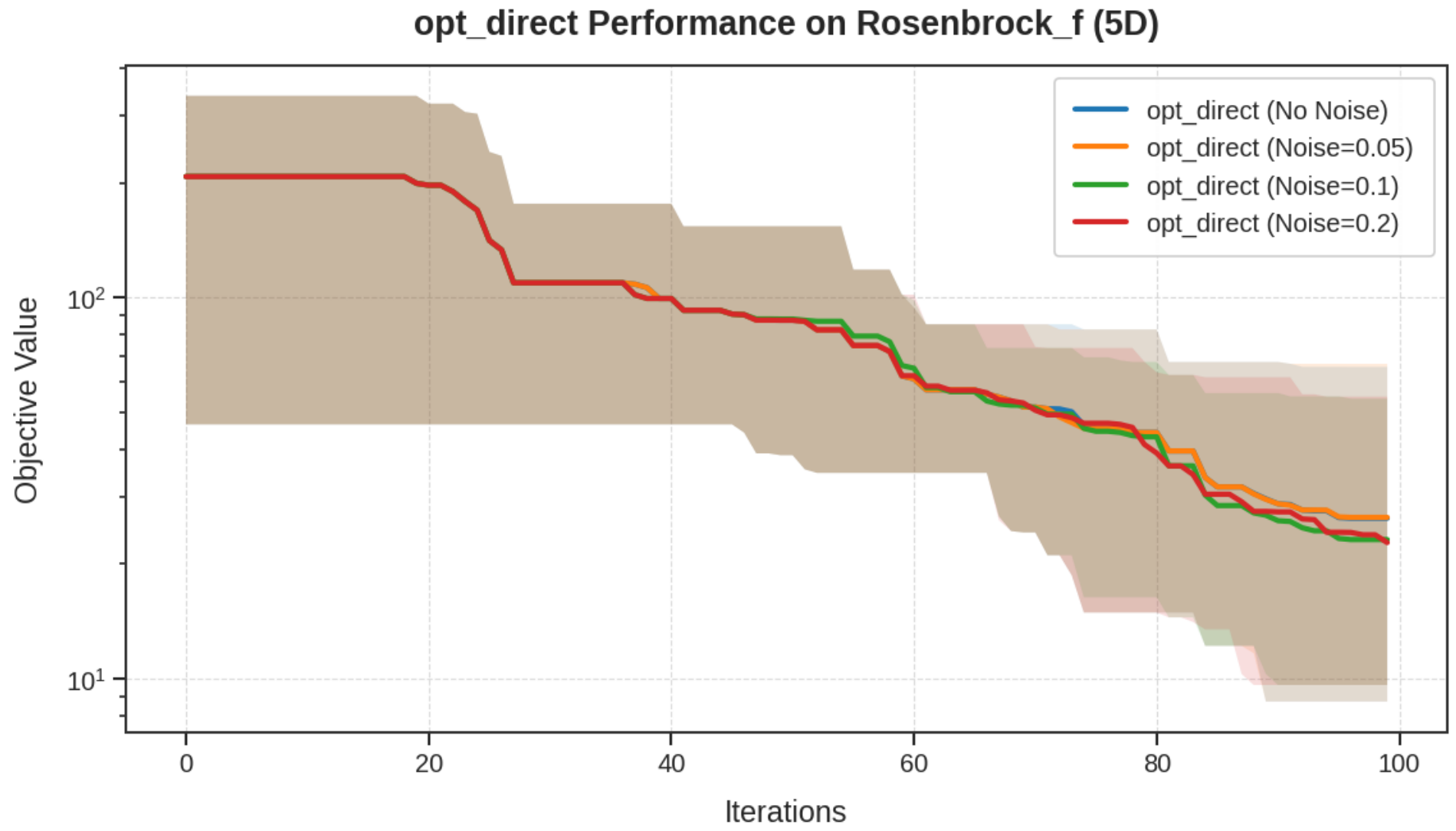






Direct derivative free methods

DIRECT global optimization algorithm



Direct derivative free methods Nelder–Mead method

Simple and efficient data-driven optimisation technique. Particularly well suited for uni-modal functions.

Evaluate $n_x + 1$ points.

1. **Order**: $f(\mathbf{x}_1) \leq f(\mathbf{x}_2) \leq \cdots \leq f(\mathbf{x}_{n_x+1})$. Calculate centroid $\mathbf{x}_0$ of all n_x points
2. **Reflection**: $\mathbf{x}_r := \mathbf{x}_0 + \alpha(\mathbf{x}_0 - \mathbf{x}_{n_x+1})$
   a) **If** $f(\mathbf{x}_1) \leq f(\mathbf{x}_r) \leq f(\mathbf{x}_{n_x})$ replace $\mathbf{x}_{n_x+1}$ with $\mathbf{x}_r$ and **Go to Step 1**
3. **Expansion**: **If** $f(\mathbf{x}_r) \leq f(\mathbf{x}_1)$, then $\mathbf{x}_e := \mathbf{x}_0 + \gamma(\mathbf{x}_r - \mathbf{x}_0)$
   a) **If** $f(\mathbf{x}_e) \leq f(\mathbf{x}_r)$ **replace** $\mathbf{x}_{n_x+1}$ with $\mathbf{x}_e$, **else replace** $\mathbf{x}_{n_x+1}$ with $\mathbf{x}_r$
4. **Contraction**: and **Shrink** **If** $\mathbf{x}_r$ and $\mathbf{x}_e$ are not good.

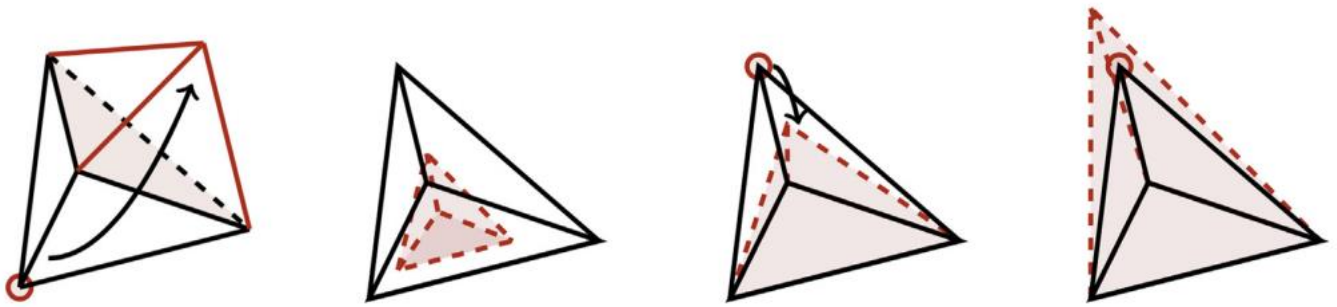
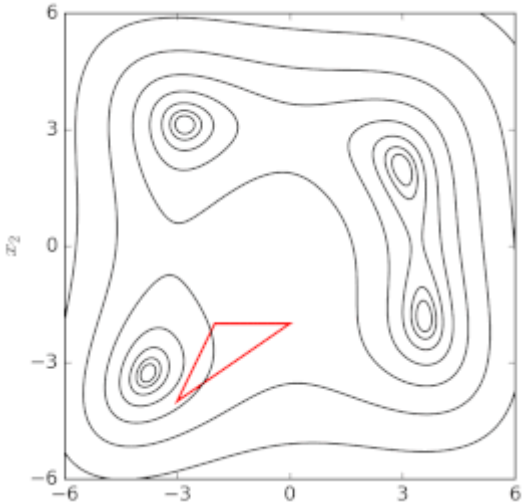
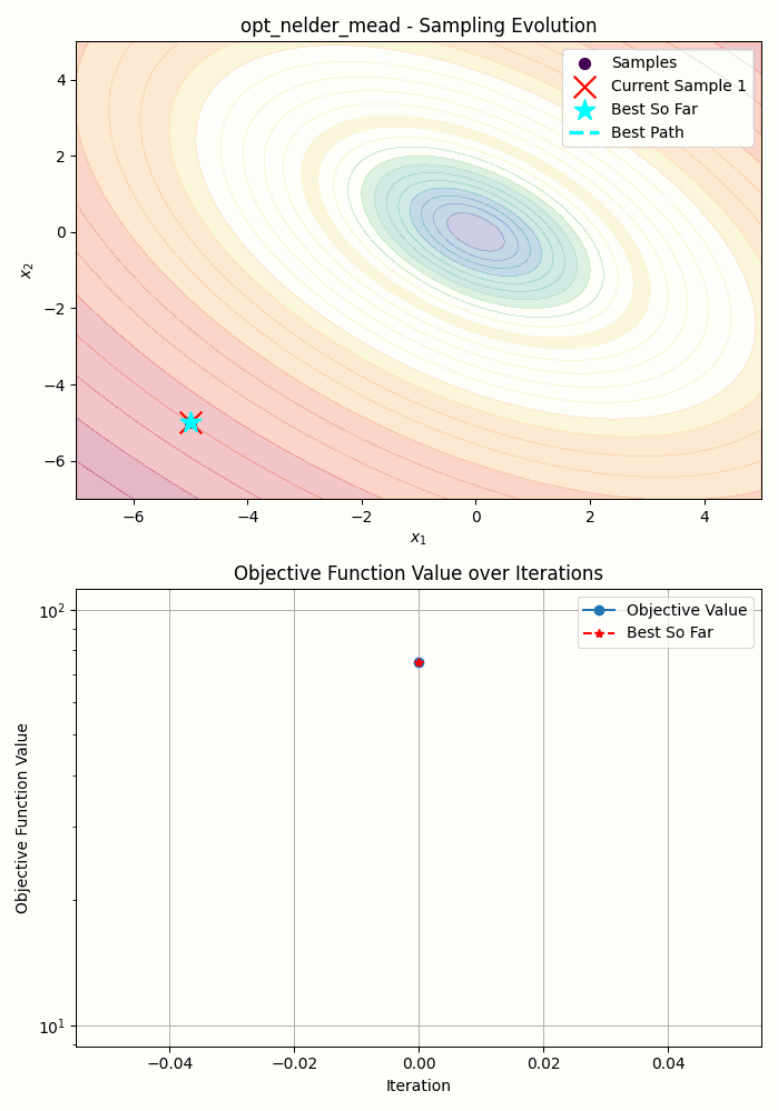


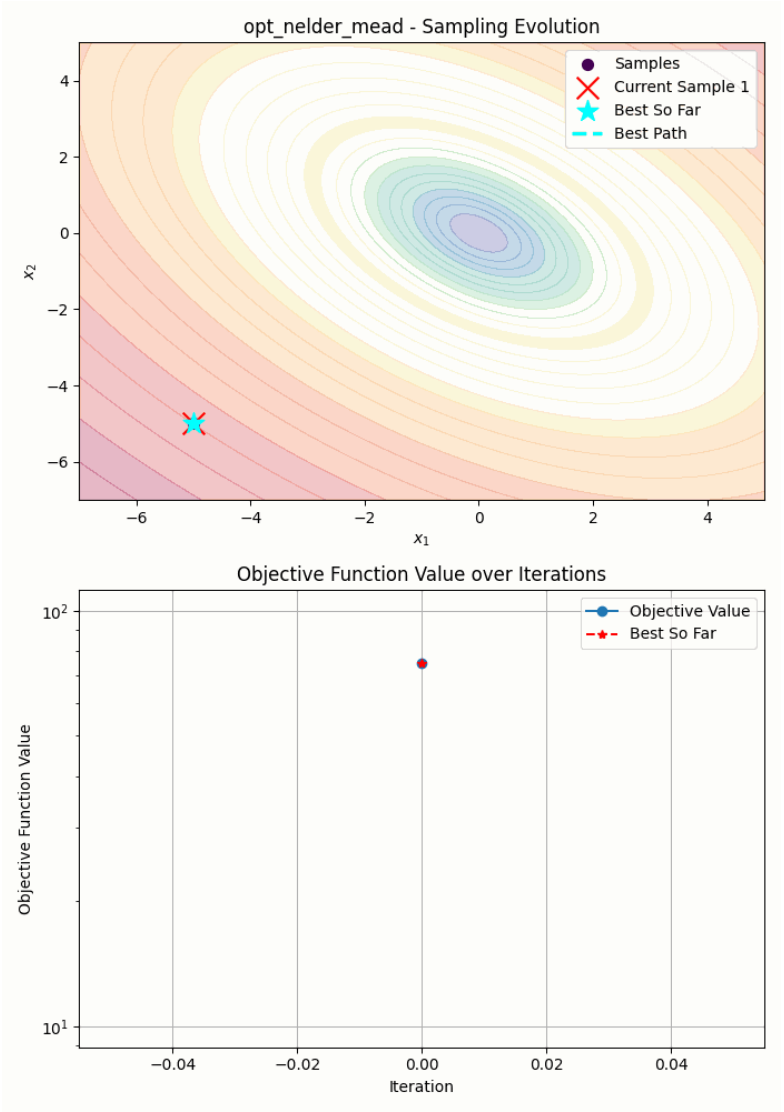
Fig. 2. The four basic simplex operations: reflect, shrink, contract and expand.

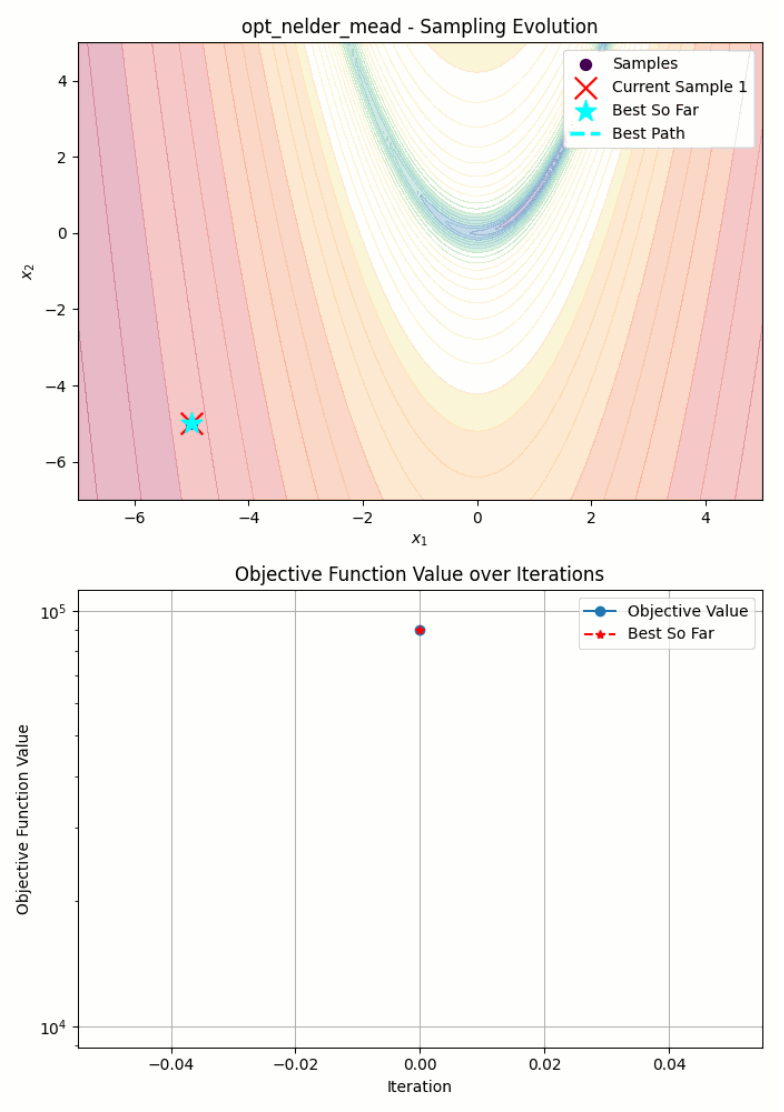


Direct derivative free methods Nelder–Mead method

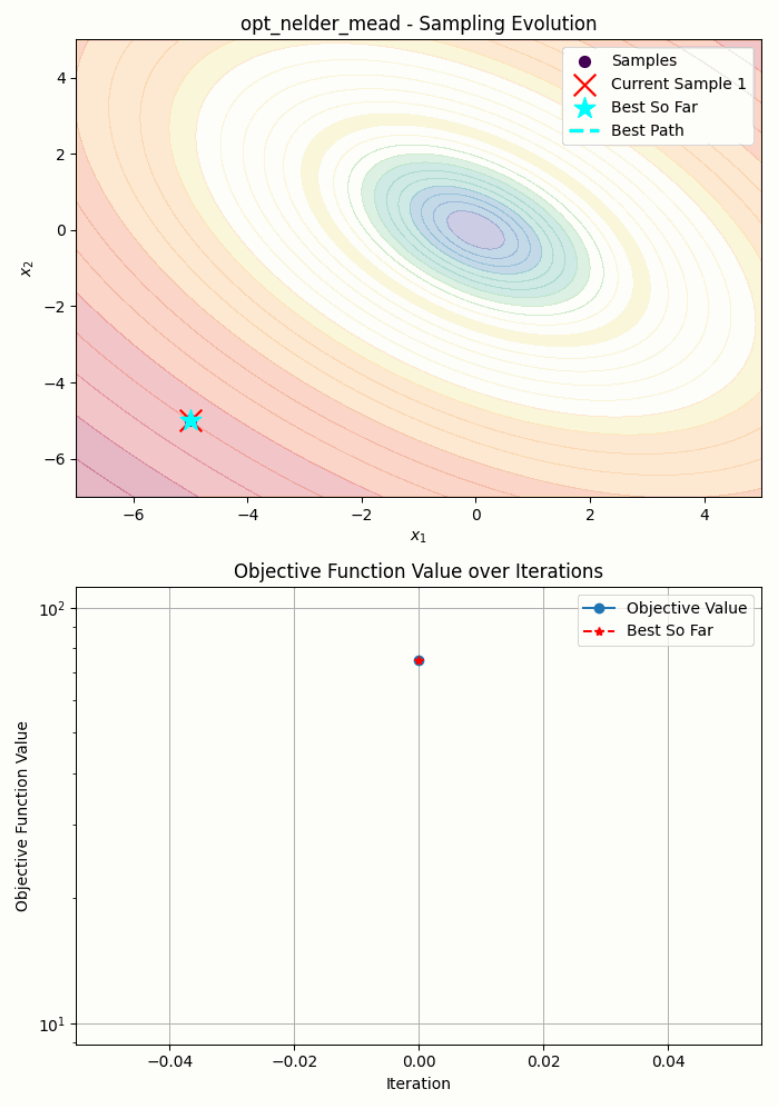


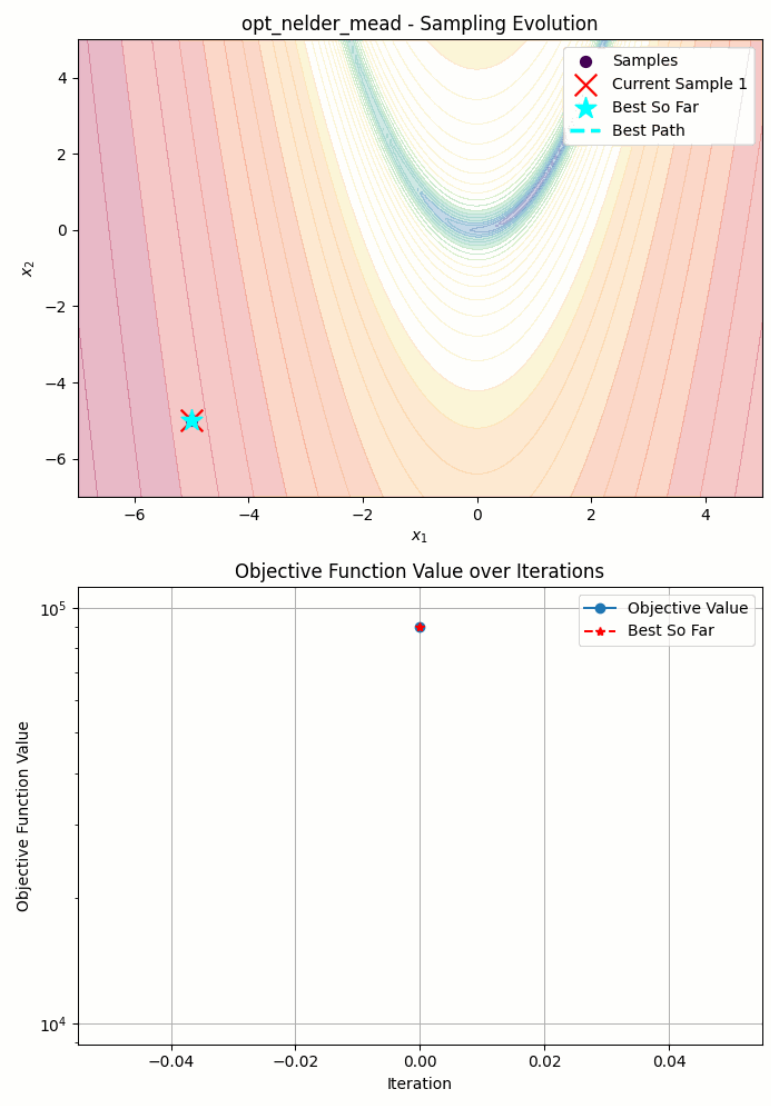
Direct derivative free methods Nelder–Mead method

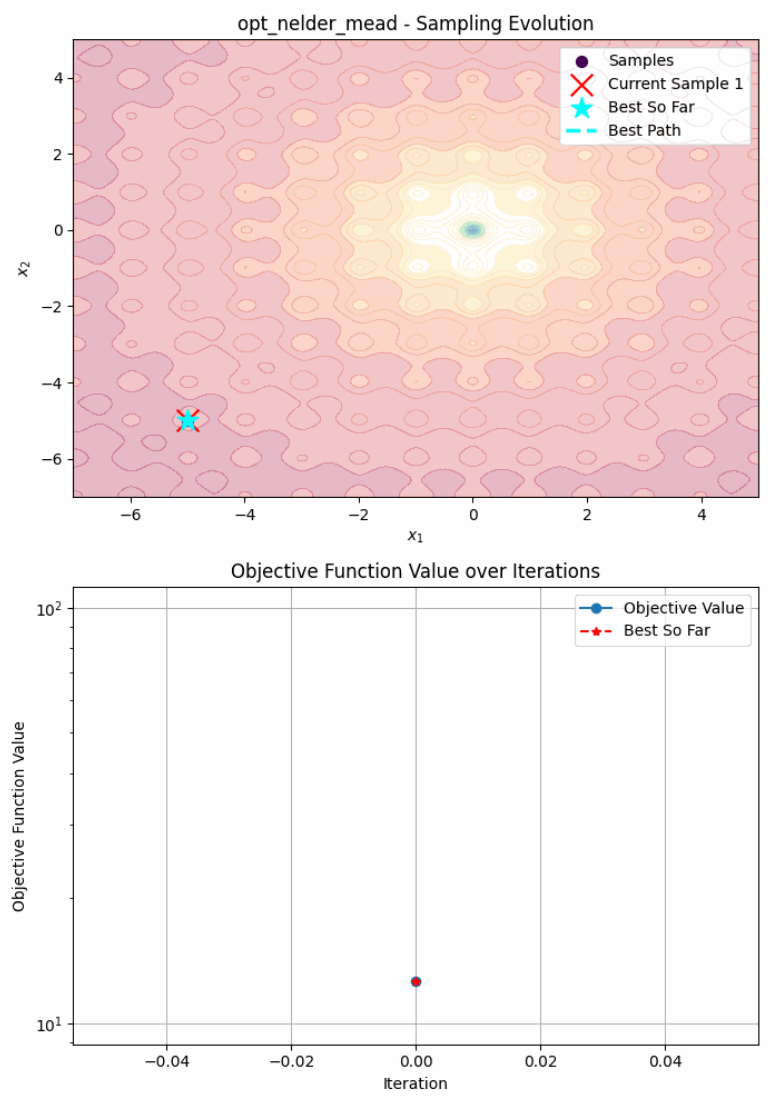




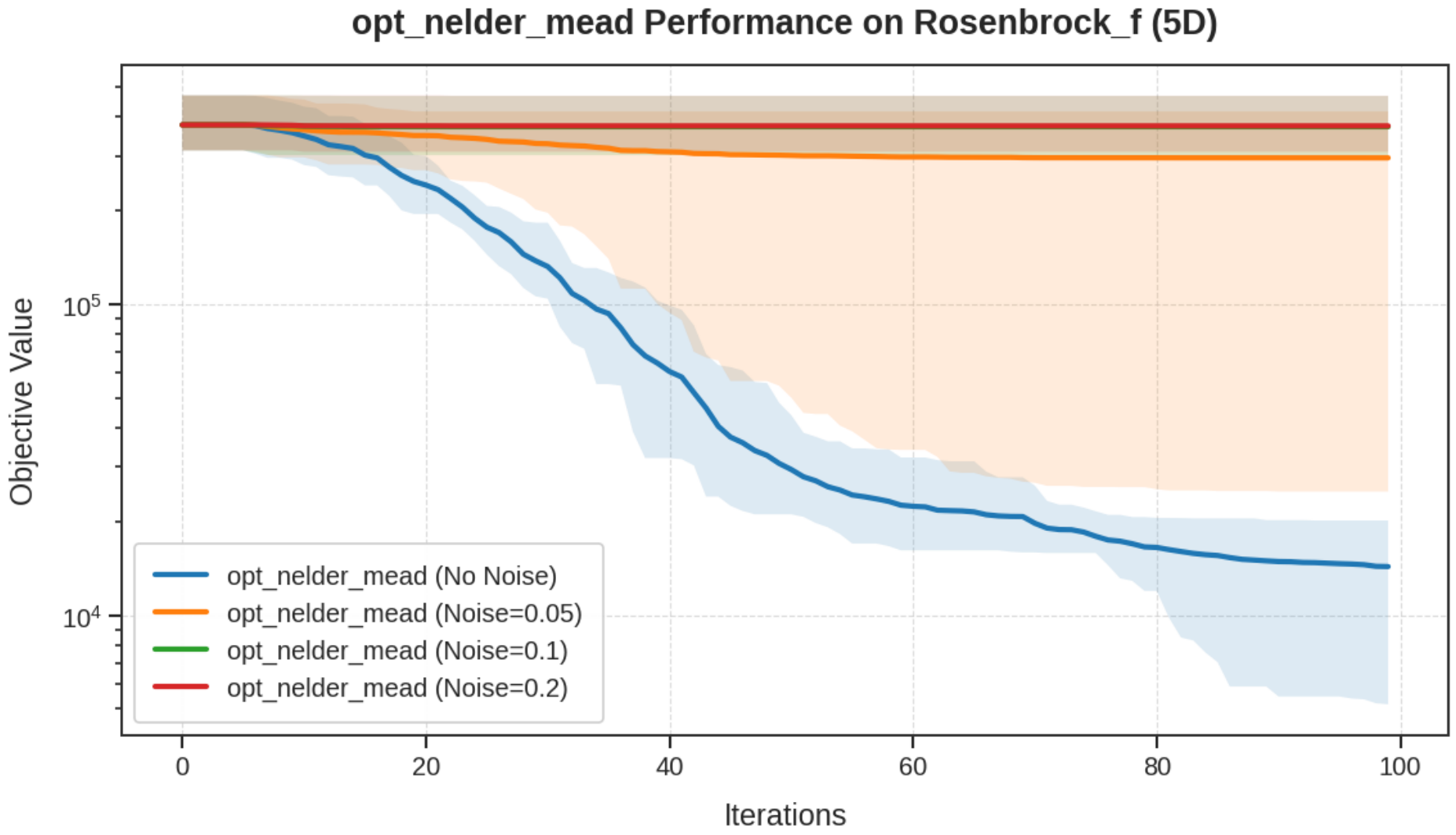
Direct derivative free methods Nelder–Mead method







Direct derivative free methods Nelder–Mead method



Direct derivative free methods

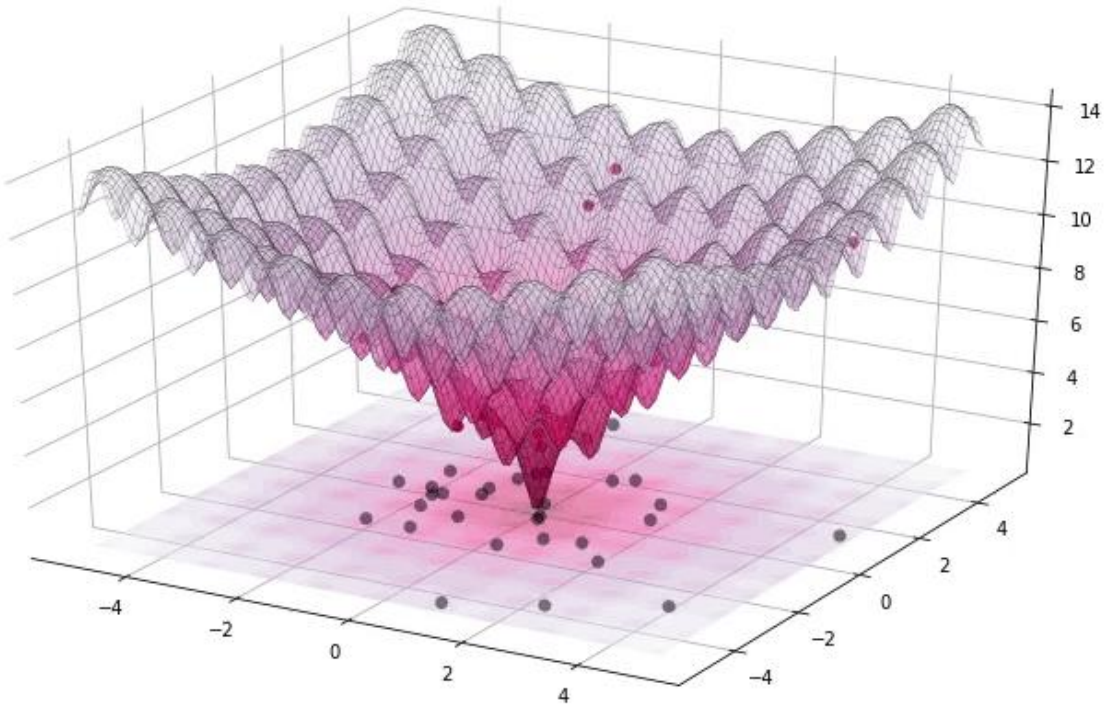
Differential evolution

Initialize: Random population $\{\mathbf{x}_1, \ldots, \mathbf{x}_{n_p}\}$ in $\mathbb{R}^{n_x}$
while not converged **do**
  for each individual $\mathbf{x}_i$ **do**
    Mutation: pick $\mathbf{x}_a, \mathbf{x}_b, \mathbf{x}_c \neq \mathbf{x}_i$
      $\mathbf{v}_i = \mathbf{x}_a + F \cdot (\mathbf{x}_b - \mathbf{x}_c)$
    Crossover: for $j = 1$ to n_x

$$u_{i,j} = \begin{cases} v_{i,j}, & \text{if rand} < CR \text{ or } j = j_{\text{rand}} \\ x_{i,j}, & \text{otherwise} \end{cases}$$

    Selection:
      if $f(\mathbf{u}_i) \leq f(\mathbf{x}_i)$ then $\mathbf{x}_i \leftarrow \mathbf{u}_i$
  end for
end while
Return: best solution $\mathbf{x}^*$

* $F \in [0,2]$



Direct derivative free methods

Differential evolution

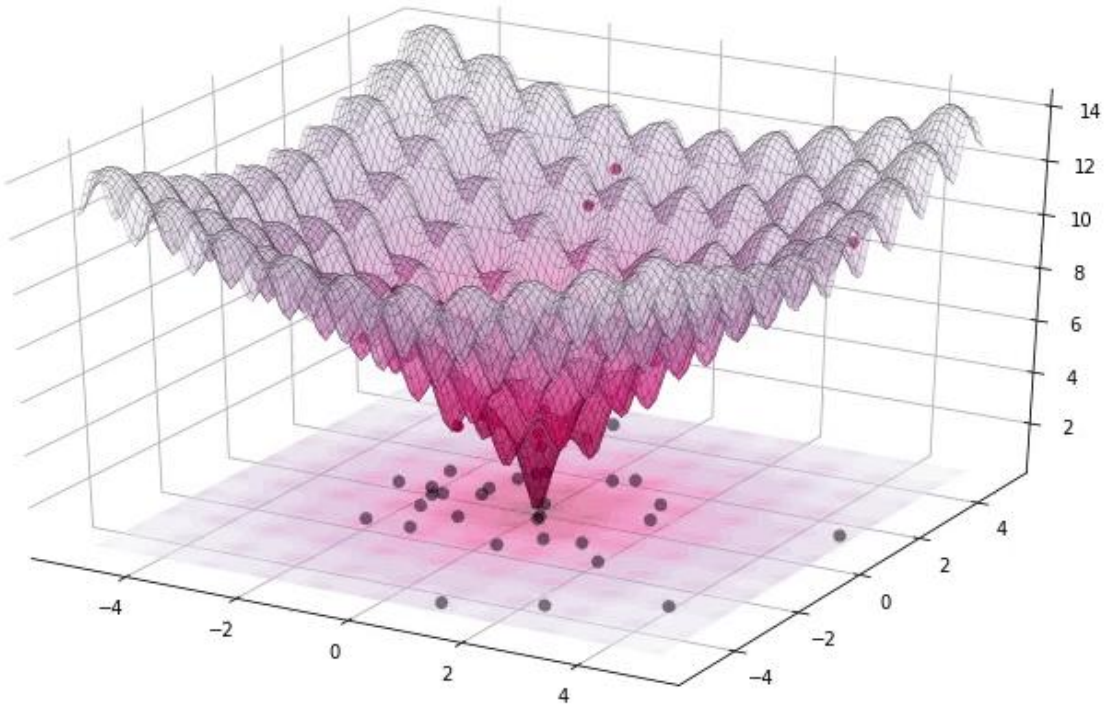
$\mathbf{x}_a$=[1,3,5] $\mathbf{x}_b$=[2,4,6] $\mathbf{x}_c$=[7,8,9]

Initialize: Random population $\{\mathbf{x}_1, \ldots, \mathbf{x}_{n_p}\}$ in $\mathbb{R}^{n_x}$
while not converged **do**
  for each individual $\mathbf{x}_i$ **do**
    Mutation: pick $\mathbf{x}_a, \mathbf{x}_b, \mathbf{x}_c \neq \mathbf{x}_i$
      $\mathbf{v}_i = \mathbf{x}_a + F \cdot (\mathbf{x}_b - \mathbf{x}_c)$
    Crossover: for $j = 1$ to n_x

$$u_{i,j} = \begin{cases} v_{i,j}, & \text{if rand} < CR \text{ or } j = j_{\text{rand}} \\ x_{i,j}, & \text{otherwise} \end{cases}$$

    Selection:
      if $f(\mathbf{u}_i) \leq f(\mathbf{x}_i)$ then $\mathbf{x}_i \leftarrow \mathbf{u}_i$
  end for
end while
Return: best solution $\mathbf{x}^*$

$\mathbf{x}_i$=[3, 5, 7]



* $F \in [0,2]$

Direct derivative free methods

Differential evolution

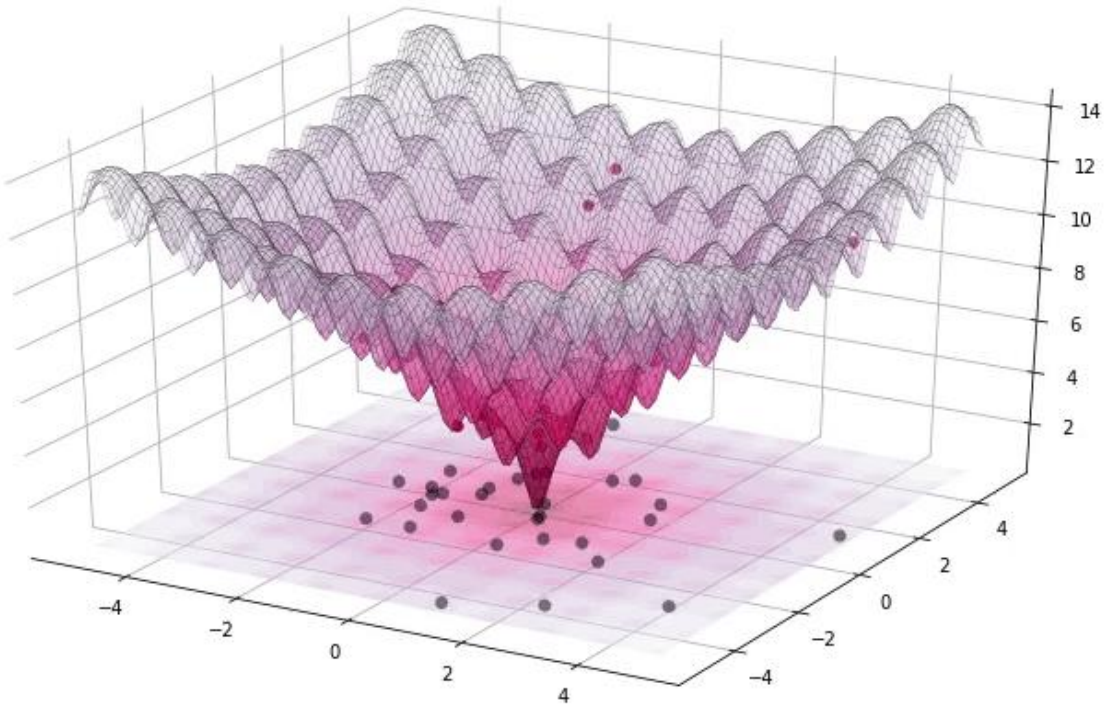
1: **Initialize:** Random population $\{\mathbf{x}_1, \ldots, \mathbf{x}_{n_p}\}$ in $\mathbb{R}^{n_x}$
2: **while** not converged **do**
3: **for** each individual $\mathbf{x}_i$ **do**
4: **Mutation:** pick $\mathbf{x}_a, \mathbf{x}_b, \mathbf{x}_c \neq \mathbf{x}_i$
5: $\mathbf{v}_i = \mathbf{x}_a + F \cdot (\mathbf{x}_b - \mathbf{x}_c)$
6: **Crossover:** for $j = 1$ to n_x
7: $u_{i,j} = \begin{cases} v_{i,j}, & \text{if rand} < CR \text{ or } j = j_{\text{rand}} \\ x_{i,j}, & \text{otherwise} \end{cases}$
8: **Selection:**
9: if $f(\mathbf{u}_i) \leq f(\mathbf{x}_i)$ then $\mathbf{x}_i \leftarrow \mathbf{u}_i$
10: **end for**
11: **end while**
12: **Return:** best solution $\mathbf{x}^*$

* $F \in [0,2]$

$\mathbf{x}_a$=[1,3,5] $\mathbf{x}_b$=[2,4,6] $\mathbf{x}_c$=[7,8,9]

$\mathbf{v}_i$=[-4,-1, 2]

$\mathbf{x}_i$=[3, 5, 7]



Direct derivative free methods

Differential evolution

1: **Initialize:** Random population $\{\mathbf{x}_1, \ldots, \mathbf{x}_{n_p}\}$ in $\mathbb{R}^{n_x}$
2: **while** not converged **do**
3: **for** each individual $\mathbf{x}_i$ **do**
4: **Mutation:** pick $\mathbf{x}_a, \mathbf{x}_b, \mathbf{x}_c \neq \mathbf{x}_i$
5: $\mathbf{v}_i = \mathbf{x}_a + F \cdot (\mathbf{x}_b - \mathbf{x}_c)$
6: **Crossover:** for $j = 1$ to n_x
7: $u_{i,j} = \begin{cases} v_{i,j}, & \text{if rand} < CR \text{ or } j = j_{\text{rand}} \\ x_{i,j}, & \text{otherwise} \end{cases}$
8: **Selection:**
9: if $f(\mathbf{u}_i) \leq f(\mathbf{x}_i)$ then $\mathbf{x}_i \leftarrow \mathbf{u}_i$
10: **end for**
11: **end while**
12: **Return:** best solution $\mathbf{x}^*$

* $F \in [0,2]$

$\mathbf{x}_a$=[1,3,5] $\mathbf{x}_b$=[2,4,6] $\mathbf{x}_c$=[7,8,9]

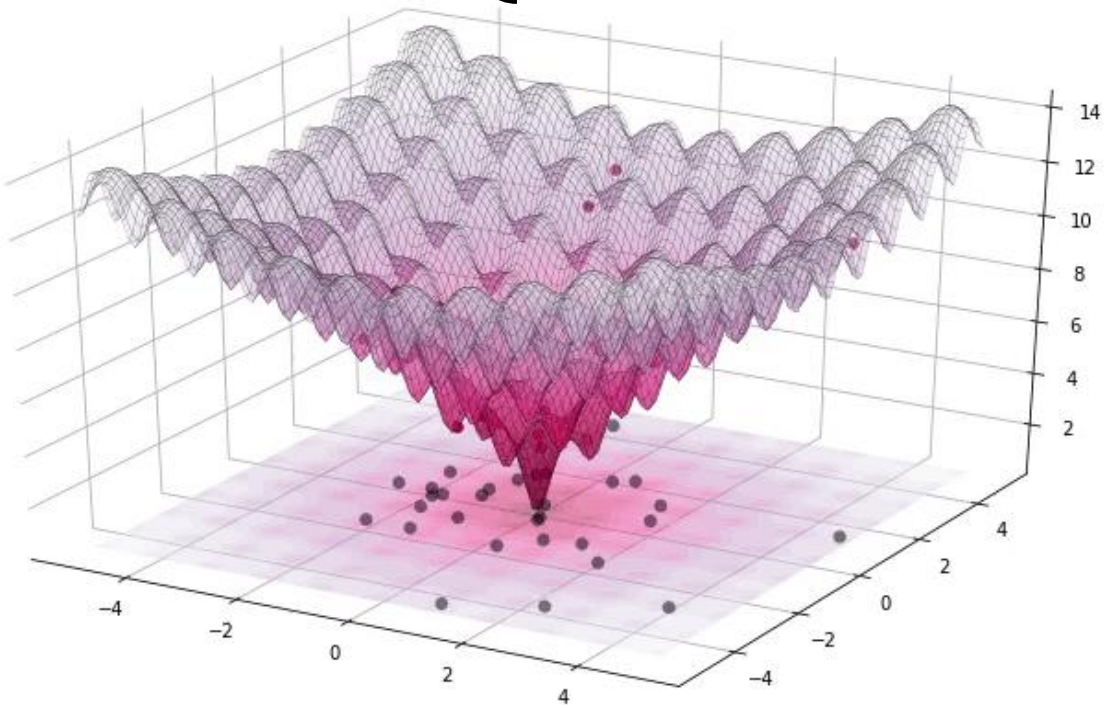
$\mathbf{v}_i$=[-4,-1, 2]

$\mathbf{x}_i$=[3, 5, 7]

$\mathbf{x}_i$=[-4, 5, 2]

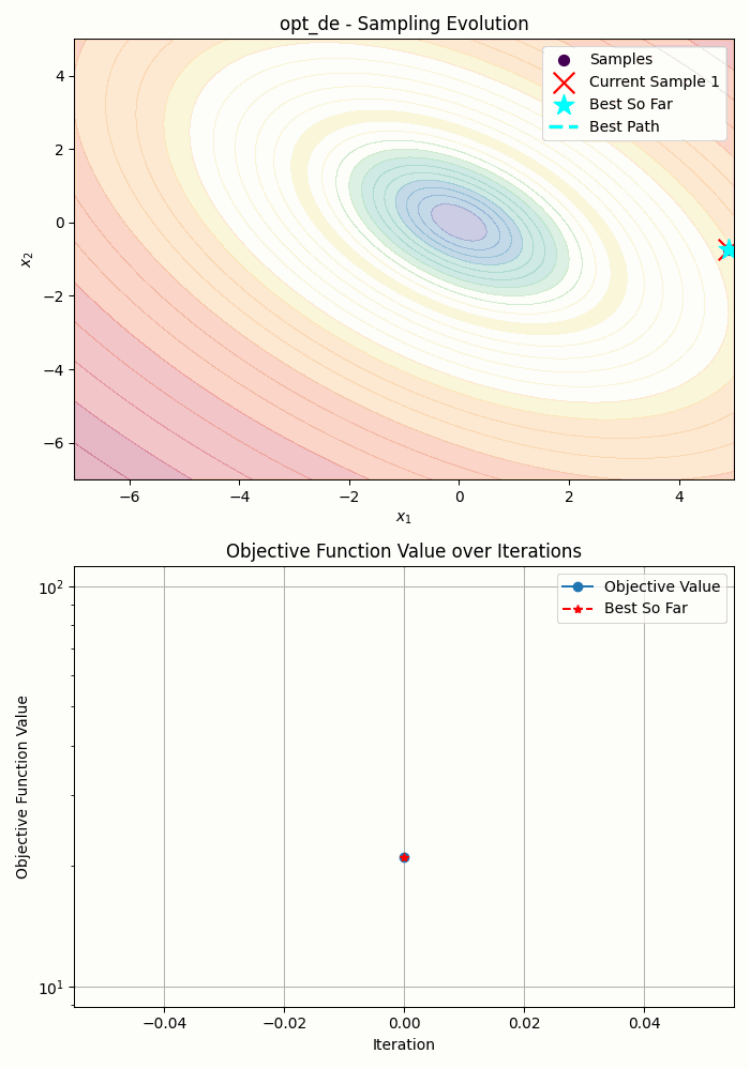
$\mathbf{x}_i$=[3, -1, 2]

$\mathbf{x}_i$=[3, -1, 7]



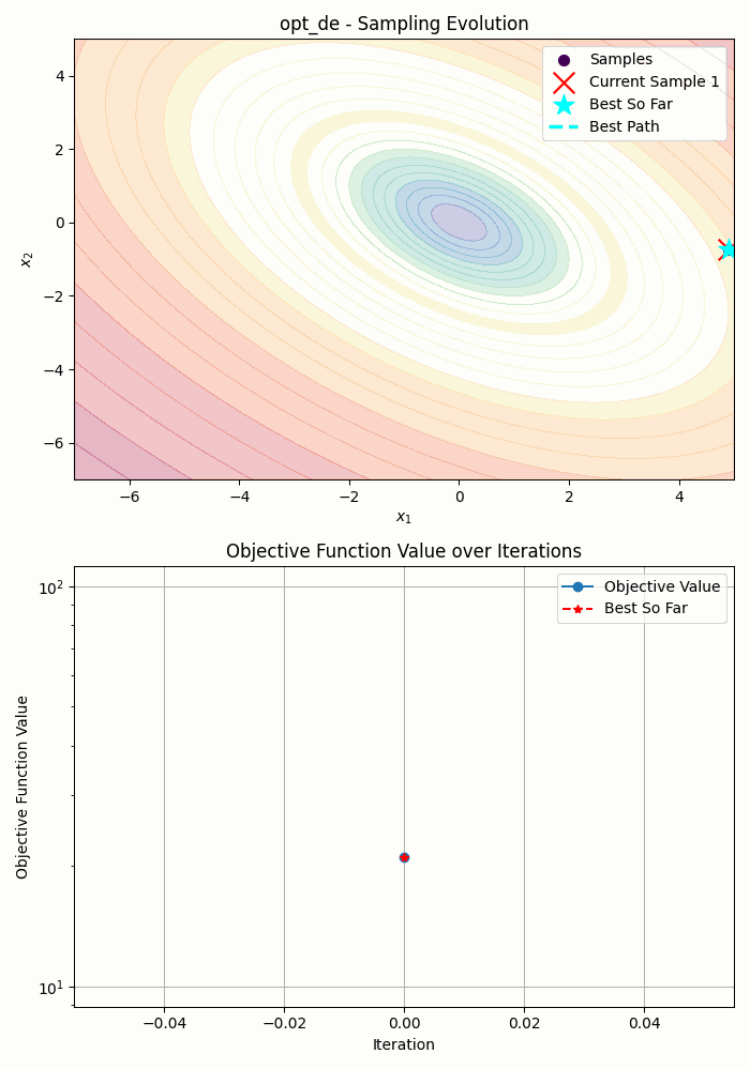
Direct derivative free methods

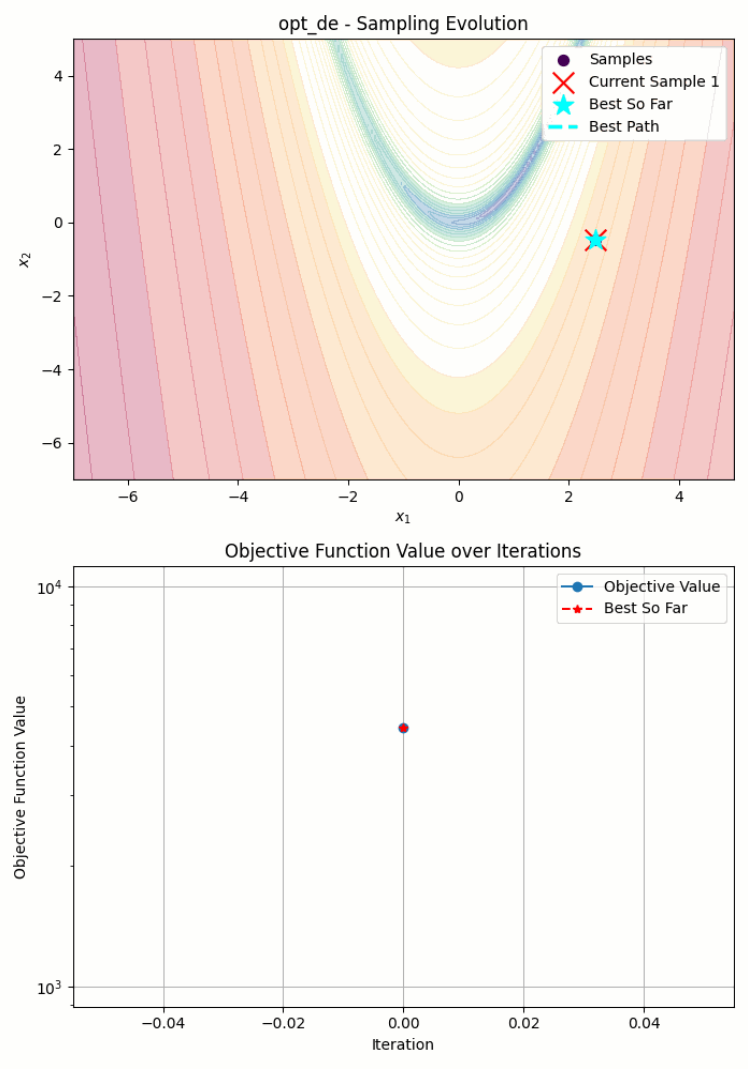
Differential Evolution



Direct derivative free methods

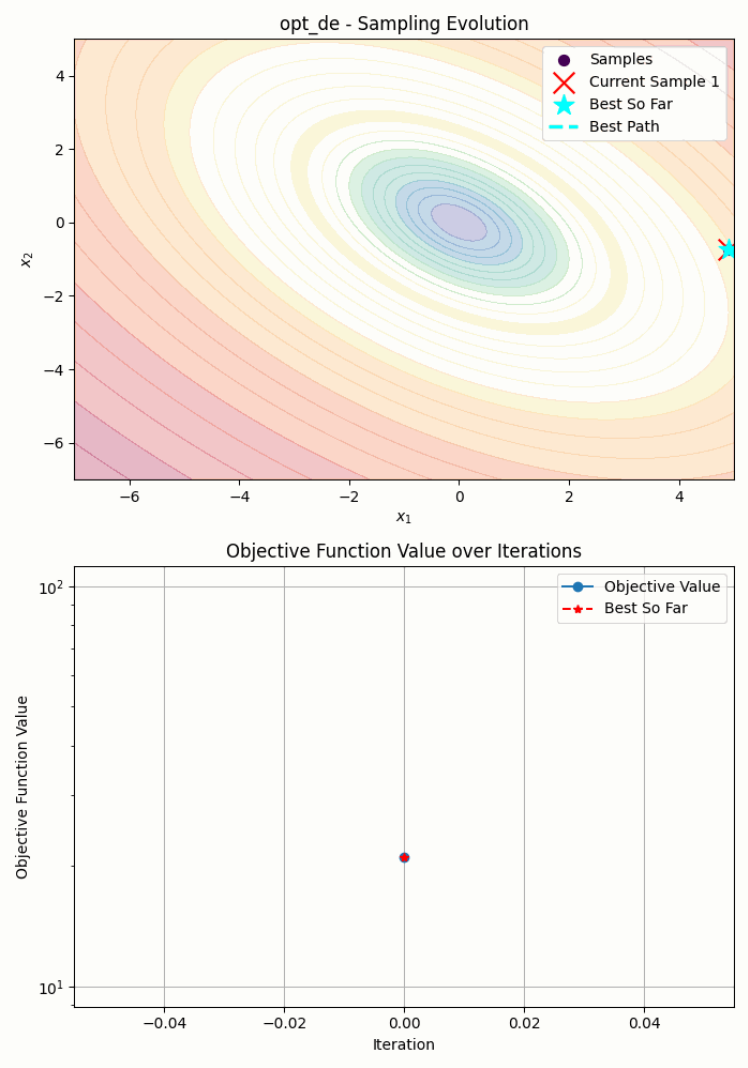
Differential Evolution

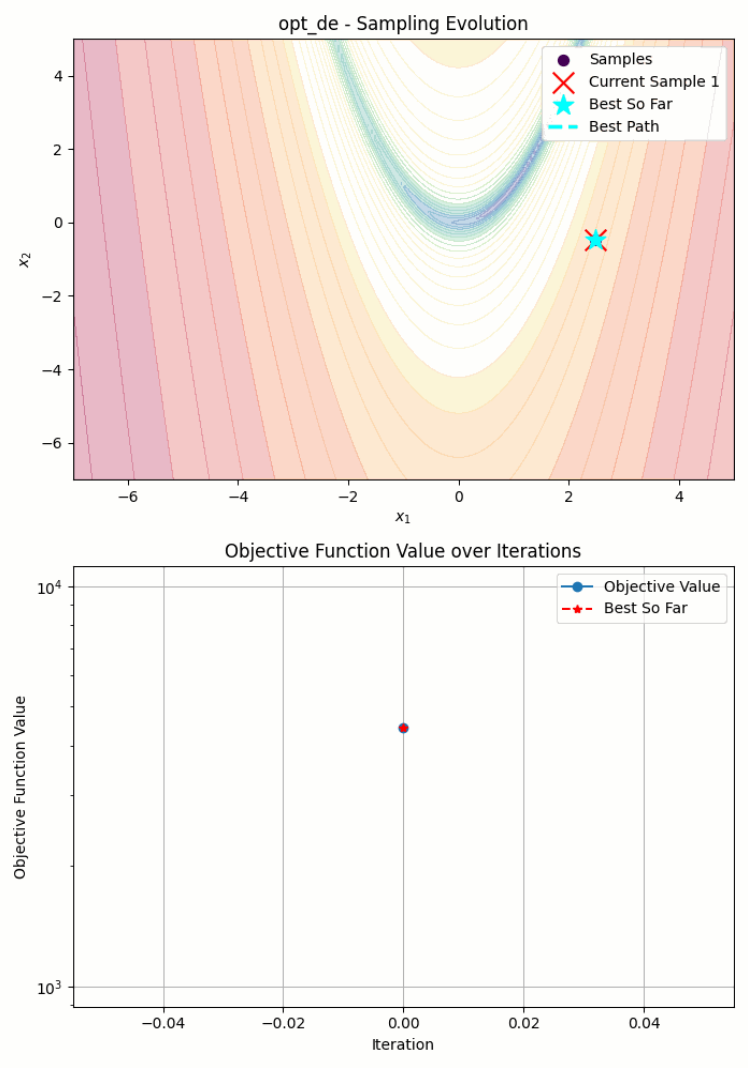


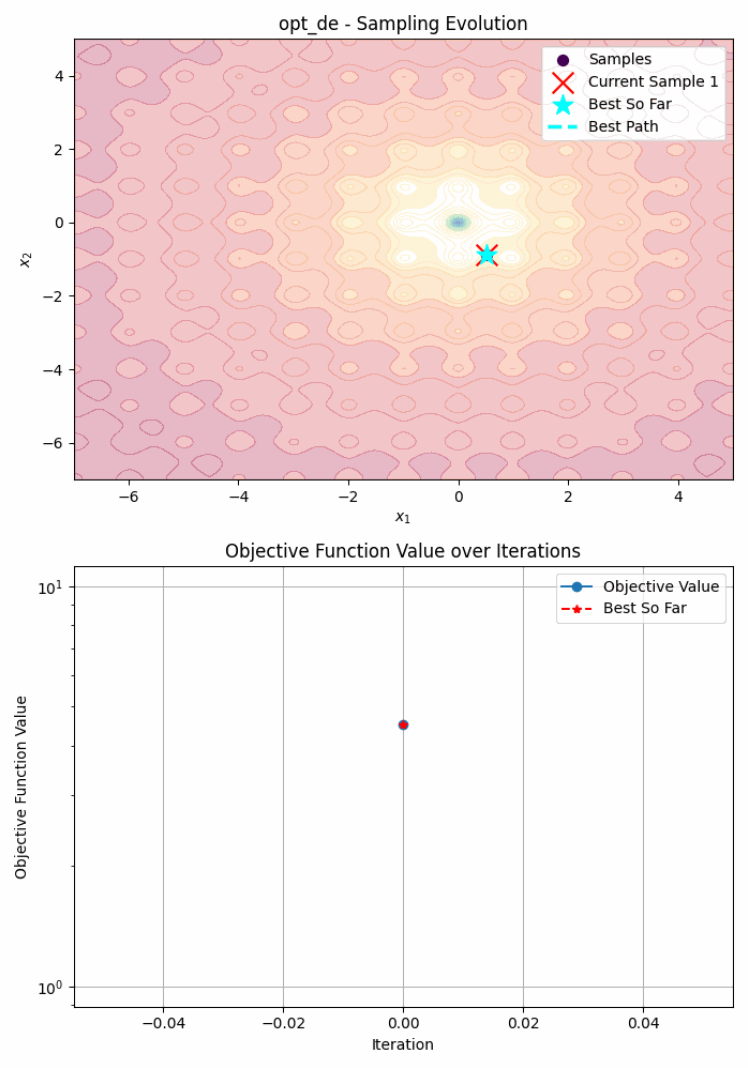


Direct derivative free methods

Differential Evolution

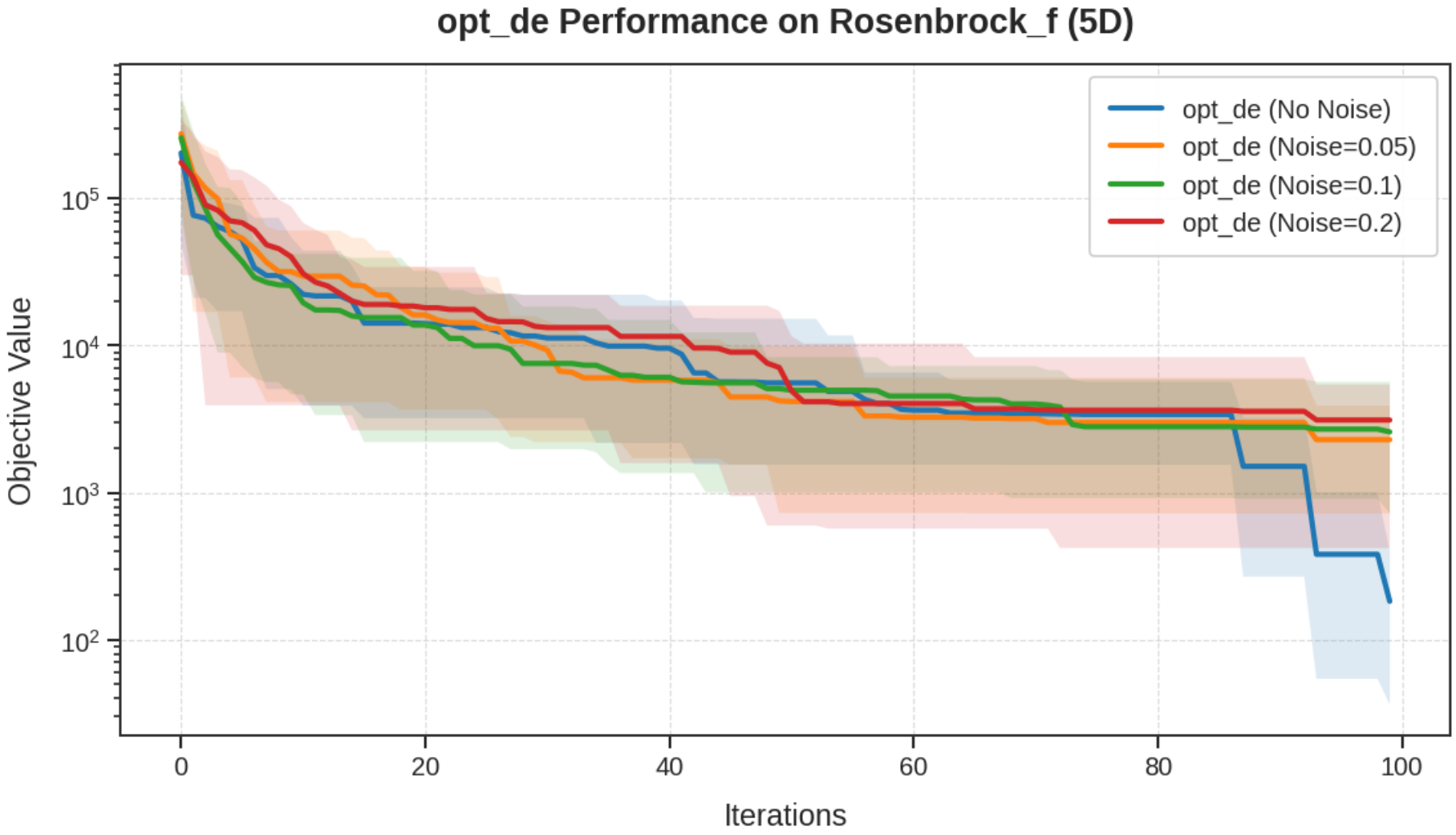






Direct derivative free methods

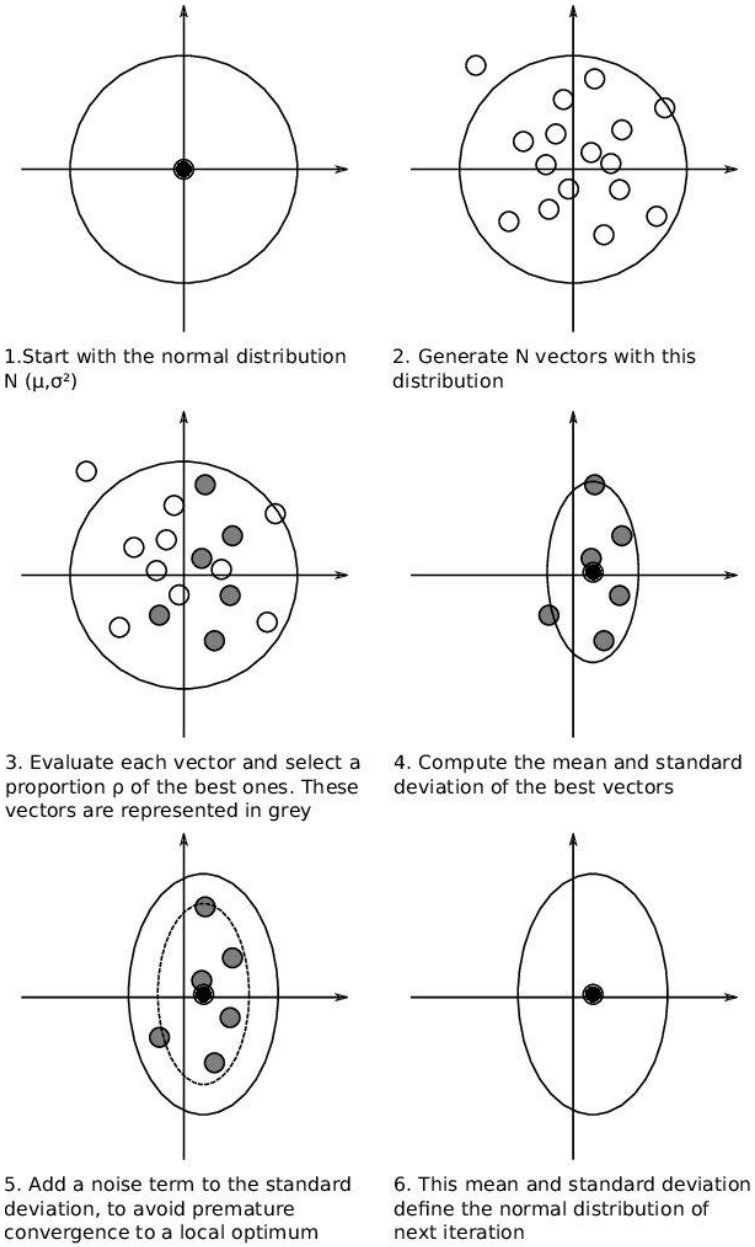
Differential Evolution



Direct derivative free methods

Cross-Entropy Method

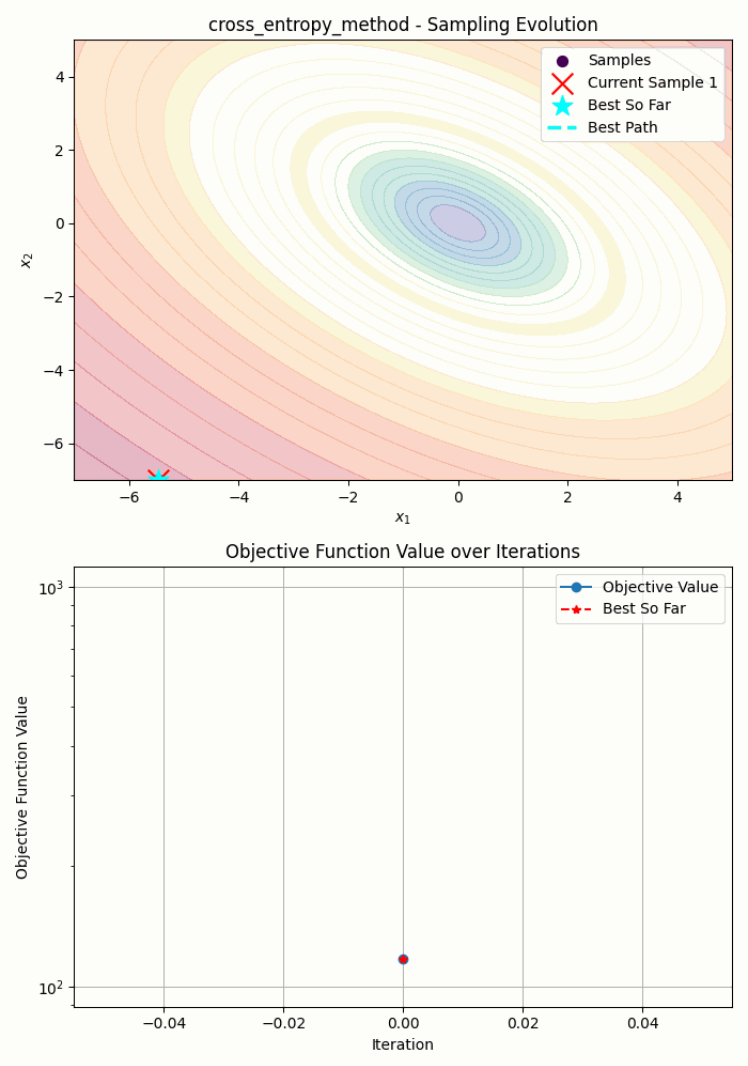
1: **Initialize:** Mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$ over $\mathbb{R}^{n_x}$
2: **while** not converged **do**
3: Sample n_p candidates $\{\mathbf{x}_1, \ldots, \mathbf{x}_{n_p}\} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$
4: Evaluate $f(\mathbf{x}_i)$ for all i
5: Select elite set $\mathcal{E} \subset \{\mathbf{x}_i\}$ with top ρ percentile
6: Update:
7: $\boldsymbol{\mu} \leftarrow \frac{1}{|\mathcal{E}|} \sum_{\mathbf{x} \in \mathcal{E}} \mathbf{x}$
8: $\boldsymbol{\Sigma} \leftarrow \frac{1}{|\mathcal{E}|} \sum_{\mathbf{x} \in \mathcal{E}} (\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top$
9: **end while**
10: **Return:** best solution $\mathbf{x}^*$



From: 10.1145/2001576.2001743

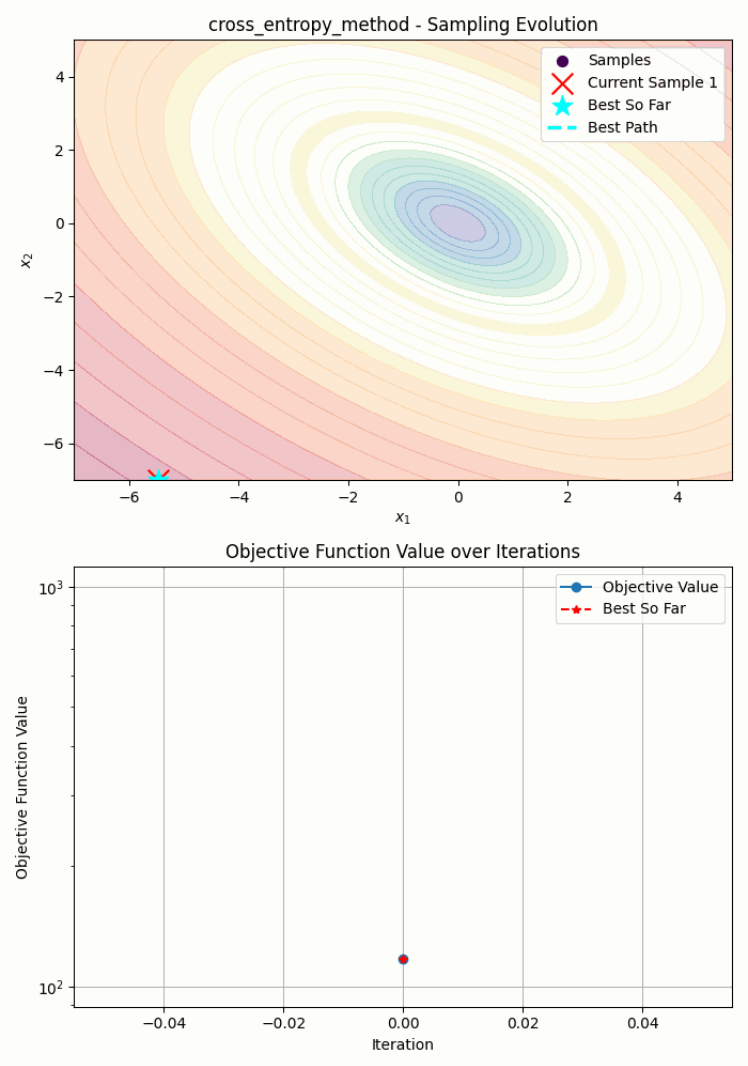
Direct derivative free methods

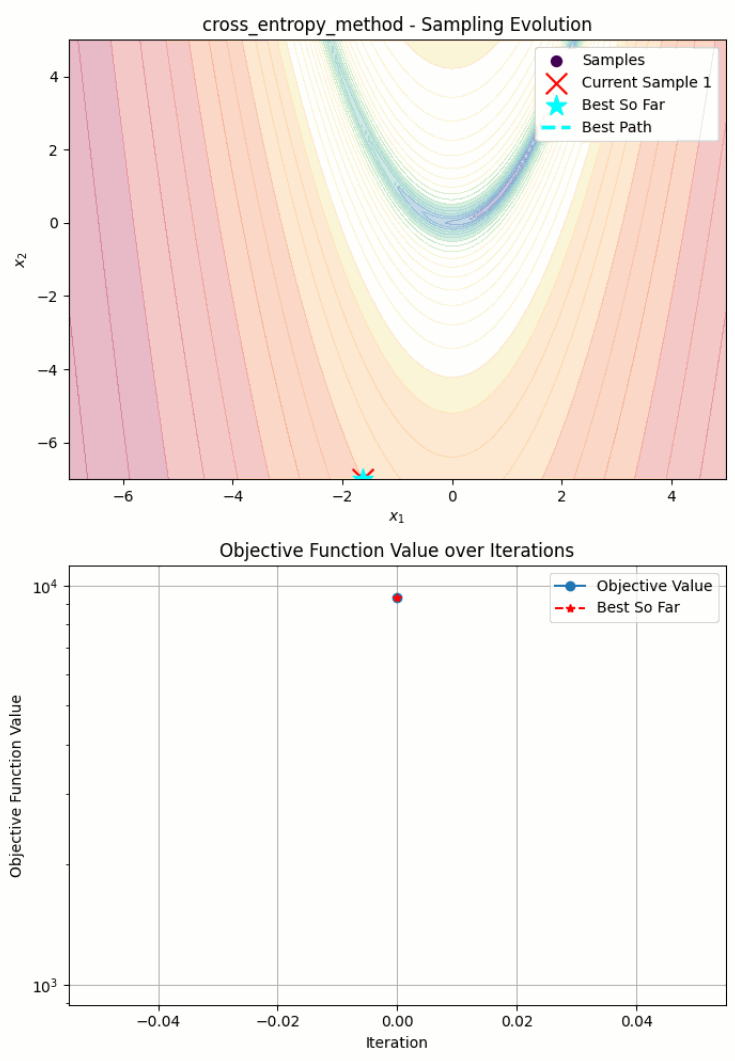
Cross-Entropy Method



Direct derivative free methods

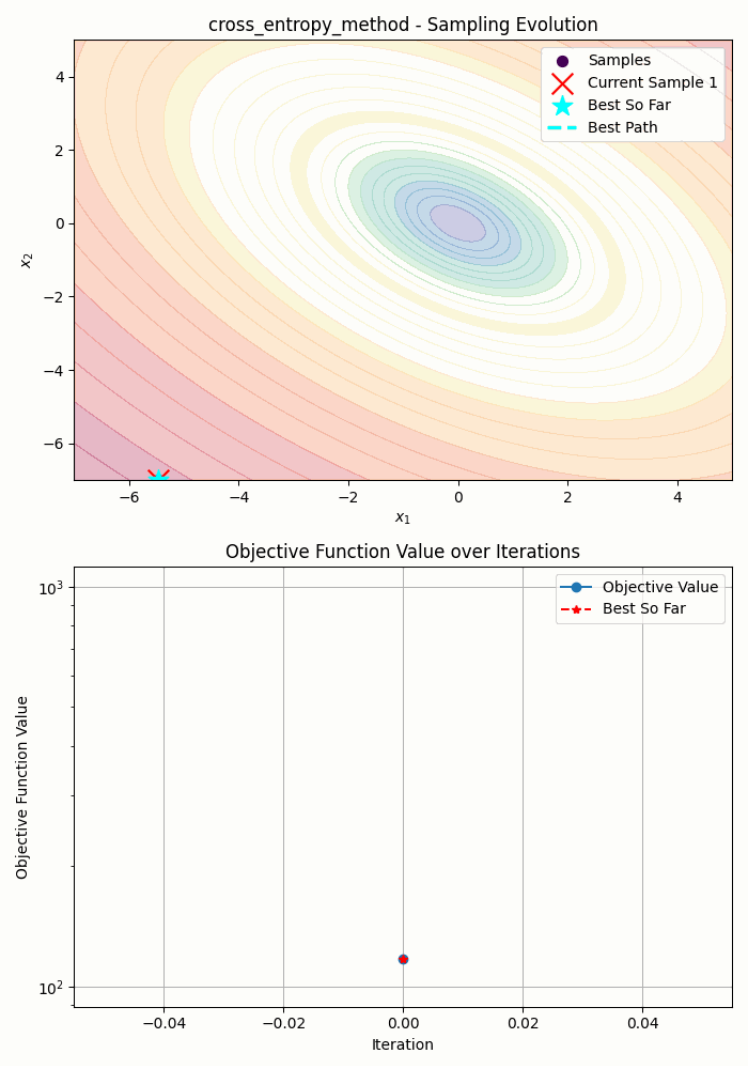
Cross-Entropy Method

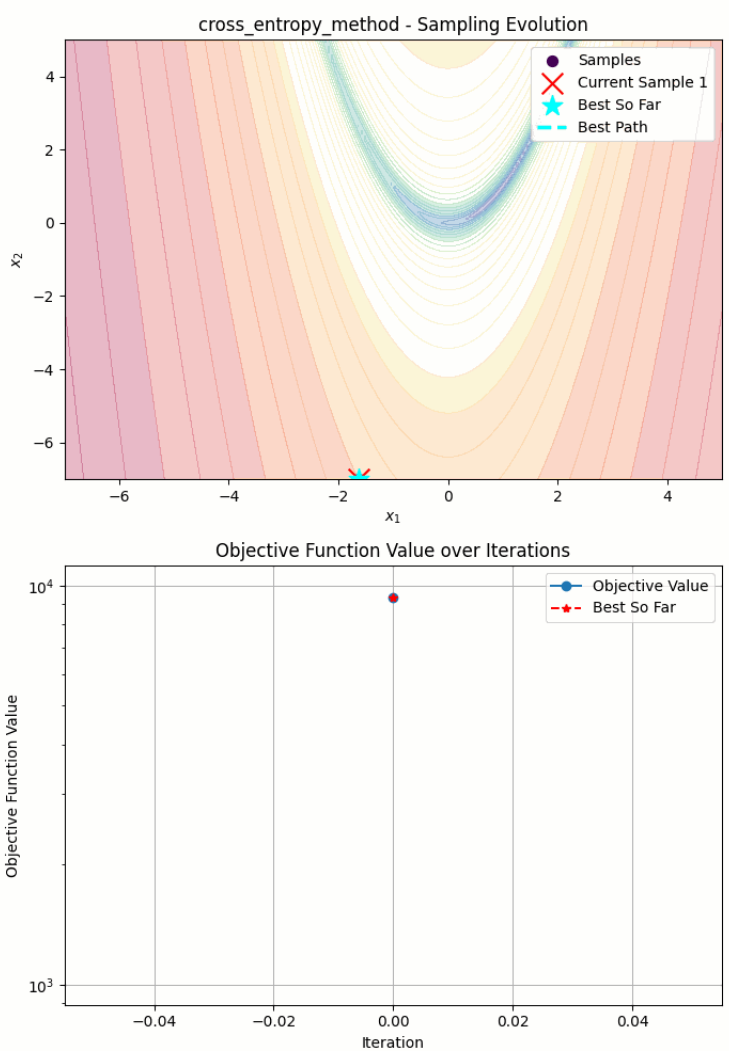


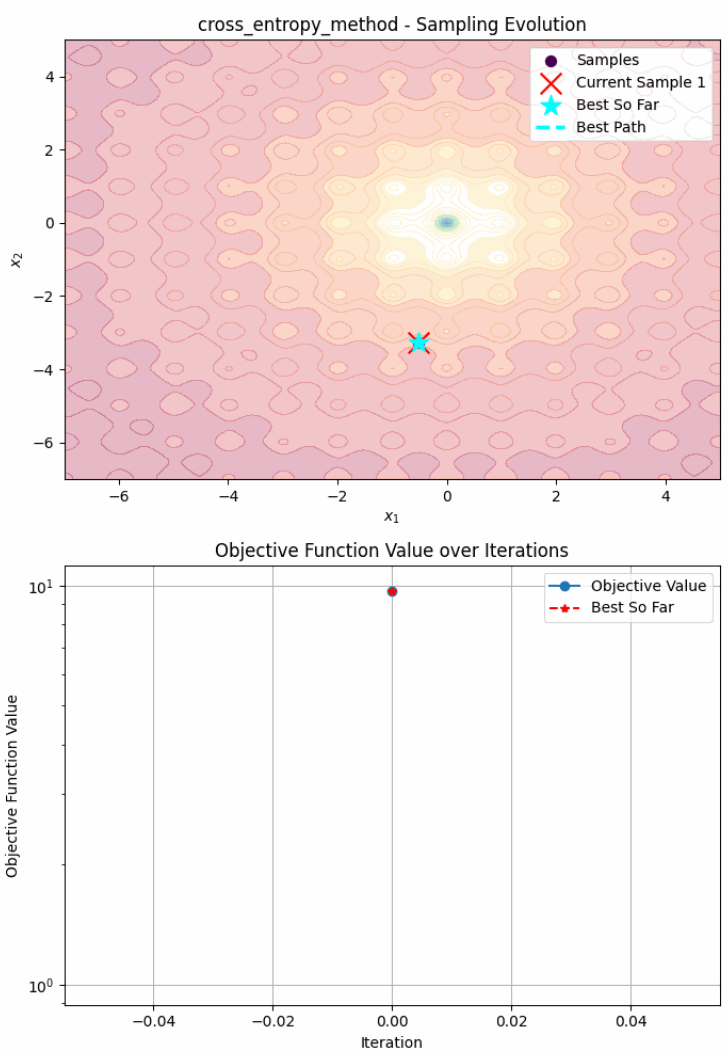


Direct derivative free methods

Cross-Entropy Method

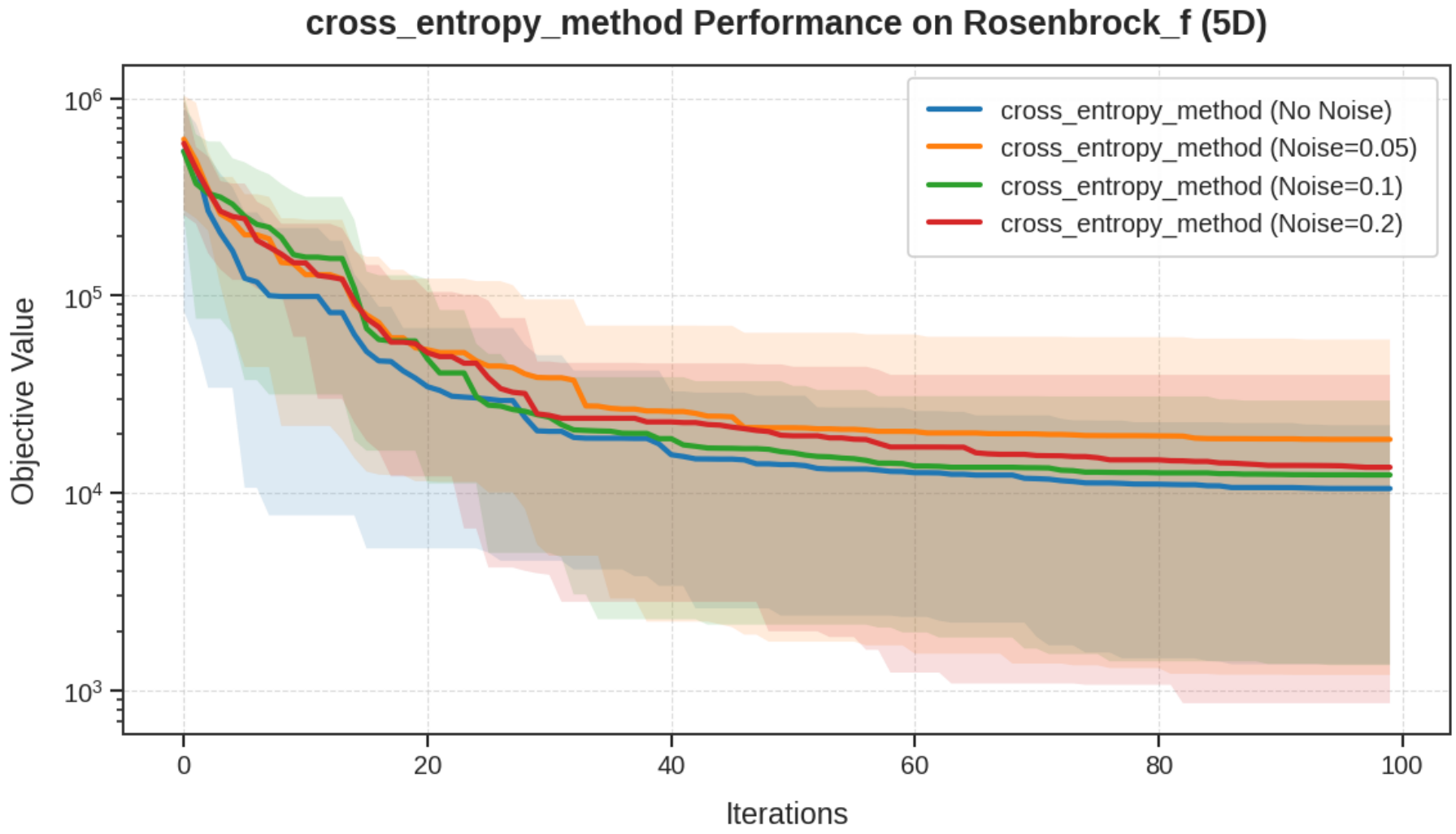






Direct derivative free methods

Cross-Entropy Method



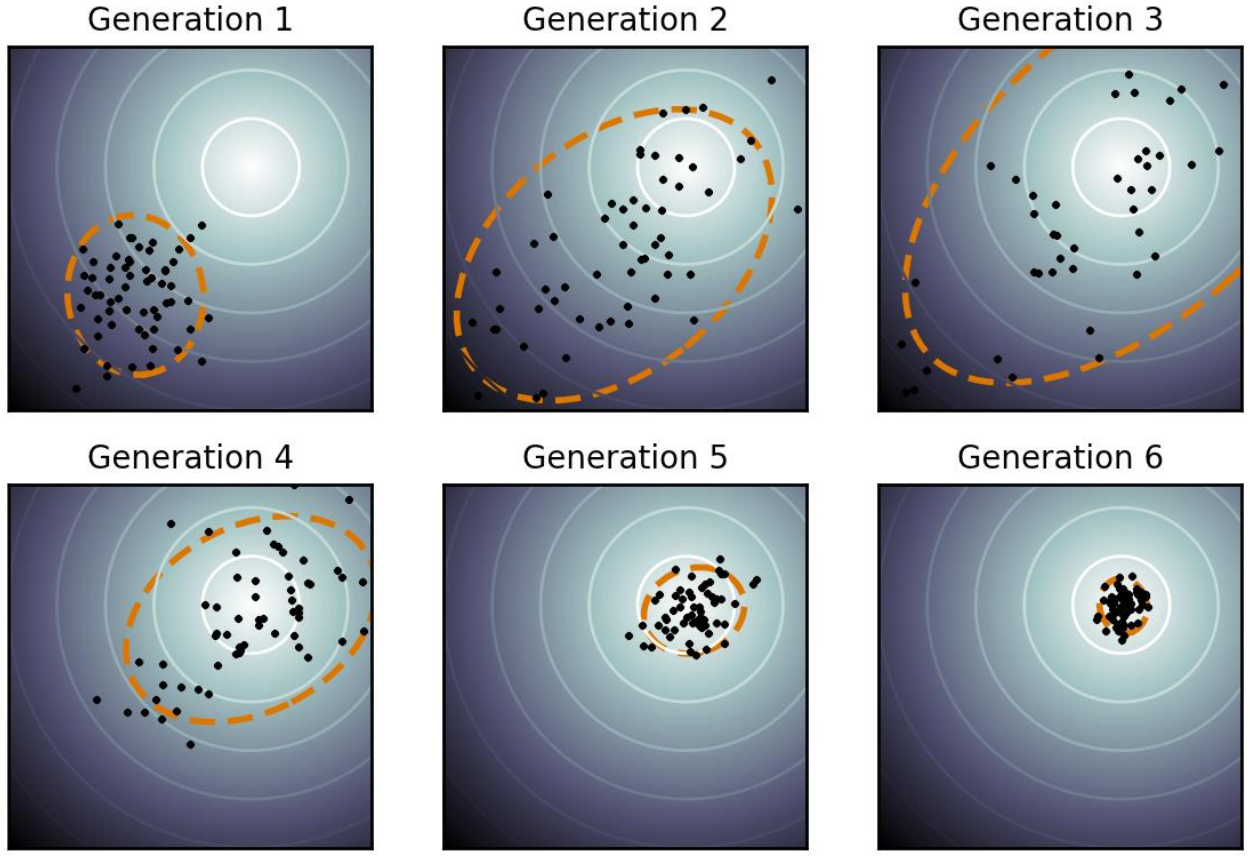
Direct derivative free methods

CMA-ES (Covariance matrix adaptation evolution strategy)

Combines elements from:

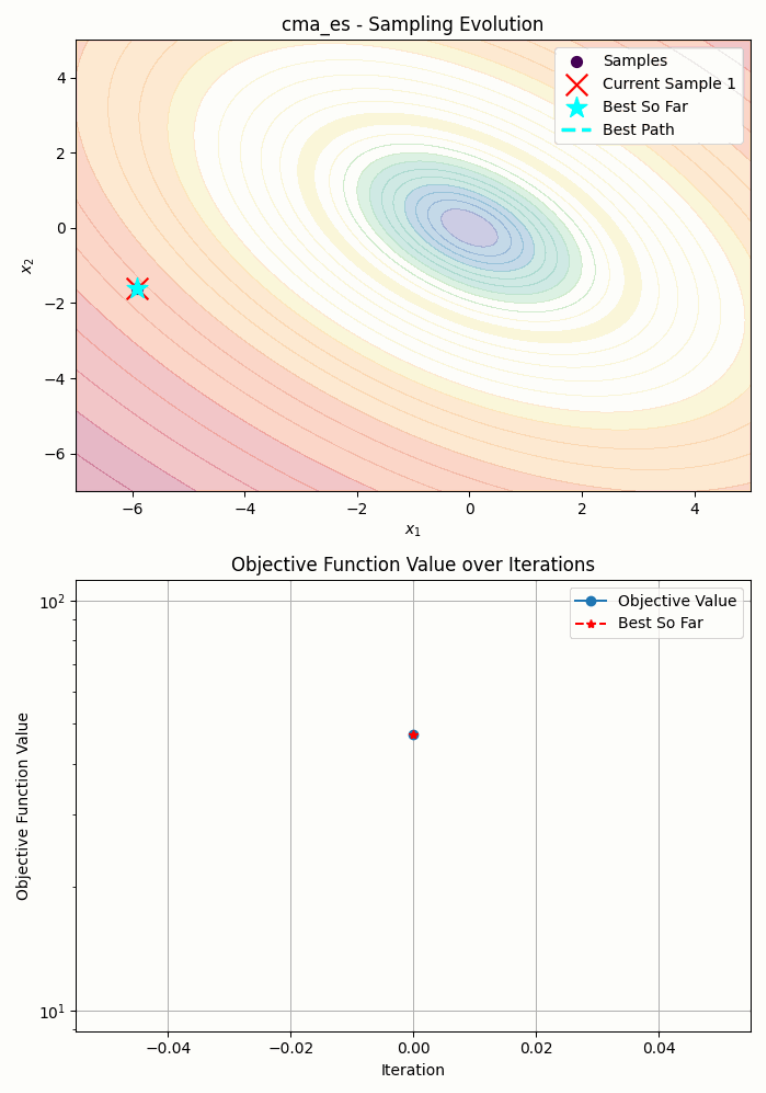
- Differential evolution
- Mean and covariance learning

One of the most widely used (and best) metaheuristic and evolutionary strategies.



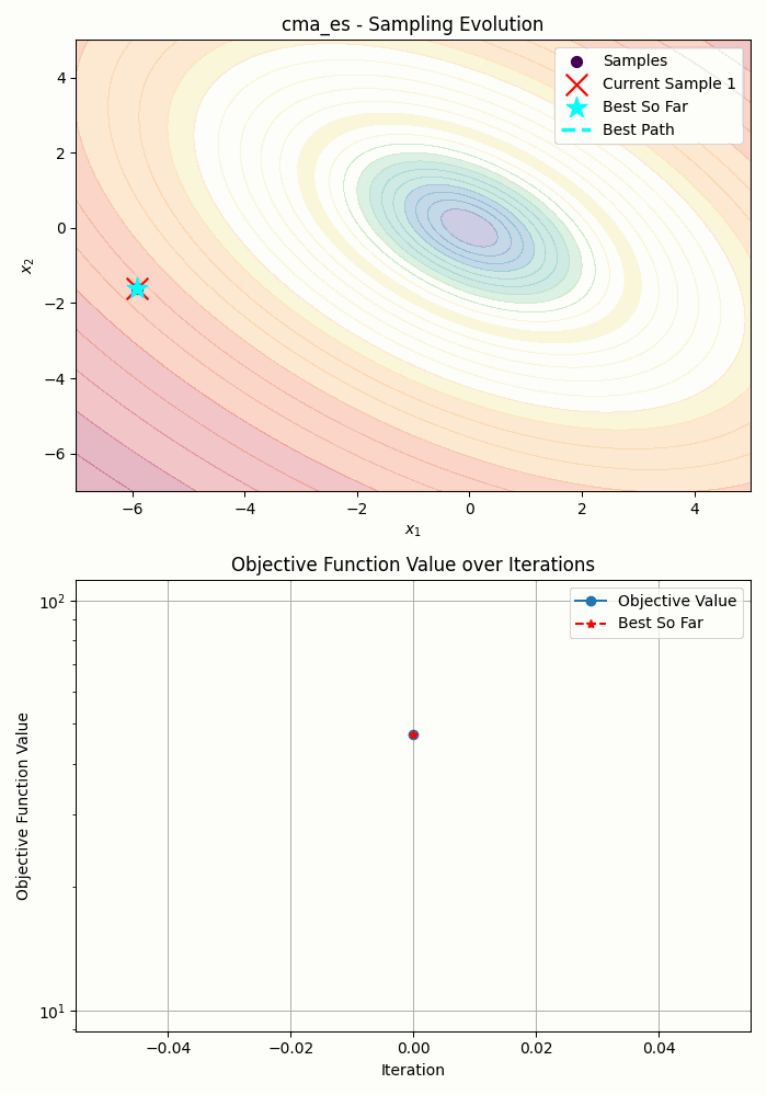
Direct derivative free methods

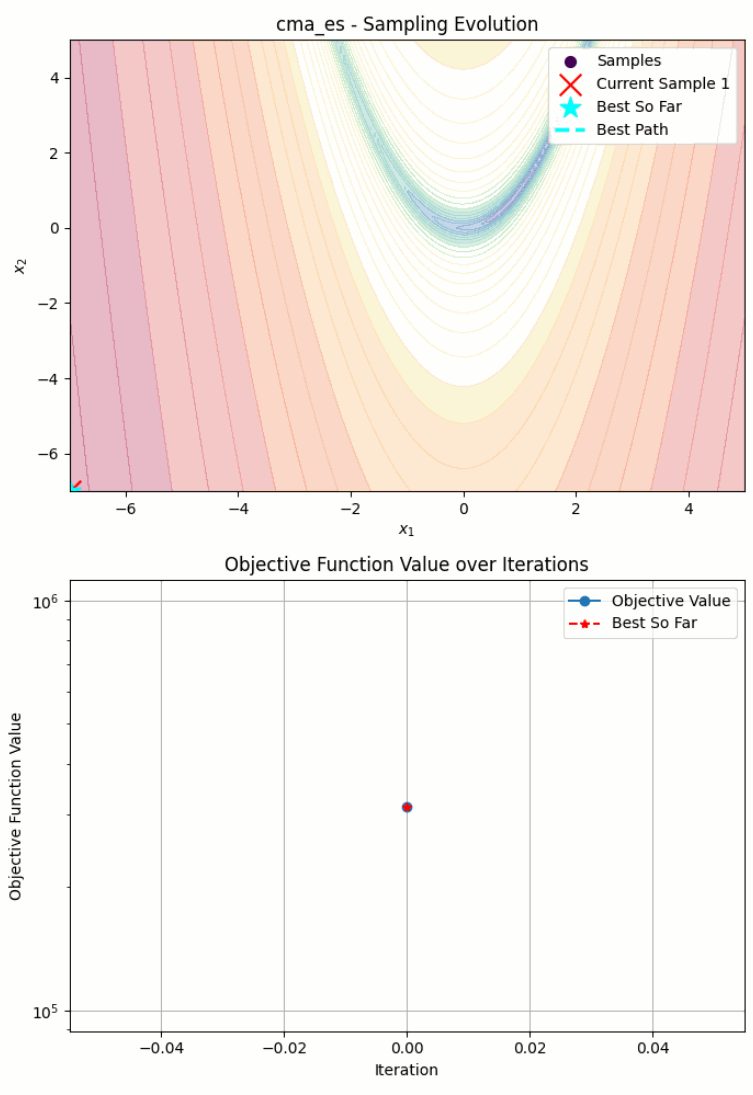
CMA-ES (Covariance matrix adaptation evolution strategy)



Direct derivative free methods

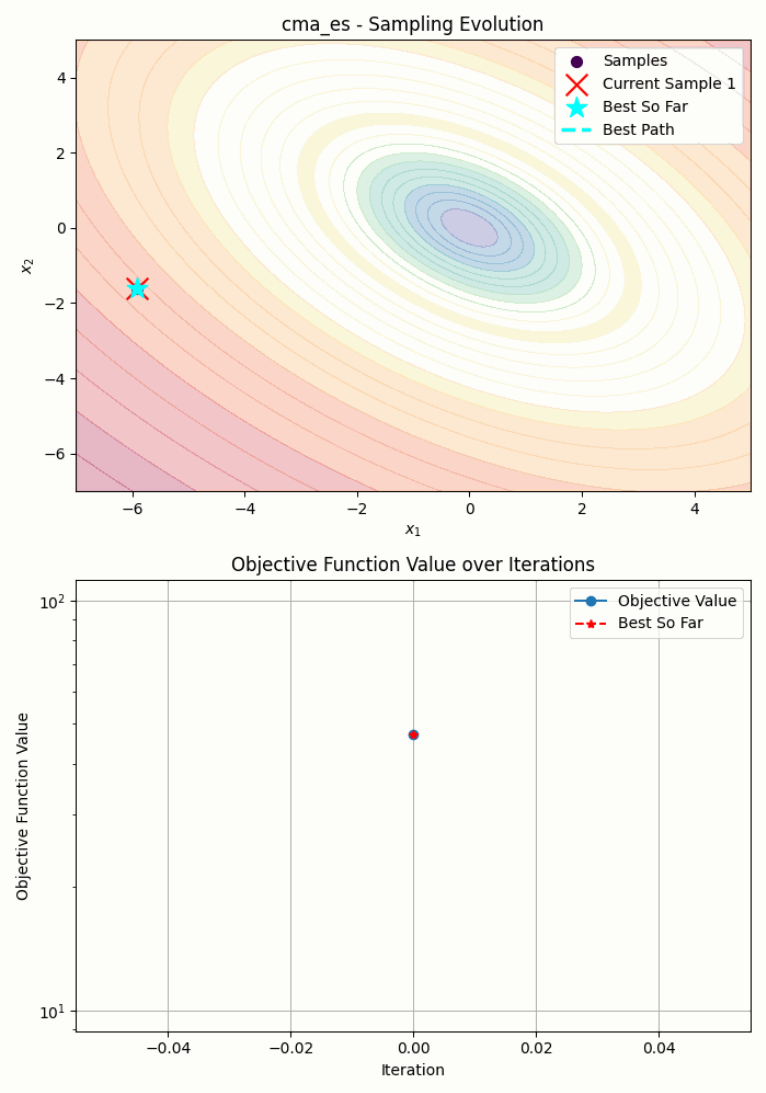
CMA-ES (Covariance matrix adaptation evolution strategy)

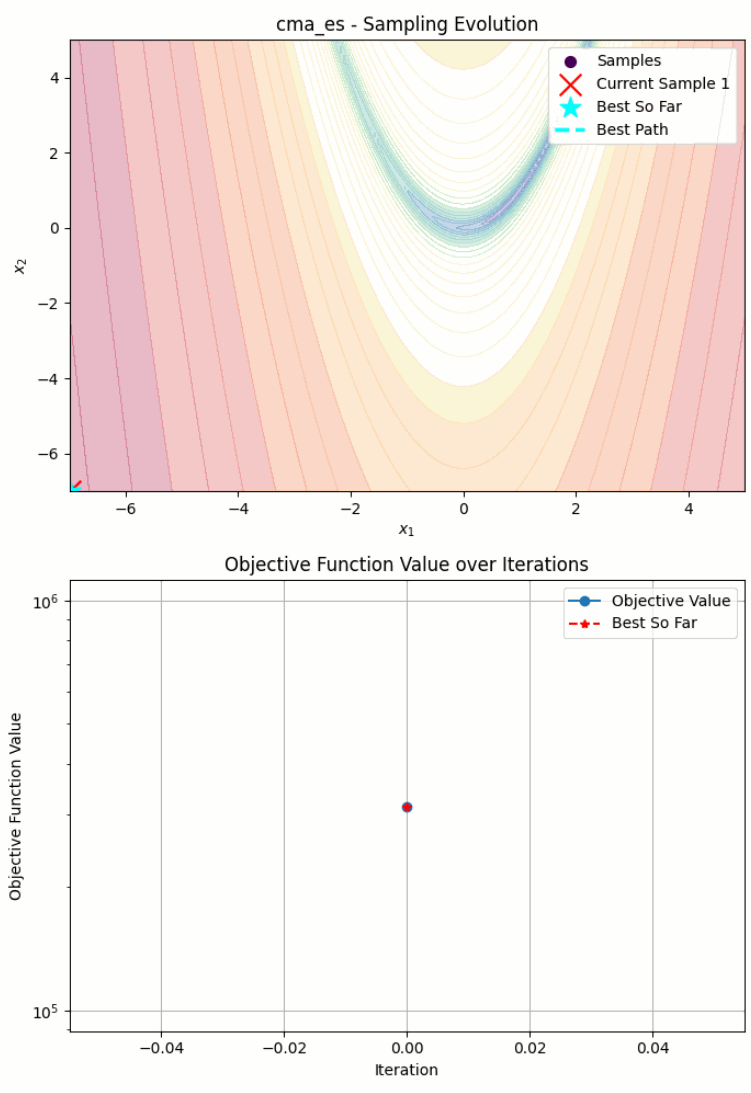


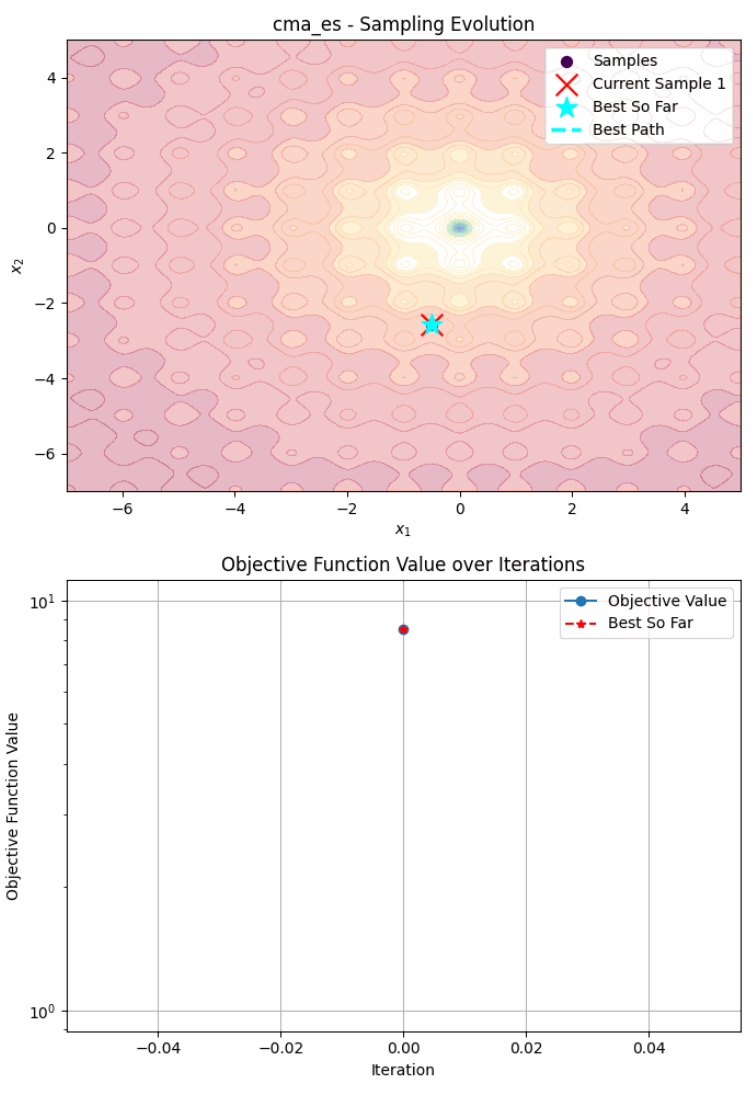


Direct derivative free methods

CMA-ES (Covariance matrix adaptation evolution strategy)

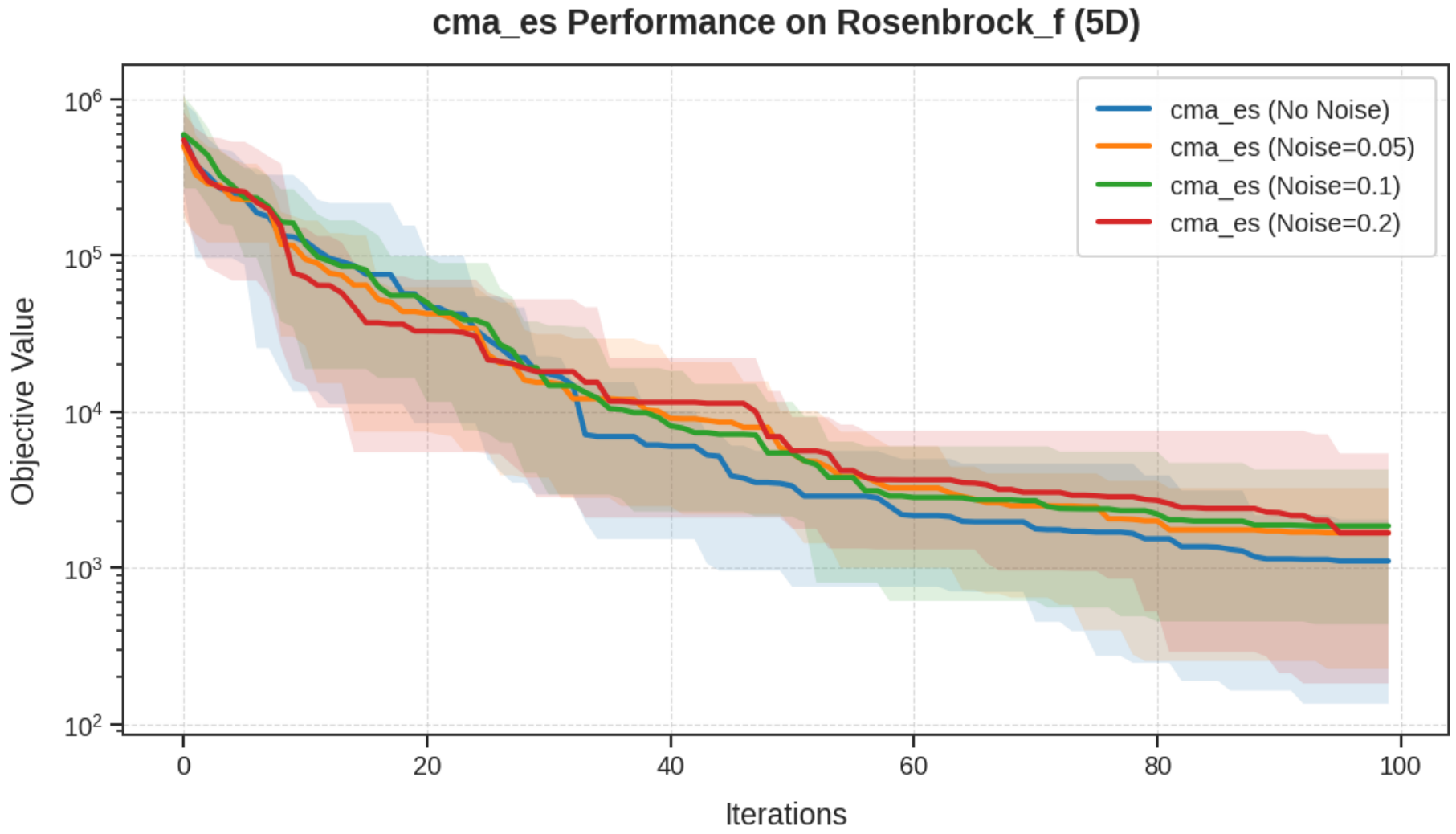






Direct derivative free methods

CMA-ES (Covariance matrix adaptation evolution strategy)



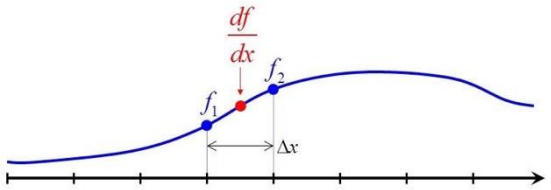
A closer look at some **finite-difference** algorithms

Finite-difference derivative free methods

Adam

Gradients are calculated via finite differences

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$ second-order accurate first-order derivative



Given $\mathbf{x}_0$, we calculate $\boldsymbol{\nabla}_x f(\mathbf{x}_0)$:

$$\boldsymbol{\nabla}_x f(\mathbf{x}_0) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_0) \\ \vdots \\ \frac{\partial f}{\partial x_{n_x}}(\mathbf{x}_0) \end{bmatrix} \approx \begin{bmatrix} \frac{\Delta f(\mathbf{x}_1)}{h} \\ \vdots \\ \frac{\Delta f(\mathbf{x}_{n_x})}{h} \end{bmatrix}$$

with n_x evaluations

Adam

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \frac{\mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1)^{\text{corr}}}{\sqrt{\mathbf{s}_k(\mathbf{x}_k, \boldsymbol{s}_{k-1}, \beta)^{\text{corr}}}}$$

α is leaning rate or step size

Algorithm: Adam

1. **Start** $\mathbf{x}_k := \mathbf{x_0}$
2. **Until** $\|\boldsymbol{\nabla}_x f(\mathbf{x}_k)\| \leq \varepsilon$
   $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha\, \mathbf{g}_k{}^{\text{corr}} / \sqrt{\boldsymbol{s}_k^{\mathbf{corr}}}$
   end

with $\varepsilon \approx 10^{-5}$

Continuous Unconstrained Optimization – Adam

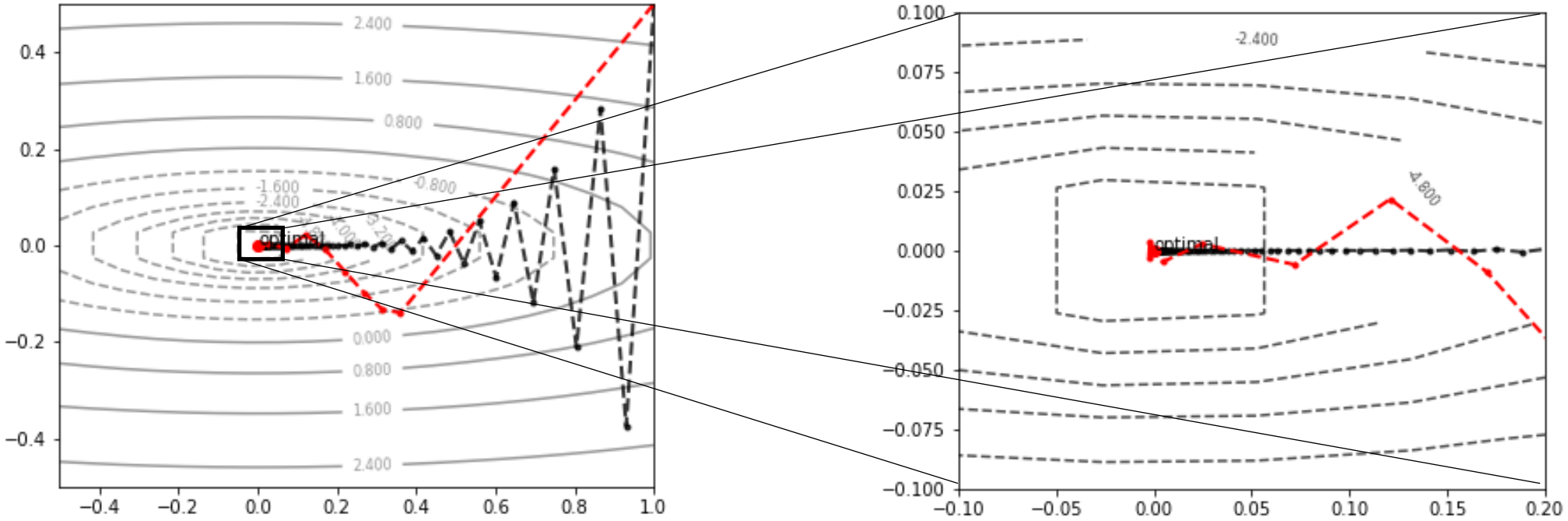
Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) \coloneqq (x_1)^2 + (5x_2)^2$$

Adam

Adam combines momentum with Adaptive learning rates (RSMProp).

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla f(\mathbf{x}_k)$$



Continuous Unconstrained Optimization – Adam

Problem statement

$$\min_{\mathbf{x}\in\mathbb{R}^{n_x}} \quad f(\mathbf{x}) \coloneqq (x_1)^2 + (5x_2)^2$$

Adam

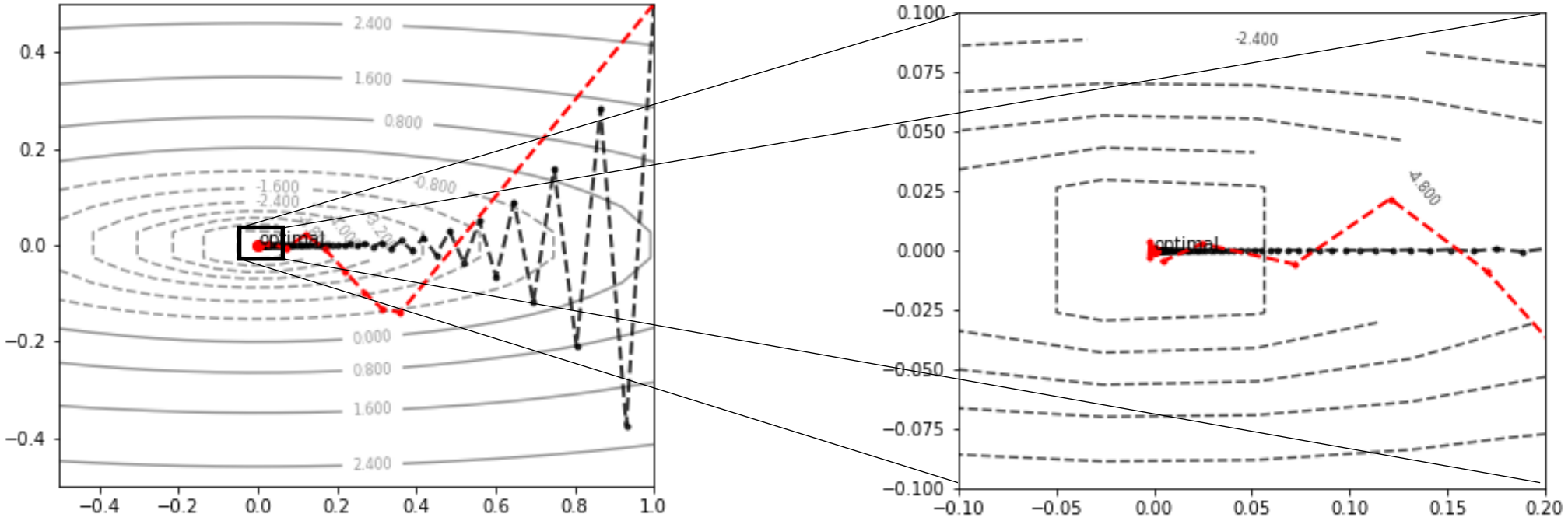
$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \frac{\mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1)^{\text{corr}}}{\sqrt{\mathbf{s}_k(\mathbf{x}_k, \mathbf{s}_{k-1}, \beta)^{\text{corr}}}}$$

momentum → $\mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1) \coloneqq (\beta_1)\mathbf{g}_{k-1} + (1-\beta_1)\nabla_x f(\mathbf{x}_k)$

adaptive rate → $\mathbf{s}_k(\mathbf{x}_k, \boldsymbol{s}_{k-1}, \beta_2) \coloneqq (\beta_2)\boldsymbol{s}_{k-1} + (1-\beta_2)\nabla_x f(\mathbf{x}_k)^2$

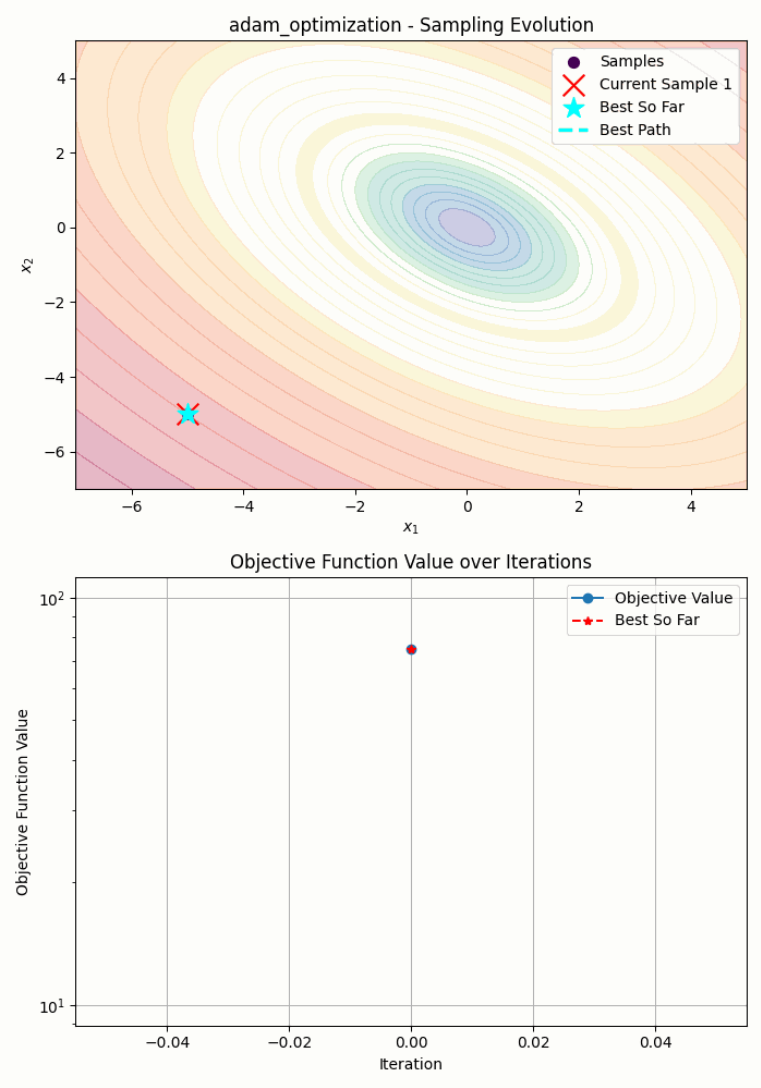
correction term

$$\begin{cases} \mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1)^{\text{corr}} = \mathbf{g}_k(\mathbf{x}_k, \mathbf{g}_{k-1}, \beta_1)/(1-\beta_1^i) \\ \mathbf{s}_k(\mathbf{x}_k, \mathbf{s}_{k-1}, \beta_2)^{\text{corr}} = \boldsymbol{s}_k(\mathbf{x}_k, \mathbf{s}_{k-1}, \beta_2)/(1-\beta_2^i) \end{cases}$$



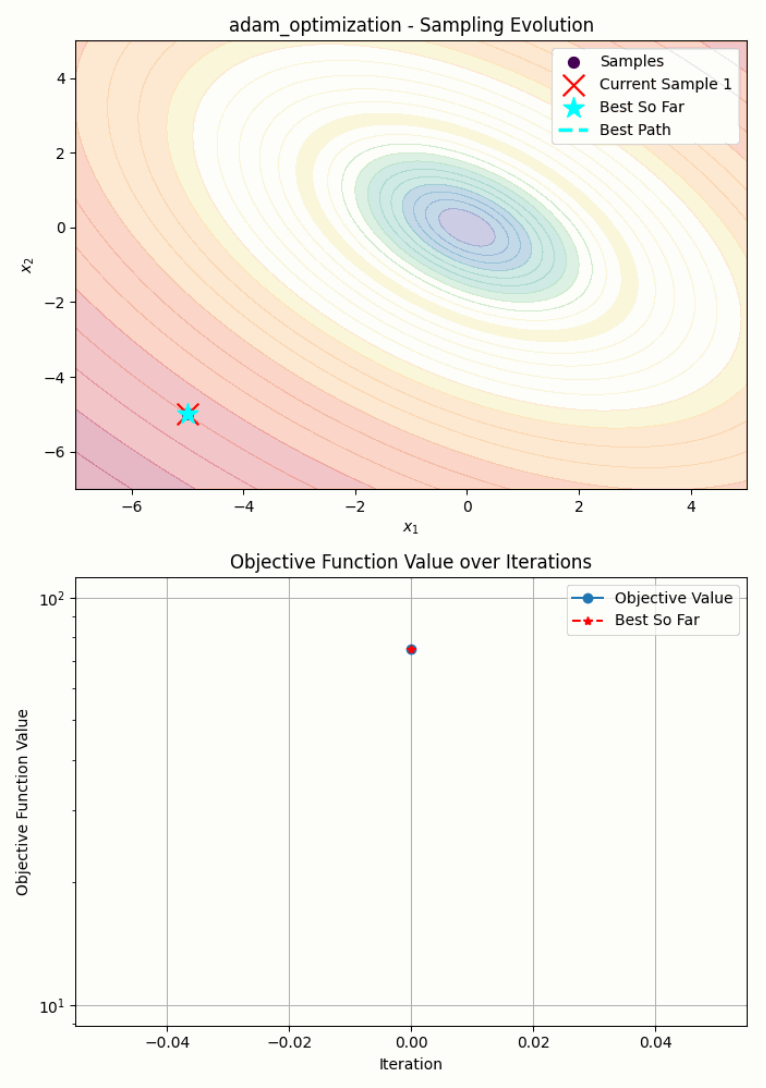
Finite-difference derivative free methods

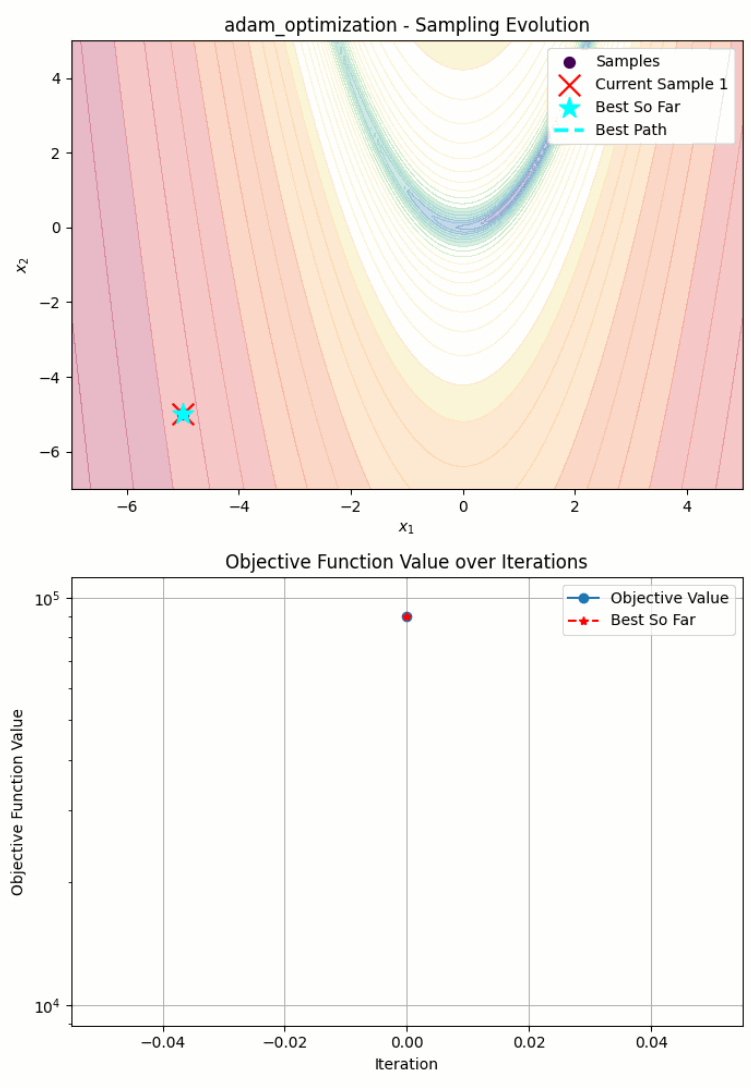
Adam



Finite-difference derivative free methods

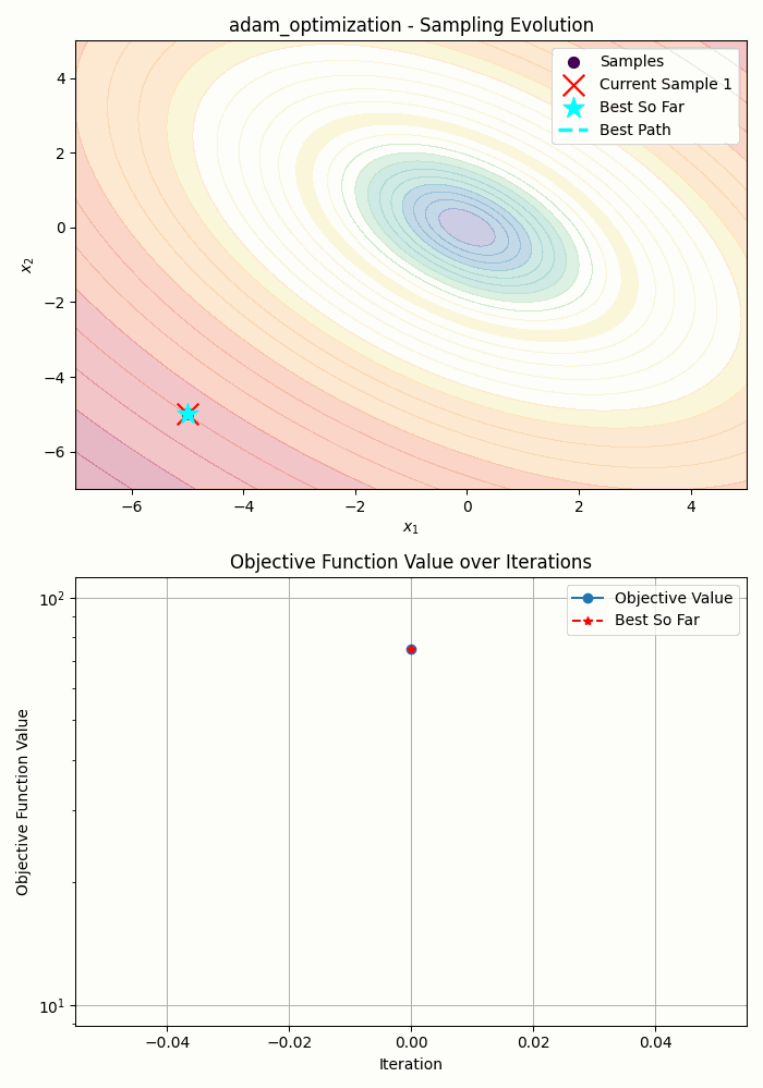
Adam

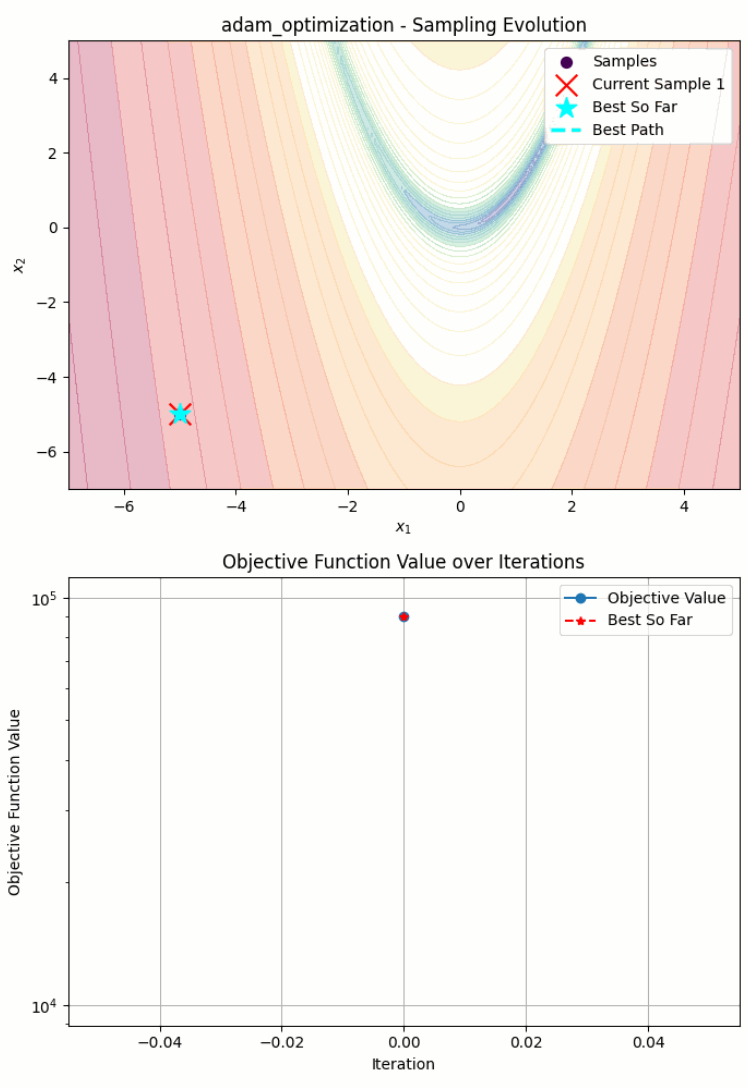


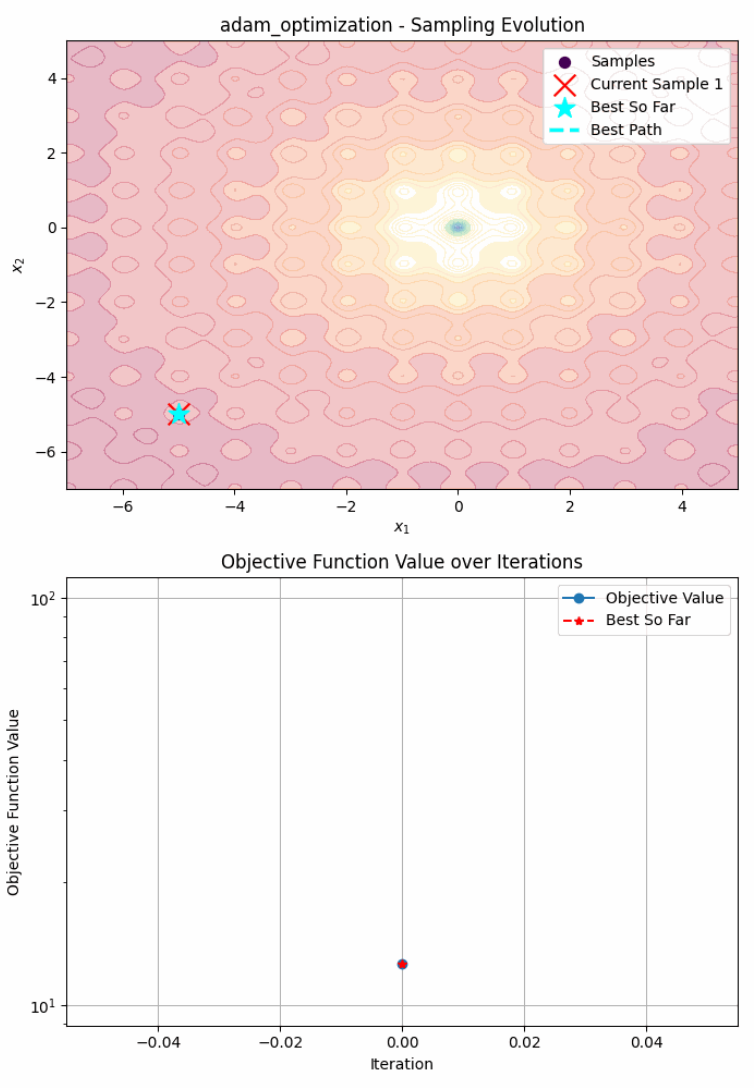


Finite-difference derivative free methods

Adam

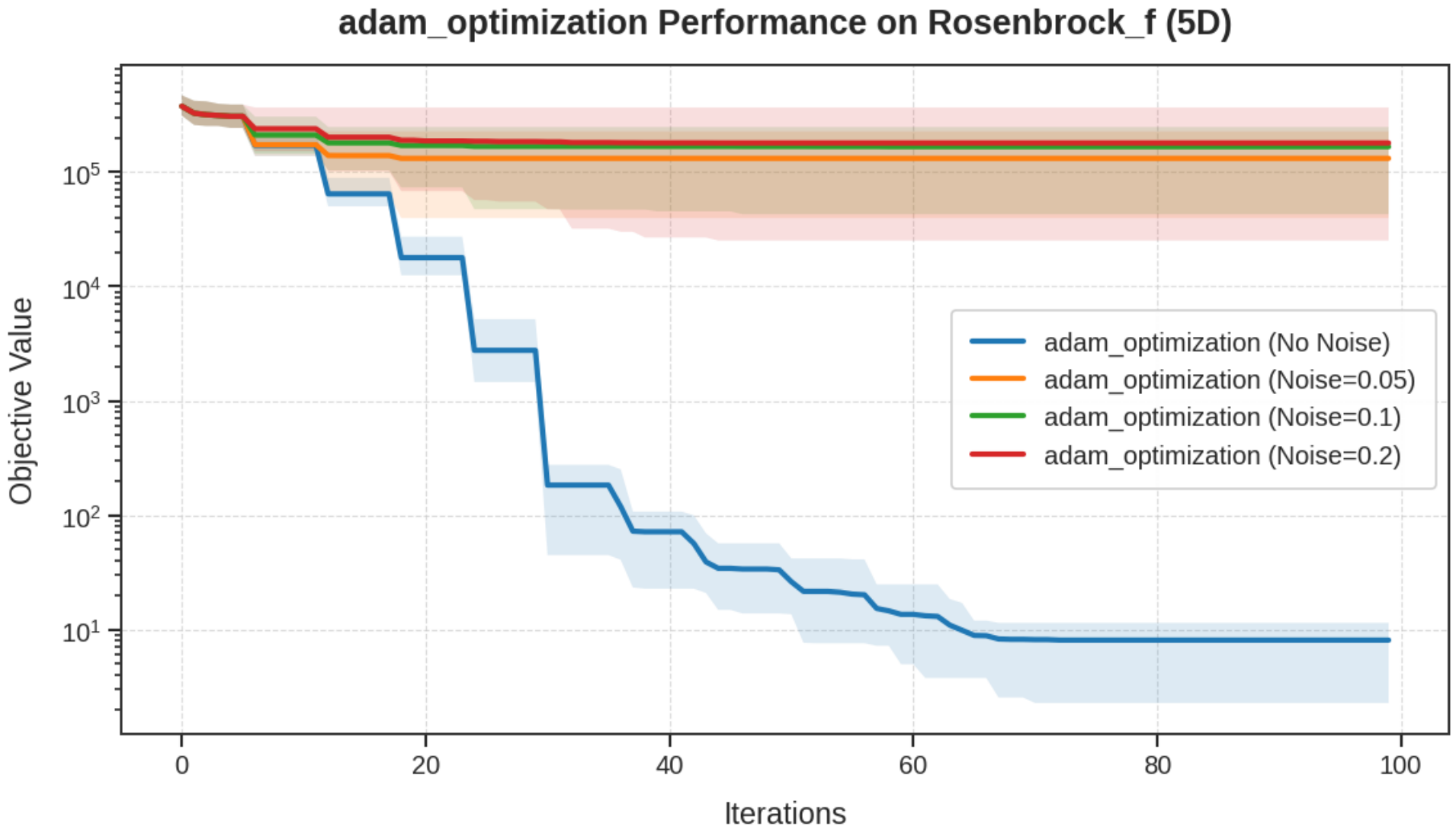






Finite-difference derivative free methods

Adam

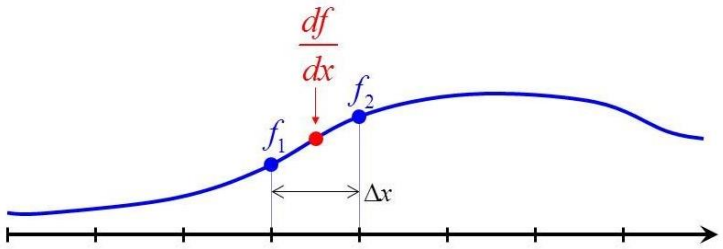


Data-driven optimization: Direct: Gradient Descent

Gradients are calculated via finite differences

$$\frac{df_{1.5}}{dx} \approx \frac{f_2 - f_1}{\Delta x}$$

second-order accurate first-order derivative



Given $\mathbf{x}_0$, we calculate $\nabla_x f(\mathbf{x}_0)$:

$$\nabla_x f(\mathbf{x}_0) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}_0) \\ \vdots \\ \frac{\partial f}{\partial x_{n_x}}(\mathbf{x}_0) \end{bmatrix} \approx \begin{bmatrix} \frac{\Delta f(\mathbf{x}_1)}{h} \\ \vdots \\ \frac{\Delta f(\mathbf{x}_{n_x})}{h} \end{bmatrix}$$

with n_x evaluations

Gradient Descent

Going in the direction of the most rapid function decrease

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \ \nabla_x f(\mathbf{x}_k)$$

α is leaning rate or step size.

On the numerical performance of finite-difference-based methods for derivative-free optimization

Hao-Jun Michael Shi, Melody Qiming Xuan, Figen Oztoprak & Jorge Nocedal

Pages 289-311 | Received 22 Feb 2021, Accepted 02 Sep 2022, Published online: 26 Sep 2022

Cite this article https://doi.org/10.1080/10556788.2022.2121832 Check for updates

DERIVATIVE-FREE OPTIMIZATION OF NOISY FUNCTIONS VIA QUASI-NEWTON METHODS*

ALBERT S. BERAHAS†, RICHARD H. BYRD‡, AND JORGE NOCEDAL§

Coursework: Data-driven optimisation